Mergeable Replicated Data Types in Sal:
Metatheory, the Embedded-Chain Family, and Sequential Specifications

Sal / FP Launchpad (KC + Claude)

July 20, 2026

one document consolidating three earlier notes (of July 7, July 14, and July 16–17, 2026),
extended July 18–19 with the runtime arc (virtual LCAs, commit GC, the stability VC) and the storage layer (re-coding, compaction, the run table),
and July 20 with the verified peer-to-peer runtime, a cross-system performance study, and a push-button SMT layer,
and July 21 with the rich-text marks arc (the document-order mark read model and the marks-layer GC)

Abstract

Mergeable replicated data types (MRDTs) resolve concurrent divergence by a *three-way* merge $\mathsf{mergeL}(l, a, b)$: the two divergent states a, b are reconciled relative to their lowest common ancestor l in a git-like version DAG. This document is the consolidated account of the Sal verification effort. Part I develops the mechanized metatheory: what an MRDT is and how to describe one; why the published soundness induction fails (a fully LCA-legal defeater) and why no lattice-style contract can replace it; the corrected architecture of canonical states and a ternary Join Lemma; the eight verification conditions that make it true, with their adequacy theorem and the discharged production catalogue; the conditioned generalization for state-dependent data types; and the factored discharge route (a generic honest-reachability metatheorem consuming per-configuration join lemmas) that the newest instances use, with the mergeable queue as its flagship. Part II presents the embedded-chain RGA, the repository's canonical sequence datatype: a tombstone-free replicated list whose sort keys are immutable birth chains carried as entropy-coded coordinates, with the encoding contract and entropy law, kernel-checked RA-linearizability parametric in the code, the compaction theorem identifying its reads with the published tombstoned RGA's, and the sided generalization that makes two-sidedness a parameter and hosts the Fugue/FugueMax arc; its extension closes the Weidner–Kleppmann maximal-non-interleaving theorem (the first mechanized such result, we believe), proves a representation impossibility naming the exact information a delete must leave behind, and adds a measured storage layer (re-coding at settled cuts, a verified concrete compaction, the run table). Part I is correspondingly extended with the runtime metatheory: virtual LCAs for criss-cross merges, commit GC with a proved keep set, and a stability VC under which state compaction is observationally invisible. Part III records the sequential-specification campaign: for every production RDT in Sal, a kernel-checked theorem that the fold of any sequentially honest single-replica history equals the output of an independent naive sequential program, together with three machine-checked refutations giving a two-axis separation between do-faithfulness and merge-faithfulness. A final part turns the design into a running system: a verified peer-to-peer MRDT runtime, datatype-parametric and content-addressed, whose garbage collectors are the executable form of the GC-safety and stability theorems, now covering the rich-text marks layer as well (dead mark anchors survive as counted retention roots, at most two per mark record, and the compaction is render-invisible by theorem), and whose git-backed store persists the commit DAG; a performance section measures the shipped artifact against

Yjs, Automerge, Loro, and list-positions on the standard editing-trace corpus, records the experiment that closed its one large loss (a three-to-four-orders apply-latency gap, diagnosed as an interface-inherited state copy and removed by a persistent-map container swap with every observable byte unchanged), and reports the design's categorical storage win, the only save that shrinks on delete; and an SMT translation-validation layer that makes the flat verification conditions push-button. Everything stated as a theorem is mechanized in Lean 4 with zero `sorry`s in the cited chains; the negative results are machine-checked countermodels, and every performance number is a labeled run of checked-in scripts.

How this document is organized. The four parts are followed by a single consolidated open-questions section and a single mechanization map. Part I is the metatheory and can be read alone; Part II depends on Part I's signature and its factored discharge route (§15); Part III depends on both, since its refutations calibrate what Part I's convergence theorems do and do not certify; Part IV is the systems pillar, a verified peer-to-peer runtime (§40) with a cross-system performance study (§41) and a push-button SMT layer (§42), and it depends on the embed kernel of Part II and the GC and stability metatheory of Part I. The document consolidates three earlier notes (the metatheory note of July 7, the embedded-chain design note of July 14, and the sequential-specification campaign note of July 16–17); reused material keeps its original wording, duplicated material appears once, and the previously separate open-question lists and mechanization maps are merged. A July 18–19 extension adds, to Part I, the runtime-facing metatheory (virtual LCAs for criss-cross merges, §16; commit GC and its keep set, §17; the stability VC for verified compaction callbacks, §18) and, to Part II, the closed maximal-non-interleaving theorem (§24), the chain-price impossibility (§25), and the storage layer with its measurements (§26). A July 20 extension adds Part IV (the verified runtime, its performance study, and the SMT layer) and closes the storage layer's mechanization (spine fusion, the merge-remap merge clause, multi-epoch composition, and the one-sided run table); the honesty-rebasing lemma $(*)$ that extension left open is since closed in Lean (the coded-forest cluster of §26.2), leaving the multi-replica protocol half (a common cut, concurrent divergent compaction, and the derivation of the per-continuation ContOK discipline from a delivery protocol) as the remaining compaction residue. A July 21 extension adds to Part II the rich-text marks arc (§27): the document-order mark read model and the marks-layer GC, with the runtime's Peritext datatype accordingly promoted from GC-refusal to certified fire. One editorial rule governs the whole document: the *rehoming* tombstone-free RGA, an earlier design demoted after its sequential refutation, receives no narrative treatment here; it appears only as the campaign's named countermodel (Part III), in one-line contrasts where the separation needs the losing side, and once as a generality stress test (§16), where its role is again that of the hardest available instance rather than a design of interest.

Contents

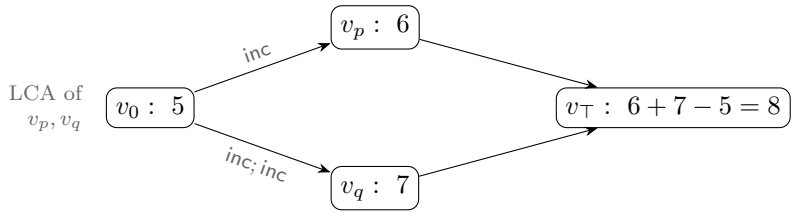


Figure 1: The counter MRDT. Both branches inherit the 5 from the fork point; the LCA argument lets the merge count each branch's *delta* exactly once. No binary merge of 6 and 7 alone can recover 8: the information "how much is shared" lives in the DAG, and the LCA argument is how the data type reads it.

Part I
The metatheory

1 Introduction

An MRDT execution looks like a git history: replicas fork, apply operations locally, and merge. Unlike a state-based CRDT, whose binary merge must reconcile two states with no memory of their common past, an MRDT's merge receives a third argument — the state at the *lowest common ancestor* (LCA) of the two versions being merged — and may use it to compute *deltas*: what each side did since the fork. The paradigmatic example is the counter,

$$\mathsf{mergeL}(l, a, b) \;=\; a + b - l,$$

which adds the two sides' increments-since-l instead of double-counting their common past (Figure 1).

The Neem methodology (OOPSLA 2025) promises a clean division of labour: the data-type designer discharges a fixed catalogue of equations over do, mergeL and the conflict-resolution relation rc — mostly by SMT — and a soundness meta-theorem, proved once, converts those VCs into *RA-linearizability* of every reachable configuration. This part documents what a Lean 4 mechanization of that meta-theorem, in the full **ternary, version-DAG** setting, found: the meta-theorem's central induction is unsound as written, and for LCA-sensitive data types several of the catalogue's merge VCs are false in principle when quantified over all states. It also develops the corrected metatheory: a set-relative linearization order, canonical states $\sigma(E)$ (each event set determines a unique state), and a ternary *Join Lemma* whose induction consumes a small, contextual VC surface. Its contributions:

1. **The paper's LCA lemma is true but its proof has a gap** (§4.1): $L(v_\ell) = L(v_1) \cap L(v_2)$ holds, but only as a *reachability invariant* of the store, whose proof needs a generator-uniqueness induction the paper silently assumes. We mechanize the transition system and the invariant.

2. **The soundness proof has a fully LCA-legal defeater** (§4.2): a reachable configuration of a legal add-wins MRDT — its final merge sitting at an honest LCA — on which every bottom-up peel of the paper's derivation rules is stuck. The meta-theorem must be rebuilt, not patched.

3. **The lattice contract does not transfer; a delta contract does** (§6): the lattice laws are equations over *free* tuples — the wrong instances for an LCA-aware merge. Side-idempotence $\mathsf{mergeL}(l, a, a) = a$ with l free fails for the counter $(2a - l)$, but its only DAG-honest instance has $l = a$ (the LCA of a version with itself *is* that version), where it holds trivially; and when two *distinct* histories reach equal side states, $2a - l$ is the *correct* merge — both branches' increments count — not an idempotence failure. The induced orders degenerate, and the honest associativity law has *four distinct LCAs* and holds only on feasible tuples. The moral: the algebra of an LCA-aware merge is an algebra of *causal histories*, not of concrete states. What replaces the lattice laws is a two-law *delta contract* — redistribution identities in which the LCA slot does, by arithmetic, the job idempotence does order-theoretically for CRDTs.

4. **Eight VCs suffice** (§9–§10): three relational VCs on rc (one of them carrying a different-replica guard — same-replica pairs are ordered by program order, and demanding rc cover them is unsatisfiable for real data types), merge symmetry, three feasibility-conditioned delta laws, and one contextual *causal-delta* equation. The adequacy theorem is per *version*: every version in the store, LCAs included, linearizes.

5. **The VC set is validated on the production catalogue** (§11): the flat production MRDTs shipped in Sal — OR-set, an optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, a tombstone RGA, Peritext, the multi-valued register and the add-wins priority queue — are discharged end-to-end, kernel-checked; the sequence data types with physical deletion go through the conditioned framework and its factored discharge route (§12–§15, Part II). The sweep produces an empirical *class map* of which data types need which strength of contract, with two surprises: tombstones are a metatheoretic trivializer, and commutativity class and delta class are independent axes.

6. **The metatheorem factors** (§15): conditioning splits as a generic honest-reachability metatheorem times a per-datatype join discharge (`HonestReach`, `JoinLemma3At`), with a uniform honesty shape (a client-checkable predicate holding of each event at the fold of its causal past). Three join species are observed in production (conditioned commutation, flat commutation, direct witness), and the mergeable queue, whose conflict graph admits no rc assignment at all, is the flagship of the direct-witness species: Peepul's merge is its own linearization certificate. The same route carries Part II's embedded-chain RGA.

7. **The model meets its runtime** (§16–§18): the criss-cross gate on Merge is lifted by a recursive virtual-LCA rule whose synthesized base provably carries exactly the intersection event set, with no new per-datatype VC for join-lemma datatypes; commit GC is proved safe against a keep set read off pairwise head intersections, with a countermodel showing head-sync is load-bearing; and state-level compaction (the GC callback into the datatype) gets a six-clause stability VC whose metatheorem makes compaction observationally invisible, with the OR-set as first instance and a refuted naive gate as the shaping countermodel.

Everything stated as a theorem below is mechanized with zero `sorry`s; §44 maps statements to Lean names and files (foundations under `Framework/`, metatheorems under `Metatheory/`, negative results under `Refutations/`, the per-RDT instances under `MRDT_Instances/`, the investigation record under `Development/`; each cited theorem kernel-checked).

2 Mergeable replicated data types, by definition and example

Data type and implementation. A replicated data type of *type* $\tau = \langle O_\tau, Q_\tau, Val_\tau \rangle$ exposes a set of update operations O_τ, a set of query operations Q_τ, and a domain of query return values Val_τ.

Definition 2.1 (flat MRDT signature)**.** *An MRDT implementation of* τ *is a tuple*

$$\mathcal{D} \;=\; \langle \Sigma,\ \sigma_0,\ \mathsf{do},\ \mathsf{mergeL},\ \mathsf{query},\ \mathsf{rc} \rangle,$$

where Σ *is the space of replica states with initial state* $\sigma_0 \in \Sigma$*;* $\mathsf{do} : \Sigma \times \mathcal{E} \to \Sigma$ *applies an event — an update-operation instance* $e = (t, r, o)$ *carrying a globally unique timestamp* t*, its origin replica* r*, and the operation* $o \in O_\tau$*;* $\mathsf{mergeL} : \Sigma^3 \to \Sigma$ *is the three-way merge, whose first argument is the state at the lowest common ancestor of the two versions being merged;* $\mathsf{query} : \Sigma \times Q_\tau \to Val_\tau$ *reads a state; and* $\mathsf{rc} : \mathcal{E} \times \mathcal{E} \to \{\mathit{Fst\_then\_snd}, \mathit{Snd\_then\_fst}, \mathit{Either}\}$ *is the conflict-resolution relation, consulted to order concurrent non-commuting operations (it is the data-type designer's declaration of who wins a race;* *Snd_then_fst* *is the conventional mirror of* *Fst_then_snd*, *and production instances do return it, so the VCs below only ever test the* *Fst_then_snd* *direction); we write* $e_1 \xrightarrow{\mathsf{rc}} e_2$ *for* $\mathsf{rc}(e_1, e_2) = \mathit{Fst\_then\_snd}$*.*[1] `MRDTSig`

Queries never change state and play no role in the metatheory; the theory below reads the implementation as $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc} \rangle$.

Describing an MRDT: the counter. To describe an MRDT one gives the five components and, when operations can race, says who wins. The increment-only counter of Figure 1 is the smallest complete example:

$$\Sigma = \mathbb{Z}, \qquad \sigma_0 = 0, \qquad \mathsf{do}(\sigma, (t, r, \mathsf{inc})) = \sigma + 1, \qquad \mathsf{mergeL}(l, a, b) = a + b - l, \qquad \mathsf{rc} = \emptyset.$$

All operations commute, so rc is empty, yet the merge still reads the shared past off its LCA: $-\,l$ subtracts the common history that $a + b$ alone would double-count. The pattern recurs throughout the catalogue.

A data type with races: the OR-set. The add-wins observed-remove set stores tagged elements: add inserts a tag unique to the adding event, rem deletes exactly the tags it has *seen*. Sequentially, a remove after an add deletes it; concurrently the designer declares the add the winner: $\mathsf{rem} \xrightarrow{\mathsf{rc}} \mathsf{add}$. The merge keeps a tag iff both sides have it (nobody deleted it) or some side added it since the fork:

$$\mathsf{mergeL}(l, a, b) \;=\; (a \cap b) \cup (a \setminus l) \cup (b \setminus l).$$

The LCA here plays the role a tombstone set plays for the CRDT OR-set: deletion is absence *relative to* l, and the state needs no graveyard.

[1] In the mechanization the ternary signature `MRDTSig` extends the binary `CRDTSig` (which contributes Σ, σ_0, do, query, rc, and a *binary* merge) with the field mergeL and one reuse contract, `merge_init_slice`: $\forall a\, b,\ \mathsf{mergeL}(\sigma_0, a, b) = \mathsf{merge}(a, b)$, pinning the inherited binary merge to the $l \coloneqq \sigma_0$ slice so the binary corpus runs unchanged on the init-LCA sub-family.

The generalized signature. Both examples' operations make sense on *every* state: any state can be incremented, any tag can be added. Such data types are *flat*, and for them the metatheory of §3–§11 quantifies its VCs over all states. But a data type whose operations carry *preconditions* — insert under a node that must exist, delete a node that must be live — breaks that quantification: its commutation equations are simply false at nonsense states. The full signature therefore extends the tuple.

Definition 2.2 (conditioned MRDT signature)**.** *A conditioned MRDT implementation extends the flat tuple with two declarations,*

$$\mathcal{D} \;=\; \langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc},\ \mathsf{Inv},\ \mathsf{app}\rangle, \qquad \mathsf{Inv} : \Sigma \to \mathrm{Prop}, \quad \mathsf{app} : \mathcal{E} \times \Sigma \to \mathrm{Prop},$$

together with an observational equivalence $\approx \subseteq \Sigma \times \Sigma$*:* Inv *is a state invariant (a shape over-approximation of reachability),* app *an applicability predicate (when an event is sensible at a state — it may read the event's timestamp, so it is a generation-time guard, not a shape predicate), and* $\approx$ *an observational equivalence (what queries can distinguish; representation residue is quotiented away). In the mechanization* Inv *and* app *are fields of the signature;* $\approx$ *is not — the quotient layers carry it as a parameter — which is why the tuple above says "together with".* `ConditionedMRDTSig`

Commutation is then required only at Inv-states where both events are applicable: the derived *conditioned commutation* (`ConditionedMRDTSig.commutesOn`) demands $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ only for σ with $\mathsf{Inv}(\sigma)$, $\mathsf{app}(e_1, \sigma)$, and $\mathsf{app}(e_2, \sigma)$. Its VCs are formalized in §13. Flat data types take $\mathsf{Inv} = \mathsf{app} = \top$ and $\approx\ =\ =$, collapsing the general signature to the plain one — one framework, of which §3–§11 is the flat instance and §12 onwards the general case.

The running hard example: sequences with physical deletion. The datatype class that forces the general signature is the replicated list underlying collaborative text editing. Characters are nodes identified by timestamps, each inserted *after* an anchor node; production RGAs keep deleted nodes as *tombstones* because later concurrent inserts may still anchor on them, and a *tombstone-free* design deletes physically. Its operations are precondition-laden: an insert names an anchor that must exist, a delete names a live node, and the flat update layer quantifies its commutation VCs over every state, including nonsense states no execution produces, where those equations are false. Worse, what makes reachable-state commutation true is a property of the *execution that generated the operation* (the issuer saw the anchor live), not of the state the operation later meets, so it is not expressible as a $\forall\sigma$ equation over the signature alone. This is the reason the general signature exists: Inv and app name the sensible states and operations, an honesty discipline makes generation-time facts available to the proofs, and $\approx$ absorbs representation residue. Part II presents the embedded-chain RGA, the sequence datatype that holds the repository's canonical seat, together with its end-to-end discharge through the factored route of §15; a predecessor design that carried recorded ancestor paths and rehomed orphans on delete was proved RA-linearizable in the same framework and was later demoted by its sequential refutation (Part III).

3 The ternary setting

This section fixes the model: the data-type signature, the replicated store and its transitions, and the specification the metatheorem must deliver. The definitions are the paper's, with one correction flagged where it occurs.

Signature. An MRDT is $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc} \rangle$ as in Definition 2.1, read flat for now ($\mathsf{Inv} = \mathsf{app} = \top$, $\approx\ =$; the general case resumes in §12). We write $e(\sigma)$ for $\mathsf{do}(\sigma, e)$, and $e_1 \rightleftarrows e_2$ when $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ for every σ (`CRDTSig.commutes`).

Formal preliminaries. Events are triples $e = (t, r, o)$ with $\mathsf{time}(e) = t$, $\mathsf{rep}(e) = r$, $\mathsf{op}(e) = o$. We write $\mathsf{fold}(\sigma, \pi)$ for the left fold $e^n(\cdots e^1(\sigma)\cdots)$ of a list $\pi = e^1 \cdots e^n$ (`applySeq`). Two events are *distinct* (`distinctOps`) when their *timestamps* differ — timestamps are globally unique, so between events of one execution this is inequality of the events — and *cross-replica* (`differentReplicas`) when their origin replicas differ. A list π *enumerates* a set E (`listPermOf`) when it is duplicate-free and contains exactly the elements of E; π *respects* a relation R (`respects`) when no pair occurs against R: $i < j$ implies $\neg R(e^j, e^i)$ (stated elementwise; R need not be transitive). The *recorded past* of an event (`downset`), relative to the ambient configuration's visibility, is

$$\downarrow e \;\coloneqq\; \{\, x \mid x = e \;\vee\; x\,(\xrightarrow{\mathsf{vis}} \wedge \not\equiv)^{+}\, e \,\},$$

the transitive closure into e of visible *non-commuting* predecessor edges together with e itself: the fragment of e's causal past that a fold can still see (commuting predecessors are invisible to states).

Convention (plumbing). Three pieces of Lean plumbing are folded out of every display below, once and for all: (i) every signature carries decidable equality on Σ and on the operation alphabet (fields `dec_state`, `dec_op` of `CRDTSig`), and framework files use classical choice; (ii) the σ-layer definitions (the set-relative order, canonical states, the join and the merge VCs) are stated over the *replica-keyed core* of a configuration — its events and visibility only — and a ternary configuration C enters them through the forgetful `Configuration.core`; the displays do not distinguish C from its core; (iii) where a display quantifies over configurations, the six core invariants of Definition 3.1 are ambient; (iv) the pruning operators of §17 carry two definedness side conditions (the pinned root and every allocated head lie outside the dropped set), discharged once by the GC run invariant, and displays write the pruned configuration without repeating them. No other hypothesis is elided.

Executions. A configuration carries a *store*: a DAG of versions, each holding a state and the set of events that produced it, with per-replica heads; transitions fork a replica, apply a fresh event at a head, merge two heads through their LCA, or query. The model is fixed by the next two definitions.

Definition 3.1 (ternary configuration)**.** *A configuration C of $\mathcal{D}$ consists of:*

- *a replica-keyed core: partial per-replica maps N (head state) and L (observed event set) and a visibility relation $\xrightarrow{\mathsf{vis}}$ over events, subject to six invariants — N and L share a domain; every $\xrightarrow{\mathsf{vis}}$-edge has its source, and its target, observed at some replica; causal closure ($a \xrightarrow{\mathsf{vis}} b$ and b observed at r imply a observed at r); timestamp uniqueness across observed events; causal monotonicity ($a \xrightarrow{\mathsf{vis}} b$ implies $\mathsf{time}(a) < \mathsf{time}(b)$, the Lamport-clock guarantee); and same-replica totality (two distinct observed events of one replica are $\xrightarrow{\mathsf{vis}}$-comparable). $E(C)$ denotes the set of events observed anywhere (`Configuration.events`);*
- *a ranked version store: a partial map ver from versions ($v \in \mathbb{N}$, the allocation rank) to pairs $(\sigma_v, L(v))$ of a state and an event set, a partial per-replica head-version map, and a*

parent list $\mathsf{parents}(v)$*, subject to: parents have strictly smaller rank; version* 0 *is* $(\sigma_0, \emptyset)$*; a replica's head version carries exactly that replica's* $(N(r), L(r))$*; every registered state satisfies* Inv*; and the lca-events shape — for all registered* v_1, v_2, v_ℓ *with* v_ℓ *an LCA of* v_1, v_2*,* $L(v_\ell) = L(v_1) \cap L(v_2)$*. The last clause is carried as a structure field; §4.1 proves it maintainable, so it is an invariant of the system, not an assumption (Remark 4.4).*

Ancestry and LCAs are read off the parent lists: $\mathsf{Reaches}(a, v)$ *is the reflexive–transitive closure of the parent edge* $p \in \mathsf{parents}(v)$ *(`Reaches`), and* v_ℓ *is an LCA of* v_1, v_2 *when it reaches both and dominates every common ancestor (`IsLCA`):*

$$\mathsf{Reaches}(v_\ell, v_1) \wedge \mathsf{Reaches}(v_\ell, v_2) \wedge \forall w.\ \mathsf{Reaches}(w, v_1) \wedge \mathsf{Reaches}(w, v_2) \Rightarrow \mathsf{Reaches}(w, v_\ell).$$

`Configuration`

Definition 3.2 (the ternary transition system)**.** *The labelled transition system* $\mathsf{Step3}$ *has four rules:*

- *CreateReplica*(r)*: requires* $N(r)$ *undefined; installs* $N(r) \coloneqq \sigma_0$*,* $L(r) \coloneqq \emptyset$*, head at version* 0*; store and visibility unchanged.*
- *Apply*(t, r, o)*: requires* r*'s head version* v *with* $\mathsf{ver}(v) = (\sigma, E_v)$*, store-wide timestamp freshness (no event observed anywhere in* C*, and no event of any registered version, has timestamp* t*), and a fresh slot* $v_{\text{new}} > v$*; allocates* $v_{\text{new}} \coloneqq (e(\sigma),\ E_v \cup \{e\})$ *for* $e = (t, r, o)$ *with parent* $[v]$*, extends* $\xrightarrow{\mathsf{vis}}$ *by* $E_v \times \{e\}$*, and moves* r*'s head to* v_{new}*.*
- *Merge*(r_1, r_2) *— the rule the metatheory lives in:*

$$\frac{\mathsf{head}(r_1) = v_1 \quad \mathsf{head}(r_2) = v_2 \quad \mathsf{ver}(v_i) = (\sigma_i, E_i) \quad \mathsf{IsLCA}(v_1, v_2, v_\ell) \quad \mathsf{ver}(v_\ell) = (\sigma_\ell, E_\ell) \quad \mathsf{ver}(v_m)\ \textit{unallocated} \quad v_1 < v_m \quad v_2 < v_m}{\mathsf{ver}(v_m) \coloneqq \big(\mathsf{mergeL}(\sigma_\ell, \sigma_1, \sigma_2),\ E_1 \cup E_2\big), \quad \mathsf{parents}(v_m) \coloneqq [v_1, v_2], \quad \mathsf{head}(r_1) \coloneqq v_m}$$

  with visibility unchanged. Merge is gated on the existence of an LCA; criss-cross configurations disable it (§16 lifts the gate).
- *Query*(r, q)*: reads* $N(r)$*; configuration unchanged.*

The initial configuration (`initConfig`) has one replica at σ_0 *observing no events and the one-node DAG* $\{0\}$*; reachability is along* $\mathsf{Step3}$ *(`labeledTS3`).* `Step3`

RA-linearizability, per version. Write $e_1 \parallel e_2$ when neither $e_1 \xrightarrow{\mathsf{vis}} e_2$ nor $e_2 \xrightarrow{\mathsf{vis}} e_1$ (concurrency).

Definition 3.3 (linearization order, set-relative)**.** *For an event set* E*, the order* lo^E *puts* e_1 *before* e_2 *iff*

$$\big(e_1 \xrightarrow{\mathsf{vis}} e_2 \ \wedge\ e_1 \not\xrightarrow{\mathsf{vis}} e_2\big) \ \vee\ \big(e_1 \parallel e_2 \ \wedge\ e_1 \xrightarrow{\mathsf{rc}} e_2 \ \wedge\ \neg\, \exists e_3 \in E.\ e_2 \xrightarrow{\mathsf{vis}} e_3 \wedge e_2 \not\xrightarrow{\mathsf{vis}} e_3\big) :$$

a visible non-commuting pair is ordered by visibility; a concurrent pair whose conflict rc *resolves is ordered by* rc *— unless* e_2 *is already absorbed (overwritten) by a later non-commuting event of* E*. The paper's order* lo *is the variant whose absorber clause ranges over the whole configuration (`Sal.Emulation.lo`; ternary mirror `Configuration.lo`); since a* lo*-edge asserts no absorber anywhere,* $\mathsf{lo} \subseteq \mathsf{lo}^E$ *pointwise, for every* E *(`loOn_of_lo`), and* lo^E *is antitone in* E *— growing the set only adds absorbers, hence removes edges (`loOn_mono`).* `loOn`

Definition 3.4 (arbitration, and RA-linearizability against it)**.** *An arbitration* arb *assigns to each event set* E *a strict order* $\mathsf{arb}(E)$ *on its operations that orders every visible non-commuting pair* ($e_1 \xrightarrow{\mathsf{vis}} e_2 \wedge e_1 \not\rightleftharpoons e_2 \Rightarrow e_1\ \mathsf{arb}(E)\ e_2$)*. A list* $\pi = e^1 e^2 \cdots e^n$ *witnesses a version holding state* s *and event set* E *against* arb *when (i)* π *enumerates* E *without repetition; (ii)* π *extends* $\mathsf{arb}(E)$*:* $i < j$ *implies* $\neg(e^j\ \mathsf{arb}(E)\ e^i)$*; and (iii)* π *folds to the state:* $e^n(\cdots e^2(e^1(\sigma_0))\cdots) = s$*, read up to the data type's observational equivalence* $\approx$ *in the general signature (flat data types instantiate* $\approx\ =\ =$*). A configuration is RA-linearizable against* arb *when every version in the store — not only the replica heads; LCAs are historical versions and the merge reads them — admits a witness.*

The canonical arbitration is lo *(Definition 3.3), built from the conflict resolver* rc*. Unqualified, "RA-linearizable" means against* lo*, and* rc *enters the definition through this one instance only: the property itself is not tied to* rc*. §7 gives the contract an arbitration must meet for a data type to be provably RA-linearizable against it, and exhibits the LWW register linearized against its timestamp order — an arbitration* rc *cannot express.* `IsRALinearizable3Arb` *(against a given* arb*),* `IsRALinearizable3` */* `IsRALinearizable3Eq` *(at* $\mathsf{arb} = \mathsf{lo}$*)*

Why a set-relative order? One definitional correction is forced at the outset: for every internal use, the absorber clause must range over the event set of *the version being linearized*, not over the whole configuration — an event can gain absorbers from another branch, silently cancelling an order edge inside a version whose own state depends on it (the defeater of §4.2 realizes exactly this). We therefore construct witnesses respecting lo^E throughout; since $\mathsf{lo} \subseteq \mathsf{lo}^E$, every such witness also extends the paper's order, and Definition 3.4 is delivered exactly as the paper states it.

Running examples. Three data types exercise every corner of the theory: the **counter** and the **OR-set** of §2, and the **enable-wins flag** — per-replica pairs (counter, flag); enabling bumps your own counter and sets your flag; disabling clears all flags; the merge compares each replica's counter against the LCA's to decide whether an enable "happened since the fork" on that replica. That the flag's merge *compares* counters rather than accumulating sets will be visible in the metatheory (§11).

4 Three facts the metatheory must respect

Before building anything, we record two boundary facts about the paper's development: its store-level LCA lemma is true but under-proved, and its soundness induction is refuted by a fully legal execution. Together they fix the shape of the corrected metatheory — resting on a mechanized store invariant (§4.1) and built around the join rather than the sides (§4.2).

4.1 The LCA lemma is a reachability invariant — with a gap in the published proof

Everything above silently used the LCA lemma: in every reachable configuration, an LCA's event set is the intersection of the two sides'. This is what lets merge VCs be stated over *event sets*: the state the merge receives as l is the canonical state of exactly the intersection of the two sides' histories. The lemma is true — but it is a *global induction over the reachable store*, not a local fact. The invariant the induction threads is:

Definition 4.1 (store invariant)**.** *For a version store* $(\mathsf{ver}, \mathsf{parents})$*, the store invariant demands:*

- *parents allocated:* $p \in \mathsf{parents}(v)$ *implies* $\mathsf{ver}(p)$ *is defined;*
- *event monotonicity:* $p \in \mathsf{parents}(v)$*,* $\mathsf{ver}(p) = (\sigma_p, E_p)$*, and* $\mathsf{ver}(v) = (\sigma_v, E_v)$ *imply* $E_p \subseteq E_v$*;*
- *origin (generator uniqueness): if* $\mathsf{ver}(v) = (\sigma, E)$ *and* $e \in E$*, then there is a version* v_0 *with* $\mathsf{ver}(v_0) = (\sigma_0, E_0)$ *and* $e \in E_0$ *such that every version* w *with* $\mathsf{ver}(w) = (\sigma_w, E_w)$ *and* $e \in E_w$ *satisfies* $\mathsf{Reaches}(v_0, w)$*.*

`StoreInv`

Lemma 4.2 (LCA lemma)**.** *Let* $(\mathsf{ver}, \mathsf{parents})$ *satisfy the store invariant (Definition 4.1), let* v_ℓ *be an LCA of* v_1, v_2*, and let all three versions be registered:* $\mathsf{ver}(v_1) = (\sigma_1, E_1)$*,* $\mathsf{ver}(v_2) = (\sigma_2, E_2)$*,* $\mathsf{ver}(v_\ell) = (\sigma_\ell, E_\ell)$*. Then*

$$E_\ell \;=\; E_1 \cap E_2.$$

`lca_events_of_storeInv`

Theorem 4.3 (the store invariant is a reachability invariant)**.** *For every* $\mathcal{D}$ *with* $\mathsf{Inv}(\sigma_0)$ *and every configuration* C *reachable from the initial configuration in the ternary transition system,* $(C.\mathsf{ver}, C.\mathsf{parents})$ *satisfies the store invariant.* `storeInv_reachable`

The published proof asserts the generator-uniqueness step (the *origin* clause) without the induction that sustains it; Theorem 4.3 is that induction, mechanized (`storeInv_init`, `storeInv_step`). An erratum-sized gap: the statement survives, the proof as written does not. The LCA, when it exists, is unique — two LCAs dominate each other and the rank order is antisymmetric (`isLCA_unique`). (A related boundary fact: *criss-cross* merges can defeat the *existence* of an LCA — they gate merge-enabledness, not the lemma. §16 lifts the gate.)

Remark 4.4 (the lca-events field)**.** *In the mechanization the LCA lemma's shape is also carried as a field of* `Configuration` *(the lca-events clause of Definition 3.1), and* `merged_store_lca_events` */* `applied_store_lca_events` *prove the field maintainable under Merge and Apply for stores satisfying the invariant — so* Step3*'s merge rule is never blocked by it. The "reachability invariant" claim above is the pair: the field plus Theorem 4.3's maintenance. The field is an invariant of the system, not an assumption smuggled into the model.*

4.2 A fully LCA-legal execution defeats the soundness proof

The generic half of the paper's contract is a *bottom-up linearization* theorem: given linearization witnesses for the two sides of a merge, construct one for the merged version by repeatedly *peeling* a final event through mergeL using the catalogue's interchange VCs. One might hope the version DAG's discipline — every merge sitting at an honest LCA — keeps the peel supplied with peelable events. It does not: one staging replica suffices to realize the intersection of two histories as an honest version, and the merge induction is stuck.

Example 4.5 (the defeater)**.** *The data type is a one-key add-wins skeleton with tagged adds:* $\sigma = (A, D) \in 2^{\mathbb{T}} \times 2^{\mathbb{T}}$ *with live set* $A \setminus D$*;* add_t *inserts its tag into* A*;* rem *kills everything it currently sees (*$D \coloneqq A \cup D$ *— the state-dependence is what makes* $\mathsf{add} \not\rightleftharpoons \mathsf{rem}$*); concurrently* $\mathsf{rem} \xrightarrow{\mathsf{rc}} \mathsf{add}$ *(add wins); the merge is componentwise union, ignoring its LCA argument. A legal MRDT. Replica* p *adds (*A_p*), replica* q *adds (*A_q*). A staging replica* s *merges both one-add*

versions, creating v_s with $L(v_s) = \{A_p, A_q\}$. Now p removes (R_p, seeing only A_p) and then merges with v_s; symmetrically q removes (R_q) and merges with v_s. Figure 2 shows the DAG. The final merge of the two heads finds $\mathrm{LCA} = v_s$*, and $L(v_s) = \{A_p, A_q\} = L(head_p) \cap L(head_q)$ — the configuration is fully legal. Now count last events. p's head lives on exactly A_q's tag, so a state-correct witness must run R_p before A_q, and neither R_p nor A_p can come last: every linearization of p's head ends in A_q. Symmetrically, every linearization of q's head ends in A_p. Yet in the merged version every tag is dead, so its linearizations must end in one of the removes. The events the induction must peel are buried in the very sides that own them; no ordering of the paper's bottom-up rules applies.*

The two removes commute ($\mathsf{rem} \not\rightleftarrows \mathsf{add}$ but $\mathsf{rem} \rightleftarrows \mathsf{rem}$), which invites a particular rescue attempt: the paper's BottomUp-2-OP rule (`inter_lca_2op`) admits the commuting case and could, one hopes, peel a remove last. It cannot, for a reason that is a one-line state fact. To peel o last from a side, that side's state must be an o-image — of the form $\mathsf{do}(\sigma', o)$ with o outermost. But in the add-wins skeleton *every* rem-output has empty live set ($\mathsf{rem}(A, D) = (A, A \cup D)$), whereas both heads carry a live tag (head_p lives on A_q, head_q on A_p — the opposite add, merged in *after* the local remove). So neither head is a rem-image, and no bottom-up rule can extract a remove from it. The tempting linearization $[A_p, R_p, A_q, R_q]$ is a valid witness of the *merged* version, but its restriction to head_q folds to the all-dead state, not head_q's actual state (live A_p): it is not assembled from a state-correct side witness, which is exactly what the induction requires. All three facts are mechanized (file `Refutations/InterLca2op_Defeater_Arbiter.lean`):

Proposition 4.6 (the defeater's three mechanized facts)**.** *In the add-wins skeleton of Example 4.5 — writing* $\mathrm{live}(A, D) = A \setminus D$*, with events $A_p = (1, p, \mathsf{add})$, $R_p = (3, p, \mathsf{rem})$, $A_q = (2, q, \mathsf{add})$, $R_q = (4, q, \mathsf{rem})$, and $\mathrm{head}_p, \mathrm{head}_q$ the two head states of Figure 2:*

1. ***(rem-images are all-dead)*** *for every state σ and every event e with $\mathsf{op}(e) = \mathsf{rem}$:* $\mathrm{live}(e(\sigma)) = \emptyset$*.* `awset_rem_output_empty`
2. ***(no 2-OP rem-peel)*** *no instantiation of the BottomUp-2-OP merge shape peels a remove from either head:*

$$\neg\, \exists\, a, b \in \Sigma,\ o_l, o_1, o_2 \in \mathcal{E}.\ \ \mathsf{op}(o_1) = \mathsf{rem} \ \wedge\ o_1(o_l(a)) = \mathrm{head}_p \ \wedge\ o_2(o_l(b)) = \mathrm{head}_q,$$

   and symmetrically with head_p *and* head_q *interchanged.* `no_inter_lca_2op_rem_peel_of_defeater`
3. ***(a valid merged witness that is not side-correct)*** *the list $w = [A_p, R_p, A_q, R_q]$ satisfies* $\mathrm{live}(\mathsf{fold}(\sigma_0, w)) = \emptyset$ *and* $\mathsf{fold}(\sigma_0, w) = \sigma_{v_\top}$ *(the merged state) — a valid witness of the merged version — yet its restriction to* head_q*'s event set, $w\restriction = [A_p, A_q, R_q]$, has* $\mathsf{fold}(\sigma_0, w\restriction) \neq \sigma_{\mathrm{head}_q}$ *and* $\mathrm{live}(\mathsf{fold}(\sigma_0, w\restriction)) \neq \mathrm{live}(\sigma_{\mathrm{head}_q})$*.* `crack1_witness`

The consequence for the design: the metatheorem cannot be patched peel-by-peel — it must be rebuilt on canonical states and a Join Lemma, with merge-side VCs that are statements *about the join*, not about how either side's state factors.

5 Canonical states and the ternary Join Lemma

We now rebuild — new proof architecture, and with it a *new set of verification conditions* to replace the paper's catalogue. The lesson of the defeater is architectural: witnesses for a merged

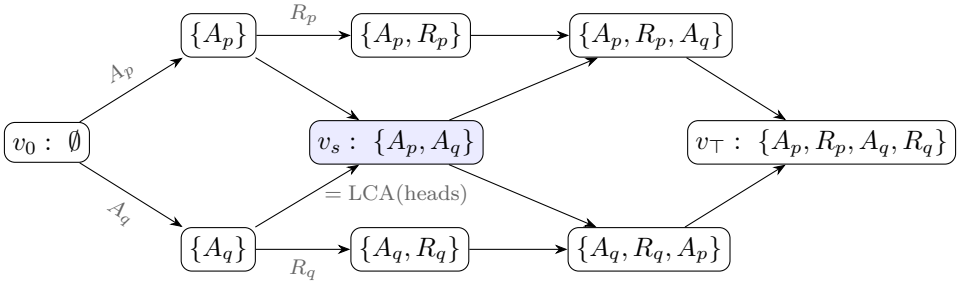


Figure 2: The defeater (Example 4.5). Nodes are versions labelled by their event sets; the shaded version v_s, created by a staging replica, realizes $L(\mathrm{head}_p) \cap L(\mathrm{head}_q)$ as an honest store version, so the final merge is LCA-legal. The removes R_p, R_q — the only events a bottom-up derivation may peel last — are buried mid-history on their own sides.

version cannot be manufactured out of witnesses for its sides, so the corrected metatheory must not traffic in witness lists at all. The rebuild has two layers. The *update layer* makes states, not sequences, the objects of the induction: it assigns to every event set E a single well-defined state, its *canonical state* (Definition 5.2 below). The *merge layer* is then one equation between canonical states — the ternary *Join Lemma* — and establishing it from per-data-type VCs becomes the entire burden of the metatheorem (§6–§9).

The update layer comes first, and a structural observation makes it cheap: this half of the corrected theory never mentions the merge. Its one theorem is stated over the update signature $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{rc} \rangle$ alone:

Theorem 5.1 (convergence)**.** *Assume the update-layer VCs (Flat VCs 1–3 of §9; Lean bundle* `UpdateVCs`*). Let C be a configuration, $E \subseteq E(C)$ an event set, and π_1, π_2 two enumerations of E each respecting lo^E. Then, for every start state σ,*

$$\mathsf{fold}(\sigma, \pi_1) \;=\; \mathsf{fold}(\sigma, \pi_2).$$

The paper-facing statement is the $\sigma := \sigma_0$ instance; the start-state genericity is consumed by the adequacy induction (Lemma 14.5). `convergence_on_u`

This is where the set-relative reading of §3 earns its keep: with the paper's configuration-wide order in place of lo^E, the theorem is *false* — the defeater's heads are already counterexamples, since two order-respecting replays of the three events of head_p reach different states. Convergence licenses a definition:

Definition 5.2 (canonical state)**.** *A state s is a canonical state of the finite event set E in C, written $\mathrm{Can}_C(E, s)$, when some enumeration of E respecting lo^E folds to it from the initial state:*

$$\mathrm{Can}_C(E,\, s) \;:\Longleftrightarrow\; \exists \rho.\ \rho \text{ enumerates } E \;\wedge\; \rho \text{ respects } \mathsf{lo}^E \;\wedge\; \mathsf{fold}(\sigma_0, \rho) = s.$$

Under the update-layer VCs the definite article is earned: a canonical state exists whenever $\xrightarrow{\mathsf{vis}}$ is transitive and irreflexive on C, $E \subseteq E(C)$, and E admits an enumeration (`isCanonicalState_exists_u`*; the required acyclicity of lo^E is Flat VC 2's contribution), and it is unique for $E \subseteq E(C)$ (*`isCanonicalState_unique_u`*, from Theorem 5.1). We then write $\sigma(E)$ for the canonical state, with $\sigma(\emptyset) = \sigma_0$, and σ_v for the state a version v happens to hold in*

the store — proving that the two coincide is precisely the metatheorem's job. Displays below keep hypothesis positions in the predicate form $\mathrm{Can}_C(E, s)$*, which does not presuppose uniqueness.* `IsCanonicalState`

With canonical states in hand, witness lists disappear as promised: a version linearizes iff $\sigma_v = \sigma(L(v))$ — the witness, when one is owed, is any respecting enumeration of $L(v)$. The induction can therefore maintain a state-level invariant ("every version holds σ of its events") instead of constructing sequences. The one genuinely ternary statement is what the merge must produce:

Definition 5.3 (ternary Join Lemma)**.** *Call an event set E weakly closed in C when it is backward-closed under non-commuting visibility: $a \xrightarrow{\mathsf{vis}} b$, $a \not\rightleftarrows b$, and $b \in E$ imply $a \in E$. The ternary Join Lemma at C (*`JoinLemma3At`*, the framework's per-configuration hook) is the statement: for all event sets E_1, E_2 and states s_0, s_1, s_2, if*

(i) $\xrightarrow{\mathsf{vis}}$ *is transitive and irreflexive on C;*

(ii) $E_1, E_2 \subseteq E(C)$*;*

(iii) E_1 *and* E_2 *are each weakly closed in C; and*

(iv) $\mathrm{Can}_C(E_1 \cap E_2,\, s_0)$, $\mathrm{Can}_C(E_1,\, s_1)$, $\mathrm{Can}_C(E_2,\, s_2)$,

then

$$\mathrm{Can}_C\big(E_1 \cup E_2,\ \ \mathsf{mergeL}(s_0, s_1, s_2)\big).$$

The ternary Join Lemma (`JoinLemma3`*) is its closure over all configurations. In σ-notation the conclusion reads $\sigma(E_1 \cup E_2) = \mathsf{mergeL}(\sigma(E_1 \cap E_2), \sigma(E_1), \sigma(E_2))$; but the closure premise (iii) is part of the contract, not scaffolding — the enable-wins-flag asymmetry of §10 turns on exactly that clause.* `JoinLemma3, JoinLemma3At`

By Lemma 4.2 the first argument is exactly the state the store hands the merge. The adequacy proof (§10) is then an induction over the transition system maintaining "every version holds the canonical state of its event set"; the merge case *is* the Join Lemma. What remains is the question this part exists to answer: which per-data-type VCs make the Join Lemma true?

6 Why the contract is a delta contract

Where should the laws that prove the Join Lemma come from? The obvious supplier is the CRDT tradition. For state-based CRDTs — binary merge, no LCA — merge-coherence is lattice-flavoured: associativity, idempotence, inflation of updates. None of the three transfers to the ternary merge: their free-tuple forms are the wrong equations for an LCA-aware merge, and their honest instances are too weak to power a lattice-style proof. The shape of what fails to transfer reveals the contract that succeeds.

Idempotence: the free-tuple form is the wrong equation. The *diagonal* law $\mathsf{mergeL}(a, a, a) = a$ — the paper's own MergeIdempotence — holds (for the counter: $a + a - a = a$), but it is not the law a lattice-style proof consumes. The binary architecture uses idempotence to absorb a duplicated *side*, $\mathsf{merge}(a, a) = a$, with nothing tying the two

copies to a common past; the ternary analogue would be $\mathsf{mergeL}(l, a, a) = a$ with l *free*. But quantifying l freely asks the merge about tuples the version DAG never produces. Merging a version with *itself* has $l = a$ — the LCA of a and a is a — where the law collapses to the diagonal and holds. And when two *distinct* histories happen to reach equal side states, feasibility puts l strictly below them and the counter's $\mathsf{mergeL}(l, a, a) = 2a - l$ is the *correct* answer: both branches' increments count. So there is no idempotence *failure* — there is no meaningful side-idempotence *law*: its honest instances are the trivial diagonal, and demanding it unconditionally would exclude precisely the LCA-sensitive data types the ternary theorem exists for.

The general principle: the algebraic properties of an LCA-aware merge are properties of the *causal history*, not of the concrete state. Two sides merge idempotently when their *histories* coincide — which is what forces $l = a$ — not when their states happen to be equal; equal states with different histories rightly merge differently. The lattice laws go wrong exactly by quantifying over states while forgetting the histories that produced them, and every law that survives below is stated over history-indexed data: canonical states $\sigma(E)$, feasible tuples (the LCA slot carrying the intersection history), downset deltas.

No usable order. The lattice-style workhorse order $x \sqsubseteq y \coloneqq \mathsf{merge}(x, y) = y$ has ternary candidates $x \sqsubseteq_l y \coloneqq \mathsf{mergeL}(l, x, y) = y$, but for the counter this relation degenerates to $x = l$; no antisymmetry survives to power order-theoretic reasoning. The single contextual VC below is accordingly an *equation*, not an inequality.

Associativity is the wrong target. The natural guess — $\mathsf{mergeL}(l, \mathsf{mergeL}(l, a, b), c) = \mathsf{mergeL}(l, a, \mathsf{mergeL}(l, b, c))$ — assumes one LCA for all three merges. In an execution, re-associating a three-way reconciliation involves *four* distinct LCAs:

$$\mathsf{mergeL}\big(l_{(ab)c},\ \mathsf{mergeL}(l_{ab}, a, b),\ c\big) \stackrel{?}{=} \mathsf{mergeL}\big(l_{a(bc)},\ a,\ \mathsf{mergeL}(l_{bc}, b, c)\big),$$

with $l_{ab} = \sigma(E_a \cap E_b)$, $l_{(ab)c} = \sigma((E_a \cup E_b) \cap E_c)$, and so on. For the counter — whose canonical states are additive measures over event sets — the two sides come to $a + b + c$ minus the two LCA measures, and they agree because

$$(E_a \cup E_b) \cap E_c = (E_a \cap E_c) \cup (E_b \cap E_c) \quad \Longrightarrow \quad \text{both LCA-sums} = |E_{ab}| + |E_{ac}| + |E_{bc}| - |E_{abc}|$$

by inclusion–exclusion. The law holds, but only *through the set-algebra of intersections*: over four unconstrained states it is false. Honest associativity is inherently a feasible-tuple fact — and, the mechanization's pleasant surprise, **the corrected induction never needs it**. Every merge the induction forms already sits at its honest LCA; the only laws consumed are two *redistribution* identities that equate two honestly-formed merges:

Definition 6.1 (the delta contract)**.**

$$\begin{aligned} \textit{LOCAL-REDISTRIBUTE}: &\quad \mathsf{mergeL}(l,\ \mathsf{mergeL}(m, x, c),\ y) \ =\ \mathsf{mergeL}(m,\ \mathsf{mergeL}(l, x, y),\ c), \\ \textit{REDISTRIBUTE}: &\quad \mathsf{mergeL}\big(\mathsf{mergeL}(m, x_0, c),\ \mathsf{mergeL}(m, x_1, c),\ \mathsf{mergeL}(m, x_2, c)\big) \\ &\qquad\qquad = \ \mathsf{mergeL}\big(m,\ \mathsf{mergeL}(x_0, x_1, x_2),\ c\big). \end{aligned}$$

Both laws are quantified over all states l, m, x, x_i, c, y — this is the unconditional form, the one the group and lattice classes satisfy outright. `DeltaVCs3`

Read c as a *delta relative to* m (in the induction, c will be "apply event e to its causal past"). LOCAL-REDISTRIBUTE says a delta application commutes past an enclosing merge through one branch slot. REDISTRIBUTE says a delta carried by *all three* components extracts *once* — the LCA slot cancels the duplicates by arithmetic. This is the delta contract's quiet coup: cancelling the duplicated shared contribution is exactly the job idempotence performs in lattice-based convergence arguments, now done without any idempotence — which is why the counter sails through (REDISTRIBUTE for the counter is the identity $\sum x_i + 3c - 3m - (x_0 + c - m) - \cdots$ collapsing on both sides).

Where the contract holds unconditionally — and where it cannot. The two laws hold *on all states* exactly for the group-like class (the counter) and the lattice-like, LCA-blind class (grow-only structures). They are *unconditionally false* for the OR-set: take an element tag present in c and l but in neither branch — the two sides of LOCAL-REDISTRIBUTE then disagree about whether the LCA's copy survives. But such a tuple can never arise: a tag in a canonical LCA state is in *both* branches unless removed, and a removal is itself an event of the branch. The failing tuples are *infeasible*, and this is a theorem-grade phenomenon, not an inconvenience:

Finding. *For LCA-sensitive MRDTs beyond the group $\oplus$ lattice classes, the merge-coherence laws are irreducibly feasibility-conditioned: true on canonical tuples at honest LCAs, false in general. Quantifying merge VCs over all states — the published catalogue's style — is not merely inconvenient for such data types; for several laws it is wrong in principle. (The enable-wins flag even falsifies the unit law* $\mathsf{mergeL}(\sigma_0, \sigma_0, s) = s$ *on an infeasible state with a set flag and a zero counter.)*

Accordingly the delta laws enter the final contract in feasibility-conditioned form (`FeasibleDeltaVCs3`, displayed as Flat VCs 5–7): quantified over canonical states of weakly closed event sets, with every mergeL node at its honest LCA. The unconditional form remains available as a one-line *on-ramp* for the group and lattice classes — it implies the conditioned form outright (`feasibleDeltaVCs3_of_delta`).

7 The arbitration contract

RA-linearizability is defined against an arbitration (Definition 3.4), with lo — the order built from rc — the canonical one. What must an arbitration satisfy for a data type to be *provably* RA-linearizable against it? That contract is this section. The eight equations of §9 are then one *recipe* for meeting it, the recipe that builds lo from rc; a data type with a different arbitration (LWW's timestamp order, below) meets the contract directly and skips them.

What the proof consumes. Trace rc through the adequacy argument (§10): it appears only through lo^E, and lo^E appears only as the arbitration, a per-event-set order. Nothing downstream inspects how that order was built. So the contract is a property of an abstract arbitration, written $\mathsf{arb}\, C\, E$ to make the configuration C explicit for the adequacy induction: the canonical state of E is the fold along any $\mathsf{arb}\, C\, E$-respecting enumeration (`IsCanonicalStateArb`), RA-linearizability is per-version canonicity against it (`IsRALinearizable3Arb`), and it has six clauses.

Definition 7.1 (arbitration family, flat)**.** *An arbitration family $\mathcal{F}$ supplies* arb *with six clauses, each a property the adequacy induction consumes at a named step:*

1. *extends-vis: every visible non-commuting pair inside E is ordered — $a, b \in E$, $a \xrightarrow{\mathsf{vis}} b$, $a \not\rightleftharpoons b \Rightarrow \mathsf{arb}\, C\, E\, a\, b$;*

2. *acyclic: $\mathsf{arb}\, C\, E$ has no cycle (maximal events exist for the peel), gated on $\xrightarrow{\mathsf{vis}}$ being a strict partial order at the reachable C;*

3. *antitone: shrinking the carrier never flips an edge — $E' \subseteq E''$, $\mathsf{arb}\, C\, E''\, a\, b \Rightarrow \mathsf{arb}\, C\, E'\, a\, b$;*

4. *vis-consistent: arb never orders an event before one it observed — $b \xrightarrow{\mathsf{vis}} a \Rightarrow \neg\, \mathsf{arb}\, C\, E\, a\, b$, on any pair;*

5. *convergent: all $\mathsf{arb}\, C\, E$-respecting enumerations of E fold to one state (`ArbConvergence`);*

6. *vis-local: $\mathsf{arb}\, C\, E$ reads only $\xrightarrow{\mathsf{vis}}$ restricted to E — two configurations agreeing on $\xrightarrow{\mathsf{vis}}$ over E agree on $\mathsf{arb} - E$.*

`ArbAdequacyReach.ArbFamily`

Three of the six were not in the design; the mechanization forced them. The definition audit (§8) named acyclic, extends-vis, and convergence. Re-threading the adequacy induction over an abstract arb surfaced *antitone* (the re-attach step `isCanonicalStateArb_snoc` needs a maximal event to stay maximal on the smaller carrier), *vis-consistent* (an arb ordering a fresh event before a *commuting* predecessor stays acyclic and convergent yet breaks apply-maximality), and *vis-local* (the createReplica and merge congruence steps need arb blind to concurrent context outside E). Each is a genuine design requirement, and both instances below satisfy all six; but the theorem is mis-stated without them.

Theorem 7.2 (adequacy over an arbitration, flat)**.** *Let $\mathcal{F}$ be an arbitration family and let the datatype supply the merge content against $\mathcal{F}$'s order — merge symmetry, and the feasible-delta and causal-delta equations of §9 read with lo replaced by arb (`FeasibleDeltaVCs3Arb`, `CDVC3Arb`). Then every reachable configuration is RA-linearizable per version against arb:* $\mathsf{IsRALinearizable3Arb}(C, \mathsf{arb}\, C)$. `ra_linearizable3Arb_of_core_feasible_cd`

The order clauses and the merge content are the two halves of adequacy, cleanly separated: the clauses make arb a usable linearization order (maximal events, one fold, stable across the transitions); the merge content makes mergeL land on that fold. lo is absent from both the statement and the proof — the fold is pinned solely by clause 5, convergence — so the engine is genuinely order-agnostic.

The conditioned family: one clause. Over the observational quotient (§13: Inv-guarded states, $\approx$ for $=$) the picture inverts. The conditioned engine does not *build* its Join from the delta laws; it takes the $\approx$-Join as a primitive obligation (VC 4). So the conditioned arbitration family carries exactly *one* genuine clause, the $\approx$-Join against arb, and the other five hold *for free*: extends-vis and vis-consistency are arb-independent (the visibility arm of $\mathsf{lo}_{\approx}$ decides them), antitone is `loOnArb_mono` for any arb, and acyclic/convergent are not levers on a Join that is assumed rather than derived.

Theorem 7.3 (adequacy over an arbitration, conditioned)**.** *Let $\mathcal{F}$ supply arb and the $\approx$-Join against it (`EqJoinLemma3C_ArbH`), with the honesty and discipline conditions of §15. Then every reachable Inv-configuration is RA-linearizable per version against arb, up to $\approx$.* `ra_linearizable3ArbEq_of_reach`

Finding (six versus one)**.** *The flat family needs six order clauses; the conditioned family needs one. The asymmetry is structural, not incidental. The flat engine derives its Join from the delta and causal-delta VCs, and each of the six clauses is a lever in that derivation; the conditioned engine is handed its Join as VC 4, so the order clauses that built it fall away. Flat pays six clauses to construct the merge fact; conditioned pays one because it assumes it.*

Completeness: the contract is forced, on reachable states. The arbitration contract is not only sufficient but, on the states that arise, necessary. Reading the adequacy hypotheses off a datatype already known RA-linearizable recovers them.

Theorem 7.4 (converse, flat)**.** *If a flat datatype's canonical merge is a fold and converges (the content of RA-linearizability), then it satisfies the four-law core of §9 — feasible unit, the two feasible-redistribute laws, and the causal-delta equation — on every reachable, causally closed event set.* `Converse.converse_core`

The converse is deliberately reachability-relative, and it is not a biconditional. RA-linearizability constrains the merge only on the tuples an execution produces; over abstract tuples the contract says strictly more, and no datatype is forced to the *specific* arbitration lo. The RESET datatype (write/write/reset) is RA-linearizable yet falsifies the conditioned swap witness VC 2 (`eqswap_not_forced`), and its invariant discipline VC 1 is a datatype choice, not a consequence (`vc_disc_extra`). The arbitration recast is thus a *sound abstraction of the sufficient condition*, exact on reachable states, not a characterization of RA-linearizability itself.

lo is one arbitration. The standard order lo^E is arb instantiated at rc: its non-commuting-visibility arm is extends-vis, its concurrent arm consults rc, and its absorber clause is what supplies antitonicity (`loOn_isArb`, at $\mathsf{arb} \coloneqq (\mathsf{rc} = \mathsf{Fst\_then\_snd})$). The next section is that instance — the eight VCs are the clauses of Definition 7.1 and the merge content of Theorem 7.2, spelled out for lo. A datatype with a different arbitration — the LWW register orders by timestamp, an order rc cannot express — skips rc and the eight VCs entirely and discharges the six clauses directly (`lwwFamily`); there vis-consistency rests on the Lamport-clock invariant, not on rc.

Worked example: the LWW register by timestamp. The recast is not idle generality. The last-writer-wins register is a datatype whose arbitration rc *cannot* express, and it routes through the adequacy engine unchanged.

The datatype. A state is $\bot$ (unset) or a winning write, a lex-ordered triple $w = (\mathsf{time}, \mathrm{replica}, \mathrm{value})$ (`LWWWrite`). Update installs a write by semilattice max, $\mathsf{do}(s, e) = \max(s, w_e)$; the three-way merge drops its LCA slot, $\mathsf{mergeL}(l, a, b) = \max(a, b)$, since the state only ever moves up. The conflict resolver is trivial, $\mathsf{rc} = \mathsf{Either}$ (`LWW_rc_either`).

lo *degenerates to nothing.* With $\mathsf{rc} = \mathsf{Either}$ and all writes commuting (max is commutative, `LWW_all_comm`), *both* arms of lo^E are dead: the visibility arm needs a non-commuting pair, the concurrent arm needs $\mathsf{rc} = \mathsf{Fst\_then\_snd}$. So lo^E is the *empty* relation on LWW (`lww_loOn_false`); the rc recipe produces the discrete arbitration, no order at all. LWW's arbitration does not live in rc — it lives in the payload, the max on lex-timestamps.

The timestamp arbitration. Take $\mathsf{arb}\, E\, a\, b \coloneqq w_a < w_b$, the total order on writes by timestamp (`lwwArb`). It discharges the six clauses of Definition 7.1, and the discharge reads off where

each clause's content sits (`lwwFamily`): *acyclic* is strictness of $<$; *extends-vis* is vacuous (no non-commuting pairs); *antitone* and *vis-local* are immediate because arb ignores the carrier E entirely; *convergent* is the max-fold being order-independent; and *vis-consistent* is the one clause with content $- \xrightarrow{\mathsf{vis}} b\,a$ forces $\mathsf{time}(b) < \mathsf{time}(a)$ by the Lamport clock (`Configuration.causal_mono`), hence $w_b < w_a$, so arb never orders a write ahead of one it observed. That clause rests on the generation clock, *not* on the apply rule (which enforces only timestamp freshness): exactly where "the timestamp order respects causality" comes from.

Theorem 7.5 (LWW through the arbitration engine)**.** *Every reachable LWW configuration is RA-linearizable against the timestamp arbitration* arb*, through the same capstone (Theorem 7.2) that certifies* lo — lo *absent, the fold pinned by* arb*'s own convergence.* *`lww_isRALinearizable3Arb_ts_via_capstone`*

LWW is the witness that the abstraction is load-bearing: a datatype the rc-based catalogue can certify only by making its order vanish is linearized, at full strength, by the arbitration contract. (LWW also carries its ordinary flat discharge, `lww_ra_linearizable3`, against that empty lo with max-convergence pinning the fold; the two agree, and the timestamp arbitration is the honest account of the order the register actually realizes.)

8 Auditing the published definition: four strikes and the absorber

Why is the linearization order really just an arbitration? The recast of §7 is the endpoint of an audit of the published RA-linearizability definition (Definition 3.4), which reads its order lo^E (Definition 3.3) as an *arbitration-acyclicity* requirement plus a fold quotient, four of its clauses exposed as devices for that content rather than parts of it. Three strikes are recorded elsewhere in this paper; the fourth, the absorber, is decided by machine.

- *Strike 1: witness extension is not part of it.* The binary specification asks a merged version's witness to *extend* its sides' witnesses; the defeater (Example 4.5) refutes that on a fully LCA-legal configuration, so Definition 3.4 asks only that each version admit *some* witness, independently. The merge layer is rebuilt on canonical states and a Join Lemma that never traffics in witness lists.

- *Strike 2:* rc*-directionality and chain-freedom are reachability-relative.* Adequacy needs only that lo order non-commuting reachable pairs and stay acyclic on reachable sets, the content the converse recovers (Theorem 7.4). rc-non-comm and no-rc-chain are sufficient devices for it; LWW (§7) chains its rc along a total order and is RA-linearizable anyway.

- *Strike 3: the all-states quantification is a conditioning completion.* The unconditional halves of the merge VCs are evaluated by adequacy only on canonical states of reachable sets; each has an RA-linearizable violator whose failure lives at an unreachable state. The off-domain content is the conditioning completion, not the specification.

- *Strike 4: the absorber.* The absorber clause *removes* edges. Does dropping it change the verdict? Decided below.

The absorber dichotomy. Write plain for lo with the absorber dropped, keeping every rc-resolved concurrent edge. Plain has a superset of lo's edges, so its respecting enumerations are a subset, so plain-RA-lin *implies* absorber-RA-lin: the absorber order is the weaker property. A separator would be a datatype RA-linearizable under the absorber but not under plain, making the absorber load-bearing.

The separator exists, and the mechanism is acyclicity, not a fold disagreement. It is the canonical add-wins OR-set. Two replicas each add then remove the same element x and merge at the empty root: events $a = \mathsf{add}\, x$, $r_a = \mathsf{rem}\, x$ on one replica, b, r_b on the other. In the merged event set $\{a, r_a, b, r_b\}$ the plain order is

$$a \to r_a \text{ (vis)}, \quad r_a \to b \text{ (rc, add-wins)}, \quad b \to r_b \text{ (vis)}, \quad r_b \to a \text{ (rc)},$$

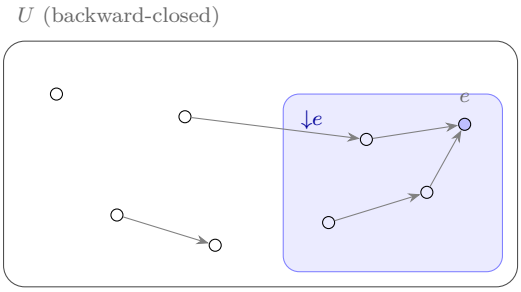
a cycle $a \to r_a \to b \to r_b \to a$ that no list extends: plain-RA-lin has *no witness at all*. The absorber removes both rc edges (each add is absorbed by its own following remove), leaving the two vis chains; every interleaving folds to the empty live set, the operational merge state. The cycle alternates rc and vis edges, so its two rc edges are not consecutive and no-rc-chain (Flat VC 2) does not forbid it: the absorber is the clause that supplies acyclicity precisely when rc opposes visibility (add-wins orders a remove before a concurrent add, visibility orders an add before its own remove, and the two close a cycle). Overwrite datatypes never trigger it — LWW's timestamp rc agrees with visibility, so no plain cycle and no separator.

The separation is certified and uniform. The exhaustive two-state, two-op-kind synthesis space (1408 specs) yields, among datatypes the metatheory certifies, **nine configuration-level separators, all via the cyclic route, none via a fold-value disagreement** (`absorber_dichotomy_check.py`, PASS+FAIL). Fold-value separator *versions* do occur, but only inside datatypes already non-RA-linearizable under the absorber order (the visible-swap specs cond-comm exists to exclude), so they never lift to a certified configuration. Plain-RA-lin on the certified route is *vacuous* (a cyclic relation has no linear extension), which is strictly stronger than an observable disagreement and not recoverable by coarsening $\approx$: coarsening the observation cannot rescue an order with no linear extension.

The invariant content. The four strikes converge on one reading, the one §7 makes primary: RA-linearizability asserts that a version's state is, up to $\approx$, the fold of some enumeration extending an *acyclic arbitration that orders every visible non-commuting pair by visibility.* The rc arm is one way to arbitrate concurrent pairs; no-rc-chain and the absorber keep that arbitration acyclic (the absorber the necessary half whenever rc opposes visibility); the all-states content is the conditioning completion; witness extension is not part of it at all. That is exactly the arbitration contract of Definition 7.1, with lo its canonical instance — the audit is why the recast re-presents the published definition rather than replacing it.

9 Deriving lo: the eight verification conditions

These eight equations are the lo instance of the arbitration contract (§7): the update-side obligations of §5 and the merge-side laws of §6 are exactly what make lo, the order built from rc, an arbitration family (Definition 7.1) and supply its merge content (Theorem 7.2). Of the six order clauses only two are carried by the update VCs — no-rc-chain (Flat VC 2) gives acyclicity, cond-comm (Flat VC 3) gives convergence — while extends-vis, antitone, vis-consistency, and vis-locality are structural properties of lo's definition (its visibility arm and its absorber), and



$A = \sigma(U \setminus \{e\})$ $\quad B = \sigma(\downarrow e \setminus \{e\})$ $\quad$ CDVC3 : $\mathsf{mergeL}(B,\, A,\, e(B)) \;=\; e(A)$

Figure 3: The causal-delta equation. The shaded region is e's causal past $\downarrow e$ (what e saw); e is lo^U-maximal. e's effect on the whole world A equals: compute e's effect on its own causal past B, then merge that delta into A *over LCA* B. The merge sits at its honest LCA: $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$. Concurrent context that e never observed cannot amplify what e does.

rc-non-comm (Flat VC 1) calibrates the concurrent arm to commutation. Flat VCs 5–8 supply the merge content. Assembled, they are a *new* VC set, replacing the paper's catalogue of twenty-four. For an MRDT $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{mergeL}, \mathsf{rc} \rangle$, the contract is the eight conditions below. In the mechanization they are carried by three bundles — CoreVCs3CD (Flat VCs 1–4: the update triple `UpdateVCs` plus merge symmetry), FeasibleDeltaVCs3 (Flat VCs 5–7), and CDVC3 (Flat VC 8) — and the canonical route is CoreVCs3CD + FeasibleDeltaVCs3 + CDVC3 ⇒ Join Lemma ⇒ RA-lin.[2]

Update layer (unconditional, SMT-shaped). Write $o_1 \bowtie o_2$ for a *conflict-eligible* pair: distinct (`distinctOps`) and cross-replica (`differentReplicas`). Only conflict-eligible pairs can race, so this guard sits on every update-layer VC below.

Flat VC 1 (rc-non-comm)**.**

$$o_1 \bowtie o_2 \implies \big(o_1 \not\rightleftharpoons o_2 \iff o_1 \xrightarrow{\mathsf{rc}} o_2 \;\vee\; o_2 \xrightarrow{\mathsf{rc}} o_1\big).$$

`UpdateVCs.rc_non_comm_directional`

The conflict table is the commutativity table, with a direction. Both guards are semantic content. The different-replica guard is a necessity, not a convenience: same-replica events are totally ordered by program order, so they are never concurrent and there is no conflict for rc to resolve. An unguarded VC would instead demand that rc order *every* non-commuting pair — unsatisfiable for real data types: the *optimized OR-set*'s same-replica same-element adds do not commute (the newer add evicts the older tag), yet they can never race, and its rc rightly ignores them. (The paper's F* artifact carries exactly this guard in its interface — `App_mrdt.fsti`: `requires distinct_ops o1 o2 /\ get_rid o1 <> get_rid o2`.)

Flat VC 2 (no-rc-chain)**.**

$$o_1 \neq o_2 \;\wedge\; o_2 \neq o_3 \implies \neg\big(o_1 \xrightarrow{\mathsf{rc}} o_2 \;\wedge\; o_2 \xrightarrow{\mathsf{rc}} o_3\big).$$

`UpdateVCs.no_rc_chain`

[2] A historical wider bundle `CoreVCs3` adds three laws (`mergeL_init`, `lem_0op3`, `merge_peel_comm3`) that the causal-delta route never consumes; it is not part of the contract.

Conflict priorities do not chain, which makes lo^E acyclic, so maximal events exist for the peel-and-recurse to select.

Flat VC 3 (cond-comm)**.** *For every state σ, events e, e', e'', and event list π: if e, e', e'' are pairwise distinct, $e \xrightarrow{\mathsf{rc}} e'$, and $e' \not\rightleftharpoons e''$, then*

$$e''\big(\mathsf{fold}(e(e'(\sigma)),\, \pi)\big) \;=\; e''\big(\mathsf{fold}(e'(e(\sigma)),\, \pi)\big).$$

UpdateVCs.cond_comm_lift

A resolved conflict is invisible once an absorber runs: swapping an rc-ordered concurrent pair deep in a fold changes nothing for any later non-commuting event e'', across arbitrary intervening events (the π). This is the engine of convergence.

Merge layer.

Flat VC 4 (merge symmetry)**.** $\mathsf{mergeL}(l, a, b) = \mathsf{mergeL}(l, b, a)$ *for all states l, a, b.* *CoreVCs3CD.mergeL_comm*

Flat VC 5 (feasible unit)**.**

$$E \subseteq E(C) \;\wedge\; \mathrm{Can}_C(E, s) \;\Longrightarrow\; \mathsf{mergeL}(\sigma_0, \sigma_0, s) \;=\; s.$$

FeasibleDeltaVCs3.feasible_init

Merging over nothing is a no-op — on states that can arise (the unconditional form is false for the enable-wins flag, the Finding of §6). The next two are the delta laws of Definition 6.1 on feasible tuples: every canonicity hypothesis pins one component to its history, and every mergeL node sits at its honest LCA.

Flat VC 6 (feasible local-redistribute)**.** *Let $\xrightarrow{\mathsf{vis}}$ be transitive and irreflexive on C; let $E_1, E_2 \subseteq E(C)$ be weakly closed in C (Definition 5.3); let $e \in E_1$, $e \notin E_2$, and let e be $\mathsf{lo}^{E_1 \cup E_2}$-maximal ($\forall x \in E_1 \cup E_2,\; x \neq e \Rightarrow \neg\,(e\ \mathsf{lo}^{E_1 \cup E_2}\ x)$). Then*

$$\begin{aligned}&\mathrm{Can}_C(E_1 \cap E_2,\, s_0) \;\wedge\; \mathrm{Can}_C(\downarrow e \setminus \{e\},\, B)\\ &\quad\wedge\; \mathrm{Can}_C(E_1 \setminus \{e\},\, t_1) \;\wedge\; \mathrm{Can}_C(E_2,\, s_2) \;\Longrightarrow\\ &\qquad \mathsf{mergeL}\big(s_0,\; \mathsf{mergeL}(B, t_1, e(B)),\; s_2\big) \;=\; \mathsf{mergeL}\big(B,\; \mathsf{mergeL}(s_0, t_1, s_2),\; e(B)\big).\end{aligned}$$

FeasibleDeltaVCs3.feasible_local_redistribute

Flat VC 7 (feasible redistribute)**.** *In the same context ($\xrightarrow{\mathsf{vis}}$ transitive and irreflexive on C; $E_1, E_2 \subseteq E(C)$ weakly closed in C), now with $e \in E_1$, $e \in E_2$, and e $\mathsf{lo}^{E_1 \cup E_2}$-maximal as above:*

$$\begin{aligned}&\mathrm{Can}_C((E_1 \cap E_2) \setminus \{e\},\, t_0) \;\wedge\; \mathrm{Can}_C(\downarrow e \setminus \{e\},\, B)\\ &\quad\wedge\; \mathrm{Can}_C(E_1 \setminus \{e\},\, t_1) \;\wedge\; \mathrm{Can}_C(E_2 \setminus \{e\},\, t_2) \;\Longrightarrow\\ &\qquad \mathsf{mergeL}\big(\mathsf{mergeL}(B, t_0, e(B)),\; \mathsf{mergeL}(B, t_1, e(B)),\; \mathsf{mergeL}(B, t_2, e(B))\big)\\ &\qquad\quad = \;\mathsf{mergeL}\big(B,\; \mathsf{mergeL}(t_0, t_1, t_2),\; e(B)\big).\end{aligned}$$

FeasibleDeltaVCs3.feasible_redistribute

Flat VC 8 (the causal-delta equation, CDVC3)**.** *Let* $\xrightarrow{\mathsf{vis}}$ *be transitive and irreflexive on* C*; let* $U \subseteq E(C)$ *be weakly closed in* C*; let* $e \in U$ *be* lo^U*-maximal (*$\forall x \in U,\ x \neq e \Rightarrow \neg\,(e\ \mathsf{lo}^U\ x)$*); and let* $\mathrm{Can}_C(U \setminus \{e\},\, A)$ *and* $\mathrm{Can}_C(\downarrow e \setminus \{e\},\, B)$*. Then*

$$\mathsf{mergeL}\big(B,\ A,\ e(B)\big)\ =\ e(A).$$

CDVC3

What an event does is determined by what it saw (Figure 3): applying e to the whole world A equals computing e's delta on its recorded past $\downarrow e \setminus \{e\}$ and merging it in over that past. The merge sits at its honest LCA: $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$.

Flat VCs 1–4 are one-liners for every data type we have discharged; Flat VCs 6–7 frequently hold unconditionally anyway (for the OR-set, REDISTRIBUTE is a pointwise Boolean tautology); essentially the entire per-data-type proof effort concentrates in Flat VC 8, whose discharge follows a uniform recipe: characterize $\sigma(E)$ (a "sandwich" invariant along canonical enumerations), then a *trichotomy* placing every event the maximal e interacts with either inside $\downarrow e$ or absorbed on a side that owns it.

Compare the published catalogue: 24 VCs per data type, plus five more the soundness proof uses silently. Seventeen of the 24 — the BOTTOMUP induction-scheme families — exist to drive the broken induction and are consumed nowhere in the corrected one; the paper's MERGEIDEMPOTENCE is likewise never used. The seventeen were, however, doing honest work of a different kind: they are an *automation layer*, Skolemizing "holds on feasible states" into base-and-step equations an SMT solver can discharge. Rebuilding that layer, aimed at the corrected target (CDVC3 rather than BOTTOMUP), is in our view the most valuable open engineering problem this work leaves (§43).

10 Adequacy

The contract delivers: a data type owes exactly the eight equations of §9.

Theorem 10.1 (soundness of the eight VCs)**.** *Let* $\mathcal{D}$ *satisfy Flat VCs 1–8 — the bundles* CoreVCs3CD $\mathcal{D}$*,* FeasibleDeltaVCs3 $\mathcal{D}$*,* CDVC3 $\mathcal{D}$ *— and let* $\mathsf{Inv}(\sigma_0)$ *hold (for flat data types, trivially). Then every configuration* C *reachable from the initial configuration in the ternary transition system (Definition 3.2) is RA-linearizable, per version (Definition 3.4):* $\mathsf{IsRALinearizable3}(C)$*.* *ra_linearizable_of_core_feasible_cd3*

The proof shape: the update layer powers the set-relative convergence theorem, hence canonical states; the transition induction maintains a five-clause invariant:

Definition 10.2 (good configuration)**.** $\mathsf{GoodConfig3}(C)$ *demands:*

1. *canonical: every registered version is canonical for its event set —* $\mathsf{ver}(v) = (s, E)$ *implies* $\mathrm{Can}_C(E, s)$*;*
2. *vis-transitivity and*
3. *vis-irreflexivity of* C*'s visibility;*
4. *universe: registered event sets sit inside the observed universe —* $\mathsf{ver}(v) = (s, E)$ *implies* $E \subseteq E(C)$*;*

5. *causal closure: registered event sets are fully* $\xrightarrow{\mathsf{vis}}$*-closed* — $\mathsf{ver}(v) = (s, E)$, $a \xrightarrow{\mathsf{vis}} b$, $b \in E$ *imply* $a \in E$.

Clause 1 delivers per-version RA-linearizability outright (`isRALinearizable3_of_good`); clause 5 is what makes full closure available to data types that need it — the asymmetry below. `GoodConfig3`

In the induction, fork and query are trivial and apply is the extend lemma; the merge case (`goodConfig3_merge_at`) reduces, via Lemma 4.2, to the ternary Join Lemma, which is proved from Flat VCs 4–8 by strong induction on $|E_1 \cup E_2|$ (`join_lemma3_of_cd_feasible`): select a $\mathsf{lo}^{E_1 \cup E_2}$-maximal e, decompose each side containing e through $\downarrow e$ (LOCAL-REDISTRIBUTE at the honest LCA $\downarrow e \setminus \{e\}$), cancel the shared delta (REDISTRIBUTE), close the principal step with CDVC3, and recurse; the join-to-RA-lin bridge is `ra_linearizable3_of_join`. The buried-event pathology of Example 4.5 dissolves structurally: no step ever asks how a *side's* state factors — every factoring happens through the join.

Everything else one might fear owing is proved once, for all data types, as an execution-model invariant: visibility transitivity, per-version backward closure, store well-formedness, and Lemma 4.2 itself. The data type owes exactly the eight equations.

A second route, and an honest asymmetry. One discharged data type does not fit the pattern above. The enable-wins flag's merge *compares counters* against the LCA, and for such merges the Join Lemma's hypotheses as stated (backward closure under non-commuting-visibility) are provably too weak: there is a closure-legal tuple — a live LCA enable, a dead post-LCA enable inflating one side's counter, the killing disable visible only to the other side — on which the production merge computes the wrong flag. Full *causal* closure (which the store invariant actually supplies) excludes it, and the flag is discharged through a full-closure variant of the join. The finding: **required closure strength is a function of the merge's shape** — commutation-closure suffices for set-shaped merges, counter-comparison merges need full causal closure.

Definition 10.3 (full-closure and closure-indexed joins)**.** *Call E fully closed in C when $a \xrightarrow{\mathsf{vis}} b$ and $b \in E$ imply $a \in E$ (all visible predecessors, commuting or not; `fullClosure`); weak closure is the predicate of Definition 5.3 (`weakClosure`).* JoinLemma3F *is Definition 5.3 with both instances of premise (iii) strengthened from weakly to fully closed (`JoinLemma3F`). Generalizing, a closure predicate is any $\mathcal{C} : (C, E) \mapsto$ Prop reading only visibility and commutation (`ClosurePred`), and the closure-indexed join* JoinLemma3C $\mathcal{D}$ $\mathcal{C}$ *is Definition 5.3 with premise (iii) replaced by $\mathcal{C}(C, E_1)$ and $\mathcal{C}(C, E_2)$. At $\mathcal{C} \coloneqq$ weak (resp. full) closure it is definitionally the Join Lemma (resp.* JoinLemma3F*) — `joinLemma3C_weak`, `joinLemma3C_full` — and it is antitone in the index: strengthening the closure demanded of the sides weakens the lemma (`JoinLemma3C.anti`).* `JoinLemma3C`

Reunifying the two routes was expected to be routine — "full closure only strengthens what a data type may assume." Mechanization shows it is not: the naive reunification — re-run the Join Lemma induction with full-closure hypotheses — is refuted, because no single-event (or block) peel preserves full closure:

Proposition 10.4 (no proper block peel)**.** *On the concrete four-event kill test — the set $U = \{e_{Ay}, e_{Rx}, e_{Ax}, e_{Ry}\}$ of the two-key datatype `K2` in its configuration C — no proper sub-block can*

be peeled from the back while preserving full closure: there is no $B \subseteq U$, nonempty and $\neq U$, that is upward closed under visibility within U ($a \in B$, $b \in U$, $a \xrightarrow{\mathsf{vis}} b$ imply $b \in B$) and exits along no lo^U-edge ($e \in B$, $x \in U \setminus B$ imply $\neg(e\ \mathsf{lo}^U\ x)$). Dually, no proper down-closed front block with no entering lo^U-edge exists. `no_proper_back_block, no_proper_front_block`

What survives is the closure-indexed contract, in which the data type *declares* its closure strength; the soundness bridge is uniform across strengths, so no reunification into one strength is needed:

Theorem 10.5 (closure-indexed soundness bridge)**.** *Let $\mathcal{C}$ be a closure predicate such that full closure implies it ($\forall C\,E$, E fully closed in $C \Rightarrow \mathcal{C}(C,E)$ — the "declares its strength" side condition: store versions arrive fully closed, by Definition 10.2 clause 5), let $\mathcal{D}$ satisfy* $\mathsf{JoinLemma3C}\ \mathcal{D}\ \mathcal{C}$*, and let $\mathsf{Inv}(\sigma_0)$ hold. Then every configuration reachable from the initial configuration is RA-linearizable, per version.* `ra_linearizable3_of_joinC`

All nine production discharges route through this bridge unchanged — the file `Development/AllMRDTs_viaContract.lean` carries the nine `*_adequate_viaContract` theorems (OR-set, optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, tombstone RGA, Peritext): eight at weak closure, the enable-wins flag at full.

11 The catalogue, discharged

Tables 1 and 2 are the empirical heart of the metatheory: the VC set is not merely adequate in principle — the data types people actually ship satisfy it. Three lessons from the sweep:

Tombstones are a metatheoretic trivializer. The catalogue's celebrated hard cases — the RGA sequence type and the Peritext rich-text type — land in the *easiest* tier. The reason is almost embarrassing: with tombstones, deletion *adds* a record; every state component is grow-only; all operations commute; the merge is a blind join. All of the RGA's genuine difficulty lives in the *read-side* traversal (turning the node soup into a sequence), which RA-linearizability of the state never touches. By contrast, tombstone-free designs (which delete physically) fall outside the eight VCs for a reason worth stating precisely: their commutation VCs are themselves reachability-conditioned (two inserts commute only on well-formed states), and the update layer above, like the paper's, quantifies over all states. The conditioned metatheory of §12 exists to host exactly this class, and the embedded-chain RGA of Part II is its canonical occupant, discharged through the factored route of §15.

Commutation class and delta class are independent axes. The multi-valued register is all-commuting, yet *not* in the unconditional tier: its merge is intersection-shaped, $(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$, and fails the delta laws on infeasible tuples exactly as the OR-set does. The boundary between tiers is drawn by *how the merge uses its LCA*, not by whether operations commute. (The compensation: all-commuting makes every punctured causal past empty, $B = \sigma_0$, so the feasible discharge needs no trichotomy at all.)

The eight VCs certify real code on its historical fault line. The enable-wins flag's repository history contains a known-broken sibling: a global-counter variant whose compositional VC (`inter_right_1op`) fails on DAG-shaped executions, the very defect per-replica counters

MRDT	contract tier	end-to-end theorem
Grow-only set σ: union of adds	unconditional	`GOSet_ra_linearizable3_eq`
Grow-only map σ: union of (k, v) records	unconditional	`GOMap_ra_linearizable3_eq`
Incr.-only counter σ: #events	unconditional	`IOC_ra_linearizable3_eq`
PN-counter σ: #inc − #dec	unconditional	`PN_ra_linearizable3_eq`
RGA (tombstone) σ: grow-only nodes + tombstones	unconditional	`rgaWithTombstones_ra_linearizable3_eq`
Peritext σ: grow-only mark records	unconditional	`peritextWithTombstones_ra_linearizable3_eq`
OR-set σ: tag live iff added, no vis-later rem	feasible	`ORSet_ra_linearizable3_eq`
OR-set (efficient) σ: … or same-replica add later (eviction)	feasible	`ORSetE_ra_linearizable3_eq`
Enable-wins flag σ: per key: ctr = #enables; flag iff unkilled enable	full-closure	`EWFlag_ra_linearizable3_eq`

Table 1: The Sal production catalogue, flat tier; each entry's second line is its one-line σ-characterization. The flat data types conclude through the ONE generic conditioned metatheorem (§12–§14) at the identity instantiation ($\approx = =$, their VC bundles feeding the flat collapse `FlatGeneric_Bridge.lean`, instance theorems in `MRDT_Instances/` (per-RDT)). The contract-tier column is each data type's Join-Lemma discharge content; the historical flat corollaries over the raw system remain in the instance directories as internal steps.

were introduced to fix. The mechanized σ-facts prove the production merge computes exactly union-liveness — *verified on precisely the corner where the sibling fails*: the metatheory explains *why* the fixed design is right, not merely that it converges.

12 The conditioned metatheorem

The eight VCs of §9 quantify over *all* states, which §11 showed is fatal for state-dependent datatypes: a sequence with physical deletion has two concurrent operations commuting only on well-formed states at which both are applicable, so over all states the commutation VC is false. The remedy is the general signature of §2: condition every VC on Inv and app, relax the witness fold of Definition 3.4 to the observational equivalence $\approx$, and prove RA-linearizability from the conditioned VCs once. This section and the next two state the conditioned VCs and the metatheorem with its pen-and-paper proof; §15 then gives the factored discharge route the production instances use.

MRDT	contract tier	end-to-end theorem
Multi-valued register	feasible	`MVR_ra_linearizable3_eq`
σ: tagged writes × overwrite log; all-comm ⇒ $B{=}\sigma_0$		
Add-wins priority queue	feasible	`AWPQ_ra_linearizable3_eq`
σ: the OR-set pattern on A; grow-only I		
Bounded counter (escrow)	all-comm	`BC_ra_linearizable3_eq`
σ: convergence flat; the *bound* is a conditioned safety theorem (`bc_version_inv`)		
FWW reservation register	all-comm	`FWW_ra_linearizable3_eq`
σ: payload arbitration (min-ts semilattice); winner characterized at every version (`fww_version_min`)		
LWW register	all-comm	`LWW_ra_linearizable3_eq`
σ: payload arbitration (max-ts semilattice), the last-writer twin of FWW; winner characterized (`lww_version_max`); *end-to-end mechanized*, unlike the flat CRDT LWW (sorry-carrying VC bridge)		
BudgetCart	or-set rc	`BCart_ra_linearizable3_eq`
σ: budgeted cart, spend derived from live instances; safety *gated* on causal canonicity (OQ 43.10)		
Mergeable queue (Peepul)	*direct join*	`queue_ra_linearizable3`
σ: no rc exists (enqueue clique); the merge itself is the linearization witness		
RGA (embedded-chain)	*direct join*	`embed_ra_linearizable3`
σ: RA-lin at every honestly reachable configuration; canonical sorted state, fold-canonical (`e_fold_canon`); code-parametric (Part II)		
RGA (sided, two-sided)	*direct join*	`sided_embed_ra_linearizable3`
σ: for every side assignment; the one-sided datatype is its all-R fragment (Part II)		
Peritext (embed kernel, fused)	*instantiation*	`peritextEmbed_ra_linearizable3`
σ: one RGA at $\alpha :=$ char⊕boundary; deleting a character never re-formats another (`renderIds_del`; Parts II–III)		

Table 2: The Sal production catalogue, conditioned and direct-join tiers (continuing Table 1). The direct-join instances conclude through the per-configuration join hook under honest reachability (§15); the conditioned instances carry their safety content as separate displayed theorems where noted.

13 The conditioned verification conditions

The conditioned signature $\langle \mathrm{State}, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{Inv}, \mathsf{app}\rangle$ is richer than the flat tuple of §9: it adds Inv and app. The four VCs below are the general contract, and the eight flat VCs of §9 are their identity instantiation ($\mathsf{Inv} = \mathsf{app} = \top$, $\approx\ =\ =$), discharged through `FlatGeneric_Bridge`. The two are one ladder, not two catalogues: a flat datatype supplies the eight, a conditioned datatype the four, never all twelve. Like the eight, the four are an arbitration instance (§7): they make $\mathsf{lo}_{\approx}$, the order from rc read up to ≈, the conditioned arbitration family of Theorem 7.3, whose one genuine clause is the ≈-Join — VC 4 below — the other five holding for any arbitration.

A datatype presents a signature $\langle \mathrm{State}, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{Inv}, \mathsf{app}\rangle$ where Inv is a predicate on states and app a predicate on (operation, state) pairs (Lean: `ConditionedMRDTSig`, `Framework/MRDTSig.lean`). It must supply:

VC 1 (Discipline)**.** $\mathsf{Inv}\,\sigma_0$ *holds;* do *preserves* Inv *on applicable operations; and generation is disciplined: every operation is* app *and fresh at its birth state, with globally unique, monotone ids. Formally, the execution model is the conditioned configuration: an event set $E(C)$ with a visibility relation $\xrightarrow{\mathsf{vis}}$ subject to*

(i) irreflexivity, transitivity, and endpoint membership (both ends of every $\xrightarrow{\mathsf{vis}}$-edge are events);

(ii) global timestamp uniqueness: distinct events of $E(C)$ carry distinct timestamps;

(iii) Lamport monotonicity: $a \xrightarrow{\mathsf{vis}} b$ implies $\mathsf{time}(a) < \mathsf{time}(b)$ (the generation clock increases along $\xrightarrow{\mathsf{vis}}$);

(iv) the two invariant clauses: $\mathsf{Inv}(\sigma_0)$, and $\mathsf{Inv}(\sigma) \wedge \mathsf{app}(e, \sigma) \Rightarrow \mathsf{Inv}(\mathsf{do}(\sigma, e))$.

App-at-birth is deliberately not a structure field: it is carried by the honesty contracts of §15, which is where "fresh and applicable at its birth state" becomes consumable. (Kernel-clean, axiom-free.) `ConditionedConfiguration, Framework/ConditionedExecutionModel.lean`

VC 2 (Conditioned commutation — the swap witness)**.** *For concurrent operations a, b and any state σ with $\mathsf{Inv}\,\sigma$ at which both are applicable in the relevant reorder sense, the swap witness holds:*

$$\mathrm{EqSwap}(a, b, \sigma) \;:\Longleftrightarrow\; \mathsf{do}\,(\mathsf{do}\,\sigma\,a)\,b \;\approx\; \mathsf{do}\,(\mathsf{do}\,\sigma\,b)\,a.$$

The states σ at which the witness is owed are pinned by VC 3. `EqSwap`

VC 3 (Threadable reorder invariant)**.** *An auxiliary invariant J on (state, pending-event) that (i) certifies applicability/the swap precondition of VC 2 at reorder states, (ii) holds at the enabling fold of each event, and (iii) is preserved by each reorder step. Only the enabled events — those whose causal past is folded — need be certified; that scope is exactly what a linearization repair ever transposes. Formally, VCs 2 and 3 are jointly the swap oracle of the convergence engine (Theorem 14.3): for a pending set E, a duplicate-free lo^E-respecting prefix list π drawn from E, and events $a, b \in E \setminus \pi$ with $a \neq b$, a and b lo^E-incomparable, and both enabled at π*

$$\big(\forall z \in E.\;\; z \neq a \wedge z\;\mathsf{lo}^E\;a \Rightarrow z \in \pi,\;\; \textit{and likewise for } b\big),$$

the oracle must supply $\mathrm{EqSwap}(a, b, \mathsf{fold}(\sigma, \pi))$. The enabledness restriction is clause (ii)+(iii) made precise: the oracle is consulted only at prefixes containing both swapped events' whole lo^E-past, exactly the prefixes a repair visits.

VC 4 (Merge–linearization bridge — the conditioned Join Lemma)**.** *For a reachable LCA state l and branches a, b obtained from l by disjoint concurrent event lists, $\mathsf{merge}\,l\,a\,b \approx \mathsf{fold}\,l\,\pi$ for a lo^E-respecting enumeration π of the branch events. Formally: at $\approx\; =\; =$ this is exactly the per-configuration ternary Join Lemma of Definition 5.3 (*`JoinLemma3At`*). At a nontrivial quotient it is the $\approx$-relaxed form (*`EqJoinLemma3C`*,* `Metatheory/GenericEqQuotient.lean`*): with canonicity up to $\approx$*

$$\mathrm{Can}^{\approx}_{C}(E,\, s) \;:\Longleftrightarrow\; \exists \rho.\;\; \rho \textit{ enumerates } E \;\wedge\; \rho \textit{ respects } \mathsf{lo}^E_{\approx} \;\wedge\; \mathsf{fold}(\sigma_0, \rho) \approx s$$

(`IsCanonicalStateEq`*; $\mathsf{lo}^E_{\approx}$ is lo^E with commutation read as $\approx$-commutation on Inv-states), the join demands: for Inv-satisfying s_0, s_1, s_2 and fully closed sides E_1, E_2, each of which, and their union $E_1 \cup E_2$, satisfies the datatype's generation-discipline predicate (a bundle parameter), canonicity of s_0, s_1, s_2 at $E_1 \cap E_2, E_1, E_2$ implies $\mathrm{Can}^{\approx}_{C}(E_1 \cup E_2,\; \mathsf{mergeL}(s_0, s_1, s_2))$. The fold is the raw update: the quotient layer's internal guard is discharged at the transfer, so the datatype states its $\approx$-join over its own do, not a guarded surrogate.*

14 The metatheorem

Theorem 14.1 (Soundness — conditioned VCs ⇒ RA-linearizability (*pen and paper*))**.** *Any MRDT satisfying VCs 1–4 is RA-linearizable (Definition 3.4, read up to ≈).*

Status of the theorem. This general form (arbitrary Inv, app, $\approx$) is a paper-level statement, proved below by two paper-level lemmas; deliberately, no single Lean declaration carries it, and none is cited on it. What is mechanized is its *instances*, which are the capstones the catalogue actually rests on: the flat instance ($\mathsf{Inv} = \mathsf{app} = \top$, $\approx\ =\ =$) is Theorem 10.5 (`ra_linearizable3_of_joinC`); the convergence engine consumed by Lemma 14.2 is Theorem 14.3 (`eq_convergence`); and the ≈-quotient layers (`Metatheory/GenericEqQuotient.lean`, `GenericEqQuotient_H.lean`, `GoodConfig3H.lean`, capstone shape `IsRALinearizable3Eq`) mechanize the adequacy induction at the quotient for the production instances of §15 and Part II.

The proof is generic — it never inspects a particular datatype — and factors through two lemmas, each proved over the abstract signature with ≈ an arbitrary congruence for do and merge.

Lemma 14.2 (Convergence)**.** *VCs 1, 2, 3 imply that all lo^E-respecting admissible enumerations of a backward-closed reachable E fold from σ_0 to ≈-equal states; hence $\sigma^*(E)$ is well defined up to ≈.*

Proof. Any two lo^E-respecting enumerations of E are related by repeatedly transposing adjacent lo^E-incomparable events. Because lo^E is non-transitive, "respecting" is the elementwise `Pairwise` condition rather than sortedness, so this order-repair is more delicate than for a total order — it is exactly what the generic engine `eq_convergence` carries out, and we appeal to it rather than re-derive it. Each transposition preserves the fold up to ≈ by VC 2, provided the swap precondition holds at the transposition point; VC 3 supplies it along the whole enumeration (base at each event's enabling fold, preserved by each step, scoped to the enabled events — the only ones the repair transposes). The engine consumes only that ≈ is an equivalence, do is ≈-congruent, and a swap oracle of the shape VC 2+3 provides (Lean: `eq_convergence`, kernel-clean). □

The engine itself is a displayable theorem, and its statement is the precise form of the two VCs' joint content:

Theorem 14.3 (the generic ≈-convergence engine)**.** *Fix an update do over a state space with an equivalence ≈ for which the fold is congruent, and any binary relation ℓ on events (the engine is order-agnostic; Lemma 14.2 instantiates $\ell := \mathsf{lo}^E$). Let E be a finite event set and π_1, π_2 two enumerations of E, each respecting ℓ, and let σ be any start state. Suppose the swap oracle holds at σ: for every duplicate-free ℓ-respecting list π drawn from E and every pair $a, b \in E \setminus \pi$ with $a \neq b$, $\neg\,\ell(a,b)$, $\neg\,\ell(b,a)$, and both enabled at π (every ℓ-predecessor in E of a, and of b, lies in π),* $\mathrm{EqSwap}(a, b, \mathsf{fold}(\sigma, \pi))$. *Then*

$$\mathsf{fold}(\sigma, \pi_1) \;\approx\; \mathsf{fold}(\sigma, \pi_2).$$

`eq_convergence`

Remark 14.4 (where the engine lives, and what is generic in the mechanization)**.** *The declaration's home is `MRDT_Instances/RGA_Rehoming/RGA_ConditionedConvergence.lean`: an*

instance directory, not `Framework/` *or* `Metatheory/`*, a bookkeeping accident of its genesis in the rehoming stress test. Its statement is payload-generic (the element type is a parameter) and order-generic in ℓ, but it is stated over that file's concrete sequence-state functor with its concrete $\approx$ (proved an equivalence with congruent fold in the same file), not over an abstract signature; the fully abstract reading is exactly Lemma 14.2's pen-and-paper step. Two mechanization details folded out of the display: the Lean statement threads an explicit length parameter for the strong induction (universally quantified, hence eliminable), and the oracle restriction to enabled prefixes is load-bearing: quantifying π over all lists is unsatisfiable, since several swap-discharge conjuncts are provably false at junk prefixes.*

Lemma 14.5 (Canonical adequacy)**.** *VC 4 and Lemma 14.2 imply that every reachable version state is $\approx\ \sigma^*$ of its delivered event set.*

Proof. Induction over the version DAG. Initial: the empty fold. Update node: appends one delivered operation to a canonical fold (definitional). Merge node with LCA l and branches a, b: by hypothesis $a \approx \sigma^*(E_a)$ and $b \approx \sigma^*(E_b)$, and $l \approx \sigma^*(E_l)$. First rewrite a, b to literal canonical folds using $\approx$-congruence of merge (so VC 4, stated for branch folds, applies); then VC 4 gives $\mathsf{merge}\, l\, a\, b \approx \mathsf{fold}\, l\, \pi$ for a lo^E-respecting π of the branch events $E_a \cup E_b$. Now E_l causally precedes those branch events (they were generated on states derived from l), so a lo^E-respecting enumeration of $E_l \cup E_a \cup E_b$ factors as (an lo^E-enumeration of E_l) followed by π; hence $\mathsf{fold}\, l\, \pi = \mathsf{fold}\, (\sigma^*(E_l))\, \pi \approx \sigma^*(E_l \cup E_a \cup E_b)$ by Lemma 14.2 (the engine is start-state generic). Backward closure keeps every enumeration admissible. □

Proof of Theorem 14.1. Immediate from Lemma 14.5: each version state $\approx \sigma^*(E_v)$, which is Definition 3.4. (Unconditioned template: `ra_linearizable3_of_joinC` plus the closure-indexed adequacy bridge, already kernel-clean with nine instances; the conditioned metatheorem generalizes it by carrying $\mathsf{Inv}/\mathsf{app}$ through the two lemmas.) □

Corollary 14.6 (Strong eventual consistency)**.** *Two versions with the same delivered set E have $\approx$ states: both $\approx \sigma^*(E)$ by Theorem 14.1, hence $\approx$ each other.*

Remark 14.7 (The flat instances)**.** *The nine flat production MRDTs (counter, OR-set, MVR, …) take $\mathsf{Inv} = \mathsf{app} = \mathsf{true}$: VC 2 is their unconditioned commutation, VC 3 is $J = \mathsf{true}$, and VC 4 is the ordinary Join Lemma — already discharged (`ra_linearizable3_of_joinC`). They inherit RA-linearizability from the same Theorem 14.1. The instances of §15 and Part II exercise the conditioned mechanisms proper.*

15 The factored discharge route: honest reachability and per-configuration joins

The conditioned instances in production do not each re-prove VCs 2 and 3 from scratch. What they share is an induction: thread a per-configuration honesty contract through reachability, and invoke a Join Lemma at each merge. This section presents that induction, extracted once, together with the instances that ride it: the mergeable queue (the flagship, whose conflict graph admits no rc at all), the bounded counter (conditioning for safety), and the FWW reservation register (payload arbitration). Part II's embedded-chain RGA is discharged by exactly this route.

The factorization: one metatheorem, three join species. The common structure across the conditioned instances is a theorem rather than a convention: the shared induction is extracted once (`Metatheory/HonestReach.lean`), as the next definition and theorem, and the queue's and the bounded counter's bespoke inductions are one-line corollaries of the extracted form.

Definition 15.1 (honest reachability)**.** *Fix a per-configuration contract H and the hypothesis* $\mathsf{Inv}(\sigma_0)$ *(displayed here and in every consumer below; flat data types discharge it trivially).* HonestReach $\mathcal{D}$ H *is the least predicate on configurations containing the initial configuration (Definition 3.2) and closed under: if C is honestly reachable, $H(C)$ holds, and $C \xrightarrow{\ell} C'$ is a* Step3 *transition, then C' is honestly reachable. The contract is checked at the pre-step configuration.* `HonestReach`

Theorem 15.2 (the factored metatheorem)**.** *Let H be a contract such that every H-configuration admits the Join at its core: for all C', $H(C')$ implies* JoinLemma3At $\mathcal{D}$ C' *(Definition 5.3). Then, with* $\mathsf{Inv}(\sigma_0)$*, every H-honestly reachable configuration is RA-linearizable, per version. Axioms:* `propext`, `Quot.sound`*. Plain reachability is the degenerate contract $H = \top$ (*`honestReach_of_reachable`*), recovering the unconditional bridge.* `ra_linearizable3_of_honest_reach`

What does *not* factorize is the join discharge, and that is the finding: the instances discharge `JoinLemma3At` by genuinely different mechanisms — *conditioned commutation* (operations commute on invariant-satisfying states, the framework's fully general quotiented route), *flat commutation* (the bounded counter: the CD route, contract $\top$), and *direct witness* (the queue: the merge is its own linearization certificate; Part II's embedded-chain family discharges the same way). Conditioning thus splits as

$$\text{generic honest-reachability metatheorem} \;\times\; \text{per-datatype join discharge},$$

and the framework's job at the boundary is to *receive* a join, not to produce one. A witness-classed generalization (`JoinLemma3AtW`, `Metatheory/WitnessClass.lean`, from the Shesha investigation of Part III) extends the hook to datatypes whose canonical states are unique only per witness-classed enumerations; and the instance layer is payload-generic where it should be (the embed instance of Part II takes its element type as a defaulted parameter, so the fused Peritext is a pure instantiation). One further regularity: the production honesty contracts all have the shape "P holds of each event at the fold of its causal past" — anchor liveness for the sequence family, slack for the counter, head-ness for the queue — with P the signature's client-checkable app (or its accuracy analogue). That shape is extracted once (`Metatheory/GenHonest.lean`):

Definition 15.3 (the generic honesty shape)**.** *For a predicate P on (event, state) pairs,*

$$\mathsf{GenHonest}\ \mathcal{D}\ P\ C \;:\Longleftrightarrow\; \forall e \in E(C).\ \forall \pi.\ \pi \text{ enumerates } \{e' \in E(C) : e' \xrightarrow{\mathsf{vis}} e\} \Longrightarrow P\big(e, \mathsf{fold}(\sigma_0, \pi)\big),$$

P of every event at the fold of every enumeration of its causal past. AppHonest $\mathcal{D} :=$ GenHonest $\mathcal{D}$ app *is the canonical instance. The companion side condition* CausalPastEnumerable *(every event's causal past admits an enumeration) holds in reachable configurations, whose event sets are finite, but is kept as an explicit hypothesis where consumed: the repository carries no generic finiteness result for reachable event sets.* `GenHonest`, `AppHonest`

Remark 15.4 (two honesty names: ∀ versus ∃)**.** *The mechanization carries two near-collision names, and both are load-bearing.* `AppHonest` *(*`Metatheory/GenHonest.lean`*) is the ∀-form above:* app *at the fold of every enumeration of the causal past.* `HonestApp` *(*`Metatheory/GenericSafety.lean`*, Definition 15.12 below) is the ∃-form:* app *at some causal fold of the causal past, the one the issuing replica materialized. The ∀-form feeds convergence contracts, whose guards are fold-order-insensitive; the ∃-form feeds the safety metatheorem, because for order-sensitive datatypes the ∀-form is unsatisfiable once a past holds two surviving candidates (different enumerations materialize different heads).*

Theorem 15.5 (the composite generic-honesty metatheorem)**.** *For any P: if every* (GenHonest $\mathcal{D}$ P)*-configuration admits the Join at its core, then, with* Inv(σ_0)*, every configuration honestly reachable under the contract* GenHonest $\mathcal{D}$ P *is RA-linearizable, per version.* `ra_linearizable3_of_genHonest_reach`

The counter's and the queue's contracts are re-derived as instantiations (`BCHonest_iff_genHonest`, `qHonest_of_genHonest`). A generic *safety* metatheorem completes the factorization; it is the subject of a paragraph below, and the counter's counting argument indeed turned out to mark where the per-datatype content sits.

The flagship direct-join instance: the mergeable queue — when no rc can exist. The queue of the certified-MRDT line of work (Soundarapandian et al., PLDI'22, *Certified Mergeable Replicated Data Types*) is the catalogue's first data type whose conflict graph is a *clique* rather than a star: two concurrent enqueues genuinely do not commute (queue order is arrival order), so VC 1's iff forces an rc-edge between them, and any orientation of a self-conflicting class composes with itself into a length-2 path — every candidate assignment violates `no_rc_chain` (Open Question 43.9). The flat engine is structurally unavailable, and the instance goes through the framework's per-configuration join hook instead (`JoinLemma3At`, `goodConfig3_merge_at`): prove the Join Lemma directly, at every reachable configuration.

The implementation is the functional queue with Peepul's merge, verbatim: the state is a list of $(tag, value)$ pairs, head first; enq v appends a fresh element tagged with the operation's own timestamp; deq t removes the element tagged t, wherever it sits; and the three-way merge keeps the LCA's elements that survive in both branches (in LCA order), then each branch's new arrivals (in branch order). The client-facing API is unchanged — dequeue() at a replica reads its local head — but the *operation* records which element that call removed: app for deq t is "t is the head of the issuing replica's state" (qApplicable(deq t, σ) $:\Leftrightarrow \exists v.$ head?$(\sigma) = (t, v)$; enqueues are unconditional). As with the sequence family, the generation discipline is the datatype's natural client behaviour, captured rather than imposed.

Definition 15.6 (the queue's honesty contract)**.** QHonest(C)*: every dequeue names a tag its issuer had observed as an enqueue,*

$$\forall e \in E(C).\ \mathsf{op}(e) = \mathsf{deq}\,t \implies \exists a \in E(C).\ a \xrightarrow{\mathsf{vis}} e \ \wedge\ \mathsf{time}(a) = t \ \wedge\ \exists v.\ \mathsf{op}(a) = \mathsf{enq}\,v.$$

The same clause read on the replica-keyed core is `QHonestCore`*; the ternary form transfers to it (*`qHonest_core`*).* `QHonest`

Theorem 15.7 (the queue's join, at every honest configuration)**.** *At every core configuration satisfying* QHonest*, the ternary Join Lemma holds:* JoinLemma3At Q C*.* `q_join_at`

Theorem 15.8 (the queue capstone)**.** *Every configuration honestly reachable under* QHonest *(the instance* QReach $\coloneqq$ HonestReach Q QHonest *of Definition 15.1, with* $\mathsf{Inv}(\sigma_0)$ *trivial) is RA-linearizable, per version.* `queue_ra_linearizable3`

The proof of Theorem 15.7 exhibits Peepul's merge as its own certificate. The Join Lemma asks for a delivery-respecting enumeration of $E_1 \cup E_2$ whose fold is $\mathsf{mergeL}(\sigma_0, \sigma_1, \sigma_2)$; the witness is the concatenation $\rho_0 \cdot (\rho_1 \setminus E_0) \cdot (\rho_2 \setminus E_0)$ of the given enumerations — exactly the merge's shape, no reordering. Three facts make the fold of that concatenation compute the merge, all consequences of the closure premise (branch event sets are backward-closed under $\xrightarrow{\mathsf{vis}}$ among non-commuting pairs), timestamp uniqueness, and the honesty contract (discharged by the app head-check via `qHonest_of_applicable`: applicability at *some* fold of the issuer's causal past suffices, the ∃-form of Remark 15.4): a fold from the empty queue is "the enqueues, in order, minus the dequeued tags" (the fold formula, which needs honesty — a dequeue preceding its enqueue would consume nothing); a delta's tags are fresh with respect to the LCA (uniqueness); and a cross-branch dequeue is impossible — a $\mathsf{deq}\, t$ in one branch pulls its enqueue into that branch by closure, so if the enqueue of t is the *other* branch's delta, it would lie in both branches, hence in the LCA, contradicting delta-ness. An LCA element then survives the union iff it survives in both branches, which is precisely the merge's first clause.

The payoff is the specification. In the PLDI'22 development the queue was the hardest case exactly because its specification had to be written in a bespoke relational language over event partial orders, with an explicit clause recovering *which element is the head*; the merge was then verified against that ad-hoc spec. Here the specification is the same generic statement as every other instance in Table 1 — every version is the fold of a delivery-respecting linearization of its event set — and head-ness, FIFO order, and once-per-element consumption are *read off* the fold rather than specified. (One semantic caveat, present equally in Peepul: two replicas that concurrently dequeue the same head both consume that one element — the merged state removes it once. RA-linearizability makes this visible instead of hiding it: both $\mathsf{deq}\, t$ events fold over the single $\mathsf{enq}\, t$.) At ∼870 lines (`MRDT_Instances/MergeableQueue/`), the instance carries positional structure at a fraction of a sequence datatype's cost.

Conditioning for safety: the bounded counter. Conditioning first entered the framework to deliver *convergence* for state-dependent datatypes. The bounded counter (per-replica escrow: grow-only tallies $(\mathsf{incs}, \mathsf{decs})$ per replica, per-slot group merge; mirror of `Sal/CRDTs/Bounded_Counter`) shows the same mechanisms delivering *safety* for a datatype whose convergence is flat: all its operations commute, the eight VCs discharge along the counter route, and the catalogue capstone (`BC_ra_linearizable3_eq`) is an identity instantiation. What the flat theory cannot even state is the property the datatype is named after. Its CRDT development is explicit: "the bound is enforced operationally by well-behaved clients" — a client refuses to issue Dec without slack — and that discipline lives outside the formal development. Conditioned, it moves inside: Inv is the per-replica escrow invariant ($0 \le \mathsf{decs}\, r \le \mathsf{incs}\, r$), app is the client's check (a Dec needs slack in the *issuing replica's own* slots — positive, and checkable against the issuing replica's state), and the contract on executions is the counter's honest-delivery contract:

Definition 15.9 (the counter's honesty contract)**.** $\mathsf{BCHonest}(C)$*: every decrement was applicable at the fold of its causal past,*

$$\forall e \in E(C).\ \mathsf{op}(e) = \mathsf{dec} \implies \forall \pi.\ \pi \textit{ enumerates } \{e' \in E(C) : e' \xrightarrow{\mathsf{vis}} e\} \implies \mathsf{app}\big(e, \mathsf{fold}(\sigma_0, \pi)\big),$$

with $\mathsf{app}(\mathsf{dec}$ *by* $r, \sigma)$ *the issuer's own-slot slack check* $\mathsf{decs}_r(\sigma) + 1 \leq \mathsf{incs}_r(\sigma)$*. This is exactly the generic shape of Definition 15.3 at* $P \coloneqq \mathsf{app}$*: the* dec*-guard is immaterial because* app *is* $\top$ *on increments (*`BCHonest_iff_genHonest`*).* `BCHonest`

Theorem 15.10 (the bound, per version)**.** *In every configuration reachable in the plain LTS (from the initial configuration,* $\mathsf{Inv}(\sigma_0)$ *trivial) whose history satisfies* BCHonest*, every registered version satisfies the escrow invariant:* $\mathsf{ver}(v) = (s, E)$ *implies* $0 \leq \mathsf{decs}_r(s) \leq \mathsf{incs}_r(s)$ *for every replica* r *(heads, LCAs, everything the store registered). Hence, for every finite replica list* rs*, the readable value is non-negative:* $0 \leq \sum_{r \in rs}\big(\mathsf{incs}_r(s) - \mathsf{decs}_r(s)\big)$ *(*`bc_value_nonneg`*).* `bc_version_inv`

The original proof is a counting argument riding the flat machinery: every version's state is a fold of its event set (canonicity), fold slots *are* per-replica event counts (the group merge is inclusion–exclusion on counts, via the LCA lemma), and the vis-maximal decrement's honesty at its own causal past — which lies inside the version by causal closure — bounds the decrement count by the increment count. At ∼600 lines, it is also a calibration point for which conditioning machinery is generic: the same invariant-at-generation shape, none of the positional structure.

The generic safety metatheorem: causal witnesses, and an escrow route. The safety half of conditioning also factorizes, but the obvious formulation is *false*, and the refutation comes from the flagship safety instance itself. One would like: if updates preserve Inv on applicable operations and the history is honest, then Inv holds at every version — proved by inducting along the version's canonical enumeration. But canonicity constrains the enumeration only by lo, and for the bounded counter — all of whose operations commute — lo is *empty*: [dec, inc] is a perfectly canonical enumeration of an honest version, and its intermediate state violates the escrow invariant. Only *endpoints* of canonical folds are guaranteed; commuting causal predecessors may float behind the event that needs them. The repair is to restrict the witness class, not the induction:

Definition 15.11 (causal folds; causal canonical witnesses)**.** σ *is a causal fold of* E *in* C *(*`CausalFold`*) when some enumeration* ρ *of* E *respecting* $\xrightarrow{\mathsf{vis}}$ *folds to it:* $\mathsf{fold}(\sigma_0, \rho) = \sigma$*. Prefixes of such an enumeration are exactly the vis-backward-closed subsets of* E*.* C *is causally canonical when every registered version carries a witness linearizing both orders:* $\mathsf{ver}(v) = (s, E)$ *implies some enumeration* ρ *of* E *respects* $\xrightarrow{\mathsf{vis}}$ *and respects* lo^E *and folds to* s*, strictly refining clause 1 of Definition 10.2.* `CausalCanonical`

Definition 15.12 (∃-form honesty)**.** *For a guard* A *on (event, state) pairs,* HonestAppOn $\mathcal{D}$ A C*: every event satisfies* A *at some causal fold of its causal past,*

$$\forall e \in E(C).\ \exists \sigma.\ \sigma \text{ a causal fold of } \{e' \in E(C) : e' \xrightarrow{\mathsf{vis}} e\} \text{ in } C\ \wedge\ A(e, \sigma).$$

HonestApp $\mathcal{D}$ *is the instance* $A \coloneqq \mathsf{app}$*: what the issuer's own materialized state witnesses. Being an* ∃*-statement it carries its own enumeration witness, so no enumerability side condition is owed; the contrast with the* ∀*-form* `AppHonest` *is Remark 15.4.* `HonestApp`

Definition 15.13 (the per-datatype safety obligation)**.** SafetyStepOn $\mathcal{D}$ I A*, over an explicit conditioning pair* (I, A)*: for every configuration* C*, event sets* E, S*, event* e*, and states* σ_S, σ_P *with*

(i) $E \subseteq E(C)$; *(ii)* E *vis-closed* ($a \xrightarrow{\mathsf{vis}} b$, $b \in E$ *imply* $a \in E$); *(iii)* $e \in E$;

(iv) $S \subseteq E$; *(v)* $e \notin S$; *(vi)* S *vis-closed;*

(vii) S *future-free for* e: $x \in S$ *implies* $\neg\, e \xrightarrow{\mathsf{vis}} x$; *(viii)* $\mathrm{past}(e) \subseteq S$: $x \xrightarrow{\mathsf{vis}} e$ *implies* $x \in S$;

(ix) σ_S *a causal fold of* S; *(x)* σ_P *a causal fold of* $\mathrm{past}(e) = \{e' \in E(C) : e' \xrightarrow{\mathsf{vis}} e\}$,

it holds that $I(\sigma_S) \wedge A(e, \sigma_P) \Rightarrow I(\mathsf{do}(\sigma_S, e))$*: applicability transfers from the causal past to any admissible prefix, fused with* I*-preservation so unconditionally-safe operations owe no guard.* SafetyStep $\mathcal{D}$ *is the instance* $(I, A) \coloneqq (\mathsf{Inv}, \mathsf{app})$. *SafetyStep*

Theorem 15.14 (generic safety, causal-witness form)**.** *If* $\mathsf{Inv}(\sigma_0)$, SafetyStep $\mathcal{D}$*, and* C *satisfies the good-configuration invariant (Definition 10.2), is causally canonical, and satisfies* HonestApp $\mathcal{D}$*, then every registered version state satisfies* Inv*. Axioms:* `propext`, `Quot.sound`. `version_inv_of_causal_canonical`

Theorem 15.15 (the causal witness is free for all-comm datatypes)**.** *If all operation pairs commute and* rc $\equiv$ *Either, then every configuration satisfying Definition 10.2 is causally canonical: a pointwise permutation argument, no reachability induction.* `causalCanonical_of_all_comm_rc_either`

The bounded counter discharges `SafetyStep` in three lines — extras in a prefix are other replicas' events and cannot touch the issuer's slots (`bc_safetyStep`) — and Theorem 15.10 is re-derived as a corollary, retiring the bespoke vis-maximal counting apparatus, which is thereby revealed as compensation for inducting over non-causal witnesses. A second, independent route generalizes that retired argument itself, with no causal witness and no Inv-preservation at all:

Definition 15.16 (measured datatypes)**.** $\mathcal{D}$ *is measured over an index type* κ *when it carries observations* $\mathsf{obs}_k : \Sigma \to \mathbb{Z}$ *and per-operation weights* $\mu_k : \mathcal{E} \to \mathbb{N}$ *with* $\mathsf{obs}_k(\sigma_0) = 0$ *and the affine update law* $\mathsf{obs}_k(\mathsf{do}(\sigma, e)) = \mathsf{obs}_k(\sigma) + \mu_k(e)$*. Folds of measured observations compute set measures, so enumeration-independence is free.* *Measured*

Theorem 15.17 (the escrow metatheorem)**.** *Let* $\mathcal{D}$ *be measured,* k_c *(consuming) and* k_f *(funding) indices, and* A *a guard, with*

(i) $\mu_{k_c}(e) \leq 1$ *for every event* e;

(ii) the guard supplies slack: $\mu_{k_c}(e) = 1$ *and* $A(e, \sigma)$ *imply* $\mathsf{obs}_{k_c}(\sigma) + 1 \leq \mathsf{obs}_{k_f}(\sigma)$;

(iii) C *satisfies the good-configuration invariant (Definition 10.2);*

(iv) consumption is serial: distinct events $a, b \in E(C)$ *with* $\mu_{k_c}(a) = \mu_{k_c}(b) = 1$ *are* $\xrightarrow{\mathsf{vis}}$*-comparable (the counter obtains this from same-replica totality, per slot);*

(v) GenHonest $\mathcal{D}$ A C *(the* $\forall$*-form: measured guards are fold-order-insensitive).*

Then every registered version state s *satisfies* $0 \leq \mathsf{obs}_{k_c}(s) \leq \mathsf{obs}_{k_f}(s)$. `escrow_version_inv`

Theorem 15.10 is derived a second time this way. The queue, by contrast, *refuses* `SafetyStep` — "t is the head" is not stable under a concurrent honest dequeue of the same head, and correctly so: the double-dequeue execution satisfies any sound obligation, and the queue's stable residue is event-level (`QHonest`), consumed by convergence rather than by a state invariant. The full analysis, including two further refuted formulations, is the pen-and-paper memo `Development/GENERIC_SAFETY_PENPAPER.md` — written, per this project's standing lesson, before any Lean. (A natural follow-up is a *conditioned step relation* — apply guarded by `app` at the head state — under which honesty becomes a theorem of the LTS rather than a contract; deferred.)

A small fourth instance: the FWW reservation register — payload arbitration, and what honesty cannot buy. A register claimable once (a seat, a username): sequentially the first claim wins; concurrently, the winner is the claim that is first in *Lamport* time. This is the positive complement to the metadata-free kill-test (Proposition 15.19 below): arbitration whose key is not in the state is impossible, and arbitration whose key *is* in the state is a semilattice triviality — the state is the minimum claim $(\mathsf{ts}, \mathsf{rep}, v)$ under the lexicographic order (or unset), update and merge are min, and the entire eight-VC discharge is min-algebra (∼300 lines, the smallest conditioned instance).

Theorem 15.18 (the FWW winner, per version)**.** *In every configuration reachable in the plain LTS (*$\mathsf{Inv}(\sigma_0)$ *trivial), every registered version* $\mathsf{ver}(v) = (s, E)$ *satisfies*

$$\big(\forall e \in E.\ s \le \mathrm{claim}(e)\big) \ \wedge\ \big(s = \top \Leftrightarrow E = \emptyset\big) \ \wedge\ \big(\forall m.\ s = m \Rightarrow \exists e \in E.\ \mathrm{claim}(e) = m\big)\ :$$

the register holds exactly the minimum-timestamp claim of its event set (a lower bound, attained), unset exactly on the empty set. No honesty hypothesis: semilattice folds are enumeration-independent. *fww_version_min*

Proposition 15.19 (metadata-free arbitration is impossible)**.** *Let* $\mathrm{canon}(\mathit{init}, L)$ *be the LWW specification value of a finite write history* L *from initial value* init *(the value of the timestamp-maximal write,* init *if* L *is empty). There is no function* $m : \mathrm{value}^3 \to \mathrm{value}$ *with*

$$\begin{aligned} m\big(\mathrm{canon}(\mathit{init}, L),\ \mathrm{canon}(\mathit{init}, L +\!\!+ \Delta_a),\ \mathrm{canon}(\mathit{init}, L +\!\!+ \Delta_b)\big) \\ =\ \mathrm{canon}(\mathit{init}, L +\!\!+ \Delta_a +\!\!+ \Delta_b) \qquad \textit{for all } \mathit{init}, L, \Delta_a, \Delta_b\ : \end{aligned}$$

two executions present the identical value triple $(10, 20, 30)$ *with opposite stamp orders, forcing* $m(10, 20, 30) = 30$ *and* $m(10, 20, 30) = 20$*. The timestamp is forced into the state by the merge, not by replay order.* *lww_merge_needs_timestamps*

Since timestamps respect causality, a causally later claim never displaces an installed one; the min rule arbitrates only among *concurrent* claimants, and it must, on pain of divergence. The instructive part is the generation discipline: `app` for a claim is "the register is unset in my causal past", and *deliberately no theorem consumes it* — two honest concurrent claimants both see unset, both claim, and each locally wins until a merge disabuses one. "Unset" is the minimal example of an applicability check that is *not stable under concurrent honest extension* (contrast the escrow counter's own-slot slack, which is — precisely the boundary the safety metatheorem's `SafetyStep` draws). A merge-based register is a reservation, confirmed only at causal stability; it is never a mutex.

The join hook is also the composition boundary. The factored interface earns one more dividend: data types compose *at the per-configuration join*, and only there. Finished capstones never compose, because reachability does not project (a second-component apply is a version-duplicating stutter for the first component's projection); per-configuration certificates do, which retroactively forces the interface of this section. The binary heterogeneous product (`Metatheory/Product.lean`) makes this a theorem pair: cross-component pairs commute definitionally, so lo^E has no mixed edges and the glued join witness is a plain concatenation of the component witnesses, with no interleaving combinatorics.

Theorem 15.20 (the product join gluing)**.** *Let $\mathcal{D}_1 \otimes \mathcal{D}_2$ be the binary heterogeneous product (`prodSig`): state $\Sigma_1 \times \Sigma_2$, operations the sum $O_1 \oplus O_2$, with* do, mergeL, Inv, *and* app *componentwise, and* rc *per component on same-side pairs and* Either *on mixed pairs (forced: cross pairs commute, so any mixed* Fst_then_snd *would violate Flat VC 1). For any product configuration C: if* JoinLemma3At $\mathcal{D}_1$ *holds at the first core projection of C and* JoinLemma3At $\mathcal{D}_2$ *at the second, then* JoinLemma3At $(\mathcal{D}_1 \otimes \mathcal{D}_2)$ C. `joinLemma3At_prod`

Theorem 15.21 (the composite product metatheorem)**.** *Let H_1, H_2 be component contracts and $H_\otimes(C) \coloneqq H_1(\pi_1 C) \wedge H_2(\pi_2 C)$ the product contract over the core projections (`prodContract`). If every H_1-configuration admits $\mathcal{D}_1$'s join and every H_2-configuration admits $\mathcal{D}_2$'s, then, with the product's* Inv(σ_0) *(both components'), every $H_\otimes$-honestly reachable product configuration is RA-linearizable, per version. Axioms:* `propext`, `Quot.sound`. `prod_ra_linearizable3_of_honest_reach`

The component certificates are consumed at the *projections* of product-reachable cores, which is exactly why they must be configuration-level, not reachability-indexed. A queue $\otimes$ counter capstone with zero bespoke proof (`MRDT_Instances/ProductDemo/`) demonstrates consumability; the safety and $\approx$-quotient kits, the boundary findings, and what remains open are collected in Open Question 43.11.

16 Virtual LCAs: merging without a common-ancestor version

The transition system of §3 gates Merge on the existence of an LCA: a version v_ℓ that reaches both heads and dominates every common ancestor. In a *criss-cross* configuration the common ancestors of two heads have two or more maximal elements, no version satisfies the predicate, and Merge is disabled. This is not an exotic corner. The shape's routine genesis is two replicas each merging the *same* pair of diverged heads: both create a version with the same event set, neither reaches the other, and every later pair of descendants has both rivals as maximal common ancestors (Figure 4). The JS runtime built on this model makes the gate explicit, and its randomized testing shows how routine the shape is under honest peer-to-peer syncing: 592 gated sync attempts in 220 randomized trials. The gap is exactly where git reaches for its *recursive* merge strategy: merge the maximal common ancestors into a virtual base, then merge the heads against that. This section develops that extension inside the metatheory and reports what it costs: nothing, for join-lemma datatypes.

The rule. Write $\mathrm{MCA}(v_1, v_2)$ for the maximal common ancestors of two versions (maximal in the reachability order, which is a partial order because a version's identifier is its allocation rank). VirtualLCA(v_1, v_2):

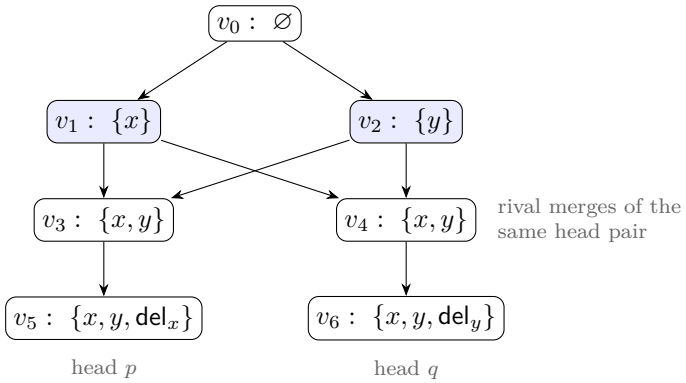


Figure 4: The criss-cross store used for the machine pins (the store of `VirtualLCA_Spot.lean`, drawn top-down; edges parent to child). Replicas p and q concurrently add x and y; each then merges the same diverged pair (v_1, v_2), creating the rivals v_3, v_4 with equal event sets; each deletes one element. The common ancestors of the heads v_5, v_6 are $\{v_0, v_1, v_2\}$ with *two* maximal elements v_1, v_2: no version is an LCA, and the gated Merge cannot fire. Note $E(v_1) \cup E(v_2) = \{x, y\} = E(v_5) \cap E(v_6)$: the antichain's union is exactly the meet, which is Proposition 16.4.

- $M := \mathrm{MCA}(v_1, v_2)$. It is nonempty (the pinned root is a common ancestor), and when the heads are incomparable every member is a strict ancestor of both.
- If $M = \{m\}$: return m. This is the existing rule, and Lemma 4.2 applies unchanged.
- Otherwise sort M ascending by rank (deterministic: version ids are allocation ranks) as $m_1 < \cdots < m_k$ and fold: start with m_1; at each step, merge the accumulator with the next member m_i through the recursively resolved LCA of the sub-pair, materializing a *scratch node* whose state is that ternary merge and whose event set is the union so far. Return the final accumulator.

The head merge then runs the ordinary ternary merge with the returned state in the LCA slot. The scratch nodes exist so the nested MCA queries are well posed; nothing references them afterwards, so they are never ancestors of real heads and cannot perturb later queries. Termination is unconditional: measuring a call on (a, b) by the number of allocated versions in the pair's joint ancestry, every version in a sub-pair's support is a common ancestor of the original heads while v_1 itself is not, so the measure strictly drops at each nesting level, and within a level the fold index drops. In the mechanization the extension lands as a conservative *new* step relation `Step3V`, with a base case embedding every old step and `mergeVirtual` the one new case; widening `Step3` in place was tried and rejected, because it breaks every exhaustive case analysis over the old relation in instance files. Only the final virtual node is allocated; the store invariant extends across the new step. The next three definitions pin the rule formally.

Definition 16.1 (common ancestors and MCAs, set-support form)**.** *The recursion pairs a scratch node, identified with its finite real support S, against a version w; $S = \{v_1\}$ recovers the head pair. Over the parent relation,*

$$\mathrm{CommonAnc}(S, w) \;=\; \{\, x \;:\; (\exists u \in S.\ \mathsf{Reaches}(x, u)) \wedge \mathsf{Reaches}(x, w) \,\},$$

and m is a maximal common ancestor (IsMCA*) of (S, w) when $m \in \mathrm{CommonAnc}(S, w)$ and every $x \in \mathrm{CommonAnc}(S, w)$ with $\mathsf{Reaches}(m, x)$ equals m. The antichain is materialized as a*

finite set (`mcaFinset`; ranks are bounded by w, so the bound is intrinsic), and LCAs are exactly the singleton case (`isMCA_singleton_of_isLCA` and its converse). `CommonAnc`, `IsMCA`

Definition 16.2 (the virtual-LCA state)**.** *Write σ_v for the state registered at v, defaulting to σ_0 (`stateD`; the canonicity theorems carry the allocation hypotheses that make the default unreachable). Sort* $\mathrm{MCA}(S, w)$ *ascending by rank as $m_1 < \cdots < m_k$; then the recursion of the rule above is (`vlcaAux`, `vfoldAux`):*

$$\mathrm{vlca}(S, w) = \begin{cases} \sigma_0 & k = 0 \\ \mathrm{vfold}(\{m_1\},\, \sigma_{m_1},\, [m_2, \ldots, m_k]) & k \geq 1 \end{cases}$$
$$\mathrm{vfold}(A, acc, [\,]) = acc$$
$$\mathrm{vfold}(A, acc, m :: ms) = \mathrm{vfold}\big(A \cup \{m\},\ \mathsf{mergeL}(\mathrm{vlca}(A, m),\ acc,\ \sigma_m),\ ms\big),$$

and $\mathsf{VirtualLCA}(v_1, v_2) \coloneqq \mathrm{vlca}(\{v_1\}, v_2)$*. The empty-antichain base is not a junk case: under the store invariant its event set, the empty union, is exactly the then-empty meet. Noncomputable but total, by the rank measure of the termination argument above.* `virtualLCAState`

Definition 16.3 (the widened transition system)**.** Step3V *embeds every* Step3 *rule (Definition 3.2) through a base constructor and adds one rule,* MergeVirtual(r_1, r_2)*:*

$$\frac{\mathsf{head}(r_1) = v_1 \quad \mathsf{head}(r_2) = v_2 \quad \mathsf{ver}(v_i) = (\sigma_i, E_i) \quad \mathsf{ver}(v_m)\ \textit{unallocated} \quad v_1 < v_m \quad v_2 < v_m}{\begin{array}{c}\mathsf{ver}(v_m) \coloneqq \big(\mathsf{mergeL}(\mathsf{VirtualLCA}(v_1, v_2),\, \sigma_1,\, \sigma_2),\ E_1 \cup E_2\big), \\ \mathsf{parents}(v_m) \coloneqq [v_1, v_2], \quad \mathsf{head}(r_1) \coloneqq v_m\end{array}}$$

with visibility unchanged: this is Merge *of Definition 3.2 with the* IsLCA *gate and its* $\mathsf{ver}(v_\ell)$ *read replaced by the recursive antichain merge in the LCA slot; with a singleton antichain the virtual state is the registered LCA state, so the rule strictly generalizes* Merge*. The widened LTS is `labeledTS3V`.* `Step3V`

The covering proposition. The Merge rule's semantic requirement on the LCA slot is Lemma 4.2: its event set must be the head intersection. The virtual construction meets it exactly, on the strength of invariants the store already carries.

Proposition 16.4 (covering)**.** *Let* $\mathsf{ver}, \mathsf{parents}$ *be a raw store with (i) the store invariant of Definition 4.1, (ii) rank-decreasing parents ($p \in \mathsf{parents}(v)$ implies $p < v$), (iii) every $u \in S$ allocated, and (iv) w allocated. Then, pointwise in the event e,*

$$\big(\exists m.\ \mathsf{IsMCA}(S, w, m) \wedge e \in E(m)\big) \iff \big(\exists u \in S.\ e \in E(u)\big) \wedge e \in E(w);$$

by extensionality,

$$\bigcup_{m \in \mathrm{MCA}(S,w)} E(m) \;=\; \Big(\bigcup_{u \in S} E(u)\Big) \cap E(w).$$

Neither finiteness nor nonemptiness of S is assumed, and the degenerate cases come for free: an empty MCA set forces an empty intersection (the $\supseteq$ direction constructs an MCA from a shared event's origin), and a singleton MCA set recovers Lemma 4.2. `mca_events_cover`

With $S = \{v_1\}$, $w = v_2$ this covers the head pair; with S the support of a scratch node it covers every sub-pair of the recursion; with a singleton MCA it degenerates to Lemma 4.2. The subset direction is event-set monotonicity along reachability, twice. The superset direction is where the store invariant earns its keep: an event in both sides has, by the *origin* invariant of §4.1 (each event appears first at a unique generator version), a single birth version that reaches every version whose event set contains it; that generator is a common ancestor, every common ancestor reaches a maximal one, and monotonicity carries the event up into it. Since the fold unions event sets, the returned virtual node has event set $\bigcup_i E(m_i) = E(v_1) \cap E(v_2)$: exactly what the Merge rule requires. An earlier forecast (recorded in the GC analysis, §17) had guessed the union approximates the intersection from below; under the carried invariants the approximation is *exact*, even though each antichain member individually sits strictly below the meet.

One delimitation marks the model boundary the proposition rests on. The origin invariant is load-bearing: in a model that can apply the *same* event at two incomparable versions (op-based redelivery, with no store-wide timestamp freshness), the event has two incomparable birth sites, neither a common ancestor, and the union strictly undershoots the intersection. Countermodel: apply e independently at the root on both branches; then $E(v_1) \cap E(v_2) = \{e\}$ while the sole MCA is the root with empty event set. The transition system excludes this by store-wide timestamp freshness (an op is born at exactly one version), so the covering invariant is not a new obligation: it is `StoreInv.origin` plus monotonicity, already carried and maintained.

Canonicity: one fold induction, and the VC surface does not move. The adequacy induction of §10 consumed two facts about the LCA slot: its event set is the intersection (now Proposition 16.4) and its registered state is canonical for that event set. The virtual construction re-supplies the second from the one per-datatype hypothesis the framework already centers on, the per-configuration Join Lemma:

Claim 16.5 (virtual join)**.** *Let C satisfy (i) the store invariant on its ranked store (Definition 4.1), (ii) the good-configuration invariant (Definition 10.2; in particular every allocated version's state is canonical for its event set), and (iii) the per-configuration Join Lemma* JoinLemma3At *at its core (Definition 5.3). Then, for any registered pair* $\mathsf{ver}(v_1) = (s_1, E_1)$, $\mathsf{ver}(v_2) = (s_2, E_2)$,

$$\mathrm{Can}_C\big(E_1 \cap E_2,\ \ \mathsf{VirtualLCA}(v_1, v_2)\big),$$

and along the fold every scratch node's state is canonical for its union event set. *`virtualLCAState_canonical`*

The proof is an induction over the ascending-rank fold: the antichain members are allocated, hence canonical by hypothesis; at each step the sub-pair's inner LCA slot is canonical by the inner recursion plus Proposition 16.4, and the Join Lemma yields canonicity at the union. Backward closure is preserved by union and intersection, so all intermediate sets stay in the hypothesis class; and canonicity is a property of a (configuration, event set, state) triple, not of allocated versions, so scratch states need no allocation to carry it. The same induction instantiates at the witness-classed join (`virtualLCAState_canonicalF`), and the merge case of the honest-reachability metatheorem gains the sibling case:

Theorem 16.6 (the widened metatheorem)**.** *Let* HonestReachV *be honest reachability (Definition 15.1) with* Step3 *replaced by* Step3V *(Definition 16.3). If every H-configuration admits*

the Join at its core, then, with $\mathsf{Inv}(\sigma_0)$*, every H-honestly reachable configuration of the widened LTS is RA-linearizable, per version: the same join obligation. The induction gains one case (`goodConfig3_mergeVirtual_at`), whose extra store-invariant hypothesis reachability itself supplies (`storeInv_of_honest_reachV`).* `ra_linearizable3_of_honest_reachV`

The headline is what did *not* change: `Framework/VC_Set.lean` is untouched. For any datatype that discharges its join at every configuration, the criss-cross gate lifts with **no new per-datatype verification condition**; the per-datatype VC surface does not move.

Two statement-hygiene facts, both machine-witnessed. First, determinism: the fold order is fixed (ascending rank), and for join-lemma datatypes order-insensitivity of the *returned* state is a theorem, since canonical states of a fixed event set are unique. Git's own recursive strategy folds in commit-date order and is known to be order-sensitive on conflicting merges; rank fold plus canonicity is what removes that defect here. Second, the insensitivity claim must be scoped to the result: on a directed three-member antichain the *intermediate* scratch states of the ascending and descending folds genuinely differ (each is canonical for its own, different, union), and only the final states coincide. Any mirror lemma stating canonicity of intermediates across fold orders is false.

The single-MCA shortcut, machine-refuted. The tempting implementation shortcut (pick *some* maximal common ancestor and use it as the LCA; no recursion, no scratch nodes) is not a simplification but a soundness bug, and the store of Figure 4 pins it. Resolving the head pair (v_5, v_6) through the virtual base folds v_1 and v_2 over the root into the state with both adds visible; the head merge then sees each delete on its own side against a base that *contains* the deleted element, and both deletions survive: the merged state is empty, the meet's fold. Picking the single MCA v_1 instead puts a base without y in the LCA slot, so y on v_6's side reads as a fresh add and the deleted y *resurrects*; picking v_2 dually resurrects x.

Proposition 16.7 (every single-MCA pick resurrects a deleted element)**.** *On the store of Figure 4 (`Metatheory/VirtualLCA_Spot.lean`, a two-element OR-pair datatype;* query_y *is the membership read of* y*,* σ_v *the registered states, and the pair resolved is the heads* (v_5, v_6)*):*

$$\mathrm{query}_y\big(\mathsf{mergeL}(\sigma_{v_1}, \sigma_{v_5}, \sigma_{v_6})\big) = \mathsf{true} \quad \wedge \quad \mathrm{query}_y\big(\mathsf{mergeL}(\mathsf{VirtualLCA}(v_5, v_6), \sigma_{v_5}, \sigma_{v_6})\big) = \mathsf{false},$$

and dually the pick v_2 *resurrects* x *(`t1f_pick_v_resurrects_x`); neither pick's merged state equals the virtual resolution's (`t1f_picks_ne`). The harness twin is the `T1F` fail companion of `virtual_lca_check.py`.* `t1f_pick_u_resurrects_y`

The recursion is not dispensable, nor is its depth boundable in practice: in a dense randomized probe (200 trials, 6 replicas, sync-heavy), criss-crosses arose in 176 of 200 trials with resolution nesting up to depth 7, because rivalry propagates upward (rival joins of rival joins); a depth-limited implementation would simply re-erect the gate.

The quotient layers, and the rehoming stress test. The claim above serves the raw layer. The framework's quotient layers (the $\approx$-indexed contracts of §14) each thread their own merge case, and their bill was probed on the hardest instance available: the rehoming RGA, the one production datatype whose proof route runs through an equivalence quotient with a reachability-anchored honesty stack (its only role here; it is the generality stress test, nothing more). Two findings. First, the dividing line at antichain unions is exactly the per-event versus prefix-state line: honesty components that speak about an event's *own* causal past (born accuracy,

dependency-prefix correctness) restrict to every scratch union for free, and the probe confirms them at every union of 590 randomized trials; components that speak about *applicability along an enumeration* (`noopFeasible`, applicable delivery) are *false* at antichain unions, in three named shapes (an insert whose anchor a concurrent kill has already removed fired 390 times across the probes; stale delete and insert paths fired 68 and 80 times), while every one of those states remained canonical for its union. So union-level obligations must be generation-discipline shaped, never applicability shaped. Second, the bill collapsed on execution: exactly *one* fold induction was owed beyond the raw layer (at the H layer, whose merge case consumes the registered slot's layer canonicity); the Eq-quotient layers contain no step destructs and re-thread mechanically (`rga_ra_linearizable3_eq_V`, and the flat production catalogue over `Step3V` via `FlatGeneric_Bridge_V`); and the NF layer is precisely the refuted `noopFeasible` route with no production consumer, so its mirror was *deliberately not built*. The refuted predicate marks a dead proof route, not a defect of the construction.

The queue, as a corollary. The mergeable queue of §15 is the instructive consumer: its join is already per-configuration (`q_join_at`), which is exactly the hook the virtual fold induction consumes, so lifting the criss-cross gate costs the queue nothing beyond restating reachability over the widened LTS. The capstone `queue_ra_linearizable3_V` (RA-linearizability at every honestly reachable configuration with virtual merges enabled) is ten lines, all of them plumbing: the framework extension multiplies across instances at the join boundary.

17 Commit GC: the keep set and the GC-safety theorem

The version store only ever grows, and merges arbitrarily far in the future reach back into it for their LCA state, so a runtime cannot reclaim "everything below the heads." The GC question is which store entries may be dropped without changing any future behavior; the answer is a keep set computable from the current heads, and a forward simulation showing pruning outside it is invisible.

The keep set. Fix a configuration C_0 (the prune point). A version is a *head* (`IsHead`) when some replica's head map points to it; write H for the current heads.

Definition 17.1 (the keep set)**.** *Over allocated versions (heads and v allocated, with the displayed event sets),*

$$\mathrm{Keep}(C_0) \;=\; \big\{\, v \;:\; \exists\, h_1, h_2 \in H \;\; (h_1 = h_2 \text{ allowed}),\quad E(h_1) \cap E(h_2) \subseteq E(v) \,\big\},$$

$$\mathrm{Keep}'(C_0) \;=\; \mathrm{Keep}(C_0) \cup \{0\},$$

and $\mathrm{Dropped}(C_0) = \{\, v \text{ allocated} : v \notin \mathrm{Keep}'(C_0) \,\}$ *is what pruning removes.* ***keepSet, keepSet', droppedSet***

This is the *upward event-set closure of the pairwise head intersections*, with the root 0 pinned separately (the model fixes $\mathrm{ver}\,0 = (\sigma_0, \varnothing)$ forever, and fresh replicas are created there). Taking $h_1 = h_2$ shows every head is kept, and with it everything event-set-above any single head. Pruning drops the store *payload* only: entries outside the keep set become unallocated, while the replica cores and the DAG skeleton (parents, a map of ids, not states) are untouched. Retaining the skeleton is forced, not a convenience: restricting the parent relation to kept versions changes

the LCA predicate itself, since removing common ancestors makes the domination clause easier to satisfy, so new LCA triples can appear whose event sets violate Lemma 4.2, and the pruned structure would fail its own invariant. The reclaimable memory is the states and event sets, which is where the memory is; the skeleton stays.

The theorem. Safety is a forward simulation, built from three lemmas, the first two packaged as a run invariant:

Definition 17.2 (the GC run invariant)**.** $\mathrm{HeadDom}(C_0, e)$*: some allocated prune-time head's event set is contained in* e*.* $\mathrm{Virgin}(C_0, v)$*: every prune-time-allocated ancestor of* v *(along the current parent relation) is the root.* $\mathrm{GCInv}(C_0, C)$*:*

(i) permanence: allocation is permanent and entries immutable, so v *allocated in* C_0 *implies* $\mathsf{ver}_C(v) = \mathsf{ver}_{C_0}(v)$*;*

(ii) monotonicity (disjunctive): every current head v *with* $\mathsf{ver}_C(v) = (s, e)$ *satisfies* $\mathrm{HeadDom}(C_0, e) \vee \mathrm{Virgin}(C_0, v)$*;*

(iii) the store invariant of Definition 4.1 at C*.*

GCInv, HeadDom, Virgin

Monotonicity is clause (ii): along any run from C_0, every current head either event-set-dominates some prune-time head, or its only prune-time ancestor is the pinned root. The disjunction is forced by replica creation: a fresh replica's head is the root, and its lineage grows from empty until it first merges with a dominating lineage, at which point it becomes dominating itself. *Intersection containment*: if a future merge fires with heads v_1, v_2 and allocated LCA v_ℓ, then v_ℓ was not dropped; when both sides dominate prune-time heads h_1, h_2, Lemma 4.2 gives $E_0(h_1) \cap E_0(h_2) \subseteq E(v_1) \cap E(v_2) = E(v_\ell)$, which is membership in the keep set, and when either side is virgin the LCA can only be the pinned root (`lca_not_dropped`). *Prune commutation*: every step of the unpruned run is matched, with the same label, on the pruned store; apply reads only the issuing head and a fresh slot (never dropped: dropped slots were allocated at prune time and allocation is permanent), and merge reads the two heads and the LCA, which the previous lemma keeps.

Theorem 17.3 (GC safety)**.** *Let* C_0 *satisfy the store invariant (supplied by plain reachability, Theorem 4.3), and let* $C_0 \xrightarrow{\ell_1 \cdots \ell_n} C$ *be any labelled* Step3 *run. Then the pruned prune point admits the same run against the correspondingly pruned endpoint,*

$$C_0 \setminus \mathrm{Dropped}(C_0) \xrightarrow{\ell_1 \cdots \ell_n} C \setminus \mathrm{Dropped}(C_0) \qquad \textit{(pruneKeep, dropVer)},$$

with the same label sequence (in particular every query, whose value rides on the label), and every replica read at every point is unchanged: $N_{C \setminus \mathrm{Dropped}(C_0)} = N_C$*. Prune-definedness is convention (iv) of §3.* *gc_safety*

One scope note, recorded because it is a genuine asymmetry: the converse simulation is deliberately not mechanized. A pruned store can exhibit steps the unpruned one cannot (a freed slot can be re-allocated; store-wide timestamp freshness quantifies over fewer entries), and the asymmetry is unobservable only via a reachability invariant (every allocated version sits below some current head) that is not part of the configuration structure. The direction GC-safety needs (nothing is lost) is the mechanized one.

The head-sync countermodel. Head-sync is the discipline that future merge arguments are current heads; the Merge rule enforces it syntactically. The keep set is calibrated to exactly that discipline, and the calibration is tight: a merge from an old interior version genuinely needs a version outside the keep set. The countermodel, worked concretely (replicas A, B; events e_1, e_2, e_3 on A, e_4 on B):

version	created by	events	parents	kept?
0	root	$\varnothing$	[ ]	pinned
1	A applies e_1 at 0	$\{e_1\}$	[0]	**dropped**
2	A applies e_2 at 1	$\{e_1, e_2\}$	[1]	kept
3	B merges A (LCA 0)	$\{e_1, e_2\}$	[0, 2]	kept
4	A applies e_3 at 2 (head)	$\{e_1, e_2, e_3\}$	[2]	kept
5	B applies e_4 at 3 (head)	$\{e_1, e_2, e_4\}$	[3]	kept

The pairwise head intersection is $E(4) \cap E(5) = \{e_1, e_2\}$; version 1, with $E(1) = \{e_1\}$, contains no pairwise intersection and is dropped, strictly (the store genuinely returns nothing at 1 after pruning). The future head merge is safe: the LCA of $(4, 5)$ is version 2, kept. But version 1 satisfies the LCA predicate for the *interior* pair $(1, 5)$, so any widened Merge that accepted a non-head argument (a rewound replica, a checkpoint restore, a replica missing from the head map) would demand the dropped entry. Every out-of-band way of resurrecting an old version breaks the keep set; head-sync is a hypothesis of the metatheorem, not pessimism. The countermodel is machine-checked:

Proposition 17.4 (head-sync is load-bearing)**.** *In the store C_{ex} of the table above (`GC_Safety.lean`, over the framework-resident conditioned grow-only set),*

$$\mathsf{IsLCA}(1, 5, 1) \ \wedge \ \neg\, \mathsf{IsHead}(C_{\mathrm{ex}}, 1) \ \wedge \ 1 \in \mathrm{Dropped}(C_{\mathrm{ex}}) \ \wedge \ \mathsf{ver}_{\mathrm{pruned}}(1) = \mathsf{none}\,:$$

the dropped interior version is the LCA of the interior pair $(1, 5)$, so a non-head-sync merge demands a pruned entry. `gc_needs_head_sync`

The revision under virtual merges. §16 widens what a merge reads: where no exact LCA exists, the resolution reads the maximal common ancestors, whose event sets sit strictly *below* the pairwise meet, while the keep set above retains versions *above* some pairwise meet. Under the widened rule the original keep set prunes away exactly what the recursion needs. The revision:

Definition 17.5 (the MCA closure, the revised keep set, and the widened invariant)**.** $\mathrm{mcasClosure}(C)$ *is the least set of versions containing every allocated head and closed under pairwise MCAs of its members (inductively: rules* HEAD *and* MCA*, the latter over* $\mathsf{IsMCA}(\{a\}, b, m)$ *for members a, b; finite, since ranks only decrease). The revised keep set is the upward event-set closure of its members, plus the pinned root:*

$$\mathrm{Keep}_V(C_0) \ = \ \big\{\, v \ \textit{allocated} \ : \ \exists w \in \mathrm{mcasClosure}(C_0) \ \textit{allocated}, \ \ E(w) \subseteq E(v) \,\big\} \ \cup \ \{0\}.$$

The run invariant GCInv_V *extends Definition 17.2 with: the prune-time store invariant; parent-list immutability at prune-time versions; the head gate (a current head allocated at prune time is a prune-time head or the root); and the gateway invariant: for every v allocated after the prune point and every prune-time-allocated w with* $\mathsf{Reaches}(w, v)$*, either $w = 0$ or some prune-time head h has* $\mathsf{Reaches}(w, h)$ *and* $\mathsf{Reaches}(h, v)$*.* `mcasClosure, keepSetV, GCInvV`

Keep_V contains the old keep set, and covers every recursion read: sub-pair MCAs are contained in real-pair MCAs, iterated. One might hope the one-layer seed (the pairwise MCAs of the current heads, without the fixpoint) suffices; it is one closure layer short, machine-witnessed: on a directed nested criss-cross, a depth-2 resolution reads the MCAs of the MCAs, pruning by the one-layer rule kills them (the sync raises a pruned-store error), and pruning by the closure rule keeps them while still pruning a genuinely dead interior commit, with the post-GC sync equal to the no-GC control. Future-proofing (the analogue of intersection containment) needs one new reachability fact, the *gateway* lemma: after the prune point, every ancestry path from a prune-time version to a post-prune version passes through a prune-time head or the pinned root, so a prune-time version that a future virtual resolution reads is an MCA of prune-time heads (layer one of the closure), and deeper reads stay in the closure.

Theorem 17.6 (GC safety, widened LTS)**.** *Let C_0 satisfy the store invariant, and let $C_0 \xrightarrow{\ell_1 \cdots \ell_n} C$ be any labelled* Step3V *run (gated and virtual merges, Definition 16.3). Then, pruning to* $\mathrm{Keep}_V(C_0)$*,*

$$C_0 \setminus \mathrm{Dropped}_V(C_0) \xrightarrow{\ell_1 \cdots \ell_n} C \setminus \mathrm{Dropped}_V(C_0)$$

as a Step3V *run with the same label sequence, and every replica read at every point is unchanged. The headline shape of Theorem 17.3 is kept verbatim; the gateway forced no statement change.* `gc_safetyV`

Bounded state: dropping the skeleton by a clock swap. The theorem above reclaims the payload but keeps the parent map, so the pruned store still costs O(commits ever minted): it retains history's shape, not its states. The skeleton was forced only for the parent-relation representation, where restricting parents mints new LCA triples that falsify lca_events (the argument of the keep-set paragraph). A change of representation removes it. The commit graph is a delta-compression of event sets, so a version's ancestry role is already carried by its event set used as a *clock*: ancestry is $\subseteq$, and the LCA of a head pair is the kept version at the *clock meet* $E(v_1) \cap E(v_2)$, which is exactly what lca_events says the structural LCA carries. The bounded store keeps only the kept payloads and sets parents $\equiv [\,]$; with no edges, structural reachability collapses to equality and lca_events holds trivially, so there is no history to store and the state is $O(|\mathrm{Keep}'|)$ (`GC_BoundedState.lean`).

Definition 17.7 (the bounded store and the clock lookup)**.** $\mathrm{clockPrune}(C)$ *prunes the payload to* $\mathrm{Keep}'(C)$ *and sets* parents $\equiv [\,]$*: its* ver *support is contained in* $\mathrm{Keep}'(C)$ *(*`clockPrune_support`*) and its skeleton is the constant empty function (*`clockPrune_no_skeleton`*).* $\mathrm{ClockLCA}(cc, v_1, v_2, u, s_u)$*: the heads are allocated with event sets e_1, e_2 and $\mathsf{ver}_{cc}(u) = (s_u, e_1 \cap e_2)$; the lookup reads event sets only, never parents.* $\mathrm{ClockCoherent}(cc)$*, the lineage guard: any two allocated versions with the same event-set clock carry the same state (the payload is a function of the clock).* `clockPrune`*,* `ClockLCA`*,* `ClockCoherent`

Theorem 17.8 (the clock lookup is complete; under the guard, sound)**.** *Completeness (guard-free): if C satisfies the store invariant and a head pair v_1, v_2 (both allocated, at heads r_1, r_2) has a structural LCA v_T with $\mathsf{ver}(v_T) = (s_T, e_T)$, then* $\mathrm{ClockLCA}(\mathrm{clockPrune}(C), v_1, v_2, v_T, s_T)$*: the swap never disables a merge the skeleton enabled (the LCA is kept,* `lca_not_dropped`*). Soundness (guarded): if* $\mathrm{ClockCoherent}(cc)$ *and both* $\mathrm{ClockLCA}(cc, v_1, v_2, u, s_u)$ *and* $\mathrm{ClockLCA}(cc, v_1, v_2, u', s_{u'})$*, then $s_u = s_{u'}$. Combined: under*

the guard, every clock LCA of a head pair in the bounded store carries exactly the structural LCA's state (`clockLCA_recovers`*).* `clockLCA_complete`, `clockLCA_sound`

Theorem 17.9 (bounded-state GC safety)**.** *Let C_0 satisfy the store invariant and $C_0 \xrightarrow{\ell_1 \cdots \ell_n} C$ be any* Step3 *run. On the bounded LTS* Step3C *(every rule of Definition 3.2 with* parents *frozen at $[\,]$, and* Merge *reading its LCA state by* ClockLCA *instead of* IsLCA *plus* $\mathsf{ver}(v_\ell)$*):*

(i) $\mathrm{clockPrune}(C_0) \xrightarrow{\ell_1 \cdots \ell_n} C \setminus_\flat \mathrm{Dropped}(C_0)$*, the whole trace, label for label (the flat drop $\setminus_\flat$ also empties the skeleton);*

(ii) every replica read at every point is unchanged;

(iii) the bounded store's payload support is contained in $\mathrm{Keep}'(C_0)$*; and*

(iv) its parent map is constantly empty,

so the state costs $O(|\mathrm{Keep}'|)$, not $O(\text{commits ever})$. `gc_safety_bounded`

The swap has exactly one hazard, and it is the lineage collision that §18 isolates. Resolving the LCA by clock alone is sound while the payload is a function of the event set, and unsound once datatype compaction breaks that: a same-clock version from an unrelated lineage, used as a merge's L, resurrects a dropped instance. The structural IsLCA sidesteps it by resolving within shared ancestry; a bare clock lookup cannot, so it is guarded by ClockCoherent (Definition 17.7), which is precisely the R_S-compatibility of §18; soundness under the guard is Theorem 17.8. The boundary is machine-checked both ways: a PASS SPOT where a future head-pair merge finds its LCA at the clock meet and reads correctly, and a FAIL SPOT:

Proposition 17.10 (the lineage collision)**.** *On the compacted store C_{coll} (*`GC_BoundedState.lean`*, two lineages compacted to the same clock),*

$$\mathrm{ClockLCA}(C_{\mathrm{coll}}, 3, 4, 1, \{10\}) \;\wedge\; \mathrm{ClockLCA}(C_{\mathrm{coll}}, 3, 4, 2, \emptyset)$$
$$\wedge\; \{10\} \neq \emptyset \;\wedge\; \neg\, \mathrm{ClockCoherent}(C_{\mathrm{coll}}) :$$

the head pair $(3, 4)$'s clock lookup returns two same-clock candidates with divergent states, so clock identification alone resurrects the dropped instance (picking version 1) or loses it (picking 2), and the guard is provably false on this store; conversely, had the guard held, Theorem 17.8 would have forced the two lookups equal (`spot_guard_would_fix`*).* `spot_lineage_resurrection`

One case is deliberately not covered: a criss-cross antichain of MCAs has no single clock meet, so the bounded form of a *virtual* merge needs the retained lineage among the MCA closure, and is left owed.

One frontier, three consumers. The meet of the current heads, $\bigcap_{h \in H} E(h)$, is the *stable cut*: the events every replica has already incorporated, below which no future merge can ever again place a concurrent event. Three parts of this document consume that one frontier. The garbage collector reads it from *above*: the keep set retains the upward closure of the pairwise meets, and what pruning reclaims is exactly the versions no future LCA lookup can name. The stability VC of §18 reads it from *below*: facts certified at the cut stay true under all future merges, which is what licenses compaction. And the storage layer of Part II (§26) consumes it as the epoch boundary for coordinate re-coding, where the in-flight-anchor argument needs visibility of all current heads. A runtime that tracks the frontier once serves all three consumers from the same bookkeeping, at pairwise rather than global granularity where GC needs it.

18 The stability VC: verified GC callbacks

Commit GC (§17) reclaims *versions*; it never touches the states themselves. But states also accumulate redundancy: an OR-set holding two add instances of the same element needs only one once both are known everywhere, and the embedded-chain RGA's coordinates carry dead history that a re-coding epoch could shed (§26). The runtime therefore wants to call back *into the datatype* so the concrete state can compact. The question is what the datatype must prove for such a `compact` to be sound, and what exactly the runtime must promise it. Both halves have sharp answers, and both were shaped by countermodels.

The naive gate is unsound: the discriminating remove. The obvious runtime promise ("the cut S is contained in every current head") does not suffice, by a near miss that shapes everything after it. Adds of element a at stamps $t_1 < t_2$, both below every head. Replica A compacts, dropping the older instance t_1. But a remove, minted long ago by a replica that had observed *only* t_2 (the remove is concurrent with the add t_1), is still at or below some head and arrives later. In the full execution the remove kills only t_2 and the element survives through t_1; in the compacted execution t_1 is gone and the element is absent. Reads diverge (Figure 5). The countermodel fires mechanically in the validation harness under the naive gate, and once (seed 2026, trial 218) it was found by the randomized twin runs rather than injected. It is also machine-checked in Lean, on the concrete merge states:

Proposition 18.1 (the discriminating remove)**.** *Let l_V hold both adds of a live (the shared LCA payload of Figure 5), a_{naive} the naively compacted head (older instance t_1 dropped), and b_{rem} the branch where the in-flight remove killed only t_2. Then*

$$a \in \text{read}\big(\mathsf{mergeL}(l_V,\, l_V,\, b_{\text{rem}})\big) \;\wedge\; a \notin \text{read}\big(\mathsf{mergeL}(l_V,\, a_{\text{naive}},\, b_{\text{rem}})\big) :$$

the control and the naively compacted merge diverge on a (`MRDT_Instances/ORSet/ORSet_Stability.lean`). `countermodel_diverges`

The interface: SettledAt. The repair is a version-level condition.

Definition 18.2 (SettledAt)**.** *Events e, a are concurrent in C (`ConcOp`) when $e \neq a$ and neither is $\xrightarrow{\mathsf{vis}}$ the other. A cut S is settled against an event set E (`SettledAtOn`) when*

(i) $S \subseteq E$*;*

(ii) *S is causally downward closed ($a \xrightarrow{\mathsf{vis}} b$, $b \in S$ imply $a \in S$); and*

(iii) *every $e \in E(C)$ concurrent with some member of S (existing anywhere in the configuration) is already in E.*

$\text{SettledAt}(C, v, S)$*: v is allocated with event set $E(v)$ and S is settled against $E(v)$.* `SettledAt`

Clause (iii) looks like a global quantification the runtime could never check, and in the design note it was carried as a hypothesis the runtime would somehow certify. It is now a *discharged theorem*: a purely local, checkable frontier predicate implies it.

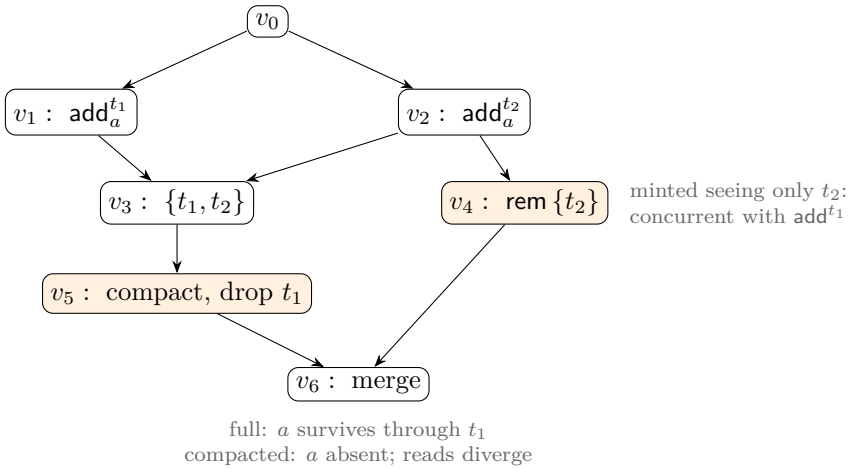


Figure 5: The discriminating remove (top-down; edges parent to child). Both adds of a sit below every head of the compacting replica, yet an in-flight remove that observed only t_2 can still arrive. Dropping the older instance t_1 under the naive "below every head" gate changes the read: the remove kills t_2 alone, and only the full state still holds a. The repair is SettledAt: compaction must wait until every event concurrent with the cut is already absorbed.

Definition 18.3 (evidence commits; the frontier predicates)**.** EvidenceCommit(C, v, S, j)*: some version c reaching v (along parents) is allocated with event set $E(c) \supseteq S$ and certifies replica j by the chain clause: every j-authored event outside $E(c)$ has all of S in its causal past ($\forall e \in E(C)$, $\mathsf{rep}(e) = j$, $e \notin E(c)$ imply $\forall a \in S$, $a \xrightarrow{\mathsf{vis}} e$). With* Registered$(C, j)$ *(replica j has a head state):*

$$\text{AllHeardSince}(C, v, S) \;:\Longleftrightarrow\; \forall j.\ \text{Registered}(C, j) \Rightarrow \text{EvidenceCommit}(C, v, S, j).$$

Separately, AllHeard(C, S)*, the all-heads frontier: every current head's event set contains S (as a gate alone this is the naive contract and unsound, Proposition 18.1; it enters only as the stability side of the runtime contract).* `AllHeardSince, AllHeard`

Theorem 18.4 (the evidence discharge)**.** *Let C satisfy the reachability bundle* ReachInvE*: (i) the store invariant of Definition 4.1, (ii) every event author is registered, (iii) the root replica* 0 *is registered; the bundle is supplied by plain or honest reachability (`reachInvE_reachable`, `reachInvE_honestReach`). If v is allocated, S is causally downward closed (the hypothesis the runtime must hand: its cut is a downward-closed set), and* AllHeardSince(C, v, S)*, then* SettledAt(C, v, S)*.* `settledAt_of_allHeard`

The argument is the evidence one made rigorous: an event e concurrent with some $s \in S$ was minted by a replica j before j saw s; the absorbed evidence commit c_j has $s \in E(c_j)$, so e precedes c_j at j and $e \in E(c_j) \subseteq E(v)$. The impossible case (were e still outside $E(v)$) would place the cut event s in e's minter's past, contradicting concurrency. This is causal stability in the sense of Baquero, Almeida, and Shoker, transplanted from op-based delivery to the commit DAG: there the argument rides on causal delivery order, here on downward closure of event sets. Three facts make it a genuine discharge rather than a relabeled assumption. The frontier the runtime maintains — the per-replica latest-absorbed commit, meeting to the largest certifiable cut — is tracked *axiom-free*: no choice, no classical step enters the version-level payoff. The gate is SettledAt *and* all-heard: settledness alone is not stable under future mints by replicas that

have not yet heard of S, so the all-heads frontier (the same one §17 tracks) keeps it stable, and createReplica is the *one* breaker — a replica joining after a compaction has heard nothing since S, falsifying AllHeardSince until it catches up, which is exactly the open-membership gate (a side condition of the theorem, not an unproved promise). And evidence is *commit*-shaped, not event-shaped: a pull that brings no new events can still bring the evidence commit that opens the gate, so a runtime that skips event-subsumed pulls silently starves the callback.

Two species of compaction. *Drop* (OR-set, MVR, remove-record GC): `compact` removes records; under a live-set merge rule a removed record propagates exactly like a deletion, so compaction spreads through merges with no protocol. *Remap* (the embedded-chain RGA's coordinate re-coding, §26): `compact` rewrites coordinates by an order isomorphism below the cut, and mixed states need translation on ingest. One VC skeleton covers both, with the simulation relation R_S instantiated to "differ only by S-redundant records" for the drop species and to the graph of the coordinate isomorphism for the remap species.

The six VCs.

Definition 18.5 (the stability VC bundle)**.** *Fix a conditioned MRDT. A stability bundle consists of data and six proof obligations. The data: a cut S; a compaction* $\mathrm{compact} : \Sigma \to \Sigma$*; a simulation relation $R_S \subseteq \Sigma \times \Sigma$ (full state on the left, compacted on the right; reflexive, so the pre-compaction run is related); a read interface* $\mathrm{obs} : \Sigma \to \mathrm{Obs}$*; an auxiliary pair invariant* Aux *on configuration pairs; a compaction gate, which must imply* SettledAt *at the gated version (field `gate_settled`); and a replica-creation policy* canCreate*. The obligations:*

- ***S1*** *Entry (`vc_entry`). Under the lifted relation and* Aux*, at a gated version v (a head, allocated on both sides with the shared event set),* $\mathsf{mergeL}(\sigma_v, \sigma_v, \sigma_v)\ R_S\ \mathrm{compact}(\hat{\sigma}_v)$*: the full side's stutter payload against the compacted callback.*
- ***S2*** *Observation (`vc_obs`).* $\sigma\ R_S\ \tau$ *implies* $\mathrm{obs}(\sigma) = \mathrm{obs}(\tau)$*.*
- ***S3*** *Step preservation (`vc_step`). Under the lifted relation and* Aux*, at a related head (v allocated on both sides with the shared event set, states $\sigma\ R_S\ \hat{\sigma}$) and a store-wide fresh timestamp t (no observed event, and no event of any registered version, carries t):* $\mathsf{do}(\sigma, e)\ R_S\ \mathsf{do}(\hat{\sigma}, e)$ *for $e = (t, r, o)$: any op minted with S in its causal past (all future ops, by stability). The design note carried a second class here, in-flight ops referencing dropped records; in the state-based ternary model those effects arrive only through merge branch states, so the class is absorbed into S4 and the step VC covers new mints alone.*
- ***S4*** *Merge congruence, argumentwise and reachability-indexed (`vc_merge`). Under the lifted relation and* Aux*, at genuine merge premises (heads v_1, v_2, an* lsLCA *witness v_T, all three registered on both sides with shared event sets):* $\mathsf{mergeL}(\sigma_T, \sigma_1, \sigma_2)\ R_S\ \mathsf{mergeL}(\hat{\sigma}_T, \hat{\sigma}_1, \hat{\sigma}_2)$*, so that mixed executions (a compacted head merging against a full LCA payload, or against a replica that never compacted) stay related. Argumentwise is forced, because mixed is the normal case. And the congruence is not free: unconditioned, it is false for the OR-set instance (a related LCA pair, an A past a remove that killed the kept twin, and a B still carrying the dropped instance live make the two merges read differently). The offending triple is unreachable (any state descending from the compacted LCA sheds the drop at its first absorption merge), which is exactly why the VC fires only under the threaded* Aux*, not as a free equation.*

S5 *Contract preservation (`vc_inv`). R_S preserves* Inv*: $\sigma\ R_S\ \tau$ and $\mathsf{Inv}(\sigma)$ imply $\mathsf{Inv}(\tau)$. This is the clause that composes compaction with the conditioned framework, and the one with no analogue in unverified systems. (The design note's wider reading, preservation of the applicable set and of the honesty contract, is not a bundle field; it is recovered observationally by the metatheorem's third conclusion, Theorem 18.7(iii).)*

S6 *Epoch coherence, observational (the standalone `EpochCoherentObs`, across two bundles V, V' with the same read interface): for every state σ,* $\mathrm{obs}(\mathrm{compact}'(\mathrm{compact}(\sigma))) = \mathrm{obs}(\mathrm{compact}'(\sigma))$*. That is: compacting at S and then at $S' \supseteq S$ lands in the same observational class as compacting at S' directly, so staggered and repeated compactions across replicas stay related. Observational is the correct strength: as a same-R-class statement it is false (a tag dropped at the first epoch whose second-epoch keeper has since died is kept by the direct coarser compaction; reads still agree).*

The bundle closes with the Aux*-threading obligations (`aux_init`, `aux_create`, `aux_apply`, `aux_merge`, `aux_compact`):* Aux *holds initially and is preserved by each matched step (creation gated by* canCreate*) and by the gated compaction.* `StabilityVC`

The metatheorem, and the stuttering-self-merge trick.

Definition 18.6 (the lifted relation, and the paired run)**.** ConfigRelS $R\ C\ \hat{C}$ *(`ConfigRelS`): the two configurations share L, $\xrightarrow{\mathsf{vis}}$,* head*, and* $\mathsf{parents}$ *verbatim; on the full side heads are allocated and headless replicas are stateless (two reachability facts carried in the relation, so the matching lemmas close without a reachability detour); event sets agree slot for slot (*ver*-projections to event sets equal); and payloads are pointwise related: $\mathsf{ver}_C(v) = (s, E)$ and $\mathsf{ver}_{\hat{C}}(v) = (\hat{s}, \hat{E})$ imply $R\ s\ \hat{s}$. The paired run* StabReach V *(`StabReach`, for a bundle V and $\mathsf{Inv}(\sigma_0)$): from the initial pair, either a matched* $\mathsf{Step3}$ *pair with mutating labels matched verbatim (queries may answer each side's own value), or a gated compaction (the full side takes the stuttering self-merge $\mathsf{merge}(r, r)$, the compacted side the callback move), each case carrying the re-established* ConfigRelS $\wedge$ Aux *(supplied by the matching lemmas `stability_match` / `stability_compact`).*

Theorem 18.7 (the stability metatheorem)**.** *Along every paired run* StabReach $V\ C\ \hat{C}$*:*

(i) ConfigRelS *$V.R\ C\ \hat{C}$ and $V.$*Aux *$C\ \hat{C}$ hold, and the full side C is ordinary* $\mathsf{Step3}$*-reachable (compactions project to stuttering self-merges): `stability_simulation`;*

(ii) reads are equal at every version: $\mathsf{ver}_C(v) = (s, E)$ and $\mathsf{ver}_{\hat{C}}(v) = (\hat{s}, \hat{E})$ imply $\hat{E} = E$ and $\mathrm{obs}(\hat{s}) = \mathrm{obs}(s)$: `stability_reads_equal`;

(iii) RA-linearizability is inherited observationally: if C is RA-linearizable per version (Definition 3.4), then every version $(\hat{s}, \hat{E})$ of $\hat{C}$ admits an enumeration π of $\hat{E}$ respecting the linearization order of $\hat{C}$'s own core with $\mathrm{obs}(\mathsf{fold}(\sigma_0, \pi)) = \mathrm{obs}(\hat{s})$: `stability_ra_inherited`.

Axioms: `propext`, `Quot.sound`.

The proof device worth recording: compaction projects, on the full side, to a *stuttering self-merge*. The LCA predicate is reflexive and $E \cup E = E$, so merging a head with itself is a legal step that allocates a new version with the same event set; the full run performs that stutter exactly where the compacted run compacts, and the twin DAGs stay *literally identical* in shape, so no version-map bisimulation is ever needed. The device is mechanized as the reflexive LCA

and the self-merge configuration (`isLCA_self`, `selfMergeCfg`, `stability_compact`). That one trick is why the metatheorem's axiom bill is {`propext`, `Quot.sound`}: no choice, no classical reasoning, just the step relation walked in lockstep.

One consequence deserves its own sentence, because it constrains future representation work: *compaction makes the payload no longer a function of the event set.* Resolving an LCA by event set alone is sound before compaction and unsound after (a same-event-set version from an unrelated lineage used as the LCA slot resurrects a dropped instance); the framework's structural LCA already resolves within shared ancestry, so the model is right as it stands, but any representation swap that identifies versions by clocks or event sets must retain enough lineage, or restrict lookup to R_S-compatible payloads.

First instance: the OR-set, and the refuted static drop. The redundancy predicate: among the *live* instances of an element, the adds in S that are not the kept witness. Under SettledAt every remove that could discriminate between two S-instances is already applied (that is Figure 5 inverted), so the surviving S-instances live and die together from here on, and dropping all but a live kept witness preserves every read. Two mechanization findings. The drop set must be *state-dependent* even under the settled gate:

Proposition 18.8 (the static drop is unsound)**.** *At the settled state s_{afterRem} (the kept twin (t_2, a) already dead before compaction fires, only (t_1, a) live), the static, twin-blind drop computed once from the cut erases the element's only surviving instance:*

$$\text{compact}_{\text{static}}(s_{\text{afterRem}})(t_1, a) = \mathsf{false} \quad \wedge \quad a \notin \text{read}\big(\text{compact}_{\text{static}}(s_{\text{afterRem}})\big),$$

while the twin-liveness-guarded callback refuses and is the identity there (`settled_refusal_id`). Newest-first is not needed for soundness: any live kept witness works. `static_drop_unsound`

And the interesting S4 case is a remove minted elsewhere against a full state whose kill set names a dropped instance: on the compacted side the kill is a partial no-op, and the states stay related because the dropped instance's fate is determined by the kept one. The instance discharges all six clauses:

Theorem 18.9 (the OR-set stability instance)**.** *For any cut specification (`CutSpec` S K nt: K marks the droppable S-adds, nt names each marked tag's kept newer twin, with the coherence clauses tying both to S: marked tags and their twins are S-adds of the same element, twins are strictly newer and themselves unmarked), the bundle with R_S := "differ only by S-redundant records" (`orRel`),* obs := *the membership read,* compact := *the twin-liveness-guarded drop (`orCompactC`), and gate* := SettledAt ∧ AllHeard *discharges all six VCs of Definition 18.5. Consequently every paired run reads identically at every version (`ORSet_stability_reads`) and inherits RA-linearizability observationally (`ORSet_stability_ra`), by Theorem 18.7.* `orStabilityVC`

The remap species' standing cut hypothesis is discharged by the same gate (`eRecode_settled_bridge`, the bridge §26 consumes).

Part II
The embedded-chain family

The embedded-chain RGA (replicated growable array) is a replicated list that keeps no tombstones, no ledger, and no event log: its entire state is the live nodes. It is a sequence MRDT in Part I's sense: state-based, reconciled by a three-way merge against the branches' lowest common ancestor version. A deleted element survives only as $\Theta(\log \delta)$ bits inside its descendants' sort keys, δ being the Lamport-timestamp gap between the element and its own insertion anchor (1 under continuation typing); that is the log of the event gap the insertion crossed, a race or a cursor jump, paid once, in space. Concurrent inserts provably cannot tie, so the merge contains no repair and never decides an order; the state is a canonical function of the event set, and convergence is literal equality. Observationally the datatype *is* the published RGA with the tombstone set compressed into coordinate entropy: a kernel-checked read-equivalence theorem, with machine lockstep at every step as its tested form. All of this follows from one idea: an element's sort key is its *birth chain*, the timestamps from the root of the birth tree down to the element, dead ancestors included. The chain is written at birth, never edited, and carried in an order-preserving self-delimiting code. Separating the datatype from its encoding makes the metatheory nearly definitional: correctness is three one-line facts about chains, plus a single faithfulness theorem about the code. The design is machine-validated (the L1–L25 anomaly battery, except L19, RGA's own, discharged by the sided generalization of §23; 420+ randomized version-DAG executions; 120/120 lockstep), and the central theorem, RA-linearizability at every honestly reachable configuration, is a kernel-checked Lean theorem (§30) discharged by the direct-join route of §15. (Working names in the validation artifact: `embed-tree` for the unary code, `embed-code` for the delta-coded one.)

19 The datatype: sort keys are birth chains

Timestamps $t \in \mathbb{N}^+$ are Lamport stamps, globally unique, and double as element identities. Causality gives: the anchor of every insert carries a smaller timestamp than the insert itself.

Convention (Part II plumbing). Four pieces of plumbing are folded out of every Part II display below, once and for all; Part I's convention (§3) remains in force. (i) *The code*: Γ ranges over ordered prefix codes (Definition 20.1), and every display is parametric in Γ unless a concrete instance (`unaryCode`, `binaryCode`, `eliasDeltaCode`) is named. Chains in displays are *positive* (every delta ≥ 1, the shape Lamport causality forces); displays write "positive chain" and elide nothing else, the mechanized statements carrying the positivity as explicit `PosChain`/`PosSChain` antecedents. (ii) *The comparator*: `key` maps a coordinate's bits into symbols ($\texttt{0} \mapsto 1$, $\texttt{1} \mapsto 2$) and appends the terminator 3; `keyLt` is strict first-difference comparison of symbol strings; the sided `sKey` (§23) appends the same terminator to symbols already banded $\{1, 2\} < 3 < \{4, 5\}$. Displays write "the display comparator" for this apparatus and before for the induced strict display order on live ids (`before`: both live, keys descending). (iii) *Axiom provenance*: displayed capstones are kernel-checked on {`propext`, `Classical.choice`, `Quot.sound`}; displayed countermodels evaluate concrete executions by `native_decide` under the document's SPOT convention (adding the compiler axioms), exactly like unit tests. Per-display axiom notes appear only where a display departs from this split. (iv) *The generation layer* (§24 only): the Fugue-family ordering theorems quantify over per-replica lists of generation records (`Know`, `KnowM`)

under the reachability relations `FugueReach`, `MaxReach`, declared once there; the record-reading functions (left/right origin, display order, liveness) are named there and never re-derived per display. (v) *The at-hand domain* (the storage layer, §26.1 onward): for a stable-prefix re-map bundle F (Definition 26.1) with at-rest domain Rest and declared beyond-cut mints MintAt, a coordinate c is *at hand* (F.Dom c) when Rest c holds or $c = \pi \cdot \Gamma.\mathrm{enc}(d)$ for some declared mint MintAt π d. The storage displays' recurring hypothesis pair, "s at hand, τ declared at F", abbreviates exactly: every record coordinate of the cut state s satisfies F.Dom, and every insert $(t, r, \mathsf{ins}(e, \pi, a)) \in \tau$ satisfies F.MintAt π $(t - a)$; both quantifiers appear explicitly, in this form, in every mechanized statement. Timestamps throughout Part II are plain Lamport stamps in $\mathbb{N}$.

The birth tree. Every insert names an anchor. The event set therefore induces a tree: the root is the front sentinel 0; the *birth parent* of each element is its anchor. The tree is append-only and never edited: an element's place in it is fixed at birth.

Birth chains. The *birth chain* of u is the sequence of timestamps from the root down to u (sentinel elided): $\mathrm{chain}(u) = \mathrm{chain}(a) \cdot u$ when u was inserted after anchor a, and $\mathrm{chain}(u) = \langle u \rangle$ at the front. Dead ancestors are *not* elided: their timestamps remain in their descendants' chains. The chain is the sort key, and everything in this section is stated in terms of it; nothing here depends on how chains are stored (§20).

Type. A state is a finite map over *live nodes only*:

$$\sigma \;:\; u \;\mapsto\; (\,\mathrm{chain}(u),\ \mathrm{char}_u\,).$$

init. $\sigma_0 = \varnothing$.

do (the operations). Two operations, with $\mathsf{rc} = \mathsf{Either}$ everywhere (§21):

- $\mathsf{ins}(x \leftarrow a,\ c,\ \pi)$: insert element x (its timestamp) with character c after anchor a ($a = 0$ for the front). The payload $\pi = \mathrm{chain}(a)$ is the anchor's birth chain, carried *for the proof alone* (§21). Application on a state where a is live (every reachable application, by honesty):

  $$\sigma[x] := \big(\mathrm{chain}_\sigma(a) \cdot x,\ c\big), \qquad \pi \text{ unread.}$$

  No head-tracking, no reading of siblings.

- $\mathsf{del}(d)$: remove d's record. Nothing else changes: survivors' chains are untouched; in particular, d's descendants still carry d's timestamp in their sort keys. If d is absent, no-op (concurrent double delete).

read. Sort survivors by *chain-lex*: compare chains timestamp-by-timestamp with *larger first*; a proper prefix precedes its extensions (a live ancestor displays immediately before its descendants). Among same-parent siblings this is exactly RGA's newest-first rule; equivalently, the canonical order is the RGA traversal of the birth tree restricted to survivors.

Definition 19.1 (the chain display order)**.** *In the mechanization a chain is its list of deltas, each entry the timestamp gap to its parent, partial sums recovering the timestamps; at a first divergence the two entries share an anchor, so the larger delta and the larger timestamp are the same verdict.* chainBefore *is the least relation closed under two rules:*

$$\text{chainBefore } ch\ (ch \cdot ext)\ \ (ext \neq \langle\rangle), \qquad \text{chainBefore } (p \cdot \langle d\rangle \cdot c_1)\ (p \cdot \langle e\rangle \cdot c_2)\ \ (e < d) :$$

an ancestor precedes everything in its subtree, and at the first divergence the larger delta (the newer sibling) precedes. Distinct chains are always comparable (`chainBefore_total`). The read above is: sort survivors by chainBefore *on their birth chains; every ordering theorem of this part is stated against this relation.* `chainBefore`

merge. merge(L, A, B), where L is the LCA (the branches' lowest common ancestor version, the merge context the MRDT model stores and supplies). The merge is *survival* (OR-set): live iff in all three, or born in exactly one branch. There is no second step at this layer: survivors' chains come with their insert events, so every input that holds a survivor agrees on its chain. The merge never compares timestamps and never decides an order.

Inv / applicable / honesty. The honesty contract is the standard RGA generation discipline: an insert is *generated* at a replica where its anchor is live (you insert after a character you can see), and ins/del are *applied* on states where they are applicable (anchor live; delete of a live or already-dead node). On every reachable state, application never consults π. Canonicity (Theorem 19.3) doubles as Inv; at the representation layer, Theorem 20.5(iii) is its stored form.

$\approx$. Equality. The state is a canonical function of the event set (Theorem 19.3), so no quotient is needed: convergence is literal state equality.

The components above are mechanized twice, one datatype, two carriers; every Part II theorem names one of the two.

Definition 19.2 (the embedded-chain RGA, as mechanized)**.** *The model layer (`Sal/MRDTs/RGA_Embed/RGA_Embed_MRDT.lean`) stores a finite map $\sigma : t \mapsto (\text{el},\ c)$ from live ids to (element, absolute coordinate) pairs, coordinates as bit strings:*

$$\mathsf{do}_\Gamma\big(\sigma,\ (t, r, \mathsf{ins}(e, \pi, a))\big) = \sigma\big[t \mapsto (e,\ \pi \cdot \Gamma.\mathrm{enc}(t - a))\big], \qquad \mathsf{do}_\Gamma\big(\sigma,\ (t, r, \mathsf{del}(x))\big) = \sigma \setminus x,$$

the insert writing the mint of its carried prefix (never reading the state; `mint`, `do_`), the delete removing the record (no path, nothing rehomed). The merge is OR-set survival on identities, values read off any input holding the id (immutable, so the choice is immaterial):

$$\mathrm{dom}\big(\mathsf{merge}(l, a, b)\big) = (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l) \qquad (\texttt{merge}).$$

rc = *Either everywhere (`rc`, §21); observational equality is extensional map equality, equal domains and equal records (`eq`). The instance layer (`Sal/ConditionedMRDTs/MRDT_Instances/EmbedRGA/EmbedRGA.lean`) is the same datatype on the canonical carrier: the state a list of records (t, el, c) strictly descending by key (`EState`); insert a sorted insertion, idempotent on a present id, delete a filter (`eUpdate`); the ternary merge the same survival formula re-canonicalized by a sorted two-merge (`eMergeL`); the fold of an operation list `eFold`. It is packaged as the conditioned signature of Part I with* $\mathsf{Inv} = \mathsf{app} = \top$ *and $\approx$ = = (honesty lives in the configuration layer, §30), parametric in Γ and in the element payload α.* `do_`, `merge`, `rc`, `E`

The metatheory, which is nearly definitional. At this layer the design's theorems all but evaporate; we record them because they are the specification the representation must meet (§20).

Theorem 19.3 (chain-level metatheory)**.** *In every reachable state:*

(i) Canonicity: $\sigma = \{\, u \mapsto (\mathrm{chain}(u),\ \mathrm{char}_u) \,\}$ *over survivors* u*; the state is a function of the event set.*

(ii) Birth constancy: $\mathrm{chain}(u)$ *is fixed at* u*'s insert and never edited by any later transition.*

(iii) Totality (no ties): chain-lex is a total order on every state's survivors.

Proof. (i) Survivorship is OR-set, hence a function of the event set, and a survivor's chain is determined by its insert event alone: by induction, ins writes $\mathrm{chain}_\sigma(a) \cdot x = \mathrm{chain}(a) \cdot x$ (the anchor is live at application, by honesty, and its stored chain is canonical), while del and the merge only delete or select records, never rewrite them. (ii) is (i) read off per node: no transition writes to an existing record. (iii) Distinct elements' chains are distinct (they end in distinct timestamps); if neither is a prefix of the other, the first difference compares two distinct timestamps in $\mathbb{N}$, and uniqueness of Lamport stamps *is* the no-tie property; prefix pairs are ancestor/descendant, ordered by the prefix rule. □

Mechanized: (i) is fold canonicity, `e_fold_canon` (§30); (ii) is definitional there (records are written once, then only removed or copied); (iii) is unique decodability plus totality of the chain order (Theorem 20.3, `chainBefore_total`). One more design-layer fact belongs beside them, the litmus g-column (non-interleaving of concurrent runs) as one lemma. It is stated one layer down, over stored coordinates, because that is where it is consumed; by unique decodability a coordinate prefix is exactly a birth-chain prefix.

Theorem 19.4 (subtree convexity, one-sided)**.** *In any state* s *(no honesty, no reachability): if* before $s\ t_1\ t_2$ *and* before $s\ t_2\ t_3$*, and a coordinate* c *prefixes both stored coordinates* $\mathrm{pos}_s(t_1)$ *and* $\mathrm{pos}_s(t_3)$*, then* c *prefixes* $\mathrm{pos}_s(t_2)$*. Instantiated at an anchor's coordinate: everything displayed between two members of a subtree is in the subtree, so runs typed under distinct anchors cannot interleave. The proof is pure lexicographic convexity of prefix sets; no property of the code is consumed.* *subtree_convex*

The one-line separation, and the conservation law. The natural first attempt at tombstone-freedom is the *rehoming* RGA: keep flat (element, anchor) records, and let a delete splice its node out, re-parenting the orphans to the nearest live ancestor (the *climb*). We built and verified that design before this one (it is RA-linearizable, in the same Lean framework), and it fails the L1 delete-reorder litmus (the $[b, a, c] \to [c, b]$ anomaly, Figure 6): the climb re-sorts a rehomed node by its own timestamp, so *its sort key is the birth chain truncated at each rehoming.* This design keeps the chain whole. That is the entire difference between the two datatypes, and it is the crisp form of the conservation law that our design sweep kept re-imposing: *the sort key must retain dead ancestors' timestamps.* Every design that forgets them has a named machine-checked countermodel (§29). What is negotiable is only how many bits the retained timestamps cost; that is the subject of §20, and the answer is: the log of each element's anchor gap, and nothing else.

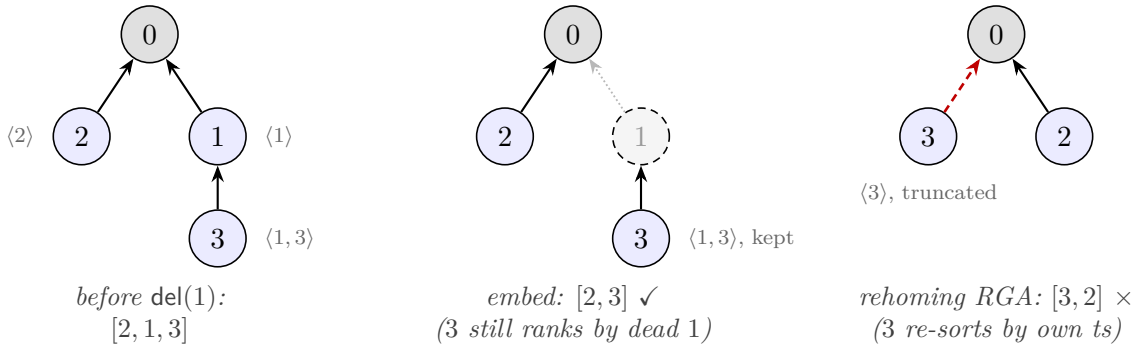


Figure 6: The one-line separation on the L1 delete-reorder litmus: insert 1 and 2 at the front, 3 after 1, delete 1. The rehoming RGA's climb (red) re-parents 3 and re-sorts it by its *own* timestamp: the truncated key $\langle 3\rangle$ beats $\langle 2\rangle$, and 3 jumps over 2. The embedded chain $\langle 1, 3\rangle$ keeps ranking 3 by its dead parent's timestamp: $[2, 3]$, the delete-order-preserving read. Arrows point from a node to its anchor; dashed grey = dead.

Worked example (Figure 7). 6 and 10 concurrently insert at the front: chains $\langle 6\rangle$ and $\langle 10\rangle$; display $[10, 6]$, no arbitration, at every replica and merge order. Type $22, 16$ under 6: chains $\langle 6, 22\rangle$, $\langle 6, 16\rangle$. Delete 6: the records of 22 and 16 are untouched and still carry 6's timestamp. A third party concurrently inserts 8 at the front: $\langle 8\rangle$ sorts below $\langle 10\rangle$ and above $\langle 6, 22\rangle$, so the display is $[10, 8, 22, 16]$ whether 8 arrives before or after 6's death (machine-checked as the *credential countermodel*, the shape that kills every timestamp-oblivious variant, §22).

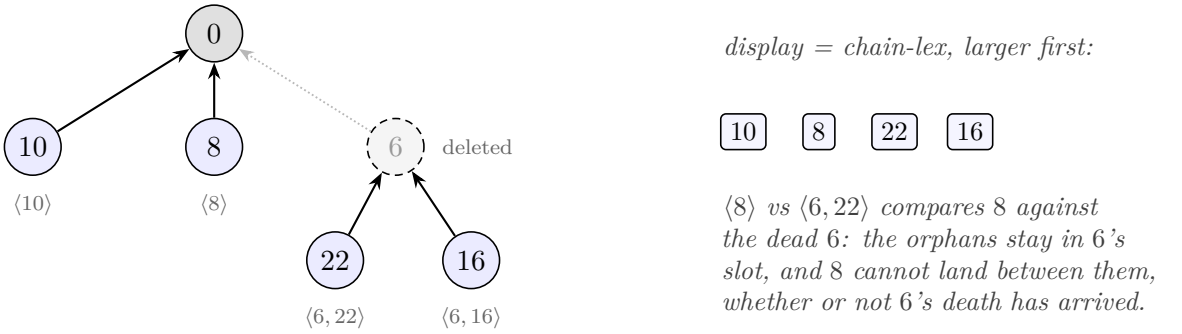


Figure 7: The credential countermodel as a birth tree. 6 and 10 race at the front; $22, 16$ are typed under 6; 6 dies; 8 arrives at the front concurrently. Survivors sort by whole birth chains, so the read is $[10, 8, 22, 16]$ at every replica and merge order. Every timestamp-oblivious variant flips this shape (§29).

20 The representation: chains as entropy-coded coordinates

Everything above is the datatype; everything below is bits. A naive carrier would store a full timestamp at every chain level (each $\sim$log(document age) bits, even for purely sequential typing) and compare whole chains at every read. This section stores the chains compactly without disturbing their order.

Vocabulary. A *codeword* is the bit string $C(\delta)$ encoding one chain level: one element's timestamp, stored as its differential δ to its birth anchor (step 1 below). A *coordinate* is the bit string encoding a whole chain: the concatenation, root to element, of one block per level (the level's codeword plus three framing bits fixed by the mint construction below), with nothing else marking the boundaries. We write $|s|$ for the length in bits of a bit string s. Read as a binary fraction, a coordinate names a dyadic interval in $(0, 1)$; codeword concatenation becomes interval nesting, and the interval is the $\alpha(u)$ of Theorem 20.5. Bit string and interval are two views of one object; we use whichever is clearer.

What the coding is for. Two jobs, only one of which is about correctness. *Job 1: cost.* The datatype layer already forced sort keys to retain dead ancestors' timestamps (the conservation law, §19); the only remaining question is how many bits that retention costs. Without entropy coding, "tombstone-free" would be bookkeeping: a full timestamp per level grows with document age however calm the history. The *unary control* (`embed-tree`: the identical datatype with a unary code in the mint, codeword length linear in the value, kept as the experiment's cost-control arm; step 2 and the entropy-law table) makes the gap measurable: 61 KB against the coded design's 0.5 KB on the table's 1000-element chain scenario. The coding is what makes the headline claim true: the dead survive as the entropy of the anchor gaps they were born across, and not more. *Job 2: faithfulness.* The chains must be stored so that plain lexicographic bit comparison reproduces chain-lex exactly, in every reachable state, forever. This is the single obligation the representation owes §19 (Theorem 20.5). Correctness lives upstairs; this layer is an implementation that must not lie. Job 2 needs no entropy optimality at all: it needs exactly two structural properties of the per-level code, stated next.

The contract. The stored form must support four operations: *mint* (insert: produce the new codeword from the two timestamps alone, coordination-free, reading no siblings); *compare* (read: plain lexicographic comparison, with no tie rule); *fold* (delete: splice a dead node out by concatenating its block into each child's stored string, exactly); and *refold* (merge: recompute survivors' coordinates by the same concatenation). Each guarantee is bought by a specific property of C:

- **Monotone** ($\delta < \delta' \Rightarrow C(\delta) <_{\text{lex}} C(\delta')$) buys correct *compare*. A chain-lex first difference always compares two same-anchor entries, where timestamp order and δ order coincide (step 1); bitwise comparison of coordinates therefore equals timestamp comparison at the first differing level precisely when C is monotone.

- **Prefix-free** (no codeword is a prefix of another) buys three things at once. *Alignment*: in a comparison of two coordinates, the first differing bit falls inside the first differing level's codewords, so levels cannot be confused across boundaries and no delimiters are needed; this is also what keeps *fold* and *refold* exact splices. *Geometry*: by the Kraft correspondence, prefix-free codewords occupy pairwise-disjoint dyadic cells, which is Theorem 20.5(i) and the reason a concurrent front-insert can never land inside a dead sibling's span (Figure 9). *No ties*: cell disjointness plus Lamport uniqueness means sibling coordinates never tie, so the merge never decides an order and the read needs no tiebreak.

- **Universal, with a cheap floor** buys Job 1: C is defined for every $\delta \geq 1$, $|C(1)| = O(1)$ so continuation typing costs a constant per level, and $|C(\delta)| = \Theta(\log \delta)$ so a level pays the

log of its anchor gap, whether the gap is a race or a cursor jump (step 1), and nothing else.

Monotone plus prefix-free is all of Job 2: any such code satisfies Theorem 20.5, which is why the Lean capstone is parametric in the code (§30) and why the unary control proves the same theorems at absurd cost. Entropy coding enters only in *which* monotone prefix-free code is chosen; the choice sets the constant in the bits-per-level bill and nothing in the correctness story. One requirement cuts across all four operations: the arithmetic is exact, never rounded (see the binary64 refutation below).

The contract's two structural properties are one Lean structure, and the coordinate is one recursion over it; these two displays are what every theorem of this part is parametric in.

Definition 20.1 (ordered prefix code)**.** *An ordered prefix code is* $\Gamma = (\mathrm{enc} : \mathbb{N} \to \{0,1\}^*)$ *satisfying, for all* d, e*:*

(m) monotone: $1 \le d$ *and* $d < e$ *imply* $\mathrm{enc}(d) <_{\mathrm{lex}} \mathrm{enc}(e)$ *(strict first-difference order on bit strings,* `0 < 1`*);*

(p) prefix-free: $1 \le d$*,* $1 \le e$*, and* $d \neq e$ *imply* $\mathrm{enc}(d)$ *is not a prefix of* $\mathrm{enc}(e)$*.*

Both laws are conditioned on positivity: enc *is only ever consumed at* $\delta \ge 1$ *(causality). The contract's third bullet (universal, with a cheap floor) is deliberately not a field: it is the cost requirement, selecting among inhabitants and entering no proof.* `OrderedPrefixCode`

Definition 20.2 (coordinate and mint, stored form)**.** *The mechanization's coordinate is the bare codeword concatenation along the delta chain, on* `List Bool`*:*

$$\mathrm{coordOf}_\Gamma(\langle\rangle) = \varepsilon, \qquad \mathrm{coordOf}_\Gamma(\langle d\rangle \cdot ch) = \Gamma.\mathrm{enc}(d) \cdot \mathrm{coordOf}_\Gamma(ch),$$

a homomorphism from chain concatenation to string concatenation (`coordOf_append`*). The mint of an insert* $(t, \mathsf{ins}(x \leftarrow a))$ *carrying prefix* π *appends one codeword:* $\mathrm{mint}_\Gamma(\pi, t, a) = \pi \cdot \Gamma.\mathrm{enc}(t - a)$ *(*`mint`*; instance form* `eCoord`*). The three framing bits of the interval realization below (the leading* 1 *and the quarter-selecting* `01`*) buy positivity and subtree interiority of the dyadic cells alone and are dropped in the stored form (the carrier paragraph below); nothing in the maintenance or faithfulness story changes.* `coordOf`*,* `mint`

Theorem 20.3 (unique decodability)**.** *For positive chains* ch_1, ch_2*:* $\mathrm{coordOf}_\Gamma(ch_1) = \mathrm{coordOf}_\Gamma(ch_2)$ *implies* $ch_1 = ch_2$*. Prefix-free concatenations decode uniquely, so distinct birth chains mint distinct coordinates and coordinates are honest names.* `coordOf_inj`

Step 1: differential coding. By causality $\delta = x - a \ge 1$. A chain-lex first difference always compares two entries with the *same* anchor (the shared prefix), and for a fixed anchor, ordering by x and by δ coincide, so each chain level may store the differential instead of the timestamp. Call δ the *anchor gap*: the events elapsed between the anchor's mint and the insert's. Continuation typing (each character anchored at its predecessor) has $\delta = 1$ at every level regardless of how long the document has lived. Two things widen the gap, and they pay the same bill: a genuine race with timestamp gap g costs $\log_2 g$ bits, and so does a *cursor jump*, the first character typed at an old anchor; its successors anchor at fresh characters and are back to $\delta = 1$. The differential is an encoding choice, not semantics: the mathematical object remains the timestamp.

Step 2: entropy coding. For $\delta \geq 1$ let L be the bit-length of δ and

$$C(\delta) \;=\; \underbrace{1\cdots 1}_{L-1}\,0\,\big(\delta \text{ with its leading bit removed}\big) \qquad (|C(\delta)| = 2L-1),$$

an order-preserving prefix-free code: exactly the contract. The length accounting: prefix-freedom makes each codeword self-delimiting, and a self-delimiting code over unbounded δ must spend bits twice, once on the value and once announcing its own length. C pays each bill at $\sim \log_2 \delta$. The header $1^{L-1}0$ is a unary announcement "$L{-}1$ payload bits follow" (L bits); the payload is δ's trailing $L{-}1$ bits, since the leading bit of an L-bit number is always 1: once the header has said L it carries no information and is dropped. Hence

$$|C(\delta)| \;=\; \underbrace{(L{-}1)+1}_{\text{header}} \;+\; \underbrace{(L{-}1)}_{\text{payload}} \;=\; 2L-1 \;=\; 2\lfloor \log_2 \delta \rfloor + 1,$$

e.g. $C(1) = 0$, $C(2) = 100$, $C(3) = 101$, $C(5) = 11001$. This is Elias γ with its header flipped: γ announces the length in *zeros*, which is prefix-free but order-reversing across length classes: at the first divergence the longer, hence larger, number shows a header 0 against the shorter's leading payload bit 1. Writing the header in ones makes that same position decide in favor of the longer code, so lexicographic order on codewords is numeric order on δ. The γ-flip is the building block, not the last word: only the *value* bill is forced at $\log_2 \delta$, and the *length* bill can itself be entropy-coded. Applying C to the length field (then the payload) gives the order-preserving flip of **Elias** δ, prefix-free and monotone by the same across-length-class argument, at $\log_2 \delta + O(\log \log \delta)$ per level; *this is the canonical mint. Any* order-preserving prefix-free code works here, and the artifact exists in three proved encodings of one datatype (`embed-tree` unary, `embed-code` the γ-flip, `eliasDeltaCode` the δ-flip); the capstone theorem is inherited at each verbatim, with zero new proof content (§30): the code-parametricity claim, machine-validated. Two honest footnotes: per level the δ-flip wins only from $\delta \geq 32$ and loses at $\delta \in \{2, 3\} \cup [8, 15]$ (kernel-checked length comparisons), and measured editing tails are thin, so on real traces the two flips are a wash; the dominance shows in race-heavy shapes (the entropy-law table below) and in the asymptotics.

Proposition 20.4 (three proved instances)**.** *Three inhabitants of Definition 20.1 are proved and packaged; every parametric theorem of this part is inherited at each verbatim.*

- *Unary (`unaryCode`):* $\mathrm{enc}(d) = 1^d\,0$*; monotonicity and prefix-freedom by two short inductions (`unaryEnc_mono`, `unaryEnc_not_prefix`);* $\Theta(d)$ *bits per level, the cost-control arm.*
- *The γ-flip (`binaryCode`):* $\mathrm{enc}(d) = 1^{L-1}\,0$ *(d minus its leading bit) with L the bit-length of d;* $|\mathrm{enc}(d)| = 2L-1$ *at $1 \leq d$ (`binEnc_length`); monotone at $1 \leq d$, $d < e$ (`binEnc_mono`) and prefix-free at $1 \leq d$, $1 \leq e$, $d \neq e$ (`binEnc_prefixFree`).*
- *The δ-flip (`eliasDeltaCode`):* $\mathrm{enc}(d) = \mathrm{binEnc}(L) \cdot$ *(d minus its leading bit), the length field itself γ-flip-coded;* $|\mathrm{enc}(d)| = L + 2\,\mathrm{len}(L) - 2$ *at $1 \leq d$ (`dEnc_length`); `dEnc_mono`, `dEnc_prefixFree`, the same hypothesis shapes. The per-level crossover of the honest footnote is kernel spot-checked at $\delta \in \{2, 8, 16, 32, 1024\}$ in the same file.*

The γ- and δ-flips are cross-validated against the Python codes by kernel `decide` on concrete codewords (4 and 10 codeword pins respectively; the unary code has no pins, its enc *being definitional); per code, this is Theorem 20.5(i).* `unaryCode`, `binaryCode`, `eliasDeltaCode`

The mint. Writing $b = 1 \cdot C(\delta)$ (a leading 1 keeps coordinates positive) and $w = 2^{-|b|-2}$, the mint is the second quarter of b's dyadic cell:

$$M(\delta) \;=\; \big(\,0.b + w,\;\; 0.b + 2w\,\big) \;\subset\; \big[\,0.b,\; 0.b + 4w\,\big).$$

The quarter placement leaves a gap below and headroom above inside the cell, and the open space $(0.b + 2w,\, 0.b + 4w)$ is never minted: it exists so that the interval has room to *contain* the node's own subtree without touching the next cell. This also pins down the per-level *block* promised in the vocabulary: $0.b + w$ written as a string is $b\,\texttt{01}$, so the block is $1\,C(\delta)\,\texttt{01}$, the codeword in its three framing bits (the leading 1 for positivity, the `01` that selects the second quarter). The block is what a node stores; read as a binary fraction it is the lower endpoint of $M(\delta)$, and flipping the trailing `01` to `10` gives the upper. Worked out for small deltas:

δ	$C(\delta)$	block $= 1\,C(\delta)\,\texttt{01}$	$M(\delta)$	bits/level
1	`0`	`1001`	(9/16, 10/16)	4
2	`100`	`110001`	(49/64, 50/64)	6
3	`101`	`110101`	(53/64, 54/64)	6
4	`11000`	`11100001`	(225/256, 226/256)	8
5	`11001`	`11100101`	(229/256, 230/256)	8
10	`1110010`	`1111001001`	(969/1024, 970/1024)	10
100	`1111110100100`	`1111111010010001`	(65169/65536, 65170/65536)	16

Three things to read off. The endpoints are consecutive dyadics at scale $2^{-(|b|+2)}$ (numerators n and $n{+}1$): the mint is one tick wide at its own scale, with the tick below it (the gap) and the two ticks above it (the subtree headroom) never minted. The rows are strictly increasing as fractions: monotonicity made visible. And the bit count is $2L + 2$ for an L-bit δ: four bits at $\delta = 1$ (the sequential-typing constant of the entropy law), sixteen at $\delta = 100$.

Figure 8: Mints $M(\delta)$ for $\delta = 1..4$ inside the parent's unit interval (cell boundaries dotted). Larger deltas sit strictly higher; distinct deltas are disjoint with gaps; each mint is a quarter of its cell, leaving room for the node's own subtree. Drawn for the γ-flip C; the construction reads verbatim over any order-preserving prefix-free code, in particular the canonical Elias-δ mint (step 2).

The stored state. The carrier stores each live node parent-relatively:

$$\sigma \;:\; t \;\mapsto\; (\,\mathrm{par} \in \mathbb{N},\;\; s \in \{0,1\}^*,\;\; \mathrm{char}\,),$$

where s is a block ($1\,C(\delta)\,\texttt{01}$ at birth; after folds, below, a concatenation of blocks) *relative to the parent*, and par $= 0$ denotes the root sentinel (which owns $(0, 1)$ and is never stored). The record holds no interval; the interval is the string's decoding,

$$(lo, hi) \;=\; \big(\,0.s,\;\; 0.s + 2^{-|s|}\,\big) \;\subset\; (0, 1),$$

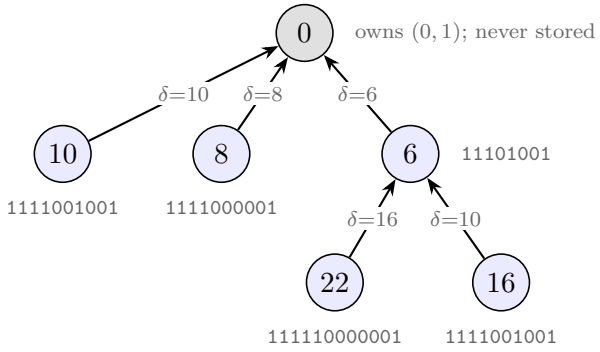
consecutive dyadics at s's own scale, one block at a time exactly as in the mint table. The maintenance below and Theorem 20.5 are stated on the intervals, and every operation they perform on an interval is a string operation on s: composition is concatenation, comparison is lexicographic. (The validation model stores the decoded pair directly, as exact fractions.) A node's *absolute* interval is the affine composition of the intervals along its parent chain. For a node v with mint $M(\delta_v) = (lo_v,\ lo_v + w_v)$, let

$$\varphi_v : (0,1) \to M(\delta_v), \qquad \varphi_v(x) \ = \ lo_v + w_v \cdot x,$$

the unique increasing affine bijection of the unit interval onto v's mint; it maps a subinterval (a, b) to $(lo_v + w_v a,\ lo_v + w_v b)$, and on strings it prefixes v's block: $\varphi_v(0.y) = 0.(s_v\, y)$. Write

$$\alpha(u) \ = \ \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}\big(M(\delta_u)\big)$$

for the composition along u's birth chain $c_1 \cdots c_k\, u$ (δ_v is v's delta to its birth parent): $\alpha(u)$ is the *encoding of* chain(u), and at the string level it decodes the concatenation of the blocks along the chain. Worked on the tree of Figure 7, redrawn before del(6) with the delta (child minus anchor) on each edge and the stored block under each node:



Writing $(n, n{+}1)/2^k$ for the interval $(n/2^k,\ (n{+}1)/2^k)$ and $\approx$ for its lower endpoint:

node	δ	stores (par, s)	decode of s	α (absolute)	$\approx$
10	10	(0, 1111001001)	$(969, 970)/2^{10}$	$(969, 970)/2^{10}$	0.94629
8	8	(0, 1111000001)	$(961, 962)/2^{10}$	$(961, 962)/2^{10}$	0.93848
6	6	(0, 11101001)	$(233, 234)/2^{8}$	$(233, 234)/2^{8}$	0.91016
22	16	(6, 111110000001)	$(3969, 3970)/2^{12}$	$(958337, 958338)/2^{20}$	0.91394
16	10	(6, 1111001001)	$(969, 970)/2^{10}$	$(239561, 239562)/2^{18}$	0.91385

The root's children store their absolute intervals already (par = 0). The grandchildren compose: 22's absolute string is its parent's block followed by its own, 11101001 111110000001 (20 bits, hence the 2^{20}), and likewise 16's (18 bits). Both absolute intervals sit inside $M(6) = (0.91016, 0.91406)$ while 8's sits above it, so the descending-α read is $[10, 8, 22, 16]$ and 8 cannot land between the orphans. Nodes 16 and 10 store the same string, both being $\delta = 10$ births; records are parent-relative, so this is no collision. After del(6) the fold rewrites the two records to (par 0, the concatenated s) and the α column is unchanged bit for bit. Figure 9 draws these intervals (widths schematic).

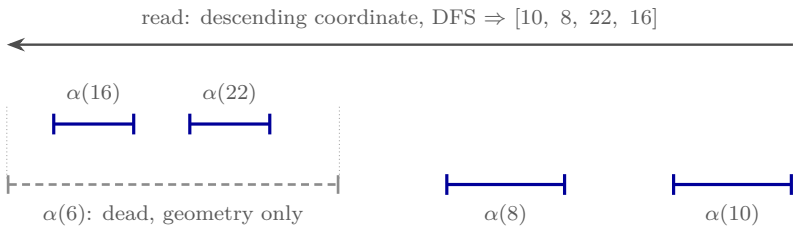


Figure 9: Figure 7 downstairs (widths schematic; true mints are the dyadic cells of Figure 8). Deleting 6 removes its record, but $\alpha(16)$ and $\alpha(22)$ are compositions *through* $M(6)$: the dead node survives as the dashed container, the tombstone compressed into geometry. 8's interval can never land inside $\alpha(6)$'s span: distinct deltas occupy disjoint cells (Theorem 20.5(i)), which is the credential verdict, now a fact about intervals.

Maintenance. How the stored form tracks the datatype:

- $\mathsf{ins}(x \leftarrow a)$ mints $M(x - a)$ relative to a: the new record depends only on the two timestamps.
- $\mathsf{del}(d)$ *isometrically folds* d into its children: each child c is re-parented to d's parent and d's record is substituted into c's. On the stored strings this is concatenation, $c.s := d.s\, c.s$; read as intervals, $c.\text{frac} := d.\text{frac} \circ c.\text{frac}$, where $I \circ J$ is the image of J under the increasing affine bijection of $(0, 1)$ onto I (the φ of the stored state, with I in place of a mint): for $I = (lo,\ lo + w)$,

  $$I \circ J \ = \ (\, lo + w \cdot lo_J, \quad lo + w \cdot hi_J \,).$$

  The substitution is exact. Upstairs, deletion changes nobody's sort key (§19); the fold is the parent-relative storage scheme *re-normalizing* while every absolute interval stays fixed (Figure 10): representation maintenance, not datatype semantics. On the running example, $\mathsf{del}(6)$ removes 6's record and substitutes it into both children: 22's record $(6,\ \texttt{111110000001})$ becomes $(0,\ \texttt{11101001 111110000001})$, whose decode is

  $$\begin{aligned}(233, 234)/2^8 \circ (3969, 3970)/2^{12} &= (233 \cdot 2^{12} + 3969,\ 233 \cdot 2^{12} + 3970)/2^{20} \\ &= (958337, 958338)/2^{20},\end{aligned}$$

  and likewise 16's becomes $(0,\ \texttt{11101001 1111001001})$, decoding $(239561, 239562)/2^{18}$. Both are exactly the α column of the worked table: the absolute intervals did not move, only the parent-relative bookkeeping did.
- The merge *refolds*: for each survivor, compute its absolute interval by folding its record to the root inside the input that holds it (a record's stored parent is always live in its own input, so the chain stays in one input; by Theorem 20.5(iii) the choice of input cannot matter, asserted at runtime and never fired); re-parent to the nearest *surviving* ancestor; re-relativize.
- The read is depth-first from the root, children in descending order of lo. No tie rule is needed (Corollary 20.6).

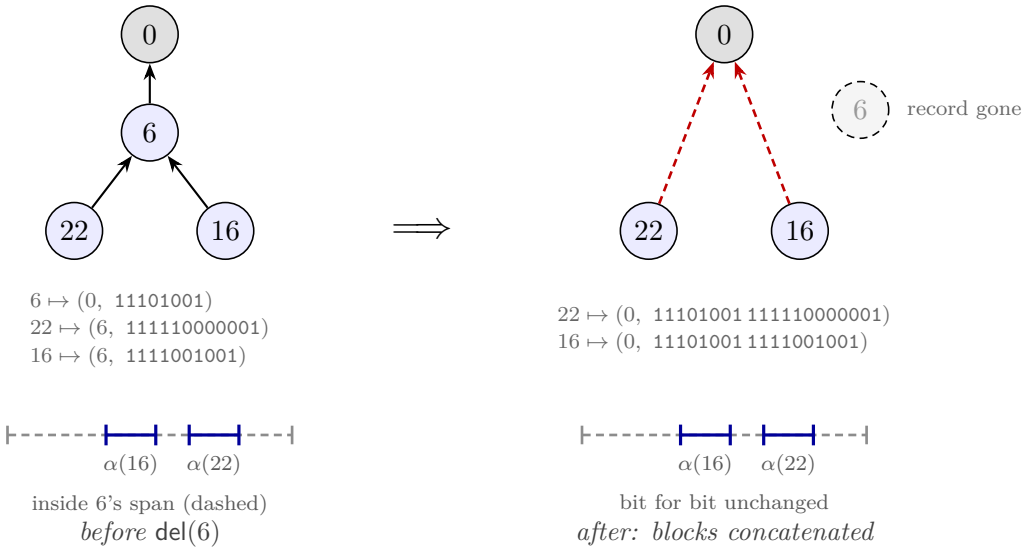


Figure 10: The isometric fold on the running example. del(6) deletes 6's record and concatenates its block `11101001` onto the front of each child's, re-parenting both to the root (red: the re-parenting, a storage move only). The stored states change: 22 and 16 now carry two-block strings and par 0. The absolute intervals $\alpha(22) = (958337, 958338)/2^{20}$ and $\alpha(16) = (239561, 239562)/2^{18}$, and with them the display, are bit-for-bit unchanged: upstairs, the sort keys still read $\langle 6, 22\rangle$ and $\langle 6, 16\rangle$. The dashed span is 6's old interval, surviving as pure geometry.

Why the codeword must announce its own end. Before the theorem, the countermodel that motivates it. A single codeword in a field of known extent needs no header, and the length looks recoverable from the string itself; plain binary shows what breaks, once per purchase in the contract. *Geometry:* $B(2) =$ `10` prefixes $B(5) =$ `101`, so $B(5)$'s cell sits inside $B(2)$'s; containment means ancestry, so the δ=5 sibling lands amid the δ=2 sibling's descendants and non-interleaving dies with everything alive. *Parsing:* once a fold erases the structural boundary between blocks, `101100…` reads as $B(2)$ then `11…` or as $B(5)$ then `00…`, and nothing outside the string can say. *Ties:* a binary fraction cannot see trailing zeros, so `10` and `100` both denote $\frac{1}{2}$; distinct siblings mint touching intervals, and the id tiebreak wakes into an arbiter that orders by id, not chain-lex. Nor can the header bits be dodged: prefix-freedom demands $\sum_\delta 2^{-\ell(\delta)} \le 1$, and plain binary puts 2^{L-1} values at every length L, so every prefix-free code pays a length announcement of order $\log L$ somewhere (out-of-band boundary tables cost the same bits and forfeit the string as a self-contained key; database key encodings inline it for the same reason). And the fold names where the boundary went: the tree node that carried it structurally is the deleted tombstone, whose ordering verdict becomes the payload and whose boundary becomes the header. "Tombstone compressed into coordinate entropy" is literal at the bit level.

The one obligation. The four theorems of the earlier drafts collapse into a single statement about the encoding.

Theorem 20.5 (encoding faithfulness)**.** *(i) Order-embedding at one level: for $\delta \neq \delta'$, $M(\delta)$ and $M(\delta')$ are disjoint with a gap; $\delta < \delta' \Rightarrow M(\delta)$ lies entirely below $M(\delta')$.*

(ii) Homomorphism: α realizes chain concatenation as affine composition and embeds chain-lex in the interval order: for distinct u, v, if $\mathrm{chain}(u)$ *is a proper prefix of* $\mathrm{chain}(v)$ *then $\alpha(v) \subset \alpha(u)$ (and, if both are live, the two are never read-time siblings); otherwise $\alpha(u) \cap \alpha(v) = \varnothing$, with the chain-lex-greater element's interval strictly higher.*

(iii) Exact maintenance: in every reachable state, every live node's stored records compose to exactly $\alpha(u)$.

(Paper-level statement, proved below; its mechanized counterparts are the code laws of Definition 20.1 with Theorem 20.3, and the stored form is Theorem 20.7.)

Proof. (i) C is prefix-free: codes of lengths $2L-1 < 2M-1$ differ at position L (the shorter's header terminator 0 against the longer's header 1); equal-length codes differ in the payload. Prefix-free codes have pairwise-disjoint dyadic cells. C is monotone as a binary fraction: within a length class the payload is δ's low bits in order; across classes the position-L bit decides in favor of the longer code. Disjoint cells ordered by their codes, and mints interior quarters of their cells, give both claims. (ii) If neither chain is a prefix of the other, they first differ on distinct deltas relative to the *same* birth ancestor (§20, step 1); (i) separates them there, and affine images of disjoint intervals are disjoint and order-preserved (each φ is increasing). If $\mathrm{chain}(u)$ prefixes $\mathrm{chain}(v)$, then $\alpha(v) \subseteq \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}(\varphi_u((0,1))) = \alpha(u)$ by construction; and if both are live, v's nearest live ancestor is u or deeper, so they never share a current parent, and sibling groups are pairwise disjoint. (iii) By induction over transitions. ins establishes it by definition. del's fold replaces $(\mathrm{par}, \mathrm{frac})$ pairs by their exact composition, leaving α unchanged. The merge recomputes precisely $\alpha(u)$ (folding a record chain to the root) and re-relativizes against another α; ties cannot exist by (i) and (ii), so no step of any transition ever rewrites an interval by anything but exact composition. □

Corollary 20.6 (display ≡ chain-lex; no tie rule)**.** *The read (DFS by descending lo) enumerates survivors in exactly chain-lex order: per level, larger delta first, so among same-parent siblings, exactly newest first; survivors rank by their own timestamps, folded heirs by their dead founder's. Sibling lo-values never tie; the read's id tiebreak is dead code. Machine checks: pairwise sibling disjointness asserted at every read of* 320 *randomized executions, all clean; over* 19,531 *multi-member sibling groups, own-timestamp order violated* 3,481 *times (exactly the post-fold heir groups; forcing own-ts there is the refuted dissolution behavior),* ***chain-lex violated*** 0 ***times****. (Paper-level corollary of Theorem 20.5; the mechanized form of the display-order claim is Theorem 20.7.)*

Theorem 20.7 (the chain-lex theorem, stored form)**.** *For distinct positive chains ch_1, ch_2:*

$$\mathrm{keyLt}\big(\mathrm{key}(\mathrm{coordOf}_\Gamma(ch_2)),\ \mathrm{key}(\mathrm{coordOf}_\Gamma(ch_1))\big) = \mathsf{true} \iff \mathrm{chainBefore}\ ch_1\ ch_2 :$$

the display comparator on stored coordinates (keys descending) computes exactly the chain order of Definition 19.1, for every ordered prefix code. This is Corollary 20.6 as one flat comparison of immutable bit strings, the form the read executes. `display_iff_chainBefore`

The entropy law. A coordinate is the concatenation of the codes along the birth chain: $\Theta(\sum_i \log \delta_i)$ bits, the entropy of the chain. Continuation typing ($\delta = 1$ at every level) costs $\sim$4 bits per level under every code here; a level minted across an event gap g, by a race or by the first character after a cursor jump, costs $\log_2 g + O(\log \log g)$ bits under the canonical Elias-δ mint ($2 \log_2 g + 1$ under the γ-flip). *The state pays for the magnitude of edit non-locality, concurrent*

or not, and for nothing else. Measured (1000 elements; the numbers are *order metadata alone*, survivors' coordinate bits read off the dyadic denominators; characters are excluded, since data costs are identical across all designs):

scenario	quarter carve (refuted)	unary mint	γ-flip mint	**Elias-δ mint**
chain then delete-all (δ=1)	2^{2001}	2^{502501} ($\sim$61 KB)	2^{4001} ($\sim$0.5 KB)	$\mathbf{2^{4001}}$ ($\sim$0.5 KB)
1000 root siblings (δ=t)	— (ties)	max 2^{1003}, 125 KB	max 2^{23}, 5 KB	**max 2^{20}, 4 KB**

The carrier. The natural representation is a **parent-relative bit string per node**: the fold is concatenation, comparison is lexicographic ($O(1)$-ish with structure sharing); the rationals are the semantics of the strings, kept for the proofs. Nothing here computes $M(\delta)$: the endpoints $0.b + w$ and $0.b + 2w$ are the strings $b\,\texttt{01}$ and $b\,\texttt{10}$, so the mint emits a block and the fold concatenates, and *hi* need not be stored at all. The Lean mechanization drops even the framing bits: its coordinate is the bare codeword concatenation on `List Bool` (`mint` appends $\Gamma.\mathrm{enc}(t-a)$ to the anchor's coordinate), with the prefix rule stated directly on strings; the quarter framing serves the interval realization alone, buying positivity and subtree interiority. The rational $M(\delta)$ is computed in exactly one place, the validation model `embed_tree.py`, as exact fractions; the entropy tables read their bit counts off those denominators. IEEE 754 binary64 is machine-refuted as the carrier, a faithfulness failure rather than a design failure: mints degenerate at $t = 52$, depths underflow at 44, and rounding voids Theorem 20.5(iii) (6/120 PBT executions die). The obstruction is a pigeonhole (chains carry $\Theta(\sum \log \delta)$ bits; a fixed-width word holds 64), but binary64 is sound as a *comparison cache* with exact fallback on float equality.

Timestamps in practice. The unique stamps in $\mathbb{N}^+$ are Lamport's 1978 construction, flattened: each replica ticks a counter per local operation, advances it to the max on merge, and breaks ties by replica id, packed as $t \cdot 2^k + r$ with k id bits. Uniqueness and causality ($t_a < t_x$ gives $\mathrm{packed}_a < \mathrm{packed}_x$ regardless of ids, so $\delta \geq 1$) both survive the packing; the proof layer only *assumes* uniqueness (the Lamport-structural part of honesty), and this construction is the witness that the assumption is realizable. Two entropy caveats. *(i)* Packing inflates every delta by 2^k, costing $\sim 2k$ extra bits per chain level, which turns the 0.5 KB sequential run above into $\sim$3 KB at $k = 10$. Cheaper: extend the mint's code to the pair, $C(\Delta t)$ followed by a k-bit id. The pair code is still order-preserving prefix-free (lex on $(\Delta t, r)$), and costs k bits only where the level is minted. A further option, unvalidated against the battery and recorded as a design note, is to order sibling ids *relative to the anchor's* (any deterministic per-anchor id order is a legitimate tiebreak; convergence needs agreement, not a uniform order), making "same author as my anchor", i.e. sequential typing, cost $\sim$1 bit, so only genuine cross-author races pay $\log R$. *(ii)* The clock must be *dense logical time, never physical*: wall-clock or HLC stamps make δ measure elapsed time rather than elapsed events: a keystroke every 100 ms over μs-resolution stamps mints $\delta \approx 10^5$, $\sim$34 bits per level for a purely sequential edit. The entropy law holds precisely because the counter ticks once per event, making δ an event-count gap; if wall-clock order is ever wanted, it belongs in a display-layer tiebreak, never in the minted coordinate.

What the storage layer now delivers. Dead chain entries concatenated into survivors' strings are the conservation law's irreducible content (§19); at the time of the original design note, reclaiming them under *causal stability* (renumber once every replica has seen the delete,

the only condition under which rewriting geometry is safe, since no verdict involving the dead can be contested again) was the deepest of the open cost items. That item is now closed, and the answer occupies its own section (§26): a verified re-coding theorem cluster at settled cuts (rank renumbering and spine fusion, gated by §18's SettledAt), and, one level further down, the run table, a lossless re-representation of the live chains that collapses the per-record cost by one to two orders of magnitude with *no* stability gate at all. The residual headroom (per-anchor context codes, Steiner-sharing dead prefixes across co-heirs, the Elias-δ constant of step 2) remains catalogued in the companion assessment `whiteboard/order-coding-compression-notes.md`.

21 The insert prefix: for the proof, and the proof alone

The insert payload π exists for verification, not execution. This section says why it must exist, and why an implementation may drop it from the wire.

Why rc must be Either. The verification framework (Part I) resolves a concurrent pair of non-commuting operations by consulting a declared conflict-resolution order rc; any *oriented* entry for an insert/delete pair, however, incurs a base commutation obligation that is machine-refuted for this design space: ordering $\mathsf{ins}(x \leftarrow a)$ against $\mathsf{del}(a)$ (in either direction) is undischargeable as a conflict entry, because rehoming is non-transitive across chains of deletions. So rc = Either: the proof must show the two *commute*, in both orders, on all applicable states.

The commutation, and where the prefix earns its keep. Order 1, $\mathsf{ins}(x \leftarrow a)$ then $\mathsf{del}(a)$: x is born with chain $\pi \cdot x$, and the delete touches no chain (downstairs: the fold re-homes x's record with composed coordinates; by exactness $\alpha(x) = \alpha(a) \circ M(x - a)$). Order 2, $\mathsf{del}(a)$ then $\mathsf{ins}(x \leftarrow a)$: a is gone; application must still be *total* and land x on the same sort key. This is what the carried prefix $\pi = \mathrm{chain}(a)$ (equivalently, the anchor's coordinate) provides: apply sets $\mathrm{chain}(x) = \pi \cdot x$ (downstairs: $\alpha(x) = \pi \circ M(x - a)$, attached to the nearest live ancestor). The two orders produce *equal* states: commutation to equality, not merely up to $\approx$, courtesy of canonicity (Theorem 19.3). Figure 11 draws the square on the smallest instance.

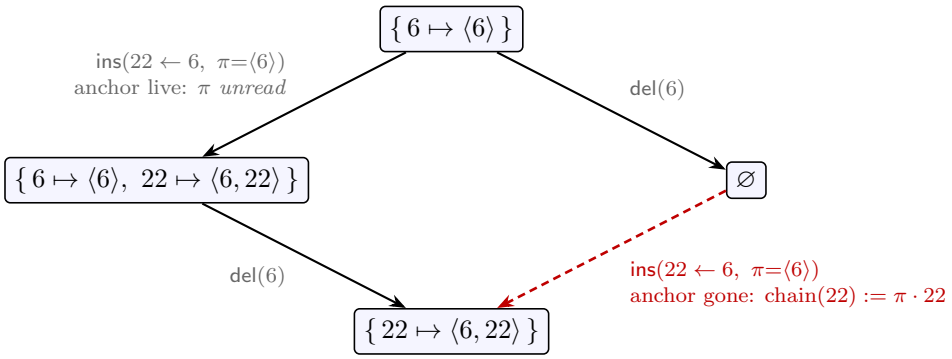


Figure 11: rc = Either discharged: $\mathsf{ins}(22 \leftarrow 6)$ and $\mathsf{del}(6)$ commute *to equality*. Down the left, the anchor is live and application never consults π (the reachable, honest order). Down the right, 6 is already gone: the state cannot supply the anchor's chain, so the carried $\pi = \mathrm{chain}(6)$ does, landing 22 on the same sort key. The right-hand state is unreachable under honesty; the verification condition quantifies over it anyway, and π exists exactly to make that corner total.

The commutation kernel is stated with its guards displayed in full: this is a *conditioned* commutation, and per the framework's standing post-mortem, a commutation whose quantifier was edited is exactly where a verification effort can fool itself, so the guard list is the content.

Definition 21.1 (conditioned commutation, embed form)**.** *For ops* o_1, o_2, $\mathrm{commutesWith}'_\Gamma(o_1, o_2)$ *demands, for every state* s *satisfying all five guards:*

- 0 *is not a stored key of* s*;*
- o_1 *and* o_2 *are each accurate at* s *(`accurate`: an insert's anchor precedes it,* $a < t$*, and the carried prefix is the anchor's stored coordinate, or the anchor is the root and the prefix empty; a delete's target is live);*
- o_1 *and* o_2 *each have fresh timestamps (`fresh_ts`: an insert's* t *is nonzero and unstored; vacuous for deletes),*

that $o_2(o_1(s)) \equiv o_1(o_2(s))$*, where* $\equiv$ *is extensional map equality (equal domains, equal records: the carrier's literal equality, not a quotient).* `commutes_with'`

Theorem 21.2 (rc = Either is honest)**.** *For all* o_1, o_2 *with distinct timestamps and distinct origin replicas:*

$$\mathsf{rc}(o_1, o_2) = \mathit{Either} \iff \mathrm{commutesWith}'_\Gamma(o_1, o_2).$$

Since rc *is Either everywhere, the content is the forward direction: every distinct cross-replica pair commutes on guarded states, to equality. The one clause with teeth is that a delete's live target is never a fresh insert's timestamp (accuracy plus freshness), ruling out the causally impossible insert-then-delete-it shape.* `rc_non_comm'` *(`Sal/MRDTs/RGA_Embed/RGA_Embed_MRDT.lean`)*

Ghost, in the strict sense. On every reachable state the anchor is live at application (honesty), and the application reads only the state and the two timestamps: π is never consulted. The Lean form is exact:

Theorem 21.3 (the prefix is ghost)**.** *If* 0 *is not a stored key of* s *and* o *is accurate at* s*, then* $\mathsf{do}_\Gamma(s, o) = \mathsf{do}^{\mathrm{run}}_\Gamma(s, o)$*: the always-*$\pi$ *transition and the state-reading transition (`do_run`, which mints from* $\mathrm{pos}_s(a)$*, or from* ε *at the root) are literally the same function on accurate ops.* `ins_prefix_ghost`

An implementation may therefore drop π from the wire entirely; the *proof* may not, because the verification conditions quantify over reorderings that visit unreachable states, and there application without π is partial. The prefix costs $O(\sum \log \delta)$ bits, the same entropy as the coordinate itself, and exists at the proof layer alone.

22 Validation

All numbers reproduce from `whiteboard/litmus/` in the accompanying Sal repository (`embed_tree.py`; harnesses `litmus.py`, `pbt.py`). Notation: $\mathrm{RGA}_\dagger$ is the published tombstoned RGA (Roh et al.; §29); the *rehoming* RGA is the convergence-verified flat tombstone-free design of §19 (since demoted repo-wide: the L1 row below was generalized into the kernel-checked sequential refutation of Part III, §36, and this design took the canonical seat). The L1–L25 battery is the litmus suite accumulated over this project's design sweep; each entry is the named countermodel that killed at least one of 16 prior candidate designs. The one exception below, L19, is discharged by the sided generalization of §23.

check	verdict
litmus battery L1–L25 (incl. every countermodel that killed 16 prior designs: ceiling escapes, tie inheritance, three-branch topology, frame mixing, fold-verdict inheritance)	clean, except one-sided L19 (backward-run interleaving, the published RGA's own anomaly)
credential countermodel (tie heirs vs mid-ts third party, death-before vs death-after topologies)	converges, no flips; $= \mathrm{RGA}_{\dagger}$'s reads
subordination-escape and retroactive-subordination CEs	survive
randomized version-DAG PBT (FLIP / CONV / LIVE / DUP)	120/120 and 300/300 clean
per-read sibling-disjointness assertion sweep	320/320 clean
chain-lex instrumentation (19,531 sibling groups)	0 violations
lockstep vs the published tombstoned RGA	**read-equal at every apply/merge/read, 120/120**
L1 delete-reorder vs the verified rehoming RGA	embed $[2,3]$ ✓ vs rehoming $[3,2]$ ×

The lockstep row is the identification: *the embedded-chain RGA is observationally the published tombstoned RGA with the tombstone set compressed into coordinate entropy*, machine-checked on 120 executions and now a kernel-checked theorem (the compaction theorem, §30): on every honest event set, any enumeration of the tombstoned RGA's visible ids in its own visible order *equals* the embed read, element for element. The L1 row is the separation from the verified rehoming RGA, and it is exactly the one-line difference of §19: that design's climb re-sorts a rehomed node by its own timestamp (the $[b,a,c] \to [c,b]$ anomaly), i.e. its sort key is the birth chain *truncated at each rehoming*; this design keeps the chain whole.

23 The sided generalization: two-sidedness as a parameter

The one blemish in the battery is L19: two replicas that each type a *backward* run at the same place (repeatedly "insert before the current head") interleave, because every insert-before anchors at the same left neighbor and both runs land in one sibling group. The fan is not a traversal artifact: no read order of the same birth tree un-fans a fan, so any fix must change where insert-before *anchors*. The Fugue family's answer is a second side: insert-before anchors at the *right* neighbor as an L-entry, so a backward run is a chain, exactly as a forward run already is.

The L19 example, concretely. It is the battery's own trace (Figure 12). The LCA document is $\langle 1 \rangle$ (one insert, stamp 1, anchored at the start marker $\diamond$). Then, concurrently, replica A types a backward run at the head, three inserts each anchored "before the current first element":

$$\mathsf{ins}(10 \text{ at head}), \quad \mathsf{ins}(30 \text{ at head}), \quad \mathsf{ins}(50 \text{ at head}),$$

so A locally reads $\langle 50, 30, 10, 1 \rangle$; replica B does the same with stamps $21, 41, 61$ and locally reads $\langle 61, 41, 21, 1 \rangle$. (The stamps interleave numerically, $10 < 21 < 30 < 41 < 50 < 61$, because the two replicas' Lamport clocks advance together.) In the one-sided datatype, "insert at the head" can only anchor at the left neighbor, which is $\diamond$ for every one of the six inserts: all six become R-children of $\diamond$, one sibling group, and the sibling tiebreak (newest first) merges the two runs by stamp:

$$\langle\, 61, 50, 41, 30, 21, 10, 1 \,\rangle$$

after the merge, the two sentences shuffled together. No traversal of that tree can do better; the information "these three form one run" is simply not in a sibling fan whose stamps interleave.

The sided datatype, and why it prevents the interleaving. The generalization does not build a second datatype. An insert names an anchor *and a side*; chains become sequences of (side, timestamp delta) entries; and the display rule generalizes from the prefix rule ("a prefix precedes its extensions") to the in-order rule ("L-extensions, then the node, then R-extensions"). One marker trick makes this ordinary lexicographic comparison again: a node's sort key is its coordinate followed by a virtual terminator, and the sided symbol alphabet is ordered

$$(R,1) < (R,2) < \cdots < \mathsf{marker} < \cdots < (L,2) < (L,1),$$

with the L band *mirrored* (among L-siblings the newer sits adjacent to the node). Marker above the R band puts a node before its R-extensions; marker below the L band puts L-extensions before the node. The three displays that carry this section:

Definition 23.1 (the sided chain and its coordinate)**.** *A sided chain is a list of entries* (side, δ)*,* $\text{side} \in \{R, L\}$*, deltas positive (`SEntry`, `SChain`, `PosSChain`). Its stored coordinate concatenates one symbol block per entry (`sBlock`, `sidedCoordOf`):*

$$\text{block}(R,\delta) = \Gamma.\text{enc}(\delta) \text{ in the } R \text{ band}, \qquad \text{block}(L,\delta) = \overline{\Gamma.\text{enc}(\delta)} \text{ in the } L \text{ band},$$

the R band mapping $0 \mapsto 1$*,* $1 \mapsto 2$*, the L band* $0 \mapsto 4$*,* $1 \mapsto 5$*, and* $\bar{\cdot}$ *the bitwise complement (`compl`: the order-reversing twin at identical lengths, prefix-freedom preserved); equivalently, one band-symbol map per side, L pre-composing the complement (`sBlock_eq`, `sideSym`). The sort key appends the marker,* $\text{sKey}(c) = c \cdot \langle 3 \rangle$*, and the bands straddle it:* $\{1,2\} < 3 < \{4,5\}$*. `sidedCoordOf`, `sKey`*

Definition 23.2 (the in-order rule)**.** *Entry divergence at a shared anchor (`sEntryBefore`): among R-siblings the newer first (*(R,d) *before* (R,e) *iff* $e < d$*); among L-siblings the older first (*(L,d) *before* (L,e) *iff* $d < e$*, the mirror); every L entry before every R entry. The display order on sided chains is the least relation closed under three rules:*

$$ch \cdot \langle (L,d) \rangle \cdot rest \prec ch, \qquad ch \prec ch \cdot \langle (R,d) \rangle \cdot rest, \qquad q \cdot \langle e_1 \rangle \cdot c_1 \prec q \cdot \langle e_2 \rangle \cdot c_2 \ \ (e_1 \text{ before } e_2):$$

L-extensions precede the node, the node precedes its R-extensions, divergences by the entry order. Distinct chains are always comparable (`schainBefore_total`). `schainBefore`

Theorem 23.3 (the sided marker theorem)**.** *For distinct positive sided chains* c_1, c_2*:*

$$\text{keyLt}\big(\text{sKey}(\text{sidedCoordOf}_\Gamma(c_2)),\ \text{sKey}(\text{sidedCoordOf}_\Gamma(c_1))\big) = \mathsf{true} \iff \text{schainBefore } c_1\ c_2:$$

two-sidedness is plain lexicographic comparison over the marker alphabet; no bespoke three-way prefix clause survives into the read. Unique decodability holds alongside (`sidedCoordOf_inj`: the side is recoverable from the first symbol's band, the delta by per-band prefix-freedom). `sdisplay_iff_schainBefore`

On the example, the Fugue anchoring rule (the generation-policy paragraph below) sends each "insert at head" to the *successor* as an L-entry, so each run is an L-chain (Figure 12, right). Writing chains as entry sequences with deltas relative to the parent's stamp ($10 = 1 + 9$, $30 = 10 + 20$, …):

$$\begin{aligned} &\text{chain}(1) = (R,1) && \text{chain}(10) = (R,1)\,(L,9) \\ &\text{chain}(30) = (R,1)\,(L,9)\,(L,20) && \text{chain}(21) = (R,1)\,(L,20) \\ &\text{chain}(50) = (R,1)\,(L,9)\,(L,20)\,(L,20) && \text{chain}(41) = (R,1)\,(L,20)\,(L,20) \end{aligned}$$

and the marker-lex comparisons (keys descending in display order) work out as follows, writing M for the marker:

- *L-extensions precede the node*: key(10) $= (R,1)(L,9)\,M$ vs key(1) $= (R,1)\,M$ first differ at position 2, $(L,9)$ vs M, and the L band sits above the marker: so 10 displays before 1. Likewise 30 before 10, 50 before 30: each backward keystroke lands immediately left of the previous one, which is the typed intent.

- *The two runs are decided once, not per pair*: key(10) vs key(21) first differ at position 2, $(L,9)$ vs $(L,20)$, and the L band is mirrored, so $(L,20) < (L,9)$: A's run displays before B's. Now *every* node of A's chain carries the prefix $(R,1)(L,9)$ and every node of B's carries $(R,1)(L,20)$, so *every* cross-pair first-differs at that same position 2 with that same verdict. The blocks cannot interleave:

$$\langle\, 50, 30, 10,\ 61, 41, 21,\ 1 \,\rangle.$$

  This is the datatype-level subtree convexity (Theorem 23.4): *any* policy that keeps a run inside one subtree gets non-interleaving from the datatype, for free.

Theorem 23.4 (sided subtree convexity)**.** *For every prefix p and sided chains c_x, c_y, c_z: if c_x and c_y extend p,* schainBefore c_x c_z*, and* schainBefore c_z c_y*, then c_z extends p. With the empty extension allowed, the node itself is a member: a subtree's display interval is L-descendants, the node, R-descendants, with nothing foreign between. A pure chain lemma: no state, no honesty, no property of the code.* `schain_subtree_convex`

The current datatype is the all-R fragment of this order, *exactly*: with L uninhabited the in-order rule degenerates to the prefix rule, and the existing key of §20 (the sym-mapped bits plus terminator) is literally this construction with the L band empty. That is a theorem, not a slogan, twice: once on chains, once on folds.

Theorem 23.5 (the all-R fragment theorem)**.** *For distinct positive one-sided chains c_1, c_2, with* liftR *tagging every entry R:*

$$\text{schainBefore (liftR } c_1\text{) (liftR } c_2\text{)} \iff \text{chainBefore } c_1\ c_2,$$

via the key identity $\text{sKey}(\text{sidedCoordOf}_\Gamma(\text{liftR } c)) = \text{key}(\text{coordOf}_\Gamma(c))$ *(`sKey_liftR`): with L uninhabited, the in-order rule is the prefix rule, symbol for symbol.* `schainBefore_liftR`

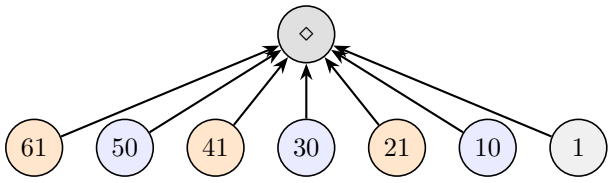
Theorem 23.6 (the fragment at the instance)**.** *For every operation list ρ of the one-sided instance,*

$$\text{sFold}_\Gamma(\text{map liftOp } \rho) = \text{map liftRec (eFold}_\Gamma\ \rho),$$

record for record. Unconditional: no honesty, no sortedness hypotheses, a pure simulation; the reads agree as a corollary (`sided_allR_read_eq`). Chained with the compaction theorem (§30), the all-R sided instance reads as the published tombstoned RGA. `sFold_liftOp`

How the encoding realizes it, for zero extra bits. The one-sided block of §20 spends a constant leading `1` per level whose only job is keeping coordinates nonempty; repurpose it as the side bit. The L band needs an order-*reversing* prefix-free code, and the bit complement $\overline{C}$ is exactly that at identical lengths (flipping every bit flips every first-difference verdict and preserves both lengths and prefix relations), so the block becomes

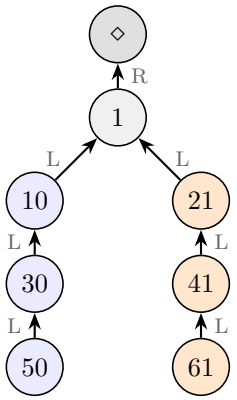
$$\text{block}(\text{side}, \delta)\ =\ \text{sideBit} \,\|\, \big(C(\delta) \text{ if } R,\ \overline{C(\delta)} \text{ if } L\big),$$



every edge R: one sibling group

⟨ 61 50 41 30 21 10 1 ⟩

one-sided: runs interleaved (L19)



⟨ 50 30 10 61 41 21 1 ⟩

sided, Fugue policy: blocks contiguous

Figure 12: L19 under the two anchoring rules, the battery's own trace. LCA document $\langle 1 \rangle$; replica A (blue) types backward $10, 30, 50$ before the head while replica B (orange) concurrently types backward $21, 41, 61$. One-sided (left): every insert-before anchors at the same left neighbor $\diamond$, both runs land in one R-sibling group, and the timestamp-sorted fan interleaves them. Sided under the Fugue policy (right): insert-before anchors at the *successor* as an L-entry, each run is an L-chain hanging off 1, and the in-order read keeps the blocks contiguous: L-extensions precede the node ($50, 30, 10$ before their parents' line ending at 1), and among the L-siblings $10, 21$ of node 1 the older run displays first (the L band is mirrored). Both display strips are outputs of the sided model harness, `embed_sided.py`.

prefix-free per band, with no framing change and no new bits. On the example, with the binary delta code ($C(1) = \texttt{0}$, $C(9) = \texttt{1110001}$, $C(20) = \texttt{111100100}$, so $\overline{C(9)} = \texttt{0001110}$, $\overline{C(20)} = \texttt{000011011}$), the stored coordinates are the concatenated blocks:

node	chain	stored coordinate (side bits marked)
1	$(R, 1)$	1̲0
10	$(R, 1)(L, 9)$	1̲0 0̲0001110
21	$(R, 1)(L, 20)$	1̲0 0̲0000011011
30	$(R, 1)(L, 9)(L, 20)$	1̲0 0̲0001110 0̲0000011011
50	$(R, 1)(L, 9)(L, 20)(L, 20)$	1̲0 0̲0001110 0̲0000011011 0̲0000011011

To compare two coordinates, parse: read a side bit; if `1`, decode one C codeword as an R-band symbol (R, δ); if `0`, decode one $\overline{C}$ codeword (decode C on the flipped bits) as an L-band symbol (L, δ); repeat, then terminate with the marker. Prefix-freedom per band delimits every block, so no length fields or framing are needed, exactly as one-sided. The complement is what makes the mirror free: within the L band, plain descending comparison of the *stored* bits is already the mirrored delta order ($\overline{C(20)} = \texttt{000011011} < \overline{C(9)} = \texttt{0001110}$ bitwise, matching $(L, 20) < (L, 9)$), so the carrier stores entropy payload only and the marker-lex of the previous paragraph is read off the bits. Faithfulness of this parse (unique decodability plus order embedding of the sided alphabet) is machine-checked by the chain-model/code-model lockstep below.

Two honest cost notes. Runs absorb the side bit like every other run-constant (paid once at the head). On *this* trace the sided coordinates are longer than the one-sided fan's (the runs'

deltas are 20 because the two replicas' Lamport clocks interleave); the compression claim is about the common case, continuation-stamped backward typing ($\delta = 1$ per keystroke, an $\texttt{0}\|\overline{C(1)} = \texttt{01}$ block per level), where today's design pays a growing-δ fan. Two-sidedness buys the semantics always and the compression exactly when backward runs are locally stamped.

Side selection is a generation policy, not a datatype choice. If insert merely "takes a side," convergence and RA-linearizability hold for any sides whatsoever: those proofs never consult the order's intent. What depends on *which* side is picked is the ordering intent. Fugue's non-interleaving needs the rule "right child of the left neighbor when it has no right children, else left child of the successor," and that rule lives at generation time, where the framework already has a slot: honesty, the same layer that polices the mint. The architecture is therefore one sided datatype, verified once, policy-free (canonicity, fold exactness, the Join, RA-linearizability), with intent theorems per generation policy: under always-R, read-equivalence to the published RGA (the fragment theorem composed with the compaction theorem); under the Fugue rule, non-interleaving (a subtree is an in-order interval). RGA and Fugue become two disciplines over one kernel rather than two datatypes.

Validation. `whiteboard/litmus/embed_sided.py`: the chain semantics and the bit carrier as lockstep twin models, both run under both policies. Every design prediction survived the first full run:

check	verdict
full gauntlet under the Fugue policy (L1–L25, credential family, subordination CEs, stale forks)	clean, **including L19** (out = $\langle 50, 30, 10, 61, 41, 21, 1\rangle$)
same gauntlet under always-R	clean except L19: the one-sided fan, reproduced
chain semantics $\sim$ bit carrier, both policies	lockstep-EXACT (unique decodability + order embedding)
all-R fragment $\sim$ the one-sided embed model	lockstep-EXACT, every scenario
randomized version-DAG PBT, all four models	120 + 300 clean

The Lean mechanization of both layers (the sided chain-lex kernel and the sided conditioned instance) has landed kernel-clean; its capstone is the policy/datatype split as a quantifier.

Theorem 23.7 (the sided capstone)**.** *Call a configuration C of the sided instance honest (`SHonest`) when (a) every delete's target has a visible prior insert, and (b) one global chain assignment exists: some* $\mathrm{chainOf} : \mathbb{N} \to \mathrm{SChain}$ *such that every insert event o of C has* $\mathrm{chainOf}(t_o)$ *positive, stored coordinate* $\mathrm{sidedCoordOf}_\Gamma(\mathrm{chainOf}(t_o))$*, and deltas telescoping to the id:* $\sum \mathrm{chainOf}(t_o) = t_o$*. Let* SReach *be honest reachability at this contract (Definition 15.1). Then every* SReach*-reachable configuration is RA-linearizable, per version, for every ordered prefix code and for every side assignment: sides are payload to convergence. The discharge is the direct-join route of §15 (`s_join_at`); the generation discipline supplies the contract (`sHonest_of_applicable`) and fold canonicity mirrors the one-sided `e_fold_canon` as `s_fold_canon`.* `sided_embed_ra_linearizable3`

§30 records the surrounding mechanization inventory.

24 Maximal non-interleaving, closed: the traversal theory and Theorem 9

The sided kernel of §23 hosts the Fugue family, and the Weidner–Kleppmann paper (TPDS'25) supplies that family's ordering specification. This section states the specification natively, presents the theory that discharges it, and records the closure: all three conditions of the paper's Definition 4 are kernel-checked for the FugueMax variant, which is, to our knowledge, the first fully mechanized maximal non-interleaving theorem for a sequence datatype. Along the way three findings recur on one theme: **the dead are load-bearing**, in stating the specification, in achieving it, and in proving it.

The specification, natively. Fix an execution. The *left origin* of an element is the element directly preceding its insertion position at the time of insertion (or a start sentinel); its *right origin* is the element directly following the left origin *in the list including tombstones* at that moment (or an end sentinel). The paper's Definition 4 (*maximal non-interleaving*) demands, over all reachable list states:

(1) *forward*: if A is the left origin of B and B displays earliest among elements with left origin A, then A and B are consecutive;

(2) *backward, with exceptions*: if B is the right origin of A and A displays latest among elements with right origin B, then A and B are consecutive, unless the exception holds: A and B have different left origins and some C in the list state sits strictly between $\mathrm{lo}(A)$ and B without being a descendant of $\mathrm{lo}(A)$ in the left-origin tree;

(3) *arbitration*: same left and right origins order by lower identifier first.

Both origin definitions and both "earliest/latest among" quantifiers range over the list *including tombstones*: that is the first of the three places the dead carry the statement. The conditions are formalized over reachable configurations of the FugueMax kernel variant (the recorded origins are the generation policy's, the display order the kernel's), with the conclusion "consecutive" read over the *visible* document, which is the property an editor's user sees.

The knowledge model (the generation-layer fold of §19, declared here once). A global configuration is a map G from replica to its list of *generation records*; a record carries $(\mathrm{ts}, \mathrm{rep}, \mathrm{op}, \mathrm{lo}, \mathrm{ro}, \mathrm{chain})$: the Lamport id, the issuing replica, the op, the recorded left origin (0 = start), the recorded right origin (the tombstone-visible successor at mint time; none = end), and the minted birth chain. For the plain Fugue policy the record list is `Know` with reachability `FugueReach` (local positional inserts with globally fresh, locally dominant Lamport ids; positional deletes; pairwise whole-knowledge sync); for the FugueMax variant, `KnowM` and `MaxReach`, same shape over the tagged records. The readers, per knowledge state K: $\mathrm{lo}(x)$ and $\mathrm{ro}(x)$ read the minted record (`mLoOf`, `mRoOf`); $x \prec y$ is "displays before, tombstones included," the display comparator on the strong-list keys (`mBefore`); $\mathrm{live}(x)$ is membership in the visible document (`mLive`); $\mathrm{minted}(K)$ the minted ids, tombstones included (`mMintedIds`); left-origin ancestry is the iterated-lo relation (`mLoDesc`). Plain-Fugue twins carry g-names (`gLoOf`, `gBefore`, …).

Definition 24.1 (adapted Definition 4, the quantified form)**.** *Over a knowledge state K (FugueMax readers shown; the plain-Fugue twins are verbatim over g-readers):*

(1) forward (`ForwardNIM`): for all minted A, B, both live, with $\mathrm{lo}(B) = A$ *and* B *display-earliest among minted elements with left origin* A *(every minted* $D \neq B$ *with* $\mathrm{lo}(D) = A$ *has* $B \prec D$*): A and B are consecutive, i.e.* $A \prec B$ *with no live* c *strictly between (`mConsecutive`).*

(2) backward, with the exception (`BackwardNIExcM`): for all minted live A, B *with* $\mathrm{ro}(A) = B$ *and A display-latest among minted elements with right origin B: if the exception fails, A and B are consecutive. The exception (`BackwardExceptionM`):* $\mathrm{lo}(A) \neq \mathrm{lo}(B)$*, and some minted C (tombstones included) has* $\mathrm{lo}(A) \prec C \prec B$ *and is not a left-origin descendant of* $\mathrm{lo}(A)$*.*

(3) arbitration (`SameOriginLowFirstM`): minted live A, B *with equal left origins, equal right origins, and* $A < B$ *display* $A \prec B$*.*

MaxNonInterleaving(K) *is the conjunction of the three; the quantified statement* FugueMaxMaximallyNonInterleaving(Γ) *demands it at every replica of every `MaxReach`-reachable G. The premise quantifiers and both origins range over minted elements, tombstones included; the conclusion is live (the visible document). The plain-Fugue twin of the exception (`BackwardException`) carries one extra conjunct, a live witness: that reading is refuted below and is retained as the negative control.* `MaxNonInterleavingM`, `FugueMaxMaximallyNonInterleaving`

The traversal theory. The paper's proofs run through a tree traversal (its Lemma 7: the list order of a forward non-interleaving algorithm is a depth-first traversal of the left-origin tree; its Lemma 8 computes origins by walking the algorithm's own tree). On the embedded-chain family the algorithm's tree is implicit in the chains, and the marker theorems of §23 already state that the display order *is* the in-order rule on chains. What the discharges consume is therefore not an emission function but an *interval* form of traversal theory (`Sided_Traversal.lean`): subtree convexity (a chain prefix's extensions display as one contiguous block around it), the side lemmas (an extension displays after the node iff its first added entry is R-headed, before iff L-headed), prefix strip-and-lift, and transitivity of the chain order. A recursive emission function was designed and deliberately not built: every consumer needs only the interval lemmas, and the emission would add a fueled recursion with no downstream client; the executable emission lives in the Python twin (`traversal_check.py`), checked against the descending-key sort on both policies.

The bridge from recorded origins to chain geometry is one shape invariant, the chain form of the paper's walk-up rule. *loShape*: every minted element x satisfies

$$\mathrm{chain}(x) \;=\; \mathrm{chain}(\mathrm{lo}(x)) \cdot \langle \text{R entry} \rangle \cdot \ell s, \qquad \ell s \text{ all-L}.$$

R mints satisfy it with ℓs empty; for L mints the content is that the tombstone-visible successor of an anchor with an R-child is itself an all-L descendant of that R-child, proved by an argmax argument (an R entry inside the rest would exhibit a minted element displaying between anchor and successor, beating the successor's maximality; the invariant and its key lemma are `loShape` and `succ_R_desc_allL`, `SidedRGA_NonInterleaving.lean`). loShape is a *generation-discipline* fact, not a kernel fact: a forged state that stores an L entry under the wrong parent displays fine (the total order does not care) but breaks the walk-up rule, and condition (1) then genuinely fails; the honesty clause excluding it is precisely that side and parent come from the policy at mint time, never from op payload.

The forward discharge. With loShape in hand, the paper's "first node the traversal visits among A's right descendants" is replaced by a *descent*: if a minted c is an R-headed extension of A, then some minted D with $\mathrm{lo}(D) = A$ displays at or before c (strong induction on chain length, comparing the two prefix decompositions of $\mathrm{chain}(c)$). Condition (1) follows: any live c strictly between A and its earliest left-origin child B lands in A's subtree by convexity, is R-headed by the side lemma, and the descent produces a D with $\mathrm{lo}(D) = A$ displaying before B, contradicting B's minimality. The one new invariant clause, the all-L successor shape for L mints, transports through insert, delete, and sync like the main bundle. Condition (3) was already the positive twin of the Fugue refutation (`fuguemax_same_origin_low_first`: at every replica of every `MaxReach`-reachable G, same-origin pairs display lower id first).

Theorem 24.2 (forward non-interleaving, FugueMax)**.** *For every ordered prefix code Γ: every replica of every `MaxReach`-reachable configuration satisfies condition (1) of Definition 24.1. No hypotheses beyond reachability; every invariant consumed is internal. Kernel axioms only.* `fuguemax_forward_ni`

Everything in the forward proof never consults the sibling order, and the same argument discharges the *plain Fugue* policy's forward condition as stated: the link layer was rebuilt as an external invariant over an untouched `FugueReach`, with roughly 85 percent of the FugueMax machinery transferring verbatim.

Theorem 24.3 (forward non-interleaving, plain Fugue)**.** *For every ordered prefix code Γ: every replica of every `FugueReach`-reachable configuration satisfies condition (1) in its g-reader form (`ForwardNI`). Kernel axioms only.* `fugue_forward_ni` *(`SidedRGA_Fugue_ForwardNI.lean`)*

Plain Fugue's conditions (2) and (3) remain false, as the paper itself proves, and the separation is machine-checked at reachable configurations:

Proposition 24.4 (the plain-Fugue refutations)**.** *At the unary code:*

- *Condition (3) fails: on the reachable two-concurrent-front-inserts configuration (`GTie`), ids 1 and 2 share both origins and display $[2, 1]$: the kernel's newest-first sibling order is the exact reverse of the paper's (by its Theorem 10, not optional) lowest-id-first arbitration. Hence* $\neg$ FugueMaximallyNonInterleaving(unary)*.* `fugue_not_maximally_noninterleaving`
- *Condition (2) fails, on the paper's own Figure-7 execution (`KFig`): the pair $(A, B) = (7, 4)$ satisfies the backward premises, the live 6 sits strictly between, and every candidate exception witness is refuted pointwise, so* $\neg \mathrm{BackwardNIExc}(\mathrm{unary}, K_{\mathrm{Fig}})$ *(`fugue_backward_gap`); lifted to the full statement at the reachable configuration as `fugue_not_maximally_noninterleaving_backward`. This is the paper's own Fugue-versus-FugueMax gap, so the separation does not hinge on the tiebreak clause.*

Both close by `native_decide` on the real fold, under the SPOT convention (§19).

The backward condition: the live-witness refutation. The Lean statement of the exception, as first adapted, required the witness C to be *live*. That reading is false, and the countermodel is the paper's own Figure-7 execution followed by one delete. In that execution (elements here named by their ids) the display is $[3, 6, 7, 4, 5]$ with $\mathrm{ro}(6) = 5$ and $\mathrm{lo}(6) = 3 \neq \mathrm{lo}(5)$; delete 4, so the visible document is $[3, 6, 7, 5]$. The premise pair $(A, B) = (6, 5)$ holds: 6 is the only minted element with right origin 5, hence trivially display-latest among them. The pair

is not consecutive: 7 sits between. The exception's first half holds. But no *live* witness exists: the only elements strictly between $\mathrm{lo}(6) = 3$ and 5 are 6 and 7 (both descendants of 3 in the left-origin tree) and the *dead* 4, which is exactly the C the paper's own proof produces for this execution. The paper's Lemma 5 quantifies C over the list state including tombstones, so the corrected definition drops the one liveness conjunct and nothing else. Second load-bearing role of the dead: without them the specification itself is not a theorem of FugueMax (the refutation fired on the directed families and on 3 of 4500 randomized delete-enabled states; the earlier insert-only sweeps could never see it). Two remarks. The corrected reading is strictly paper-faithful and strictly weaker as a guarantee: a pair excused by a dead witness is a pair the visible document may interleave; that is forced, not chosen. And the *conclusion* keeps its live reading (no live element strictly between): the visible-document property is the one proved. Per this project's standing rule, the goal was corrected in the open, forced by a countermodel and matching the paper's own quantification, never silently renamed.

Replacing causal minimality. The paper's condition-(2) proof exhibits its exception witness by choosing an in-between element "not causally later than any other," then consumes that minimality three times. A reachability-invariant formulation cannot replay this: the invariant carries per-record facts about the current knowledge, not a well-founded causal order to minimize over. The replacement is geometric. Three mint-time clauses on R-mint records (a supplementary invariant bundle, transported exactly like the existing ones): a root-anchored R mint records no successor; a recorded successor displays after the anchor; and the two *RSucc* clauses pin, for an R mint whose anchor is itself an L or R mint, a stable witness (the anchor's parent, a later L-sibling, or the anchor's own recorded successor) whose key bounds the recorded successor's (the `RBk` bundle, `SidedRGA_Backward.lean`). With those, a deterministic, state-definable witness function replaces the causal choice: it inspects the in-between element's chain, walks at most two steps (four routes, exercised as α 253 times, β_1 22, β_0-direct 2, β_0-ladder 2 across 4500 validation states, the β_0 routes reachable only by directed construction), and returns a minted element strictly between $\mathrm{lo}(A)$ and B that the left-origin walk can never reach. The methodological point: a *universal* fact about the minter's knowledge does not transport to later states, but an *existential* witness does, because minted records, chains, and keys are stable under knowledge growth. Third load-bearing role of the dead: the constructed witness may itself be a tombstone (it is, in the Figure-7 family), and the intermediate lemmas are deliberately stated over minted, not live, elements so the ladder case can call them.

The capstone. The backward discharge (`SidedRGA_Backward.lean`, `fuguemax_backward_ni`) closes the arc.

Theorem 24.5 (Theorem 9, mechanized)**.** *For every ordered prefix code* Γ*:* FugueMaxMaximallyNonInterleaving(Γ)*, that is, all three conditions of Definition 24.1 hold at every replica of every* `MaxReach`*-reachable FugueMax configuration. No hypotheses beyond reachability and the code; kernel-clean on* {`propext`, `Classical.choice`, `Quot.sound`}*. Assembled from conditions (1) and (3) (Theorem 24.2,* `fuguemax_same_origin_low_first`*) and the backward condition (*`fuguemax_backward_ni`*) by the reduction theorem* `fuguemax_theorem9_of_backward`*.* `fuguemax_maximally_noninterleaving`

The reduction theorem and the forward file recompiled with zero edits after the definition correction, as the design phase predicted. SPOTs pin the witness values on all four delete-bearing directed variants, PASS and FAIL shaped.

The convergence capstone, and the TagsOK finding. Ordering intent is only half the variant's bill. The FugueMax realization writes records over a *different block format* (R entries carry the mint-time successor's key as an order-reversing tag before the complemented delta code), so RA-linearizability for the tagged datatype was owed separately and is paid: same canonical sorted-list state, same mergeable-queue route, fold canonicity mirrored as `f_fold_canon`. The honesty contract needs one clause the sided contract did not:

Definition 24.6 (wellformed tags)**.** *A right-origin tag t is wellformed when it is the end tag $\langle 0\rangle$, or a real key: $t = (h \cdot c) \cdot \langle 3\rangle$ with head $h \notin \{0, 3\}$, $h \leq 5$, and every symbol of c marker-free and bounded ($\neq 3$, ≤ 5). The obligation applies to R entries (L entries carry no tag);* TagsOK(ch) *demands it of every entry of the chain.* *`TagOK`, `TagsOK`*

Theorem 24.7 (FugueMax convergence)**.** *Call a configuration C of the tagged instance honest (`FMHonest`) when (a) every delete's target has a visible prior insert, and (b) one global chain assignment exists: some* chainOf *with, for every insert event o of C, the chain positive, its tags wellformed (*TagsOK(chainOf(t_o))*, the new clause), the stored coordinate equal to the chain's tagged coordinate (`fmCoordOf`), and deltas telescoping to the id. With* FMReach *honest reachability at this contract (Definition 15.1): every* FMReach*-reachable configuration is RA-linearizable, per version, for every ordered prefix code. The right-origin tags ops carry are payload to this theorem, exactly as sides were to Theorem 23.7.* *`fuguemax_ra_linearizable3` (`SidedRGA_FugueMax_RA_Lin.lean`)*

The finding worth a name: **the right-origin tag is convergence-load-bearing**, not merely intent-load-bearing. Unique decodability of the variant coordinate format (`fmCoordOf_inj`), which key injectivity and hence fold canonicity run through, is only unambiguous on wellformed tags: the generation layer supplies TagsOK for free, but a hand-forged tag would break convergence itself, not just the display intent. Convergence is otherwise insensitive to what the coordinates contain, exactly as the policy/datatype split predicts: coordinates are injectively minted, immutable, and consumed only through key comparisons.

24.1 Where the family sits in the published table

The Weidner–Kleppmann paper's Table 1 is the nearest published designs × anomalies matrix: roughly ten list CRDTs classified by hand against the interleaving conditions. The placement this repository contributes is not a new row but a change in how rows are occupied: **one kernel datatype, verified once, whose generation policy selects the row, each placement by theorem**.

policy / variant	row occupied	by which theorem
always-R	RGA	fragment theorem + compaction theorem: read-equal to the published RGA, anomalies *included* (L19 inherited by theorem)
Fugue policy	Fugue	forward NI as stated (`fugue_forward_ni`); *not* maximally non-interleaving: the paper's own separation, machine-checked on its Figure-7 execution
FugueMax variant	FugueMax	Theorem 9 mechanized (`fuguemax_maximally_noninterleaving`); convergence separately (`fuguemax_ra_linearizable3`)

Two readings of the table. First, the one-sided embedded-chain RGA occupies the RGA row *by theorem*: the compaction theorem (§30) makes it the published RGA's reads verbatim, so every property Table 1 records for RGA, favorable or not, transfers; the design does not get to keep the good rows and disown L19. Second, the convergence column is policy-independent (sides and tags are payload to the merge; the sided capstone quantifies over every side assignment) while the interleaving column is per-policy, which is the policy/datatype split of §23 rendered as table structure. What no policy can do is leave the family's *delete-insensitive* rows altogether: display keys are immutable and deletion is pure removal for every policy, so a policy selects among the retention-faithful orders and nothing else. The row a splice-based design would occupy is unreachable by policy, and the next section proves that unreachability is not an accident of this kernel but an information-theoretic fact.

25 The chain price: what delete must remember

The conservation law of §19 (the sort key must retain dead ancestors' timestamps) was stated as an empirical regularity of the design sweep: every design that forgets them has a named counter-model. This section upgrades it to a characterization. The question that forced it (KC, 2026-07-18): the embed RGA works, but its proof rests on keeping a deleted node's memory alive inside live descendants' coordinates; could a *sibling-edge* implementation, which splices a dead node out and lifts its children a level up, be shown equivalent to the embed RGA with no such retention? The answer is no, by a four-event fooling pair (`Refutations/SiblingSplice_Fooling.lean`), and the counterexample names exactly the information the splice destroys.

The mechanism. In the embed order, a lifted child displays at its dead ancestor's rank: the subtree keyed by the ancestor's stamp holds its *slot*, and the child sits in that slot. Splice-and-lift erases the ancestor and re-ranks the child among its new siblings by the child's *own* stamp. Whenever a concurrent sibling's stamp lies strictly between the dead ancestor's and the child's, the two rankings disagree, and no trace of the disagreement survives in the spliced state.

The fooling pair, walked. Timestamps only; slot keys shown. Both worlds share the LCA and the concurrent branch B; only branch A differs, and its two variants land on *equal* spliced states.

	world 1	world 2
shared LCA	ins a (stamp 1) at the root	same
shared branch B	ins c (stamp 2) at the root, concurrent with all of A	same
branch A	ins b (5) *under* a; del a (6)	del a (4); ins b (5) at the root
spliced A-state	the one-node forest $[\,b(5)\,]$	$[\,b(5)\,]$: **equal**, machine-checked
b's slot key (embed)	1 (dead a's slot)	5 (its own)
owed read after merge	$[\,c,\,b\,]$ $(1 < 2)$	$[\,b,\,c\,]$ $(5 > 2)$

The knob is the stamp crossing $a < c < b$ with b and c on *different* branches, and it rides on concurrency: B saw only the LCA, so its stamp 2 is Lamport-honest, and every stamp on A exceeds everything its issuer observed ($1 < 5 < 6$; $1 < 4 < 5$). Sequentially the crossing is impossible (a later insert has a larger stamp than everything visible, the dead anchor included), which is why no sequential test can see this and why the pair had to be built on a version DAG.

The impossibility is then three lines: a ternary merge function on the splice representation receives *identical* arguments in the two worlds (the LCA and branch B verbatim, the A-states by the machine-checked equality) and owes *different* answers, so there is none.

Proposition 25.1 (the fooling pair: no merge function over spliced states)**.** *Let w_L and w_B be the spliced LCA and branch-B states, w_{1A}, w_{2A} the two worlds' branch-A states, and ρ_1, ρ_2 the two worlds' embed histories (unary code; payloads* 101/105/102 *for a/b/c;* read *is the element projection of the fold). The supporting pins, expected values hand-derived:*

$$w_{1A} = w_{2A}, \qquad \text{read}(\text{eFold } \rho_1) = [102, 105], \qquad \text{read}(\text{eFold } \rho_2) = [105, 102]$$

(`states_equal_A`, `embed_read_w1`, `embed_read_w2`; the FAIL pin `embed_reads_differ` is the inequality of the two reads). The impossibility, quantified over every merge function on the splice representation: for all $f : \text{St} \to \text{St} \to \text{St} \to \text{List } \mathbb{N}$,

$$\neg\big(\, f\ w_L\ w_{1A}\ w_B = \text{read}(\text{eFold } \rho_1)\ \wedge\ f\ w_L\ w_{2A}\ w_B = \text{read}(\text{eFold } \rho_2)\,\big).$$

The proof is the three lines above; the pins close under the SPOT convention (fold (iii), §19). `sibling_splice_no_merge_function`

Finding (the retention characterization)**.** *The fooling pair (Proposition 25.1) is a lower bound: any state representation that forgets a lifted child's dead slot key admits the pair, so no merge over it can reproduce the embed's reads. The capstone is the matching upper bound: keeping the whole dead chain suffices for everything (`embed_ra_linearizable3`, the compaction theorem, the sequential spec, delete-order preservation; §30). Between them: the chain of dead slot keys is exactly the information content of delete. Deletion's semantic content in this family is "the subtree keeps its slot," and the slot chain is both necessary and sufficient to honor it. What remains negotiable is only where the memory lives (coordinate prefixes here, carried spine paths in the validated sibling-edge design, tombstone records in the published RGA: three encodings of the same bits) and how many bits it costs (§20, §26); whether it lives is not. This is the pairing of two mechanized bounds, not a single Lean name.*

The two-by-two. Retention crossed with display rank sorts the four named designs of this document's arc:

design	retains the dead chain?	lifted child ranks by	verdict
embed RGA	yes: coordinate prefix	dead ancestor's slot key	capstone + seq. spec
$\text{RGA}_\dagger$ (tombstoned)	yes: tombstone records	dead ancestor's slot key	read-equal to embed (compaction theorem)
rehoming (flat TF)	truncated at each rehoming	own stamp, after the climb	RA-lin; seq. spec *refuted* (Part III)
Shesha (splice)	no	own stamp, after the splice	no merge function (the fooling pair)

Reading down the rows: full retention with slot-key display gives the two read-equal designs (tombstone-free and tombstoned differ only in encoding); truncating the chain at each rehoming preserves convergence but breaks the sequential specification; discarding it altogether leaves nothing for any merge function to work with. The verdicts degrade exactly as the retained information does.

The delete-sensitivity separation. §24.1 showed the generation policy selecting rows of the interleaving table. Here is what no policy can select.

Theorem 25.2 (delete never moves survivors)**.** *For every ordered prefix code* Γ*, every state* s*, and every delete operation:* eUpdate Γ s $(t, r, \mathsf{del}\ x)$ *is a sublist of* s *(the survivors, in unchanged order). No hypotheses: no honesty, no reachability, no policy. The proof is one line (*`List.filter_sublist`*); the theorem carries the separation because it is unconditional.* `eUpdate_del_sublist`

For every policy over the retaining kernel, display keys are immutable and deletion is record removal, so a delete never moves survivors (Theorem 25.2 is policy-free): the whole family is *delete-insensitive*. Shesha's row is delete-*sensitive* (the splice moves survivors), and the fooling pair shows the sensitivity is not a stylistic difference but an information-theoretic one: the splice state cannot support the retention family's reads at all. Policies span the interleaving axis; nothing spans the retention axis. The two degrees of freedom are orthogonal, and only one of them is free.

No coding escapes. Could a cleverer coordinate code rescue the splice? No, in both directions. Downward: the impossibility theorem quantifies over *every* merge function on the splice states, and an encoding is a function of the state, so any coded splice inherits the fooling pair verbatim (the two worlds' encodings are equal bit for bit, being encodings of equal states). Upward: the positive theorems are parametric in the code (§30: one proof, three instantiated codes), so the entire coding freedom provably lives in what retention *costs*, never in whether it happens. Coding moves bits; it cannot manufacture information the state has discarded. That is the chain price: the storage layer of the next section spends considerable effort making the price small, and none trying to evade it.

26 The storage layer: re-coding, compaction, and the run table

The datatype's costs were settled upstairs (§20: pay the entropy of the anchor gaps at mint time) and its obligations downstairs (§25: the dead chain must be retained). This section is the layer between: what a running system does about accumulated weight. The arc has three rungs, and stating them up front keeps the bookkeeping honest. *Chains carry the semantics*: birth constants, never edited, the objects of every correctness theorem. *Runs carry the representation*: an invertible re-encoding of the current state, valid with no hypotheses at all. *Stability is needed only for forgetting*: epoch maps that genuinely discard dead history rewrite geometry, and rewriting is safe only at a settled cut (§18). Every measurement below reproduces from the cited harnesses, and each table's accounting model is stated with it.

26.1 Re-coding at a settled cut: the theorem cluster

The epoch problem. Coordinates are birth constants and never shrink: a live record whose birth chain passed through nine deleted ancestors carries all nine dead codewords as a prefix forever. Tombstone-freedom removed the dead *records*; their *codewords* survive inside every descendant, so long editing accumulates coordinate weight proportional to total history, not to the live document. A GC epoch should re-code: assign live records fresh, short coordinates inducing the same display order, and let new mints extend the fresh coordinates.

Lazy translation, and why it is the only nontrivial formulation. Replicas cannot switch codes in lockstep: a message minted against old coordinates may be in flight while the receiver has already compacted. The model is therefore *lazy translation*: the re-map is a pure function on coordinates, applied once to the state at rest and to each incoming record or op minted in the old code (the op's carried anchor prefix is translated; its delta codeword is not touched). Formally, fix a set S of *stable* coordinates, downward closed under the ancestor relation. Downward closure is not a design choice but a fact about cuts: a coordinate's proper codeword-prefixes are exactly its ancestors' coordinates, ancestors causally precede their descendants, and any causally downward-closed notion of stable therefore contains the ancestors of everything it contains. Given a compaction ρ on S, the induced re-map factors every coordinate at the cut:

$$\hat{\rho}(c) \;=\; \rho(\mathrm{stab}\ c) \,\|\, \mathrm{rest}\ c,$$

where stab c is the longest stable prefix (it is exactly one chain prefix, again by downward closure) and the unstable tail is kept verbatim. The theorems consume two hypotheses, both relativized to the coordinates *at hand* (convention (v), §19), never to all bit strings; injectivity is free. The Lean bundle (`EmbedRGA_Recoding.lean`, payload- and code-generic, kernel-clean):

Definition 26.1 (the stable-prefix re-map bundle)**.** *A stable-prefix map for* Γ *is* $F = (f, \mathrm{Rest}, \mathrm{MintAt}, \mathrm{ext}, \mathrm{ord})$*: a coordinate map* f*, the at-rest domain* Rest *(the cut state's records), the declared beyond-cut mints* MintAt*, and two laws:*

(H3) extension commuting, guarded by MintAt*:* $\mathrm{MintAt}\ \pi\ d$ *implies* $f(\pi \cdot \Gamma.\mathrm{enc}(d)) = f(\pi) \cdot \Gamma.\mathrm{enc}(d)$*;*

(H2) order preservation on the domain: for c*,* c' *at hand,* $\mathrm{keyLt}(\mathrm{key}(f\,c))\,(\mathrm{key}(f\,c')) = \mathrm{keyLt}(\mathrm{key}\ c)\,(\mathrm{key}\ c')$*, a Boolean equation carrying both directions of the display comparator.*

"At hand" ($F.\mathrm{Dom}\ c$*) is* $\mathrm{Rest}\ c \vee \exists \pi\, d,\ \mathrm{MintAt}\ \pi\ d \wedge c = \pi \cdot \Gamma.\mathrm{enc}(d)$*. There is no injectivity field.* `StablePrefixMap, StablePrefixMap.Dom`

Lemma 26.2 (H1 is derived)**.** *For* c*,* c' *at hand with* $f\,c = f\,c'$*:* $c = c'$*. (H2 at the pair, plus totality and irreflexivity of* keyLt *and injectivity of* key*: a strict first-difference order admits no two-sided tie except equality.)* `StablePrefixMap.injOn`

Writing remap_F for the translation of records, states, and ops by $F.f$ (`eRemapSt`, `eRemapOp`: an op's carried anchor prefix is translated; its delta codeword and deletes are untouched):

Theorem 26.3 (T1: cut-parametric fold congruence)**.** *For a stable-prefix map* F*, cut state* s*, and continuation* τ *with* s *at hand and* τ *declared at* F*:*

$$\mathrm{applySeq}\ (\mathrm{remap}_F\ s)\ (\tau.\mathrm{map}\ \mathrm{remap}_F) \;=\; \mathrm{remap}_F\ (\mathrm{applySeq}\ s\ \tau)\,:$$

folding the translated continuation over the translated cut state is the translation of the untranslated fold. `eRecode_applySeq`

Proposition 26.4 (the global-collapse negative)**.** *One might hope to state T1 over the whole history (map every op ever issued, re-fold from the initial state); that statement forces H3 at every historical mint. If a coordinate map* f *satisfies H3 at every positive mint (*$\forall \pi\, d,\ 1 \le d$ *implies* $f(\pi \cdot \Gamma.\mathrm{enc}(d)) = f(\pi) \cdot \Gamma.\mathrm{enc}(d)$*), then for every positive chain* ch

$$f(\mathrm{coordOf}_\Gamma\ ch) \;=\; f(\varepsilon) \cdot \mathrm{coordOf}_\Gamma\ ch\,:$$

a constant-prefix prepend, which compacts nothing. Re-coding is inherently an epoch operation on states, not a history rewrite; the lazy, cut-parametric formulation is the only nontrivial one. A finding, mechanized; kernel-clean (a theorem over all such maps, not a SPOT). `eRecode_ext_global_collapse`

Theorem 26.5 (T2: reads identical)**.** *Under exactly the hypotheses of T1 (s at hand, τ declared at F), the query of the translated run equals the untranslated one:*

$$\mathrm{query}\big(\mathrm{applySeq}\ (\mathrm{remap}_F\ s)\ (\tau.\mathrm{map}\ \mathrm{remap}_F)\big)\ =\ \mathrm{query}\big(\mathrm{applySeq}\ s\ \tau\big)\ :$$

compression is invisible to every observer. `eRecode_reads_identical`

The contentful half of T2 hides in T1: the translated side re-sorts by the *new* keys, so equality of reads is exactly the statement that the new keys induce the old order, H2 doing its job. The negative control earns its keep here: an injective, H3-satisfying but order-breaking map genuinely flips the read (the `badMap` FAIL companion of the file's SPOT), pinning H2 as the load-bearing hypothesis.

Theorem 26.6 (T3: the capstone's witness statement transports)**.** *Hypothesis:* EReach Γ C *(honest reachability, §30); no at-hand hypothesis. For every registered version* $C.\mathrm{ver}\ v = (s, E_v)$ *there is a witness enumeration π with: π a permutation of E_v (*`listPermOf`*); π respecting the delivery order of C's core; and*

$$\mathrm{remap}_F\ s = \mathrm{remap}_F\ (\mathrm{eFold}_\Gamma\ \pi), \qquad \mathrm{query}(\mathrm{remap}_F\ s) = \mathrm{query}(\mathrm{eFold}_\Gamma\ \pi).$$

Canonicity, sortedness, and version-level sortedness transport too, each under its own side conditions: canonicity needs EWf *of both enumerations, a membership condition, and declared mints (*`eRecode_fold_canon`*); sortedness needs the at-hand hypothesis of convention (v) (*`eRecode_sorted`*); version-level sortedness needs EHonest in addition to EReach (honesty at the endpoint, which honest reachability alone does not supply) plus at-hand (*`eRecode_version_sorted`*).* `eRecode_ra_transport`

Honest scope note. The transport is at the level of states and witnesses, not configurations: the re-coded history is *not* itself an honest configuration of the same datatype (compacted coordinates deliberately forget dead chain segments, so the chain-generation honesty clause fails for them). Re-basing honesty across the epoch, so that the capstone re-invokes natively in the new epoch, was the deferred obligation of this cluster's first revision and is now closed: the (∗) cluster of §26.2 (Definition 26.21 through Corollary 26.25); what stays open downstream is the multi-replica half (agreement on a common cut, Open Question 43.8). The cut hypothesis itself is discharged by §18's SettledAt gate: `eRecode_settled_bridge` (in `EmbedRGA_Stability_Bridge.lean`, with the stability import, not in the re-coding file) packages T1+T2 at a settled cut against the named residue `AnchorsFactorBeyond`, and is superseded by the residue-free honest form (Theorem 26.11).

Measurement 1: the microbenchmark. Directed, hand-computed worked example (unary code; history: insert a at the root, b under a, c under b, delete a and b; then, beyond the cut, insert d under c and e at the root). Accounting: the numbers are *coordinate bit lengths of live records*, order metadata alone, identical convention to the entropy tables of §20; characters are excluded.

live record	original coordinate	re-coded
c (two dead codewords carried)	6 bits	2 bits
d (minted beyond the cut, under c)	10 bits	6 bits
e (minted beyond the cut, at the root)	8 bits	8 bits
total	24 bits	16 bits

Reads are identical on both sides (hand-checked and pinned as SPOTs); the dead-chain share of c and d's bits is gone entirely, and e, which never passed through the dead region, is untouched. The same harness's delete-heavy directed case (a 200-deep chain, binary code) drops live coordinate weight from 200 bits to 1 bit, the **200**× figure quoted in the arc's motivation; 500 randomized histories (random cut, two replicas forked beyond the cut with concurrent ops and a merge at the end) pass reads-identical and the T1 record-for-record correspondence at every step, and the order-breaking negative control flips the read exactly as predicted.

26.2 The verified concrete compaction, and what real traces say

compactEliasDelta. The theorem cluster is parametric in the re-map; the concrete v1 compaction instantiates it (`EmbedRGA_CompactEliasDelta.lean`): *(a) drop dead ranges* (a stable chain is carried forward iff a live descendant or a declared in-flight coordinate passes through it; everything else is dropped and its sibling rank reclaimed) and *(b) rank-renumber* (in each sibling group the surviving deltas are renumbered to $1..k$ order-isomorphically and re-encoded with the same ordered prefix code). Its H2 discharge runs the per-group order isomorphism through the chain-lex theorem, with renumbered ranks dominated by Lamport-fresh future deltas; the headline discharges everything at a settled cut of an honest configuration.

Theorem 26.7 (the concrete compaction preserves every read at a settled cut)**.** *Let C be a configuration at the Elias-δ code with (i)* EReach *C (honest reachability, §30), (ii)* GenHonest *at* eApplicable *(the generation discipline), and (iii)* CausalPastEnumerable *C; let (iv) C.*ver *$v = (s, E_v)$ be a registered version with (v)* SettledAtOn *C E_v S a settled cut (§18). Let tree, live, inflight be the compactor's inputs with (vi) every tree chain positive (h_{tpos}); (vii) tree exactly the settled inserts' chains, in both directions (h_{Tree}: every settled insert's coordinate is some tree chain's coordinate; h_{TreeS}: every tree chain is some settled insert's); (viii) h_{Live}: a tree chain whose insert no delete of S targets carries live =* true*; and (ix) h_{Inflight}: every insert of E_v escaping S is declared in inflight with its (positive) chain. Then there is a stable-prefix map F with*

$$F.f = \mathrm{fStab}\ \big(\mathrm{rED}\ (\mathrm{ePruneKeep}\ \mathit{tree}\ \mathit{live}\ \mathit{inflight})\ \mathit{inflight}\big)\ (\mathrm{ePruneKeep}\ \mathit{tree}\ \mathit{live}\ \mathit{inflight})$$

(drop dead ranges, rank-renumber outside `skipAt`-guarded groups, re-encode) such that every continuation τ of events existing beyond the cut ($o \in C$.events, $o \notin E_v$) satisfies both the T1 fold congruence and T2 reads-identical of §26.1. `compactEliasDelta_settled_reads`

Remark 26.8. *Display compressions, for the record: the mechanized statement phrases (vii)–(ix) per operation, over* eIsIns *$o =$* true *with coordinate matching* coordOf *$k =$* eCoord *o; each per-operation quantifier there additionally ranges over $o \in C$.*events *(a conjunct the display drops: it is redundant under hypothesis (i), which bounds every registered version's events by C.*events *through* GoodConfig3*, but it is present in the mechanized statement); and (viii)'s undeleted clause compares payloads, $\forall dl \in S$, $dl \neq \mathsf{del}(o.1)$ read at the payload of dl. The theorem is*

the Elias-δ instantiation of the code-parametric engine `compactRanked_settled_reads`*, whose hypothesis list is identical.*

The in-flight wrinkle, pinned both ways. Renumbering a sibling group is only an order isomorphism against the deltas it can see. If an op minted in the old code is still in flight with a delta in that group, dense renumbering can slide a surviving sibling past it: the FAIL SPOT pins exactly this (an in-flight root insert; the unguarded renumbering sends a surviving delta $9 \mapsto 2$, and the delivered op lands on the wrong side: the pinned read flips). The v1 resolution is the guard `skipAt`: sibling groups with a known in-flight child delta are skipped this epoch. Both pins are hand-derived, PASS and FAIL shaped.

Proposition 26.9 (the in-flight wrinkle, pinned both ways)**.** *On the file's directed cut (root sibling group* $\{1, 2, 3, 4, 9\}$*, survivor* 4*, dead-but-kept* 9*; an epoch-0 in-flight root insert with chain* [6]*, payload* 110*; hand-derived control read* $[106, 110, 104]$*):*

FAIL the unguarded dense renumbering (the tempting degenerate, `rED` *fed an empty in-flight list) yields*

$$\mathrm{read}(\textit{unguarded}) = [110, 106, 104] \;\neq\; [106, 110, 104] = \mathrm{read}(\textit{control}) :$$

$9 \mapsto 2$ *slides the compacted survivor under the in-flight* 6 *and the late op jumps to the front (*`unguarded_flips_order`*);*

PASS the guarded construction skips exactly the root group (`skip_guard`*:* skipAt = true *there,* false *at the in-flight-free* [9] *group), keeps the read pinned at* $[106, 110, 104]$*, and still compresses: coordinate weight* 19 *against the control's* 23 *(*`guarded_skips_group`*).*

`unguarded_flips_order`*,* `guarded_skips_group`

AnchorsFactorBeyond: the residue is a theorem. The bridge from SettledAt to the cluster left one named residue: every op minted with the cut in its causal past must carry an anchor prefix that factors at the map. This is *not* dischargeable from bare honest reachability (countermodel: the chain-generation clause constrains the minted coordinate, not the carried split, so a dishonest split satisfies it while defeating the factorization). It *is* a theorem of the generation discipline: under the framework's honest generation (the same hypotheses the capstone's honesty discharge consumes), every insert's carried prefix is the stored coordinate of an actual anchor record in the fold of its causal past, so the factorization sits on a codeword boundary by construction, and the bridge is restated residue-free.

Theorem 26.10 (the residue is a theorem of the discipline)**.** *Call* (π, δ) *a canonical chain-aligned mint beyond the cut* S *(*`ChainMintBeyond`*) when* $1 \le \delta$ *and* $\pi = \mathrm{coordOf}_\Gamma(ch_A)$ *for a positive chain* ch_A*, witnessed by an actual insert event* $o \in C$.events *with* S *entirely in its causal past (*$\forall a \in S$, vis a o*) and* $\mathrm{chainOf}(o.1) = ch_A \cdot \delta$*. Hypotheses: (i)* EAnchored Γ C chainOf *(the anchor certificate the honesty discharge produces); (ii)* F.MintAt *contains every canonical chain-aligned mint beyond* S*. Conclusion:* AnchorsFactorBeyond Γ F C S*, i.e. every op minted with the whole cut in its causal past carries an anchor prefix that factors at the map. The containment (ii) is definitional for the stage-2 construction; the residue is a theorem of the anchored geometry, not an extra hypothesis.* `anchorsFactorBeyond_of_anchored`

Theorem 26.11 (the bridge, residue-free)**.** *Hypotheses: a stable-prefix map F;* GenHonest *at* eApplicable*;* CausalPastEnumerable *C; C.*ver *$v = (s, E_v)$;* SettledAtOn *C E_v S; s at hand for F; and the construction-checkable containment: for every anchored assignment* chainOf *certified by* EAnchored*, the canonical chain-aligned mints beyond S lie in F.*MintAt*. Conclusion: every continuation τ of events existing beyond the cut folds and reads identically under the lazy translation (T1 $\wedge$ T2). The* AnchorsFactorBeyond *hypothesis of* `eRecode_settled_bridge` *is gone; no* vis*-side condition survives in the interface.* `eRecode_settled_bridge_honest`

Scope, honestly delimited: the single-epoch statement, with cut-side data addressed by coordinates and chains, never by event ids resolved through prefix sums (the runtime twin's erratum: renumbered coordinates no longer telescope to ids, so id-addressed cuts break after epoch one). Concurrent divergent compactions (an epoch DAG with common-refinement translation) remain the deferred protocol half.

Spine fusion, mechanized. Rank renumbering alone leaves the *spine depth*: typing runs mint δ=1 chains, and every level costs at least one codeword even at ordinal 1 (Measurement 2 quantifies this). Fusion removes the dead levels themselves. A *fusible spine* is a maximal chain of settled dead nodes, each with exactly one child branch (counting every known coordinate, in-flight prefixes included) and no in-flight op anchored on it; it collapses to a single level at the spine head's codeword, so a typing-run-then-delete chain of depth k costs one codeword instead of k. Composed after the rank pass this is the full iteration-two map, and reads survive it (`EmbedRGA_Fusion.lean`, kernel-clean).

Theorem 26.12 (fusion composes with the rank pass, reads intact)**.** *Let F be the rank-pass map. Fusion data: a positive spine Q ($h_{Q\text{pos}}$), its surviving head Q' a prefix of Q ($h_{Q'}$), and a chain domain $\mathcal{D}$ with (i) every $\mathcal{D}$ chain positive ($h_{\mathcal{D}\text{pos}}$) and (ii) through-traffic passes the whole spine (h_{thru}: a $\mathcal{D}$ chain extending Q' extends Q). Let the two maps be compatible at a surviving domain* (Rest′, MintAt′) *(*CompatOn*, Definition 26.14), and let the cut state s be at hand and the continuation τ declared at the composite F.*comp (fuseSPM Γ Q Q' $\mathcal{D}$ …)*. Then the reads of any beyond-cut continuation survive the two-pass translation* fuseBits $\circ$ *$F.f$:*

$$\text{query}\big(\text{applySeq (remap } s) \ (\tau \textit{ translated})\big) = \text{query}\big(\text{applySeq } s\ \tau\big).$$

(`fuseSPM` *packages the fusion as a stable-prefix bundle over* fuseBits*, its bit-level map; the fusion-alone form is* `fuse_reads_identical`*; both are direct re-invocations of T2.)* `compactThenFuse_reads`

Order preservation is a three-class H2 argument: (1) within a fused block the common prefix is replaced wholesale, relative order untouched; (2) a block against the spine head's siblings is decided at the head's level, whose codeword fusion keeps; (3) the anchor-versus-extension corner is vacuous, since dead spine nodes hold no records and no key ends at a fused-away level. The mechanization sharpens class 2 into triviality: because fusion runs *after* the rank pass, the head codeword it sees is already the ordinal, so the fusion map keeps the head verbatim and drops only the strict interior, and the class-2 comparison is unchanged by construction rather than by a separate "block inherits the rank" argument. The guard is conservative (any known in-flight branch or anchor blocks its node from every spine); through-traffic below a blocked node still fuses.

The merge clause, discharged (VC-S4 for the remap species). The stability bundle's argumentwise merge congruence (§18, VC-S4) is, for the coordinate re-map, the statement that lazy translation commutes with the ternary merge (`EmbedRGA_MergeCongr.lean`).

Theorem 26.13 (the merge clause)**.** *For a stable-prefix map F and states l, a, b with a and b at hand for F (no hypothesis on the LCA l):*

$$\text{eMergeL}\ (\text{remap}_F\ l)\ (\text{remap}_F\ a)\ (\text{remap}_F\ b)\ =\ \text{remap}_F\ (\text{eMergeL}\ l\ a\ b).$$

The LCA enters the survival rule solely through its id set, which the re-map preserves; this is what lets a compacted head merge against a full LCA payload, the mixed case §18 calls the normal one. The VC-S4 packaging: the simulation relation eRemapRel *(compacted = translation of full, full at hand) is preserved by the ternary merge (`eRemapRel_merge`), literally the `vc_merge` conclusion for the remap species; `embed_merge_remap_congr` restates the congruence at* $(E\ \Gamma\ \alpha)$.mergeL. `merge_remap_congr`

Multi-epoch composition. A single re-coded configuration is not a native honest configuration (T3's gap), so a *second* compaction cannot naively re-invoke the cluster. The data-plane half is now closed (`EmbedRGA_MultiEpoch.lean`, kernel-clean): the semantic residue of one epoch is a stable-prefix map, and maps compose.

Definition 26.14 (epoch composition at the surviving domain)**.** *For maps F (earlier) and G (later),* CompatOn $F\ G$ Rest′ MintAt′ *carries the composite's own at-hand domain, the coordinates surviving to G's epoch, and demands four boundary clauses:* restF *(*Rest′ c *implies* F.Dom c*),* restG *(*Rest′ c *implies* G.Dom $(F.f\ c)$*), and* mintF*/*mintG *(the same two clauses for declared mints, the later mint at the translated prefix $F.f\ \pi$). Under them* F.comp G *is again a stable-prefix map: $f = G.f \circ F.f$ over* $(\text{Rest}', \text{MintAt}')$*; H3 chains through both maps'* ext *(an epoch-*1 *mint (π, d) once remapped is an epoch-*2 *mint $(F.f\,\pi, d)$, the delta codeword untouched); H2 chains because order-isomorphisms compose; H1 is derived as always. The no-reclaim special case* Compat *(every F-at-hand coordinate stays G-at-hand after remapping, every F-mint stays a G-mint) collapses the four clauses to two at F's full domain. The n-fold closure* chainSPM *composes a whole* CompatChain *of maps into one, carrying the earliest map's domain. `CompatOn`, `StablePrefixMap.comp`, `chainSPM`*

Theorem 26.15 (the n-epoch headline)**.** *For a compatibility chain* CompatChain $(F :: F_s)$*, cut state s at hand for the first map F, and continuation τ declared at F:*

$$\begin{aligned}&\text{query}\big(\text{applySeq}\ (\text{remap}_{\text{chainSPM}\,(F::F_s)}\ s)\ (\tau\ \textit{translated through every epoch's map})\big)\\ &\quad = \text{query}\big(\text{applySeq}\ s\ \tau\big)\ :\end{aligned}$$

after arbitrarily many settled-cut compactions, a beyond-all-cuts continuation, its lagging ops translated through every epoch map, reads exactly as the uncompacted run. (The fold-congruence twin is `multiEpoch_applySeq`; both are direct re-invocations of T1/T2 on the composite.) `multiEpoch_settled_reads`

The subtlety that makes this non-trivial is *rank reclaim*, pinned by the first of two companion negatives; the second records the runtime twin's erratum.

Proposition 26.16 (rank reclaim defeats naive composition)**.** *Unary code. With the epoch-*1 *map* gc_1 *(*enc $3 \mapsto$ enc 1, enc $7 \mapsto$ enc 2*) and the reclaiming epoch-*2 *map* gc_2 *(*enc $2 \mapsto$ enc 1*):*

$$(gc_2 \circ gc_1)(\text{enc } 3) = (gc_2 \circ gc_1)(\text{enc } 7) \quad \textit{while} \quad \text{enc } 3 \neq \text{enc } 7 :$$

a dead coordinate and a live one collide on the reclaimed rank, H1 fails, and the composite is not a stable-prefix map on the full first-epoch domain. This is exactly why comp *carries the surviving domain.* `naive_composition_collides`

Proposition 26.17 (the id-addressing erratum)**.** *After epoch one a renumbered coordinate no longer telescopes to its event id: with the unary telescope decode* unaryId *(the count of* 1*-bits),* $\text{unaryId}(\text{enc } 7) = 7$ *but* $\text{unaryId}(gc_1(\text{enc } 7)) \neq 7$*, and an id-addressed second cut at* $\{7\}$ *resolves to no record of the renumbered state while the coordinate-addressed cut resolves the record exactly. Second cuts must be coordinate- or record-addressed.* `id_addressing_breaks`

The diamond: incomparable cuts, and the barrier removed. Everything above composes cuts that are *comparable* (each a refinement of the last). The distributed question is the incomparable case: two replicas compact at settled cuts S_1 and S_2 that neither refines, then merge. The result is that they merge with *no* coordination, and the finding is that this is not a new algebraic obstruction at all. Since $S_1 \subseteq W$ and $S_2 \subseteq W$ for the join $W = S_1 \cup S_2$, each leg $S_i \to W$ is a comparable (nested) step, hence a genuine StablePrefixMap by the same comp closure; the only new content is that the two legs and the one-shot W-compaction *agree*, and agreement is determinism of the W-map in its certificate. The join map is produced by a builder that is a function of a JoinCert alone (surviving coordinates, settled-dead, declared mints), so two replicas that computed the same certificate install the same map:

$$c_1 = c_2 \;\Rightarrow\; \mathcal{C}(c_1) = \mathcal{C}(c_2) \qquad (\texttt{certMap\_deterministic}).$$

This `congrArg` *is* the "without coordination" content: equal certificates give equal maps by definition, not by reconciliation. Agreement then upgrades to bit-identical states.

Theorem 26.18 (the epoch diamond at $s1$)**.** *For maps* $\text{spm}_{S_1}, \text{spm}_{S_2}, \text{rel}_A, \text{rel}_B, \text{spm}_W$ *and a carried domain* D*, if every record of the cut state* s *has its coordinate in* D*, and both relative composites agree with the one-shot on* D *(*$\text{rel}_A(\text{spm}_{S_1}\, c) = \text{spm}_W\, c$ *and* $\text{rel}_B(\text{spm}_{S_2}\, c) = \text{spm}_W\, c$ *for all* $c \in D$*), then the two-leg compactions and the one-shot compaction produce bit-identical states, not merely equal reads:*

$$\text{remap}_{\text{rel}_A \circ \text{spm}_{S_1}}\ s = \text{remap}_{\text{spm}_W}\ s = \text{remap}_{\text{rel}_B \circ \text{spm}_{S_2}}\ s,$$

and their reads coincide.

`diamond_confluence` *(via* `diamond_confluence_comp` *through* comp*)*

The one genuinely new obstruction is the carried domain. It must be the join cut's kept id-set transported to epoch-0 coordinates *by record identity*, $\text{transportedDom}\ s\ \text{kept}_W = \{\, c : \exists x \in s,\ \text{kept}_W(x.\text{id}) \wedge x.\text{coord} = c \,\}$ (`transportedDom`, coordinate-nodup on a sorted state by `transportedDom_coords_nodup`, so it never decodes a renumbered coordinate and the id-addressing erratum does not apply). The naive membership pullback aliases, the incomparable-cut twin of Proposition 26.16:

Proposition 26.19 (naive pullback aliases)**.** *Unary code. A dropped coordinate falls through a leg's map verbatim onto a kept later coordinate:* (rel ∘ *gc*)(enc 3) = (rel ∘ *gc*)(enc 4) *while* enc 3 ≠ enc 4. *Intersecting the two legs' kept sets would admit the aliased coordinate; the record-identity transported domain excludes it.* ***naive_pullback_aliases***

Two more obligations close the protocol. Dropping a superseded epoch's map is a fold no-op once that map fixes every live coordinate and every continuation anchor (the ack-plus-`AllHeardSince` certificate supplies exactly these hypotheses; the ack-only shortcut is machine-refuted as its FAIL companion):

$$\text{applySeq } (\text{remap}_F\ s)\ (\tau.\text{map remap}_F) = \text{applySeq } s\ \tau \qquad (\texttt{mapDrop\_sound}),$$

and continuation freshness survives the join epoch (dense renumbering never grows a delta, so a fresh delta stays key-distinct from every renumbered ordinal, `contOK_root_key_fresh`, `rED_fresh_dominates_join`) — the theorem face of the 11029/0 `ContOK` observation in the model harness. The model (`whiteboard/epoch_diamond_check.py`) validated the whole diamond at $s1$ across 2400 trials before any of this was stated.

Remark 26.20 (what the runtime realizes, and an honest gap)**.** *The shipped runtime (`runtime/src/epoch.js`, task #112) turns the per-replica epoch integer into a cut-indexed DAG and replaces the old cross-epoch throw with the certificate-determined join. Building it surfaced that the model's "relative-compact each side to W, merge in epoch W" is unsound under partial straggler visibility: a settled record's group freezes on the side holding a straggler and renumbers on the side without it, so the two sides disagree on shared coordinates. The sound protocol lifts both heads down to their common base (the greatest lower bound) via inverse coordinate maps and merges there; this is read- and fold-equivalent to the W-merge by Theorem 26.18, and coordination-free (both sides lift deterministically to the same frame). The merged head therefore de-compacts to the base frame, recovering compactness only on the next certified compaction: a compaction frame is minted solely by the content-addressed `compactStable`, never re-derived at a merge from divergent local holdings, which is arguably the correct discipline rather than a compromise. The twin property-based test is clean across the* 2000*-trial validation sweep, of which* 600 *are committed to CI (`runtime/test/epoch.test.js`).*

Freshness across epochs is representation-agnostic (`rED_le_self`, `rED_fresh_dominates`: a delta fresh against original sibling deltas stays fresh against renumbered ordinals without inspecting their meaning), so the order-iso half composes whether stored deltas are timestamp differences or epoch-k ordinals. The `StablePrefixMap` bundle is not text-specific: the rich-text marks layer instantiates the same interface a second time as its keep-set relabeling, and composes through the same route (§27).

The honesty-rebasing lemma (∗), closed. Two residues above — the single-epoch honesty re-basing (T3's gap: the re-coded history is honest only *modulo* the coordinate order-embedding) and the multi-epoch CompatChain residue — are the *same* obligation. Its causal-stability half (`nested_cuts_settled`, from settledAt_of_allHeard and cut-antitonicity of the frontier, `EmbedRGA_CompatChain.lean`) and its order-iso freshness half (`rED_hocc_fresh`) closed first; what remained was one lemma, which we called (∗): exhibit an epoch-2 configuration honest *modulo* the order-embedding F_1, so that anchored-existence re-derives at the epoch boundary. (∗) is now discharged (`EmbedRGA_HonestyRebase.lean`, kernel-clean), and the shape of the discharge is itself the finding: honesty is not *re-derived* at epoch 2, it is *re-based*. The invariant

that crosses the boundary is not EHonest (false of a compacted state, whose coordinates are no longer birth-chain telescopes) but a history-free *coded-forest* invariant I over the live coordinate set: honest configurations satisfy I (S1), every chain-aligned stable-prefix map preserves I (S2), and I alone discharges the epoch-boundary obligations of the next epoch's map (S4), with no epoch-2 honesty assumption anywhere.

Definition 26.21 (the coded anchored forest invariant I)**.** *For a code* Γ *and a list* L *of live coordinates,* $I(L)$ *(*CodedAnchoredForest Γ L*) has two fields: (nodup)* L *is duplicate-free; (decode) every* $c \in L$ *is* $\mathrm{coordOf}_\Gamma(ch)$ *for some nonempty positive birth chain* ch*. History-free: a predicate on the coordinate set alone, which is the whole point — a re-coded configuration has no honest history but still carries* I*. (The nearest-live-ancestor forest is reconstructed from* I *by prefix-freeness alone:* `caf_ancestor_factor`*,* `caf_ancestor_shorter`*,* `caf_chain_inj`*.)* `CodedAnchoredForest`

Theorem 26.22 (S1: honest configurations satisfy I)**.** *Hypotheses:* GenHonest *at* eApplicable*;* CausalPastEnumerable C*; a well-formed enumeration* EWf Γ ρ*; every op of* ρ *in* C.events*. Conclusion:* I *of the live coordinate set of* eFold_Γ ρ*. (The anchor certificate gives each live insert's coordinate as its chain's; nodup is sortedness; nonemptiness is the nonzero id.)* `ehonest_implies_I`

Call a map F *chain-aligned* via φ (`ChainAligned`) when $F.f(\mathrm{coordOf}_\Gamma\ ch) = \mathrm{coordOf}_\Gamma(\varphi\ ch)$ on positive chains and φ preserves positivity and nonemptiness: the stable/unstable split lands on a codeword boundary and the tail rides verbatim. Both shipped passes are chain-aligned (`chainAligned_compactSPM`, `chainAligned_fuseSPM`), and chain-alignment composes (`chainAligned_comp`).

Theorem 26.23 (S2, load-bearing: stable-prefix maps preserve I)**.** *Hypotheses:* F *chain-aligned via* φ*;* $I(L)$*; every* $c \in L$ *at hand for* F*. Conclusion:* $I(L.\mathrm{map}\ F.f)$*. Decodability survives through* φ*; distinctness survives because* F *is injective on its domain (H1, Lemma 26.2). Honesty rebases: no re-derivation of* EHonest *on the re-coded set.* `stablePrefixMap_preserves_I`

Theorem 26.24 (S4: the epoch boundary discharged from I)**.** *Hypotheses: epoch-1's map* F_1 *chain-aligned via* φ_1*; a domain* $\mathcal{D}_1$ *of positive chains; and the two construction-shape clauses, true of the shipped compaction and fusion maps:* $h_{\mathrm{RestShape}}$ *(every* F_1.Rest *coordinate is* coordOf *of a* $\mathcal{D}_1$ *chain) and* $h_{\mathrm{MintShape}}$ *(every declared mint* (π, d) *has* $\pi = \mathrm{coordOf}(ch_A)$ *with* ch_A *positive,* $ch_A \cdot d \in \mathcal{D}_1$*,* $1 \le d$*). Conclusion:*

$$\mathrm{CompatOn}\ F_1\ (\mathrm{rebaseSPM}\ \Gamma\ \mathcal{D}_1\ \varphi_1\ \ldots)\ F_1.\mathrm{Rest}\ F_1.\mathrm{MintAt}:$$

the epoch-boundary obligations restG/mintG *of Definition 26.14 are met by epoch 2 built over the rebased image domain (*`rebaseSPM`*,* `rebasedDom`*: the* φ_1*-images of epoch-1's at-hand chains), a survivor's image at hand by chain-alignment and a beyond-cut mint's image a declared mint by the snoc-commute (*`chainAligned_snoc`*). No epoch-2 honesty assumption; this is* $(*)$*, closed. (*I *is also exactly the at-hand certificate a next-epoch compaction's domain needs:* `I_implies_atHand`*, S3.)* `twoEpoch_compatOn_of_I`

Corollary 26.25 (I rebases across two epochs)**.** *For* F_1*,* F_2 *chain-aligned via* φ_1*,* φ_2*,* $I(L)$*, and the two domain coverages:* $I((L.\mathrm{map}\ F_1.f).\mathrm{map}\ F_2.f)$ *— the invariant survives both compactions, so the second epoch operates on the* I*-carried rebased domain, not by transport through the first.* `I_preserved_two_epoch`

What is left downstream is the protocol layer: deriving the per-continuation hypotheses from a delivery protocol, and everything multi-replica (agreement on a common cut, concurrent divergent compaction), Open Questions 43.7, 43.8.

Measurement 2: real traces, and where the bits actually sit. The full epoch stack (rank renumbering plus *spine fusion*: collapse each maximal settled dead chain with exactly one child branch and no in-flight anchor into a single level) was run over the standard collaborative-editing replay corpus (the josephg traces). Accounting: per record, the sum over its kept-chain levels of $|D(\delta)|$ bits, D the flipped Elias-δ code of §20; totals divided by live characters; character payloads excluded; "fused" is after the renumber-plus-fusion epoch at a settled cut.

trace (one-sided)	chain b/ch, raw	chain b/ch, fused	ratio
friendsforever	1622.5	856.5	1.9×
clownschool	2488.7	1471.0	1.7×
seph-blog1	2974.9	917.2	3.2×
automerge-paper	2304.5	1279.4	1.8×

Three findings, in causal order. *(i)* Rank renumbering alone recovers only 1.1–1.8×: typing runs mint δ=1 chains, and every level costs at least one bit even at ordinal 1, so the residual cost is *spine depth*, which is what fusion owns. *(ii)* Fusion then removes essentially everything removable: surviving dead levels cost 3 to 76 bits per character in total, and the fused column equals the code cost of the *live tree shape* alone, verified by a per-level attribution that accounts for every bit. *(iii)* The few-bits-per-character hope fails for a reason worth the number: the live shape is itself expensive. Real traces anchor typing runs in *live* chains of mean depth 835 to 1468 levels at about one bit per level, and 97 to 99.8 percent of the post-fusion bits sit on live ancestor levels that no epoch map may touch (they are the retained semantics, §25). History independence holds three ways (staggered cuts, one final cut, and from-scratch compaction land on bit-identical totals), and the fused display spells the trace's final content on every sequential trace. Conclusion: the epoch stack removes the dead-history share; what survives is a *representation* cost, which is the run table's brief (§26.3).

Measurement 3: the sided instantiation. The same stack, band-aware (renumber within each parent-band group; fusion carries the head's band and codeword), replayed under the Fugue policy. Same accounting, plus one side bit per level, which is precisely the point:

trace (sided, Fugue)	chain b/ch, raw	chain b/ch, fused	ratio
friendsforever	2582.0	1735.7	1.5×
clownschool	4058.5	2988.1	1.4×
seph-blog1	5559.1	1919.8	2.9×
automerge-paper	4448.0	2583.7	1.7×

The ratios sit *systematically below* the one-sided 1.7–3.2×, and the naive expectation that the ratio would be preserved is falsified by its own arithmetic: adding the same flat per-level side cost to numerator and denominator pulls the quotient toward the *level-count* ratio, which is below the payload ratio. Where the sides pay: the fused sided state costs about twice the fused one-sided state, and the premium is *entirely the flat side flag*; the sided delta payload exceeds the one-sided payload by only 1.0 to 4.7 percent (on these traces the Fugue tree is payload-equivalent

to RGA's). The flag costs a flat 1.000 bit per level against a true side entropy of 0.19 to 0.27 bits (L-mints are 3.0 to 4.6 percent of mints under Fugue on real traces): a **4–5× overpay** on the side channel, load-bearing for lexicographic band separation, and 94 to 98 percent of the side bits sit on live levels no epoch map can touch. Both residues (live depth, flat flags) point at the same lever: a different representation of the same immutable chain data.

26.3 The run table: the representation lever

Runs. Work over the *kept tree*: the live records plus their dead ancestors (every other node has been removed outright; tombstone-freedom is unchanged). Call the edge from node p to child c *fusible* when (1) c is the unique kept child of p, (2) $\delta(c) = 1$, (3) c's side is R (vacuous one-sided), and (4) c and p have the same liveness. A *run* is a maximal chain of fusible edges. Fusibility is a per-edge predicate and condition (1) gives each node at most one fusible out-edge, so the runs are a canonical partition of the kept tree into chains: a function of the state, independent of arrival order. A table entry is one run, with header (liveness flag, parent entry, offset in parent, side, head delta, length); members after the head implicitly have delta 1 and side R, and liveness is uniform along a run. A record's address is *(run id, offset)*. Dead entries carry no records, only the structure live descendants' coordinates pass through. The design's starting observation: what typing mints *is* exactly a fusible chain, so real documents decompose into few long runs.

Definition 26.26 (the run table)**.** *Over the kept tree of a live chain set L (`keptL`: the live chains plus their nonempty prefixes, deduplicated), the edge into c is fusible (`fusible`) when c and its parent are kept, c's label is* 1*, c is the parent's unique kept child, and c and the parent have equal liveness ($c \in L \leftrightarrow c.\mathrm{dropLast} \in L$); a head (`isHead`) is a kept node whose incoming edge is not fusible. Runs are the fibers of the climb `headOf`; every member is its head extended by a block of* 1 *labels. An entry (`RTEntry`) is the header* $\langle \mathrm{par} : \mathrm{Option}(\textit{parent index} \times \textit{offset}),\ \mathrm{live},\ \delta,\ \mathrm{len} \rangle$*; the table (`tableOf`) maps the canonical head enumeration (`headsList`, sorted by the short-lex `hLt`: shorter first, so parents precede children, `keyLt` within a length) through the entry compiler (`entryOfHead`); the abstraction (`stateOf`) recovers each live entry's members from headers alone. `Canonical` recognizes the image: positive lengths, backward parent references, tail attachment (`par_tail`), maximal fusion (`no_fuse`), dead leaves pruned (`leaves_live`), sorted head enumeration.* `RTEntry`, `tableOf`, `stateOf`, `Canonical`

Two properties before any numbers. *No stability gate*: the run table is an invertible re-representation of the current state, whatever that state is, unsettled suffixes included, and in particular the freshly typed hot end of the document where the depth cost is being minted; the PBT exploits this by rebuilding the table mid-flight between merges after every event. *Lossless in the strong sense*: from the table alone one recovers every kept node's delta (head delta from the header, 1 elsewhere), side, liveness, and pre-epoch timestamp (deltas summed along the contracted path), hence every coordinate, hence the state; the harness gates exactly this, with zero tolerance, on every record of every trace.

Theorem 26.27 (tail attachment)**.** *In the canonical table, every entry attaches at its parent entry's last member. Kernel form: if h is a head, the edge into $h.\mathrm{dropLast} \cdot 1$ is not fusible,*

$$\mathrm{isHead}\ L\ h = \mathrm{true} \implies \mathrm{fusible}\ L\ (h.\mathrm{dropLast} \cdot 1) = \mathrm{false}.$$

(If an entry attached at an interior member m, then m's run successor and the entry's head would be two kept children of m, contradicting fusibility of m's outgoing run edge.) No honesty, no stability, no reachability hypothesis. `tail_attachment`

Three consequences: the "offset in parent" header field is derivable (always parent length minus one; the accounting still counts it and reports it recoverable); the cross-run comparator only ever diverges at a tail or inside a shared run, so it consults entry chains and header fields only and *never materializes a coordinate*; and it is cheap to assert, which the harness does on every table built.

The comparator. Within a run, display order is offset order (each member is the R delta-1 child of its predecessor, hence displays directly after it). Across runs, compare the two entry chains from the root; by the lemma every step exits at a tail, and at the first difference either one chain is a prefix of the other (the deeper record is inside the shallower's continuation subtree; one-sided the shallower displays first, sided an L branch displays first) or two branch entries hang at the same tail and the branch headers decide (one-sided newest first; sided by the banded order). The $(\text{ts}, \text{agent})$ concurrency tiebreak is *not* encoded in either representation's counted bits; the tie oracle rides outside the representation on both sides and is charged to neither, preserving parity with the chain measurements. The cost of a comparison is the *contracted* depth: one entry per run on the path instead of one level per node.

Mutation, and the forced coalesce. Splits mutate the table, never the semantics: stored chain data is immutable, only the partition changes. Insert at anchor a: if a is interior, split its entry after a; attach the new record as a singleton; then *coalesce* (a fusible singleton that is the tail's unique attachment fuses into the parent entry), which is how typing extends a run in $O(1)$ table work. Delete of x: split to make x a tail; if x still has kept children, carve it out as a dead singleton (a liveness split); childless, it simply vanishes, dropping newly unkept dead tails upward and re-coalescing the parent with a now-unique fusible child. Coalesce is the inverse of split *and it is not optional*: without it, deleting a concurrent interloper leaves the two halves of the old run split forever and the table drifts from the canonical partition (found during design, pinned by a directed case and by the PBT's incremental-equals-canonical-rebuild gate after every event). Delivery under a vanished anchor (the anchor chain passed through nodes deleted while childless here) re-materializes the missing levels as dead entries from the op's carried coordinate: the run-table cost of tombstone-freedom, paid with information the embed op already carries. Merges deliver the other side's events through the same two rules; epochs rebuild the table over the compacted tree (sound because the table is a function of the state).

The accounting model. Applied identically to both representations; every pointer and every header bit counted, no hidden structure. D is the flipped Elias-δ code; $W = \lceil \log_2(N_{\text{entries}} + 1) \rceil$ is the id width.

$$\begin{aligned} \text{per record:} &\quad W + |D(\text{offset} + 1)| \\ \text{per entry:} &\quad 1 + W + |D(\text{poff} + 1)| + |D(\text{head } \delta)| + |D(\text{len})| \quad (+\ 1 \text{ side bit, sided}). \end{aligned}$$

Totals are per live character, reported with the full per-field breakdown so every number is auditable back to a field. Not charged on either side: the tie oracle (above). Two fields are counted although the canonical table makes them recoverable, and the breakdown lets a reader subtract them: the parent offset (the tail-attachment lemma) and the per-record run id when records are stored grouped by run (it is then positional).

Measurement 4: the run table. Same traces and replay as measurements 2 and 3. Columns: the raw chain representation ("chain"), the chain after its best epoch ("fused," the previous subsection's current best), the run table over the *raw uncompacted* chains with no epoch and no settled cut ("rt"), and the run table rebuilt after the epoch ("rtc").

trace	family	chain	fused	rt	rtc	fused/rtc
friendsforever_flat	1-sided	1622.5	856.5	22.13	22.30	38.4×
friendsforever_flat	sided/F	2582.0	1735.7	22.54	20.98	82.7×
clownschool_flat	1-sided	2488.7	1471.0	22.42	22.64	65.0×
clownschool_flat	sided/F	4058.5	2988.1	24.11	21.20	140.9×
seph-blog1	1-sided	2974.9	917.2	26.00	23.16	39.6×
seph-blog1	sided/F	5559.1	1919.8	27.21	24.99	76.8×
automerge-paper	1-sided	2304.5	1279.4	23.76	23.92	53.5×
automerge-paper	sided/F	4448.0	2583.7	25.21	24.58	105.1×
friendsforever	1-sided	1243.2	856.5	20.92	22.30	38.4×
friendsforever	sided/F	2189.2	1735.6	22.36	20.98	82.7×
clownschool	1-sided	1863.1	1471.0	20.85	22.64	65.0×
clownschool	sided/F	3421.3	2988.1	22.29	21.20	140.9×

All values in bits per live character under the accounting model above; the `_flat` traces are the concurrent traces replayed single-author, their unsuffixed twins the concurrent originals. The verdict: **20.9 to 27.2 bits per character, 38 to 141 times below the fused chains**, confirming the design prediction (per-record cost collapses from Θ(depth) to $O(1)$ amortized plus a log-sized offset) with margin. Where the remaining bits sit, from the audited breakdown:

- The per-record *run id* now dominates (10 to 14 of the 21 to 27 bits, the flat id width): the deep-chain cost did not shrink to the offset code, it shrank to *one pointer*. An implementation storing records grouped by run makes this field positional and free, putting totals at 9 to 14 bits per character; the model charges it because the pre-registered accounting does.
- Offsets cost 5.2 to 12.1 bits per character; all header fields together amortize to 0.6 to 6.4.
- The side channel collapses from one flat bit per live level (867.8 to 1494.1 bits per character after fusion) to one bit per *entry*: **0.061 to 0.174 bits per character**, three to four orders of magnitude, and below even an entropy-coded per-level flag (H(side) times 850 to 1500 levels); the explicit L-entry-list alternative is computable from the reported entry counts and is dearer at these sizes.
- The epoch stack becomes *bit-wise marginal* under this representation: the composed column beats raw by at most 2.9 bits per character (sided) and loses to raw by up to 1.8 (one-sided, where longer post-renumber runs make offsets dearer than the headers they save). The run table alone, with no settled cut at all, already delivers the collapse. What the epoch still buys is *structure*, not bits: entries drop up to 7×, the id width shrinks, raw cursor-jump boundaries vanish under renumbering, and the contracted depth drops up to 11.8×.
- The honest limit: the *bit* cost per record is $O(1)$ amortized plus the offset code, but the comparator's *time* is still Θ(contracted depth) per cross-run comparison (mean 19.6 to 261 runs per path against chain depths of 850 to 1500). Collapsing comparison time further (an order index over the contracted tree) is a separate, orthogonal layer; nothing here needs it for the metadata claim.

Gates, all passing: display identity on all six traces, both families, raw and composed (walk order equals the chain display; text equals the final content); about 198k sampled pairwise comparator verdicts matching display positions with antisymmetry checked; losslessness exact on every record; the tail-attachment invariant on every entry of every table; and a 150-trial PBT (8,186 events, three replicas, concurrent edits and merges) gating walk identity, the all-pairs comparator, and incremental-equals-canonical after every event, while exercising 872 merges, 315 mid-run splits, 438 liveness splits, 551 coalesces, and 26 vanished-anchor materializations.

The Lean mechanization of this subsection, owed in the previous revision, is now in place (`RunTable.lean`, state-level and hypothesis-free except where noted); the load-bearing kernel is Theorem 26.27. Only the epoch-composition clause (`runTable_epoch`: sorted state, at-hand domain, chain representation; walk and read identity after the epoch) carries a stability hypothesis. Task #73 has since closed the remaining theorem bill: the four mutation rules are promoted from SPOT-pinned perturbation lemmas (canonicalization under mutation: `tableOf_congr`) to *closed* `tableOf` equations, and the walk is promoted to a literal, table-direct reading procedure. The sided run table has its own subsection (§26.4); the one face still open is the wire format (Open Question 43.14).

Theorem 26.28 (the representation iso, over labels and liveness)**.** *Losslessness: for* $[] \notin L$, stateOf(tableOf L) *is duplicate-free and membership-exact,*

$$(\text{stateOf}(\text{tableOf } L)).\text{Nodup} \ \wedge \ \forall c,\ c \in \text{stateOf}(\text{tableOf } L) \leftrightarrow c \in L$$

(`stateOf_tableOf`; this membership form is the load-bearing declaration, its unconditional sibling `stateOf_tableOf_eq` being the flatMap computation the proof runs on). Canonicity: for Canonical T, tableOf(stateOf T) $= T$, *field for field (`tableOf_stateOf`); without `no_fuse` this direction is false (a run split in two re-canonicalizes to one entry, the drift the coalesce rule exists to prevent). Image: every* tableOf L *is canonical (`canonical_tableOf`), so the pair is a genuine bijection between live-chain sets and canonical tables. Stated over labels, sides (vacuously R), and liveness, not timestamps: the exact correction the implementation forced, rank ordinals forgetting time after the first epoch.* `stateOf_tableOf`, `tableOf_stateOf`, `canonical_tableOf`

Theorem 26.29 (the contracted comparator equals the key order)**.** *For every ordered prefix code* Γ*: if every chain of* L *is positive, and* x, y *are kept nodes with* $x \neq y$, *then*

$$\text{cmpTable } L\ x\ y \ = \ \text{keyLt } (\text{key}(\text{coordOf}_\Gamma\ y))\ (\text{key}(\text{coordOf}_\Gamma\ x)),$$

the display comparator of the coded fold. The chain form (`cmpTable_iff_chainBefore`) is the two-case rule of the design prose; the label order stands where the (ts, agent) *tie oracle rides, on both sides identically.* `cmpTable_eq_keyLt`

Theorem 26.30 (the structural walk is the display)**.** *For* $[] \notin L$*:* walk L *is duplicate-free, contains exactly the live chains, and is pairwise in* chainBefore *order (via `walkE_sound`/`walkE_complete`), which pins it to the display sequence, strict orders having unique sorted enumerations. Key form: descending in* keyLt *on the coded coordinates for every ordered prefix code (`walk_pairwise_keyLt`, adding the positivity hypothesis). No stability hypothesis.* `walk_display`

The mutation rules, as closed equations. Each rule is now an equation: `tableOf` of the mutated state equals an explicit rewrite of pre-mutation data. New heads enter the enumeration by sorted insertion, removed heads leave by filtration, surviving entries keep their fields except the announced parent rehoming and length arithmetic, and parent references are re-indexed into the new enumeration (itself closed, being the equation's own head list). No stability or honesty hypothesis appears in any of the four. The engine is a pair of characterization lemmas that force the right-hand sides rather than check them: any strictly sorted enumeration of the heads *is* the canonical head list (`headsList_eq_of_iff`), and a run's length follows from an explicit offset enumeration of its members (`lenOf_eq_of_iff`). One equation displayed, the other three grouped.

Theorem 26.31 (D1: the mid-run split, closed)**.** *Hypotheses, exactly two:* fusible $L\ (a{\cdot}1) = \text{true}$ *(the anchor a is run-interior: its run successor is fused) and* $a \cdot d \notin \text{keptL}\ L$ *(the delivered record is fresh); $d \neq 1$ is derived (`splitc_d_ne_one`), not assumed. Writing* $g := \text{headOf}\ L\ a$, $j := |a| - |g|$ *(the anchor's offset),* $\ell := \text{lenOf}\ L\ g$*: the new table* $\text{tableOf}(L + [a \cdot d])$ *is the old one with*

g :	$\text{len} := j + 1$ *(the anchor run cut at the offset)*,
$a \cdot 1$:	$\langle\, \text{par} = (g,\, j),\ \delta = 1,\ \text{len} = \ell - j - 1 \,\rangle$ *(the suffix, re-headed)*,
$a \cdot d$:	$\langle\, \text{par} = (g,\, j),\ \delta = d,\ \text{len} = 1 \,\rangle$ *(the fresh live singleton)*,
h attached in g's run :	$\text{par} := (a \cdot 1,\ \text{off} - (j{+}1))$ *(rehomed to the suffix)*,
every other h :	*unchanged*,

the two new heads entering at their sorted positions and the suffix keeping the run's (uniform) liveness; by tail attachment the rehomed entries are exactly those at the old tail. (The childless-anchor happy path stays the O(1) typing equation, grouped below.) `tableOf_insert_split`

Remark 26.32. *Display dictionary: in the mechanized equation the new enumeration is literally* $(\text{headsList}\ L + [a \cdot 1, a \cdot d])$ *re-sorted, parent references re-indexed by index-of into it, and the suffix liveness is $a \cdot 1 \in L$; the table above is that term with g, j, ℓ substituted. Nothing else is elided.*

Theorem 26.33 (the remaining closed equations: typing, D2, D4, D5)**.** *Each is an equation with all right-hand data pre-mutation; none carries a stability or honesty hypothesis.*

- *Typing (`tableOf_insert_extend`). Hypotheses: $a \in L$, $[] \notin L$, a childless in the kept tree ($\forall d$, $a \cdot d \notin \text{keptL}\ L$). Appending the delta-1 child re-canonicalizes to the same table with ONE* len *field bumped, at* $\text{headOf}\ L\ a$*: O(1) table work per keystroke.*

- *D2, the delete liveness split (`tableOf_delete_split`). Hypotheses: $x \in L$, both edges of x fused (*$\text{fusible}\ L\ x$, $\text{fusible}\ L\ (x{\cdot}1)$*), and $L' = L$ minus x (membership-exact). The kept tree is untouched; the run cuts in three at the liveness break: the live prefix keeps the old head with length cut to x's offset, x survives as a dead singleton entry, the live suffix re-heads at $x \cdot 1$, and old-tail attachments rehome to the suffix. The boundary shapes (x a run head, x the tail) split in two rather than three and are the same construction with one empty side.*

- *D4, the coalesce (`tableOf_coalesce`), the closed delete-inverse of D1. Hypotheses: $a \neq []$, $d \neq 1$, the interloper $a \cdot d \in L$ and childless in the kept tree, the suffix $a \cdot 1$ kept with a's liveness ($a \cdot 1 \in L \leftrightarrow a \in L$), a's only kept children $a \cdot 1$ and $a \cdot d$, and $L' = L$ minus $a \cdot d$. Both the interloper's entry and the suffix entry are removed, the anchor's run absorbs the suffix (*len *the sum of the two), and suffix-tail attachments rehome into the merged run.*

- *D5, directed materialization (`tableOf_materialize`). Hypotheses:* $[] \notin L$*, the vanished anchor's parent* m.dropLast $\in L$ *and childless in the kept tree. Delivering* $m \cdot d$ *inserts two singleton entries at their sorted head positions, the dead materialized anchor* m *and the live record below it, parent references re-indexed. This is the directed single-anchor shape; the general vanished-path cascade (a whole dead spine re-materializing, with cuts at non-unit labels and a possible mid-run split at the re-attachment point) is deliberately presented as the composite of this rule, applied once per vanished level, with D1-shaped splits: a genuine cascade, not forced into one equation.*

The necessity of D4 is pinned by `spot_d4_coalesce_necessity`. `tableOf_insert_extend`, `tableOf_delete_split`, `tableOf_coalesce`, `tableOf_materialize`

The literal walk. The structural walk behind Theorem 26.30 still consults the state. Its promoted form, `tableWalk`, consults nothing but the table: entry headers, parent references, and address arithmetic (each entry's head recovered by summing deltas along the contracted parent path, each run emitted as that head followed by its offset extensions, recursion into the entries attached at the run's tail, newest label first). `stateOf` is never materialized, and the recursion fuel (one more than the total of the length fields) is proved sufficient.

Theorem 26.34 (the literal walk is faithful)**.** *For* $[] \notin L$*:*

$$\text{tableWalk}\ (\text{tableOf}\ L)\ =\ \text{walk}\ L\,:$$

build the table, read it back table-directly, obtain the display of the live-chain set verbatim. The simulation at matched fuel is `walkT_eq_walkE`*; the canonical-table form is* `tableWalk_eq_walk`*. No stability hypothesis.* `tableWalk_tableOf`

This is the formal face of the reading claim the measurement bullets priced: a full read is one pass over the entries in walk order, emission per record with no coordinate materialized. The Θ(contracted depth) comparator time of the honest-limit bullet above is a *random-access* cost; a sequential read never pays it, because the table itself is the order index. The instance file pins the generic procedure on the D1 state (`spot_d1_tableWalk_generic`: the concrete SPOT and the faithfulness theorem name the same reading procedure).

26.4 The sided run table: the same lever under Fugue

Measurements 3 and 4 already ran the sided (Fugue) family through the same accounting and reported the decisive number: the side channel collapses from one flat bit per live level to one bit per entry, 0.061 to 0.174 bits per character. This subsection is that family's verified counterpart (`SidedRunTable.lean`): the full mirror of the one-sided development of §26.3, over the sided chain kernel of §23, state-level and hypothesis-free throughout (no stability gate, no honest-delivery hypothesis).

Entries, runs, and where the L data lives. A sided entry adds one field to the one-sided header: (parent entry and offset, liveness, *side*, head delta, length). Fusibility condition (3) of §26.3 (the child's side is R), vacuous one-sided, is now live.

Theorem 26.35 (runs are uniformly R; L nodes are heads)**.** *A fusible sided edge's last step is* $(R, 1)$ *with the parent's liveness:* sfusible $L\ c$ = true *implies the side of* c*'s last element is* R

(`sfusible_side_R`), so runs are uniformly R and member sides cost nothing. And every kept node whose last element is L-sided fails condition (3) (`Lentry_not_fusible`), hence is a head with its own explicit entry: `Lentry_isHead`, with hypotheses $c \in$ skeptL L and c's last side $= L$. No further hypotheses. `sfusible_side_R`, `Lentry_isHead`

The entire L content of a state is therefore the side bits of the entry headers, which is the verified shape of the measured collapse: the representation stores L data nowhere else.

The canonical order and the iso. The head enumeration is sorted by `shLt`, short-lex over an *injective* flattening of the sided word stream: each head chain flattens to its alternating (side, delta) word, injectivity makes the order strict and total, parents strictly precede children (flattened lengths double), and the tie-break is inherited wholesale from the one-sided head order. Canonical tables (`SCanonical`) carry the side-aware `no_fuse` condition: only an R, delta-1, same-liveness *lone* attachment counts as a missed fuse, so an L singleton hanging alone at a tail is canonical.

Theorem 26.36 (the sided representation iso)**.** *Losslessness: for* $[] \notin L$, sStateOf(sTableOf L) *is duplicate-free and membership-exact for* L *(`sStateOf_sTableOf`). Canonicity: for* SCanonical T, sTableOf(sStateOf T) $= T$, *field for field, the side field included (`sTableOf_sStateOf`). Image: every sided table of a state is canonical (`scanonical_sTableOf`). Over labels, sides, and liveness; deliberately not over timestamps, for the same reason as one-sided: rank ordinals forget time, so a timestamp-preserving iso is false after the first epoch, and the iso is stated over exactly what survives one.* `sStateOf_sTableOf`, `sTableOf_sStateOf`, `scanonical_sTableOf`

The comparator: the offset-vs-tail exit-side rule. The two-case structure of §26.3 survives, licensed by the sided tail-attachment lemma (`stail_attachment`), and the divergence case is still a header verdict: two branch entries at the same tail compare by the banded entry order (L before R; among L siblings the older label first, among R the newer first). The new structural content is in the prefix case. One-sided, an ancestor simply displays before its continuation subtree. Sided, the comparator consults the ancestor's offset against its run length: a strictly interior ancestor precedes its subtree *regardless of the exit side*, while exactly at the run tail the exit side decides (an R exit displays after the ancestor, an L exit before). This is the run-table shadow of the kernel's in-order display rule: runs are uniformly R, so strictly inside a run the only continuation is the $(R, 1)$ step toward the tail, an R extension, and the ancestor wins; an L edge can hang only at a tail, where every attachment lives, so sidedness surfaces exactly there.

Theorem 26.37 (the sided comparator equals the sided key order)**.** *For every ordered prefix code* Γ*: if every chain of* L *is a positive sided chain, and* x, y *are kept with* $x \neq y$, *then*

$$\text{scmpTable } L\ x\ y\ =\ \text{keyLt } (\text{sKey}(\text{sidedCoordOf}_\Gamma\ y))\ (\text{sKey}(\text{sidedCoordOf}_\Gamma\ x)),$$

refining the kernel's in-order marker theorem (§23) to the storage layer; the chain form is `scmpTable_iff_schainBefore`. `scmpTable_eq_keyLt`

The cost is unchanged from one-sided: contracted depth, header fields only, no coordinate materialized.

Congruence and the typing equation. `sTableOf_congr`: the sided table is a function of the live sided-chain set, so merges land wherever delivery order put them and the table cannot tell. `sTableOf_insert_extend`: appending an $(R, 1)$ child to a childless live node changes exactly one length field, O(1) sided table work per keystroke, exactly as one-sided.

The in-order walk, and its literal form. The sided walk of one run: interior members in offset order, then the L subtrees at the tail (ascending, older label first), then the tail member itself, then the R subtrees (descending, newer label first); at the virtual root, the L-rooted subtrees ascending, then the R-rooted descending.

Theorem 26.38 (the sided walk is the in-order display)**.** *For* $[] \notin L$*:* swalk L *is duplicate-free, membership-exact for the live sided chains, and pairwise in* schainBefore *order (`swalk_display`); descending in* keyLt *on the sided coordinates for every ordered prefix code (`swalk_pairwise_keyLt`, adding the positivity hypothesis). The literal form, `stableWalk`, consults entry headers, parent references, and the same address arithmetic only, with* sStateOf *never materialized; its faithfulness theorem is `stableWalk_sTableOf`, mirroring Theorem 26.34: build the sided table, read it back, obtain the in-order display verbatim.* `swalk_display`

The all-R lockstep bridge. On an all-R live set the sided development projects to the one-sided one in lockstep.

Theorem 26.39 (the all-R lockstep bridge)**.** *Table half, no hypotheses:*

$$\text{sTableOf } (L.\text{map liftR}) \;=\; (\text{tableOf } L).\text{map liftEntry} :$$

the sided table of a lifted all-R state is the one-sided table with the side bit stamped R on every entry (`sTableOf_liftR`). Walk half, hypotheses every chain of L positive and $[] \notin L$*:*

$$\text{swalk } (L.\text{map liftR}) \;=\; (\text{walk } L).\text{map liftR},$$

consuming the kernel's fragment theorem `schainBefore_liftR` (§23). `sTableOf_liftR`, `swalk_liftR`

This is the storage-layer analog of the convergence-layer "sides are payload" theorem (§30): the one-sided datatype is the L-uninhabited fragment of the sided one, so every one-sided storage result of §26.3 transports to the sided table on that fragment with no re-proof.

Per house discipline the file closes with hand-derived PASS+FAIL executions: the L child that does not fuse where the R child does; the band order pinned both in the entry order and in the coordinate keys; the L-band mint displaying L band, then node, then R band; the band *direction* with its FAIL companion (sorting the L band newest-first, like the R band, provably flips the display: the tempting degenerate order is itself refuted); the table-direct walk on the mint case; and the all-R lockstep pinned on the one-sided D1 state, table half and walk half. Every capstone named in this subsection and in §26.3 passes its `#print axioms` audit at `propext/Classical.choice/Quot.sound` or below; `native_decide` is confined to the SPOTs.

26.5 From projection to shipped bytes, and the cross-system question

Two external facts close the representation arc; both are measured, and both are laid out with their full methodology in the performance section (§41), so only the headline belongs here.

First, the run-table projection of §26.3 is no longer merely measured-in-model: task #104 ships a lossless run-table serializer (`runtime/src/serialize.js`), whose `accountingBits` routine reproduces the projection's bit totals *bit-for-bit* on every trace, while the shipped bytes land *below* the projection (the model charges the recoverable positional fields — per-record run id and offset, and the tail-attachment parent offset — that a real encoder drops). Over a settled-cut compacted state the serializer reaches 1.1 to 1.3 bytes per character on the josephg traces, at production save size and an order of magnitude below the packed absolute-chain estimate. Second, placed against production CRDT libraries this closes the save-size gap to rough parity with the smallest state-only production saves, and it confirms the design's one categorical storage win: tombstone-freedom means the save *shrinks* on delete where the history-carrying systems grow. The full cross-system matrix — per-character apply latency, merge and sync, load, all four save representations, sync payload, and the grows-on-delete reproduction — is developed in §41.

27 The marks layer: document-order rich text, and its GC

Rich text is the family's first *effect-coupled* extension: marks (bold, links, comments) are anchored in the sequence but are not sequence elements. The repository holds two mark encodings. Part III's fused Peritext (§34) encodes a mark as a pair of in-sequence boundary inserts; the model of this section is the other encoding, the one the runtime ships as its third datatype (§40): characters live in the embed order, and marks are *separate immutable records* that name their boundary anchors by id. Two campaigns are recorded here. Task #55 built the read model and its theorems (`Peritext_Embed/PeritextEmbed_MarkIntent.lean`), replacing a retracted claim; task #110 built its garbage collector (design and validation `whiteboard/marks-gc-note.md` with harness `whiteboard/litmus/marks_gc_check.py`; mechanization `PeritextEmbed_MarksGC.lean` and `PeritextEmbed_MarksGC_A3.lean`; runtime `runtime/src/compact-peritext.js`), moving the runtime's Peritext state GC from a justified refusal to a certified fire. Verdicts below are labeled by provenance, as everywhere in this document: *proved* is kernel-checked Lean, *measured-in-model* is the checked-in Python harness, *measured* is the shipped JavaScript runtime's checked-in tests.

27.1 The document-order mark read model

State. Text is an insert-only shadow: a map from character id (a Lamport stamp, globally unique, exceeding the id of the insert's anchor) to its immutable birth coordinate, plus a *grow-only set of deleted ids*. Text deletes are logical because the resolver below needs the birth positions of *dead* anchors; that retention is exactly what the GC half of this section must then contend with. Marks are a grow-only map from mark id *mid* (the same Lamport space) to an immutable record $(mid, mtype, op, startId, endId, startSide, endSide)$ with $op \in \{\mathsf{add}, \mathsf{remove}\}$: a removeMark is a first-class negative record, and per (character, *mtype*) the covering mark with the highest *mid* wins (LWW), a winning remove suppressing the formatting. A per-mark value (a link url, a colour) rides outside the LWW key. Reading order is the embed order of §19 (birth-chain lexicographic, newer children first); the live order is that order filtered by the deleted set.

Boundary resolution. Each boundary resolves to a live position. For a live anchor the side is gravity, and "newer than the mark" means character id $>$ *mid*; *growth is end-side only*, the

one design decision carried in from validation: an end with *endSide* = *after* grows right over the newer-than-mark run (typing at the end of a bold span stays bold), while a *before* start does *not* grow left, else it grabs unrelated newer siblings and diverges from the Peritext paper's semantics. For a dead anchor the boundary *rehomes*: the nearest surviving character in reading order, scanning from the anchor's birth position in the side's direction. The eight cases:

boundary	anchor	side	resolution
start	live	*before*	the span starts at the anchor
start	live	*after*	starts after the anchor, skipping the newer-than-mark run
start	dead	*before*	rehome right, the span starting at the survivor; scan exhausted: the mark collapses
start	dead	*after*	rehome left, then as the live case (the skip applies to the survivor); exhausted: document start
end	live	*after*	ends at the anchor, then *grows* right over the newer-than-mark run
end	live	*before*	ends before the anchor, stepping back over the newer-than-mark run
end	dead	*after*	rehome left, then as the live case (the growth applies to the survivor); exhausted: the mark collapses
end	dead	*before*	rehome right, then as the live case (the step-back applies to the survivor); exhausted: document end

Definition 27.1 (the mark-record state and the document-order read)**.** *The document is* $d = (shadow, deleted)$ *(`DocD`): an insert-only embed state (every insert ever applied, payload the codepoint) plus the grow-only deleted-id list. The birth order is the shadow's id sequence in embed reading order (`DocD.birthIds`); the live order is the birth order filtered by deleted (`DocD.liveIds`). A mark is the immutable record* $(mid, mtype, op, value, startId, endId, startSide, endSide)$ *(`MarkD`; two frozen ancestry snapshots ride along for the retracted tree control alone and are ignored by the resolver). The resolver returns option-valued live indices:* startIncl $d\ m$ *is the first covered index and* endExcl $d\ m$ *the exclusive upper index, each computed by the eight-case table above, in resolve-then-side order: a dead anchor first rehomes by the side's nearest-survivor scan from its birth position (exhaustion giving the side's collapse or document sentinel), and the boundary anchor, live or so rehomed, then takes its side's rule (gravity; the newer-than-mark run — contiguous live ids* $> mid$ *— skipped at an after start and grown over at an after end);* none *is the collapsed span. Coverage is the half-open interval:*

$$\text{markCoversPos}\ d\ m\ k\ =\ \big(f \le k \wedge k < e\big)$$
$$\textit{when}\quad \text{startIncl}\ d\ m = \text{some}\ f,\ \text{endExcl}\ d\ m = \text{some}\ e,$$

and false when either boundary collapses. Per $(position, mtype)$ *the covering mark with the largest mid wins (`bestCover`), and the position is formatted iff that winner is an* `add` *(`fmtAt`). The render* renderMarksDoc d *marks mt is the live order with each character's flag,* renderFlagWith d (markCoversPos d)*; the retracted control* renderMarksTree *is the same render*

over treeCoversPos *(climb the frozen path to the nearest live ancestor and cover the inclusive interval between the two resolved anchors, sides ignored).* `DocD, MarkD, renderMarksDoc`

The positive theorem, and the retraction it replaces. An earlier composed-Peritext read resolved a dead boundary by climbing *tree* ancestry (frozen path snapshots), and carried a `mark_*_no_leak` theorem asserting that formatting never leaks under deletion. That claim was **false** and is retracted: a tree ancestor sits *earlier in reading order* than its descendants, so a dead boundary anchor migrates backward in the document and formats text that was never in the span. The document-order model is built so that the replacement is provable: rehoming scans *reading* order, so a single-replica rehome can only shrink or hold a span, never migrate it backward.

Theorem 27.2 (no backward leak, document order)**.** *For every document d, mark m, type mt, and live indices f, k: if* startIncl $d\ m =$ some f *and* $k < f$*, then*

$$\text{fmtAt}\ (\text{markCoversPos}\ d)\ [m]\ mt\ k\ =\ \mathsf{false}:$$

no character strictly before the mark's rehomed start carries that mark's formatting. Stated at the singleton mark list $[m]$*, which is exactly the per-mark claim:* markCoversPos *is list-independent, so in any larger list m still covers nothing before f, and a position there can be formatted only by another mark's own span. Proved on axioms* {propext} *alone, below even the document's kernel-clean baseline.* `doc_no_backward_leak`

Proposition 27.3 (the tree-ancestry read leaks backward)**.** *There exist a document d, a mark m, and a live position k with*

$$\text{markCoversPos}\ d\ m\ k = \mathsf{false}\quad\wedge\quad\text{treeCoversPos}\ d\ m\ k = \mathsf{true}:$$

the retracted claim's own resolver formats a character the document-order read does not, so no analogue of Theorem 27.2 holds for the frozen-path read. The witness is concrete: delete a bold span's start anchor; document order rehomes the start forward to the next survivor (`leak_doc_shrinks_doc`*), while the tree read climbs to the anchor's parent and formats text that was never bold (*`leak_doc_shrinks_tree`*), with the $\neq$ companion* `leak_doc_shrinks` *pinning that the two renders genuinely differ.* `tree_backward_leak_refutes_noleak`

The Litt et al. pins. The model is pinned against the worked examples of the Peritext paper (Litt et al. CSCW'22), each a concrete SPOT with a hand-derived expected render and a $\neq$ companion refuting the tempting degenerate: insertion within a bold span is formatted (Ex 1); overlapping bold and italic carry both on the overlap (Ex 2); marking a span, deleting the whole span, and reinserting yields plain text under document order, while the tree control leaks the formatting back (Ex 3, the leak made concrete); concurrent add versus removeMark resolves LWW by *mid*, and swapping which is newer flips the verdict (Ex 5); typing at the end of a bold span is grabbed exactly when the typed text is newer than the mark (Ex 7); and the gravity contrast (Ex 8): on the same insertion, a bold span (*endSide* = *after*) expands and a link (*endSide* = *before*) does not. Convergence is watched at the same grain: the render is a function of the mark *set* (permutation invariance, with a dropped-mark FAIL pin); character order converges by the embed capstone.

The honesty theorem: the atomicity cost, stated. The substrate is tombstone-free and live, so the mark-positioning trilemma (atomic mark placement, tombstone-freedom, live state: pick two; §34) charges this model too, and it pays on the growth rule.

Proposition 27.4 (a delete can re-span: the trilemma horn)**.** *There is a four-character document and a bold mark — concretely: A, B marked (endSide = after), C older than the mark blocking the growth run, D newer, with the hand-derived render pins* `respan_before` *(A, B bold; C, D plain) and* `respan_after` *(A, B, D bold) — such that deleting the plain, boundary-untouched C (id 3) changes an untouched survivor's formatting:*

$$\text{renderMarksDocIds } d_{\text{after}}\ [m]\ \mathsf{bold} \neq (\text{renderMarksDocIds } d_{\text{before}}\ [m]\ \mathsf{bold}).\text{filter}\ (\lambda e.\ e.1 \neq 3) :$$

the post-delete render is not the pre-delete render minus the deleted character's entry, because C's deletion makes D contiguous with the span and the after-end growth run reaches it. `doc_delete_can_respan`

This is a membership re-span, not a backward leak (Theorem 27.2 is untouched), and the contrast with the fused encoding is exact: the fused design has no growth rule and proves `renderIds_del` (a character delete re-formats nothing); the marks model trades that theorem for the paper's expand-at-the-end behaviour, and the price is stated as a theorem rather than hidden.

27.2 The GC problem, and why refusal was correct

The state-GC pass of §26.2, as shipped in the runtime (§40), drops settled-dead records and re-codes coordinates. For plain text that is sound; for this datatype, pruning a dead anchor's record destroys exactly the birth position a mark needs to rehome, and the runtime therefore *refused* to compact Peritext at all. The refusal was correct, and the justification is a directed flip (D6 in the note's numbering, hand-derived). Geometry, reused throughout this section: R (id 1) and L (id 2) minted concurrently at the root, so L is newer and reads first; d (id 3) a child of L; reading order $L\,d\,R$. Mark [start L *before*, end d *after*], *mid* = 5; delete d; compact at a settled cut with no retention. The never-compacted twin: the dead end anchor d scans left and finds L; covered $\{L\}$, so L is bold. The unretained subject: d has no record, the resolver cannot place the scan and degrades to the collapse branch; covered $\{\}$, L is plain. The read flips. The companion pins the cause: the same compactor with all mark anchors *live* reads twin-identical, so the refusal's reason is precisely dead mark anchors, nothing else. Randomized (measured-in-model): the no-retention control flips 351 of 600 randomized DAG trials.

27.3 Two attacks, falsifiable forms first

The settled-cut contract is the text layer's (§26.2): every replica has delivered everything at or below the cut, and every op concurrent with the cut that is not yet delivered is *declared* in flight (its id and anchor id). The compaction map is also the text layer's: drop settled-dead records outside the keep-set, renumber each sibling group of the kept tree order-preservingly except groups receiving a declared in-flight delta (frozen verbatim, the `skipAt` guard), and let beyond-cut mints derive their coordinate from the re-coded anchor record plus the fresh delta (the extension law H3 realized as derive-at-ingest). The two attacks differ only in the keep-set and the mark treatment, and each was stated as a falsifiable reads-identical claim before any trial ran.

H-A: retention roots, validated. Keep-set = live ids ∪ the boundary anchor ids of every present mark record (add and remove records both) ∪ declared in-flight anchors. Dead boundary anchors survive as dead records with re-coded coordinates; the marks map is untouched. Falsifier: any version, on any beyond-cut continuation (text and mark ops on any replica, merged in any order, declared stragglers delivered late, successive cuts composed), whose render differs from the never-compacted twin. Verdict (measured-in-model): **validated**, 2000/2000 randomized DAG trials (4048 cuts, 2214 with declared stragglers), zero divergences. Cost (measured-in-model): 0.296 retained dead records per mark record, maximum 2.000 against the structural bound of 2 (a mark has two boundaries); 30.6 percent of settled-dead records dropped under a deliberately mark-heavy random workload (real traces mark far fewer characters per edit and would drop a larger share).

H-B: early re-anchoring, refuted. At the cut, rewrite each *dead* mark boundary to the resolver's own cut-time rehoming target (side kept; an exhausted scan becomes the side's collapse sentinel), then retain nothing for marks. The rewrite commits at the cut to information the resolver would otherwise read at render time (the dead anchor's position among *later* arrivals), and the minimal countermodel family (D1, all cells hand-derived, then machine-confirmed cell by cell) makes that exact. Same geometry as D6; one continuation character n: (a) a beyond-cut insert, id 10 (newer than the mark), anchored at L; (b) a declared straggler, id 4 (*older* than the mark), anchored at L, delivered after compaction; (c) the same straggler anchored at d itself. Covered sets, twin versus H-B subject, per boundary combination (the mark's boundary at d, the other on a survivor):

combo (boundary at d)	(a) id 10 at L	(b) id 4 at L	(c) id 4 at d
start *before* (inner)	$\{R\} = \{R\}$	$\{R\} = \{R\}$	$\{n, R\}$ vs $\{R\}$: **diverges**
start *after* (outer)	$\{R\} = \{R\}$	$\{R\}$ vs $\{n, R\}$: **diverges**	$\{n, R\} = \{n, R\}$
end *after* (inner)	$\{L, n\} = \{L, n\}$	$\{L, n\}$ vs $\{L\}$: **diverges**	$\{L\} = \{L\}$
end *before* (outer)	$\{L\} = \{L\}$	$\{L, n\} = \{L, n\}$	$\{L\}$ vs $\{L, n\}$: **diverges**

Worked sample, end *after*, variant (b). Twin: birth order $L\,n\,d\,R$; the dead end anchor d scans left and finds n live, so the span ends at n: covered $\{L, n\}$. H-B rewrote the end at the cut, before n existed, to L; at render the end sits at L and end-growth does not cross n, whose id 4 is below the mark's *mid* 5: covered $\{L\}$. The straggler sits inside the twin's span and outside the subject's.

The compensation finding. Column (a) is the negative space of the countermodel, and it is exact rather than lucky: with only a beyond-cut *newer* insert, all four combinations agree. Every character that appears between the rewrite target and the dead anchor's old position after a settled cut is either a beyond-cut mint (id above every settled id, hence above the mark's *mid*, so the newer-run growth and skip rules cross it on *both* sides of the comparison) or a declared straggler, the only source of ids below the mark's *mid*. And nothing beyond-cut can land between a settled-dead anchor and its reading-order successor without anchoring in the dead subtree, which a settled cut makes invisible to future minters; only a straggler anchored at the dead node itself can, which is column (c). So the naive recency candidate (a beyond-cut newer insert alone) *provably cannot fire*: the H-B countermodel needs a straggler older than the mark (columns b, c) or the death of the rewrite target. The latter is D1(d), hand-derived: chain L (1) at the root, d (2) a child of L; mark [start L *before*, end d *after*], *mid* = 5; delete d; cut

(H-B rewrites the end to L); beyond the cut insert n (10) under L, then delete L. Twin: birth $L\,n\,d$, live $\{n\}$; the start rehomes right to n, the dead end d scans left and finds n: covered $\{n\}$, n is bold. Subject: the rewritten end anchor L is now dead too and scans *left* from L's position, where nothing lives (n sits to L's right, unreachable by that scan): the mark collapses and n is plain. It diverges with no straggler anywhere: the rewrite also forgot that d sat to the *right* of L, so the re-rehoming scans from the wrong birth position.

The weaker true statement, labeled as what it is. The rewrite is computed once at the common settled cut, so every compacted replica shares it, and all compacted replicas *converge among themselves*: machine-checked separately, 2000/2000 trials. This is **licensed divergence**, a semantic change at a settled cut, and it is never reads-identical: 156 of the 2000 randomized trials diverged from the never-compacted twin, on top of the directed family above. It must not be stated as H-B succeeding, and is recorded here as the price a re-anchoring design would pay, not as a result.

D3: a dead run keeps its internal order. Two boundaries inside one dead run are observably *ordered*. Chars A (1) at the root, x (2) under A, y (3) under x, B (4) under y; mark m_1 covers $[A..x]$ (*mid* 10), m_2 covers $[A..y]$ (*mid* 11), both inner sides; delete x and y (one dead run); cut; declared straggler g (5) anchored at x, delivered after compaction. g is a newer child of x than y, so birth order is $A\,x\,g\,y\,B$. Twin: m_1's end (x, dead) scans left to A: covered $\{A\}$; m_2's end (y, dead) scans left to the live g: covered $\{A, g\}$. So g carries m_2 and not m_1: the two boundaries' order inside the dead run is observable. H-B rewrote both ends to A at the cut (the run's left survivor): both marks cover $\{A\}$ and g loses m_2. H-A retains x and y as dead records whose re-coded coordinates keep their relative order, and reads exactly as the twin; that order preservation on retained-dead pairs is the A1 obligation the mechanization below discharges.

27.4 The growth window: the guarded root-freeing drop

Plain H-A retains a mark's anchors as long as the mark record exists, and mark records are grow-only, so a heavily reformatted document accumulates (add, remove) pairs whose anchors are retained forever. The A3 refinement drops such a pair and frees its retention roots: an add m and a remove r of the same *mtype* with *equal boundary tuples* (ids and sides) and $r.mid > m.mid$ resolve to the same covered interval forever, and r beats m on every covered character, so dropping both looks render-neutral. It is sound only under *three* guards, each with a hand-derived countermodel:

- *Removal settled, declared in-flights included* (the α flip): a concurrent same-type add s with $m.mid < s.mid < r.mid$, still in flight at the drop, is suppressed by r on the overlap while the pair is present and formats it once the pair is dropped.
- *No LWW shadowing*: no other same-type mark with *mid* below $r.mid$ may exist, because spans move under later edits, so a no-overlap check *now* does not persist; dropping r could resurrect the older mark on a future overlap.
- *The growth window* (the β flip, the discovery of this phase): no settled or declared id may lie strictly inside $(m.mid, r.mid)$. A character w with $m.mid < w.id < r.mid$ is grabbed by the add's end-growth ($w.id > m.mid$) but *not* by the remove's ($w.id < r.mid$ fails its newer test), so w is bold with the pair present and plain after the drop: the pair is not

render-neutral even though its boundaries are identical. Beyond-cut mints are Lamport-fresh (above $r.mid$), so checking the settled and declared ids at the cut is sufficient.

Both unguarded flips and the guarded positive (with a genuinely dead root freed that plain H-A would have retained) are hand-derived and machine-checked. Guarded, on top of H-A (measured-in-model): 600/600 twin trials with 44 pair-drops fired; each drop removes two mark records and frees their anchors for the next cut.

27.5 The mechanization

The note's obligations O1–O4 are mechanized in `PeritextEmbed_MarksGC.lean` (O1–O3) and `PeritextEmbed_MarksGC_A3.lean` (O4); O5, the settled-cut protocol half, is deliberately *not* taken on, being the already-tracked residue below.

O1: the keep-set is a stable-prefix map, not a new notion.

Definition 27.5 (the keep-set stable-prefix map)**.** *A keep-set is* $K = (\text{live}, \text{markAnchor}, \text{inflight})$ *(`KeepSpec`), three coordinate predicates, with* $K.\text{kept}\ c = \text{live}\ c \vee \text{markAnchor}\ c \vee \text{inflight}\ c$*: the live records, the boundary anchors of present and declared in-flight mark records (dead anchors included — the retention roots), and the anchors of declared in-flight text ops. Given a relabeling* f *and declared mints* Mint *satisfying*

(h_{ext}*)* $\text{Mint}\ \pi\ d$ *implies* $f(\pi \cdot \Gamma.\text{enc}(d)) = f(\pi) \cdot \Gamma.\text{enc}(d)$*, and*

(h_{ord}*) for* c*,* c' *each kept or mint-shaped (*$\exists \pi\, d$*,* $\text{Mint}\ \pi\ d \wedge c = \pi \cdot \Gamma.\text{enc}(d)$*):* $\text{keyLt}(\text{key}(f\ c))\ (\text{key}(f\ c')) = \text{keyLt}(\text{key}\ c)\ (\text{key}\ c')$*,*

keepSPM $\Gamma\ K\ f$ Mint *is the existing stable-prefix bundle of Definition 26.1 with* $\text{Rest} := K.\text{kept}$ *and* $\text{MintAt} := \text{Mint}$*:* h_{ext} *is H3 and* h_{ord} *is H2, field for field. H3's* Mint *covers fresh mints and declared stragglers, whose frozen sibling groups are exactly the text layer's skipped-groups side condition, with the FAIL companion `freeze_guard_violation` pinning the flip (deltas* $2 < 5$ *in-flight* < 9 *renumber to* $1, 2$ *and the frozen* 5 *jumps the ex-*9*).* `KeepSpec`, `keepSPM`

Lemma 27.6 (A1, with H1 derived)**.** *For kept coordinates* c*,* c'*:*

$$\text{keyLt}(\text{key}(f\ c))\ (\text{key}(f\ c')) \;=\; \text{keyLt}(\text{key}\ c)\ (\text{key}\ c')\ :$$

the relabeling preserves the display comparator on the whole kept set, retained-dead pairs included, which is exactly what the D3 observation (§27.3) consumes. Injectivity on kept pairs is derived through the bundle (`keepSPM_injOn_kept`), not assumed. `keepSPM_A1` *is proved with no axioms at all.* `keepSPM_A1`

O2: scan factorization, a pure list lemma.

Lemma 27.7 (scan factorization)**.** *For every split* $l_1 \cdot a \cdot l_2$ *of the birth order, deleted list* del*, and keep predicate* p *with* $p\,a = \mathsf{true}$*,* $a \notin l_1$*, and every survivor of* l_2 *kept (*$\forall x \in l_2$*,* $x \notin del \Rightarrow p\,x = \mathsf{true}$*):*

$$\begin{aligned}&\text{scanRight}\ \big((l_1 \cdot a \cdot l_2).\text{filter}\ p\big)\ del\ \big(\text{idxOf}\ ((l_1 \cdot a \cdot l_2).\text{filter}\ p)\ a\big)\\ &\quad= \text{scanRight}\ (l_1 \cdot a \cdot l_2)\ del\ \big(\text{idxOf}\ (l_1 \cdot a \cdot l_2)\ a\big)\ :\end{aligned}$$

the first survivor strictly right of the retained element a is the same in the filtered and the full sequence, because every dropped element is a non-survivor the scan skips anyway. Mirror statement `scanLeft_filter`*, with the kept-survivors hypothesis on l_1. No datatype content:* `List.filter` *and* `find?` *over an arbitrary split.* `scanRight_filter`, `scanLeft_filter`

O3: the render-congruence capstone. The load-bearing observation: `renderMarksDoc` is a function of the live id sequence, the boundary anchors' positions relative to it, and the id order; it never reads a coordinate. Its hypothesis structure is displayed first:

Definition 27.8 (the continuation discipline)**.** ContOK $\Gamma\ s\ \tau$*, for a cut state s and continuation τ, has four fields:*

(freshId) every insert $(t, r, \mathsf{ins}(e, \pi, a)) \in \tau$ has $t \notin \mathrm{ids}(s)$;

(nodupIns) τ's insert ids are duplicate-free;

(freshKeySt) every insert of τ mints a key $\mathrm{key}(\pi \cdot \Gamma.\mathrm{enc}(t - a))$ *distinct from every stored record's key;*

(freshKeyPair) distinct inserts of τ mint distinct keys.

This is what any honest beyond-cut continuation satisfies (Lamport freshness plus unique decodability); deriving it, together with the straggler declarations, from honest reachability at a certified cut is nothing marks-specific: it is the note's O5, the per-continuation face of the settled-cut protocol residue. The honesty-rebasing half of that residue, the $(*)$ lemma, is closed for the text layer (Definition 26.21–Theorem 26.24, §26.2); what remains is the protocol half — Open Questions 43.7 and 43.8 — and the marks layer adds no new obligation to it. `ContOK`

Theorem 27.9 (O3: render congruence under retention-roots compaction)**.** *For a stable-prefix map F; cut state s; continuation τ; keep predicate kp; deleted list del; mark list $marks$; type mt; under the hypotheses:*

(h_{sort}, h_{nd}) s is strictly key-sorted and its ids are duplicate-free;

(h_{ok}) ContOK $\Gamma\ s\ \tau$ *(Definition 27.8);*

(h_{kp}) every insert of τ is kept (kp true at its id);

*(h_{dom}) every kept record of s is at hand (F.*Dom *of its coordinate);*

*(h_{mint}) every insert $(t, r, \mathsf{ins}(e, \pi, a)) \in \tau$ is a declared mint (F.*MintAt $\pi\ (t - a)$*);*

(h_{dead}) every dropped record's id is recorded dead (kp false at $r.1$ implies $r.1 \in del$);

(h_{anchor}) every present mark's two boundary ids are kept (the retention roots),

the compacted-and-continued document renders equal to the never-compacted twin:

$$\mathrm{renderMarksDoc}\ \big(\mathrm{applySeq}\ (\mathrm{remap}_F\ (s.\mathrm{filter}\ kp))\ (\tau.\mathrm{map}\ \mathrm{remap}_F),\ del\big)\ marks\ mt = \mathrm{renderMarksDoc}\ \big(\mathrm{applySeq}\ s\ \tau,\ del\big)\ marks\ mt,$$

for every mark and every type. Kernel-clean. The proof chains the re-coding cluster's T1 (Theorem 26.3 transports the compacted fold), coordinate invisibility of the render, commutation of the continuation with the drop, and O2 (Lemma 27.7) plus the retention roots. `marksGC_render_congr`

Theorem 27.10 (O3, two epochs)**.** *For maps F_1, F_2 with* CompatOn F_1 F_2 Rest′ MintAt′ *(Definition 26.14); keep predicates kp_1, kp_2; continuations τ_1, τ_2; under Theorem 27.9's hypothesis list taken twice: epoch 1's clauses (h_{ok_1}, h_{kp_1}, h_{dom_1}, h_{mint_1}) verbatim at (s, τ_1, kp_1, F_1), and epoch 2's at the once-continued state and the composite: h_{ok_2} at* applySeq s τ_1*; h_{kp_2}, h_{dead}, h_{anchor} at the conjunction $kp_1 \wedge kp_2$; $h_{\mathrm{dom}G}$, $h_{\mathrm{mint}G}$ at F_1.*comp F_2*'s surviving domain* $(\mathrm{Rest}', \mathrm{MintAt}')$*. Conclusion: compact at a settled cut, continue, compact again on the already re-coded records, and deliver the twice-lagging τ_2 translated through the composite; the final render equals the never-compacted twin's (the display elides the nested remaps; the mechanized statement carries them in full). Kernel-clean. The composite is the existing* `StablePrefixMap.comp` *carried at its own surviving domain: a retention root freed between the epochs (by an A3 pair-drop) is exactly the rank-reclaim collision shape* `naive_composition_collides` *pins (§26.2), and restricting the composite's domain to the survivors is the fix that file already provides. n-epoch towers follow by the same* `chainSPM/CompatOn` *route.* `marksGC_render_congr_twoEpoch`

O4: the guarded drop.

Theorem 27.11 (O4: the guarded pair-drop)**.** *For a document d, mark list marks, and $m, r \in$ marks with $m.op =$* add*, $r.op =$* remove*, $r.mtype = m.mtype$, $m.mid < r.mid$, equal boundary tuples ($r.startId = m.startId$, $r.endId = m.endId$, $r.startSide = m.startSide$, $r.endSide = m.endSide$), and marks's mids duplicate-free; under the three guards:*

(G1) removal settled, declared in-flights included: the statement is over the final delivered document and mark list, so a cut-time drop is licensed exactly when the settled marks plus the declared in-flight marks exhaust the final list's candidates — beyond-cut Lamport freshness (every later mid above $r.mid$) covers the rest, which is the α flip's discharge;

(G2) no LWW shadowing (h_{others}): every other same-type mark $o \in$ marks (with $o.mid \neq m.mid$, $o.mid \neq r.mid$) has $r.mid < o.mid$;

(G3) the growth window (h_{window}, the β clause, the discovery of this phase): every birth id c of d satisfies

$$c \leq m.mid \ \vee \ r.mid < c :$$

no id in the half-open window $(m.mid, r.mid]$ (the formula also excludes $c = r.mid$): such a character is grabbed by the add's end-growth but not the remove's, so the pair is not render-neutral even with identical boundaries,

dropping the pair preserves the render:

$$\begin{aligned} &\mathrm{renderMarksDoc}\ d\ \big(marks.\mathrm{filter}\ (\lambda o.\ o.mid \notin \{m.mid, r.mid\})\big)\ mt \\ &\quad = \mathrm{renderMarksDoc}\ d\ marks\ mt. \end{aligned}$$

Only $r.op =$ remove *is load-bearing for the verdict (the* add *hypothesis names the shape, the proof never consults it).* `a3_guarded_drop`

The SPOT block is PASS+FAIL shaped throughout: the guarded positive fired *through the theorem*, the freed-root payoff (the next cut drops the ex-anchor's record and still matches the twin that holds both marks), a surviving-rival-mark variant, and both flips (α, β) each with a guard-refuses companion.

One honest flag. The Lean model and the Python harness diverge on one degenerate branch: a boundary anchor with *no record at all*. The Lean resolver's scan then starts past the end of the birth order (finding the last, respectively first, survivor); the Python harness degrades to the side's collapse sentinel. The two disagree on the degenerate *value*, but D6 flips against the twin either way, which is the only load-bearing fact, pinned on both sides (`d6_no_retention_flip`; the harness's D6), and the positive obligations never touch the branch: retained anchors always have records. The general lemmas and both capstones are kernel-clean; the SPOTs use `native_decide` under the document's SPOT convention.

27.6 The runtime: refuse becomes fire-with-certificate

The shipped compactor. `compact-peritext.js` transliterates the harness semantics. The keep-set is as above; the re-coding, the order-preserving rank renumbering, and the in-flight freeze guard reuse the text layer's `compactEliasDelta` verbatim over the kept shadow; marks reference *ids*, which re-coding never rewrites, so the mark map itself needs no translation; retained dead anchors stay listed in the deleted set. `compactiblePeritext` plugs the hooks into `DistributedReplica`: `compactStable` for Peritext now *fires* under the same frontier certificate as the text layer (§40), where it previously refused, and the A3 pair-drop runs first so freed roots leave the keep-set in the same epoch.

The delete-must-settle refinement. One contract refinement goes beyond the harness. Peritext deletes are logical, so a record may be freed only once its *delete* is settled, not merely its insert: an id whose delete is still propagating is live somewhere and may yet be marked or anchored on. The harness's full-exchange barrier settles both at once and cannot see the distinction; the runtime's certificate separates them (settled insert ids versus settled delete ids, both extracted from the certified meet), and a directed test pins the refusal while a delete propagates and the fire once it settles.

Deviations, all conservative. Under a certified cut every in-flight field is empty: every op concurrent with the cut is already delivered (SettledAt), so passing empty declarations is proved sufficient rather than asserted, exactly as for the text layer. A declared in-flight insert whose anchor has no kept record throws rather than guesses. And the justifying countermodels stay executable against the shipped code as negative controls (`noRetention`, `unguardedPairDrop`, the unguarded renumber), so the flips above are reproducible in the artifact, not only in the model.

Measured. With the marks GC wired, the full runtime suite passes 103/103 and the p2p-demo suite 10/10. The dedicated file (`runtime/test/peritext-gc.test.js`) carries the directed PASS+FAIL family (D6, the D1 straggler cells, the frozen-group order flip through the Peritext path, D3, both A3 flips with their guard-refuses companions), certified refuse-then-fire with wire catch-up across the compaction commit and a *new* mark anchored on a retained dead record in the new epoch, the delete-must-settle directed pair, and a 150-trial multi-epoch twin PBT with declared stragglers (300 cuts, 179 with stragglers, 217 of 730 settled-dead records dropped, 10 guarded pair-drops): zero divergences from the never-compacted twin. Measured cost on this workload: 444 retained dead records over 1780 mark records, 0.249 per mark record, per-cut maximum 2.000 against the structural bound of 2. The 60-trial no-retention control flips 35 trials: the old refusal stays justified against the shipped code.

The stack, closed. §25 characterized what a tombstone-free sequence must remember: the dead chain's slot keys, surviving as order ordinals inside live coordinates, with the fooling pair as the lower bound. The marks layer adds one further debt, and pays it counted. With retention-roots GC and the guarded pair-drop, the entire Peritext state is $O(\text{live})$: live characters, present mark records, and dead information surviving *only* as (i) the provably necessary order ordinals of §25 and (ii) at most two mark-anchor retention roots per present mark record (the structural bound; measured 0.25 to 0.30). Everything else, dead text and settled mark pairs alike, is dropped, and the invisibility of the drop is a kernel-checked theorem plus a twin-checked measurement, not a hope.

28 Comparison with Eg-walker

Eg-walker (Gentle & Kleppmann, EuroSys 2025) is the closest system in spirit, and the two designs can be read as the *operational* and *denotational* realizations of one thesis, **pay for concurrency, not for history**:

	Eg-walker	embedded-chain RGA
model	op-based; replicas exchange events	MRDT: three-way merge with an LCA
durable state	**full event graph on disk**, deletes included, retained while concurrent ops may arrive; document state metadata-free	**live nodes only**; no event log anywhere; dead survive as $\Theta(\log \delta)$ bits inside live coordinates
merge mechanism	*replay*: walk the event graph, rebuild a transient CRDT structure (tombstones reappear transiently), discard at causally-linear points	*refold*: $O(\text{survivors})$ coordinate recomputation; no replay, no transient structure, no order decisions
sequential-editing cost	$\approx$ zero (plain edits; no metadata)	$\approx$ zero order metadata ($\sim$4 bits/level for continuation typing; a cursor jump pays log-gap once, at the run head)
concurrency cost	replay time proportional to the concurrent region, at every affected merge	coordinate bits proportional to log(ts gap), once, in space
ordering semantics	Yjs-variant (FugueMax-adjacent); maximal non-interleaving *conjectured*, not proved; two-sided (no L19 analogue)	$\equiv$ published RGA (machine lockstep); one-sided: L19 backward-run interleaving inherited; removed by the sided generalization (§23), RGA kept as the all-R fragment
correctness artifact	informal proof (Appendix C) + property testing	L1–L25 battery + DAG PBT + lockstep vs $\text{RGA}_\dagger$; **RA-linearizability kernel-checked in Lean** (§30); the design was chosen so the merge proves itself (no order decisions to verify)
op payload	integer origin positions (compact, human-meaningful)	two timestamps + ghost prefix (droppable on the wire; §21)

Four sentences of honest positioning. *(i)* Eg-walker achieves its clean document state by keeping the entire history and re-deriving order on demand; the embedded-chain RGA achieves it by making order derivation unnecessary: the sort keys are written at birth and never edited, and the coordinates are their encoding (§20), precomputed at mint time and exact forever. *(ii)* Eg-walker's ordering target was stronger where L19 is concerned (Fugue-style two-sidedness removes backward interleaving, while the one-sided design inherits RGA's anomaly by construction, being read-equal to RGA); the sided generalization of §23 adopts exactly that two-sidedness as a parameter, closing the gap while keeping the one-sided datatype as its all-R fragment; the Fugue-Max variant's maximal non-interleaving, which Eg-walker's shipped ordering only conjectures, is here a kernel-checked theorem (§24). *(iii)* The verification gap runs the other way: Eg-walker's replay-transform-clear loop is algorithmically sophisticated and only informally proved; here the merge performs no arbitration at all, the metatheory is one-line at the chain layer plus a single faithfulness theorem at the encoding layer, and RA-linearizability is a kernel-checked theorem in a framework that had already verified a dozen-plus replicated datatypes end-to-end (§30). *(iv)* The cost models part company at cursor jumps: a single-user history that hops back to type at an old anchor is free for Eg-walker (nothing concurrent to replay) but mints log-gap bits here, once per jump, at the run head; on measured single-user editing traces the jump tail is thin (code bits $\approx$ 1.4–1.8 per level; see the companion note of §20). In the MRDT setting there is also a model fit: Eg-walker must reconstruct merge context by replay because the op-based model gives it none; the MRDT model hands the merge its LCA, which is exactly the context replay rebuilds.

29 Related work

The verified sweep of nearby systems lives in `whiteboard/litmus/related-work.md` (per-claim verdicts, primary sources); this section positions the datatype.

Retention taxonomy. What nearby systems keep for the dead: tombstone records (RGA with cemetery under causal stability; WOOT; Yjs's per-deletion GC structs, kept forever; Fugue, "we cannot remove a deleted element's node entirely"; Peritext, "the positions must be remembered"); the full history (Eg-walker's event graph, Chronofold/causal trees where the log *is* the text); consensus-gated compaction (Treedoc's flatten requires Paxos-commit); or **dead identifiers inside live position keys** (Logoot/LSEQ, Weidner's position-strings/list-positions, the Fugue id space). The embedded-chain RGA is in the last class *by information content* (dead ancestors' timestamps persist inside survivors' sort keys, at live-reachable scope), but with three deltas to its classmates: the retention is *delta-coded at the entropy of the anchor gaps* (a sequential history costs a constant per element where Logoot-family keys grow with allocation policy), the *birth tree kept whole* gives forward non-interleaving and delete-order preservation (position-key designs interleave, the PaPoC'19 anomaly, and Figma-style fractional indexing is unsound under concurrency by its own account), and the semantics is pinned to the best-studied sequence CRDT by a machine-checked lockstep rather than being a new point in anomaly space.

The impossibility backdrop. Attiya et al. (PODC 2016) prove $\Omega(D)$ metadata for push-based protocols meeting even the weak list spec: the published form of "ordering requires remembering the dead." This design does not escape the bound; it *relocates and minimizes* the memory: $\Theta(\log \delta)$ bits per dead-chain element inside live coordinates (plus the MRDT version store, which

the model provides). The sharper, machine-witnessed form from our design sweep is the *conservation law* (§19): the sort key must retain dead ancestors' timestamps (the verdict information survives every encoding change: stamp, spine, tombstone, or denominator bits), and ties force separate *metadata* only under timestamp-oblivious minting; the timestamp-faithful mint carries the same bits inside the coordinate. Eight timestamp-oblivious mutation variants each have a named machine-checked countermodel (fold-verdict erasure; repair non-locality); the credential countermodel separates this design from all of them.

Verified datatypes. Isabelle RGA (Gomes et al. OOPSLA'17, op-based convergence), OpSets (list semantics specified by a total order of identified ops: arbitration-by-identity as specification), VeriFx (SMT, convergence properties), MET (TLA+, list CRDT bugs). In the MRDT line (Kaki et al. OOPSLA'19; Peepul PLDI'22; RA-lin OOPSLA'25, this framework), no verified sequence with sequence-CRDT-grade ordering guarantees existed; our verified rehoming RGA (§19) is RA-linearizable against its own fold yet fails the sequential delete-order spec, since RA-linearizability certifies convergence to the datatype's *own* semantics and ordering intent must be specified and proved separately. The embedded-chain RGA delivers the missing artifact: RA-linearizability is a kernel-checked Lean theorem (§30); delete-order preservation, display stability and forward non-interleaving are proved at the model layer; and the RGA read-equivalence (the compaction theorem, §30) closes the intent question against the best-studied sequence semantics.

Treedoc's lesson, inverted. Treedoc re-balances geometry and must pay consensus for it; the principle that emerged from our ledger-based sibling design study (*reprojection without consensus is safe iff geometry has been demoted to a rendering*) is here taken to its fixed point: geometry is never reprojected at all. It is immutable, so it may safely be the only thing.

30 Mechanization: the conditioned discharge and the compaction theorem

The verification framework is Part I's metatheory; file paths below are relative to the Sal repository.

The capstone is proved (2026-07-15, kernel-clean on $\{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$). Its honesty hypothesis is displayed in full, never folded:

Definition 30.1 (embed honesty, and honest reachability)**.** EHonest Γ C *has two components. (Delete provenance): for every* $e \in C$.events *with* $e = (\cdot, \cdot, \mathsf{del}(x))$ *there is an* $a \in C$.events *with* C.vis a e, $a.1 = x$, *and* a *an insert. (Chain generation): there is a global assignment* chainOf : $\mathbb{N} \to$ List $\mathbb{N}$ *such that every insert* $o \in C$.events *satisfies*

$$\text{PosChain (chainOf } o.1), \qquad \text{coord}(o) = \text{coordOf}_\Gamma \text{ (chainOf } o.1), \qquad \text{(chainOf } o.1).\text{sum} = o.1 :$$

the carried coordinate is a positive birth chain's, and the chain's deltas telescope to the op's own id. EReach Γ *is* HonestReach $(E\ \Gamma\ \alpha)$ (EHonest Γ)*: LTS reachability where every step is taken from a configuration with an honest history, instantiating Part I's generic metatheorem exactly as the mergeable queue does.* `EHonest, EReach`

Theorem 30.2 (the embed capstone)**.** *For every ordered prefix code* Γ *and configuration* C*:*

$$\text{EReach } \Gamma\ C\ \rightarrow\ \text{IsRALinearizable3 } C,$$

RA-linearizability per version at every honestly reachable configuration, parametric in the code Γ*: the unary code, the binary delta code, and the flipped Elias-*δ *code are all proved* `OrderedPrefixCode` *instances, three encodings on one proof (the Elias-*δ *inheritance corollary* `embed_ra_linearizable3_eliasDelta` *is one line).* `embed_ra_linearizable3`

The factoring is exactly this part's design–encoding split, which is itself evidence for the split:

1. **Model layer** (`Sal/MRDTs/RGA_Embed/`, six files, 0 sorry, plus the read-equivalence order core and the sided kernel noted below): the code-parametric kernel; commutations *to state equality* under `rc` = `Either` (`rc_non_comm'`); `ins_prefix_ghost`; coherence and chain-generation closures; display stability, with step stability (S2, delete-order preservation at its `del` instance) proved *without any axioms* and merge pairwise stability (S4, the adopted contract); unique decodability (`coordOf_inj`); **the chain-lex theorem** (`display_iff_chainBefore` = Corollary 20.6); non-interleaving as subtree convexity (`subtree_convex`); Theorem 20.5(i) for all three codes (`binEnc_mono`/`binEnc_prefixFree`, `dEnc_mono`/`dEnc_prefixFree`, cross-validated against the Python codes by kernel `decide`); `native_decide` SPOTs where the rehoming RGA's delete-reorder witness gets the opposite verdict.

2. **Conditioned instance** (`Sal/ConditionedMRDTs/MRDT_Instances/EmbedRGA/`, nine sections, 0 sorry, plus the Elias-δ inheritance corollary `EmbedRGA_EliasDelta.lean`), proved via the *mergeable-queue route* of §15: a generic join lemma (`JoinLemma3At`: the merge exhibits its own linearization witness) plus an honest-reachability metatheorem (`HonestReach`), rather than the rehoming RGA's 55-file bespoke chain. The route applies exactly when states are canonical per event set, which is Theorem 19.3(i). Landed: the canonical sorted-list state; strictly-sorted extensionality (`esorted_ext`: sorted lists with the same members are equal, replacing any canonical-list formula); **fold-canonicity**, the Join, and the honesty discharge (Theorems 30.3, 30.4, 30.5 below); the honesty core and its bridge to well-formedness; the merge membership characterization (`e_mergeL_mem`, OR-set survival with honesty closing the cross-branch-delete corner); the capstone (Theorem 30.2); and the instance intent theorems: delete-order preservation *is* sublist stability (`eUpdate_del_sublist` is literally `List.filter_sublist`: a delete's successor state is a sublist of its predecessor, so the surviving display order is untouched; the merge preserves each input's survivor order on both sides).

Theorem 30.3 (fold canonicity: Theorem 19.3(i) mechanized)**.** *Call an enumeration* ρ *well-formed (*$\text{EWf } \Gamma\ \rho$*) when its insert ids are duplicate-free, no id is deleted before its insert, and distinct inserts mint distinct keys — the three consequences honesty supplies on any enumeration of a closed event set. For* ρ*,* ρ' *with* $\text{EWf } \Gamma\ \rho$*,* $\text{EWf } \Gamma\ \rho'$*, and* $\forall o,\ o \in \rho \iff o \in \rho'$*:*

$$\text{eFold}_\Gamma\ \rho\ =\ \text{eFold}_\Gamma\ \rho',$$

well-formed enumerations of one event set fold to the same state. `e_fold_canon`

Theorem 30.4 (the Join: the merge is its own linearization witness)**.** *For every configuration whose core history is honest (*EHonestCore Γ C*, the two components of Definition 30.1 stated at the core):* JoinLemma3At (E Γ α) C*. The witness enumeration for a merged pair is the LCA's enumeration, then branch one's delta in branch order, then branch two's news; its delivery-respect obligation falls to closure (a later event sitting in an earlier block would have been pulled into the earlier event set), the within-block orders transferring verbatim because the local order is event-set independent under* rc = *Either.* `e_join_at`

Theorem 30.5 (the honesty discharge: applicable generation forces honesty)**.** *The issuer-side guard* eApplicable o s*: for an insert* $(t, r, \mathsf{ins}(e, \pi, a))$*,* $a < t$ *and either* $a = 0 \wedge \pi = \varepsilon$ *(the root) or some record* $(a, el, \pi) \in s$ *— the anchor stored with exactly the carried prefix as its coordinate; for a delete, the target id is stored. If every* $e \in C$.events *admits an enumeration* π *of its causal past (*listPermOf π $\{e' \in C.\text{events} \mid C.\text{vis}\ e'\ e\}$*) with* e *applicable at* $\text{eFold}_\Gamma\ \pi$*, then* EHonest Γ C*: per-op honesty needs no bespoke oracle. The delete component is fold provenance; the chain component builds the global assignment by strong induction on ids (anchors have smaller stamps, and the anchor's record in any fold of the issuer's past is op-determined, forcing the carried prefix to be the anchor's chain's coordinate). The framework form:* GenHonest *at* eApplicable *plus* CausalPastEnumerable C *supply exactly this hypothesis (*`eHonest_of_genHonest`*).* `eApplicable`, `eHonest_of_applicable`

The compaction theorem is proved (2026-07-15, kernel-clean): $\text{read} \circ \text{embed} = \text{read} \circ \text{RGA}_\dagger$ on honest executions.

Theorem 30.6 (the compaction theorem)**.** *Hypotheses: the generation discipline* EAppDiscipline Γ C *(every op applicable at some fold of its issuer's causal past — the same hypothesis Theorem 30.5 discharges honesty from, named); an enumeration* ρ *with* listPermOf ρ C.events *and* EWf Γ ρ*; and a list* L *that meets the published RGA's own relational read specification at the tombstoned fold:*

$$L \text{ pairwise-sorted by } \text{visible\_lt } (\text{rgaFold } \rho), \qquad \forall t,\ t \in L \iff \text{visible } (\text{rgaFold } \rho)\ t.$$

Conclusion: L = eIds ($\text{eFold}_\Gamma\ \rho$) *— any enumeration of the tombstoned fold's visible ids in the published order is the embed fold's id sequence — with element agreement: every record* $(t, el, c) \in \text{eFold}_\Gamma\ \rho$ *satisfies* readSeq_visible (rgaFold ρ) t el *(*`embed_rec_readSeq`*, hypothesis* EWf Γ ρ *alone). The consumed relations are the published side's own:* `visible_lt` *here is the inductive of* `Sal/MRDTs/RGA_with_tombstones/RGA_ReadSide.lean` *(parent-child, newest-first siblings, left-descendant, transitive); the file is named because the repository carries four homonymous* `visible_lt` *declarations across its CRDT and MRDT trees, and the compaction theorem consumes exactly this one.* `rga_read_eq_embed_read`, `embed_rec_readSeq`

The proof is *against the published definition*: the order core (`Sal/MRDTs/RGA_Embed/RGA_Embed_ReadEquiv.lean`) shows `visible_lt` coincides with chain-lex on the shared birth tree (soundness by induction on derivations, completeness by realizing every chain prefix as an `afters`-reachable ancestor), and en route proves the published relational order total and asymmetric on the birth tree, facts the published side never had. This converts the design into a theorem *about* the published RGA: it admits a strictly-live compaction preserving every read. The lockstep rows of §22 are this theorem's tested form.

The sequential specification is proved, and the design holds the canonical seat (2026-07-16). The campaign of Part III proved `embed_seq_sound`: on every sequentially honest single-replica history the canonical sorted state is the naive insert-at-position text buffer, record for

record, with the adjacency lemma (`chainBefore_snoc_iff`: a fresh insert's delta beats every existing child's by sum-telescoping) as the geometric heart (§35). The published RGA$_\dagger$ inherits the same buffer spec through the compaction theorem (`rga_seq_read_eq_buffer`), and the same statement is refuted for the rehoming design (§36), which demoted it repo-wide: the embedded-chain family is the repository's canonical sequence RDT, and the canonical fused Peritext was re-based onto this kernel (`Peritext_Embed/`: capstone by pure instantiation of the payload-generic instance; `renderIds_del`, deleting a character never re-formats another, the theorem the rehoming kernel provably lacks; and the tier-4 editor theorem `peritextEmbed_seq_sound`). One caveat is recorded as Open Question 43.12: per-replica soundness and convergence do not compose into "the merged read is the naive buffer on some ordering of the union"; the dead-anchor corner (§38) is where stamp-sorted replay diverges from the fold.

The sided generalization is proved (2026-07-15, kernel-clean), in the same two layers as §23 presents it. The sided chain-lex kernel (`Sal/MRDTs/RGA_Embed/Sided_ChainLex.lean`): the sided marker theorem `sdisplay_iff_schainBefore` (display comparison of sided coordinates IS the in-order rule), totality of the display relation *without any axioms*, unique decodability (`sidedCoordOf_inj`: the side is recoverable from the first symbol's band, the delta by prefix-freedom within the band), the all-R *fragment theorem* `schainBefore_liftR` (all-R chains order exactly as one-sided chains: "RGA is the all-R fragment," mechanized), and in-order-interval subtree convexity (`schain_subtree_convex`, the datatype half of the L19 discharge). The conditioned instance (`MRDT_Instances/SidedRGA/`, nine sections mirroring the one-sided instance with the sided key): fold-canonicity (`s_fold_canon`), the Join by the same mergeable-queue route, and the capstone:

Theorem 30.7 (the sided capstone, restated from Theorem 23.7)**.** *For every ordered prefix code* Γ *and configuration* C*:*

$$\mathrm{SReach}\ \Gamma\ C\ \rightarrow\ \mathrm{IsRALinearizable3}\ C,$$

where $\mathrm{SReach}\ \Gamma = \mathrm{HonestReach}\ (S\ \Gamma)\ (\mathrm{SHonest}\ \Gamma)$ *and* SHonest *is the verbatim sided mirror of Definition 30.1 (delete provenance; sided chain generation, the mint a positive sided chain's coordinate with telescoping deltas). The theorem holds for every side assignment: the guard constrains the anchor, never the side, so sides are payload to convergence — the policy/datatype split of §23 as a theorem — with* `sHonest_of_applicable` *discharging honesty from the generation guard exactly as one-sided (Theorem 30.5).* `sided_embed_ra_linearizable3`

The per-policy intent theorems have since landed, and the policy story is a machine-checked three-act arc (2026-07-16/17). *Act one*:

Theorem 30.8 (act one: the all-R fragment at the instance)**.** *For every history* ρ *— no honesty, no sortedness hypotheses, a pure simulation:*

$$\mathrm{sFold}_\Gamma\ (\rho.\mathrm{map}\ \mathrm{liftOp})\ =\ (\mathrm{eFold}_\Gamma\ \rho).\mathrm{map}\ \mathrm{liftRec},$$

the sided fold of an all-R-lifted history IS the one-sided fold, record for record (the sort keys coincide definitionally), so every one-sided theorem transports; the reads corollary `sided_allR_read_eq` *agrees id-and-element for id-and-element, and chained with tier 3 and Theorem 30.6, the all-R sided instance = the naive text buffer = the published tombstoned RGA.* `sFold_liftOp`

Also in act one: `sided_fold_subtree_convex`, subtrees display as contiguous blocks in any chain-generated fold, so runs that chain never interleave (`fugue_reachable_runs_no_interleave`). *Act two*: the Weidner–Kleppmann maximal-non-interleaving Definition 4 was formalized over reachable configurations (`SidedRGA_Fugue.lean`) and REFUTED twice for this Fugue policy on reachable traces (the recency tiebreak reverses the paper's lowest-ID-first; their Figure-7 execution): the paper's own Fugue-vs-FugueMax separation, landing on this implementation. *Act three*: FugueMax realized as a kernel variant (`SidedMax_ChainLex.lean`): its sibling order depends on the right ORIGIN, not expressible by any re-banding of deltas (the mirror control machine-witnesses this), so R-entries carry the mint-time successor's key as an order-reversing tag; condition (3) is proved in full (`fuguemax_same_origin_low_first`, the positive twin of the refutation), and full Theorem 9 has since been **closed** (§24: the traversal theory discharges condition (1), the corrected backward exception discharges condition (2), and `fuguemax_maximally_noninterleaving` is kernel-clean). The sided sequential spec also landed (`sided_seq_sound`: a sentinel-rooted two-sided buffer, R-inserts splice after their anchor, L-inserts immediately before). The tag has a COST: at contested inserts the coordinate embeds the successor's key, $\Theta(|\text{key}|)$ where plain Fugue pays zero, and whether that retention is a lower bound for tombstone-free maximal non-interleaving is an open question with a Myhill–Nerode shape (task #89; Open Question 43.17): the entropy law of §20 meeting the ordering intent.

Provenance. `whiteboard/litmus/embed_tree.py` (the design, the timestamp-oblivious control, the credential countermodel, the IEEE 754 measurements; `python3 embed_tree.py` / `fp` / `code`); `litmus.py`, `pbt.py` (battery and DAG harness); `contest_tree.py` (lockstep harness, CE regressions); `whiteboard/retention-countermodel.pdf` (the retention arc that forced this design); `whiteboard/delta-tree-design.pdf` (the ledger-based sibling design and the invariants I1–I3); `whiteboard/litmus/related-work.md` (verified source list: Roh et al. JPDC'11; Attiya et al. PODC'16; Kleppmann et al. PaPoC'19; Weidner & Kleppmann TPDS'25 (Fugue); Gentle & Kleppmann EuroSys'25 (Eg-walker, arXiv:2409.14252); Preguiça et al. ICDCS'09 (Treedoc); Grishchenko & Patrakeev PaPoC'20 (Chronofold); Litt et al. CSCW'22 (Peritext); Gomes et al. OOPSLA'17; OpSets 2018; Kaki et al. OOPSLA'19; Peepul PLDI'22; RA-lin OOPSLA'25; VeriFx ECOOP'23; Figma/Wallace fractional indexing; Weidner position-strings).

Part III
Sequential specifications: convergence is not correctness

Replication-aware linearizability certifies that every version of a replicated data type is the fold of some delivery-respecting linearization of its events. The reference point of that guarantee is the datatype's *own* fold: a bug in `do` appears identically on every replica, converges perfectly, and passes every convergence theorem. This part records the campaign that closes the gap. For every production RDT in Sal there is now a kernel-checked theorem that the fold of any single-replica history equals the output of a straightforward sequential program, the one a programmer would write with no replication in mind. The campaign's unifying observation is the *witness discipline*: sequential interfaces address state positionally (`dequeue()` is nullary, `insert` names an index), replicated operations cannot, and every replicated operation therefore carries the identity of what its issuer *observed* the positional referent to be; the honesty side conditions of the theorems are exactly the faithfulness of those witnesses. The campaign's negative results carry as much information as its theorems: the rehoming tombstone-free RGA is refuted at this specification on a four-op single-replica trace, its fused Peritext inherits the refutation at the render, and Shesha passes sequentially while failing at the merge, so do-faithfulness and merge-faithfulness are independent axes, and both separations are kernel-checked. A final section states what the campaign does *not* give: per-replica soundness and convergence do not compose into "the merged state is some user's sequential result," and the precise corner where composition fails is the dead-anchor corner the sequence-CRDT trilemma names. Every claim below names its Lean identifier.

31 The specification gap

The framework's central guarantee is RA-linearizability at every reachable configuration: each version the store registers is fold(ρ) for some enumeration ρ of the version's event set that respects delivery order. This is the right guarantee for *replication*: the merge invented nothing, reordered nothing it was not entitled to reorder, and all replicas agree.

It says nothing about whether the fold is *right*. The reference sequence is the datatype's own `do`; if `do` is wrong, every replica is wrong in unison (Figure 13). Two incidents made this concrete in this repository. First, a mark-extent defect in an early Peritext render passed every convergence obligation and was caught only by eye. Second, and decisively: the rehoming tombstone-free RGA carries a fully general, kernel-clean RA-linearizability capstone (`rga_ra_linearizable3_eq`), and its `do` nonetheless reorders surviving text when an interior element is deleted, with no concurrency anywhere in the history (§36). Both defects are invisible to convergence because convergence is self-referential.

The remedy is an independent specification. For sequential behaviour there is a canonical one: *the sequential program a programmer would write for the same operations with no replication in mind*. A counter is addition. A queue is a list. A text buffer is insert-at-position and delete. A marked-text editor is a buffer of characters and boundary tokens with one left-to-right walk.

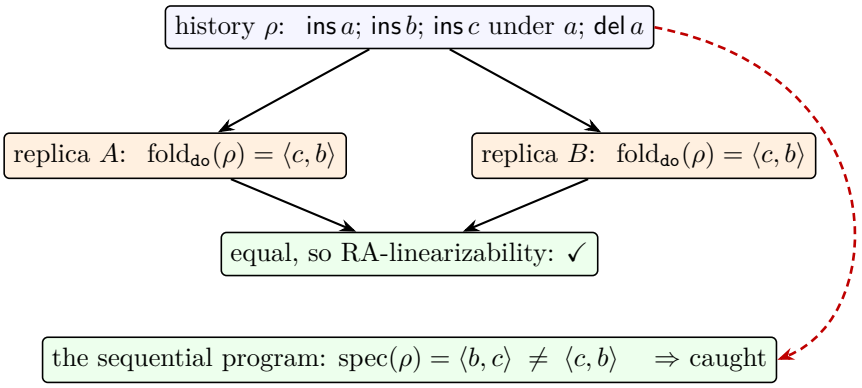


Figure 13: Why convergence cannot see a `do` bug. A defective fold is applied identically everywhere, so all replicas agree and every convergence theorem passes over the shared mistake. Only a reference computed by something *other than* `do` (dashed) can disagree. The history shown is the real refutation trace of §36.

32 The obligation, and the rules of engagement

Convention (Part III plumbing). Three pieces of plumbing are folded out of every Part III display below; Part I's convention (§3) and Part II's (§19) remain in force. (vi) *The single-replica fold*: histories ρ are plain lists of events (t, r, o), and the fold is always seqFold of Definition 32.1; "the view" of a datatype means its named user-visible projection: the view column of Table 3 for the flat tier and, for the queue and the sequence datatypes, the value map and the record projections eProj/sProj. (vii) *The prefix-split honesty shape*: every honesty predicate below quantifies over decompositions $\rho = \sigma +\!\!+ o :: \tau$ in the one shape of Definition 33.1, so per-datatype displays state the two clauses at the split and elide the (identical) quantification. (viii) *Snoc/prefix plumbing*: the step lemmas (`seqFold_snoc` and the per-spec `*SpecFold_snoc`) and the prefix-closure lemmas (`*OK_prefix`) are induction plumbing, extending Part I's `applySeq` fold, and never appear in displays.

Definition 32.1 (the sequential fold, and the campaign obligation)**.** *For a datatype D and a single-replica history ρ, the sequential fold is the datatype's own replay from its initial state,*

$$\text{seqFold } D\ \rho \ =\ \mathsf{fold}(\sigma_0, \rho) \quad (\textit{the } \texttt{applySeq} \textit{ fold of §3, at } D),$$

and the campaign's obligation, per datatype, is the schema

$$\textit{for every sequentially honest single-replica history } \rho : \\ \text{view}\,(\text{seqFold } D\ \rho) \ =\ \text{spec}(\rho),$$

with spec *a sequential program written from the datatype's user-facing intent and* view *its user-visible projection. Every `*_seq_sound` theorem of this part is an instance of the schema.* `seqFold`

The obligation is proved by discovering an inductive invariant of the fold, never by weakening the spec to match the implementation. "Sequentially honest" is the datatype's own issuing discipline specialized to a linear history (fresh, monotone stamps; each operation applicable at the state it executes against), so the theorems attack no strawman: the histories quantified over

are exactly the ones a single well-behaved replica produces, and in every case the side condition reuses a contract the convergence proofs already consume.

Three method rules were fixed in advance and held throughout. *Falsifiable statement first*: each tier states its equality before proof work begins, and a refutation is a first-class outcome (three landed, §36). *The spec is external*: the sequential program is written from the datatype's user-facing intent, never read off the implementation; where the two disagree, the disagreement is the result. *Tests carry both polarities*: every concrete test block pairs its expected values (hand-derived, never evaluated out of the code under test) with should-fail companions, so the harness demonstrably discriminates; the refutation files pair each negative with the positive prefix on which the two sides still agree, isolating the departing operation exactly.

33 The observation gap: positional interfaces, witnessed ops

Writing out the specifications exposes one structural pattern, and it is the campaign's most instructive artifact. A sequential structure addresses its state *positionally or implicitly*: `dequeue()` takes no argument because "the head" is unambiguous; `insert(i, e)` names an index; `write(v)` implicitly overwrites everything before it. A replicated datatype can afford none of this, because positions and implicit referents are not stable under concurrent edits: by the time an operation is applied elsewhere, "the head" may be a different element and index i a different character. So the replicated operation carries a **witness**: the identity (tag, stamp, id, coordinate, snapshot) of what the issuing replica *observed* the positional referent to be, at its own state, at issue time (Figure 14).

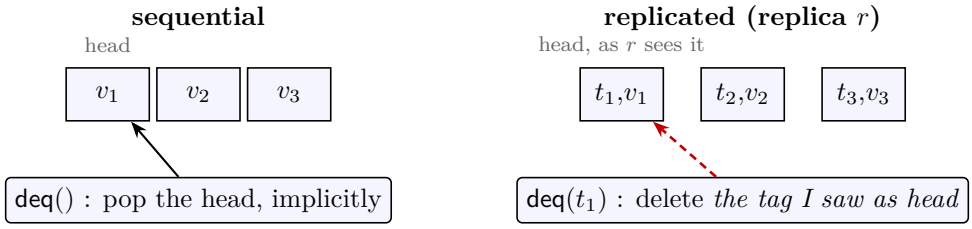


Figure 14: The observation gap, on the queue. The sequential `dequeue` is nullary; the replicated one names the tag its issuer read as the head of its own replica (dashed: the witness). Honesty is the faithfulness of the witness: `qOK` states that the fold at the issue point is literally $(t_1, v_1) :: \mathit{rest}$.

The sequential-honesty side conditions of §32 are then exactly the statement *the witness is what the sequential operation would have seen*, and each campaign theorem says: under that discipline, the witnessed-identity program implements the positional program. Where no witness is needed the alphabets coincide and the spec is a view homomorphism. The whole catalogue of §34 sorts by this principle, and the summary is one sentence: *every honesty condition in the campaign is the faithfulness of a witness that replaced a positional argument.*

Definition 33.1 (sequential honesty is witness faithfulness)**.** *Every campaign honesty predicate has one shape: for every decomposition* $\rho = \sigma +\!\!+ o :: \tau$*,*

(freshness) *o's stamp is new at* seqFold $D\ \sigma$*, and*

(witness faithfulness) *o's carried witness is what* seqFold $D\ \sigma$ *shows.*

The six mechanized instances:

predicate	*freshness*	*witness faithfulness*
`stampsMono`	*stamps strictly increase:* $\rho.\text{Pairwise}\,(o.1 < o'.1)$	*none: the stamp itself is the arbitration payload*
`mvrOK`	*o's stamp staked under no value*	*the snapshot O lists exactly the visible stamps*
`qOK`	*the* `enq` *tag is unstaked*	`deq` t*: the fold is literally* $(t, v) :: \mathit{rest}$
`eSeqOK`	*the stamp exceeds every prior insert stamp*	`eApplicable`*: the anchor is live, carrying exactly* π *(Definition 34.8)*
`sSeqOK`	*same*	`sApplicable`*, two-sided (Theorem 35.5)*
`StepsHonest`	`fresh_ts` *at the state*	`accurate`*: the claimed path is the true ancestor chain (Definition 36.1)*

`stampsMono` *constrains stamps pairwise rather than by split (equivalent to freshness at every split), and* `StepsHonest` *recurses along the history rather than quantifying over splits: the same content in structural clothing. The remaining four are literally the displayed* $\forall$*-split shape.* `stampsMono`*,* `mvrOK`*,* `qOK`*,* `eSeqOK`*,* `sSeqOK`*,* `StepsHonest`

34 The specifications, datatype by datatype

34.1 No gap: the alphabets already coincide

For ten of the thirteen flat datatypes the RDT operation is literally the sequential operation; the state carries invisible bookkeeping (tags, per-replica slots) and the spec is the evident program on the user-visible view. Table 3 is the tier's display, one row per datatype, every theorem cell a resolving Lean identifier (`SeqSpec_Flat.lean`, thirteen datatypes, fourteen theorems: the AWPQ's spec is two-part). No honesty hypothesis appears in the first ten rows: those theorems quantify over *all* histories, each view a fold homomorphism, no invariant needed. The three rows below the rule are the gap cases of the next two subsections; their honesty column names the one extra hypothesis each theorem carries.

34.2 Gap: arbitration by stamp instead of program order

LWW and FWW registers. The sequential register is `write` v with *program order* deciding who wins; the spec programs are "the last write's value" (`lwwSpecFold`) and "the first write's value" (`fwwSpecFold`). The RDT operation carries its Lamport stamp *as the arbitration key in the payload* (a max- resp. min-semilattice on stamped writes), because concurrent replicas have no shared program order to appeal to. The gap is a transfer of ordering authority from control flow to data, and the honesty condition is precisely its repair: `stampsMono` (stamps strictly increase along the history, the sequential Lamport condition), under which the arbitration key agrees with program order and `lww_seq_sound` / `fww_seq_sound` recover last- and first-write semantics. Without `stampsMono` the statements are false, and should be: a replica that backdates a write is asking LWW to disagree with program order.

34.3 Gap: overwrite by snapshot instead of "everything"

Multi-valued register. The sequential `write` v implicitly overwrites everything before it. The RDT operation is `write` (v, O) where O is a **prepare-time snapshot**: the write-stamps the issuer

datatype	view	spec program	honesty	theorem
Counter	the state ($\mathbb{Z}$)	the history length	—	`counter_seq_sound`
Incr.-only counter	the state	the history length	—	`ioc_seq_sound`
PN-counter	the state	`pnSpec`: $\#\mathsf{inc} - \#\mathsf{dec}$	—	`pn_seq_sound`
Grow-only set	membership	some op added x	—	`goset_seq_sound`
Grow-only map	membership at (k, v)	some op added (k, v)	—	`gomap_seq_sound`
G-set (conditioned)	`gsetView`	some op added x	—	`gsetcond_seq_sound`
OR-set	`orView`	`orSpecFold`: a plain add/remove set	—	`orset_seq_sound`
OR-set (efficient)	`orEView`	`orSpecFold`, the same program	—	`orsete_seq_sound`
Enable-wins flag	`ewView`	`ewSpecFold`: one boolean	—	`ewflag_seq_sound`
Add-wins PQ	`awpqMemView`; the inc component	`awpqSpecFold` (inc inert); the issued-increment log	—	`awpq_mem_seq_sound`, `awpq_inc_log_sound`
Multi-valued reg.	`mvrView`	`mvrSpecFold`: the last write	`mvrOK`	`mvr_seq_sound`
LWW register	`lwwView` $\circ$ `lwwSt`	`lwwSpecFold`: the last write	`stampsMono`	`lww_seq_sound`
FWW register	`fwwView` $\circ$ `fwwSt`	`fwwSpecFold`: the first write	`stampsMono`	`fww_seq_sound`

Table 3: Tier 1, the flat datatypes (`SeqSpec_Flat.lean`). Each theorem states: the view of seqFold equals the spec program's result, under the honesty column's hypothesis (none above the rule). Views are the file's named projections; the two counters read the state itself, and `counter_seq_sound` / `ioc_seq_sound` conclude seqFold $D\ \rho = |\rho|$ directly.

currently sees, named explicitly so that a *concurrent* write (absent from O) survives, which is the point of an MVR. The sequential spec is still the plain last-write register (`mvrSpecFold`: the final write's value, none on the empty history).

Definition 34.1 (MVR sequential honesty)**.** mvrOK ρ*: for every decomposition* $\rho = \sigma +\!\!+ o :: \tau$*,*

$$\forall v'.\ (\text{seqFold MVR } \sigma).1\,(o.1, v') = \mathsf{false},\ \textit{and}$$
$$\forall w\, O.\ o\ \textit{carries}\ \mathsf{write}(w, O) \to \big(\forall n.\ n \in O \iff \text{mvrVis}\,(\text{seqFold MVR } \sigma)\ n\big) :$$

the stamp is fresh (staked under no value), and the overwrite snapshot O *lists exactly the issuer's visible stamps (`mvrVis`: staked, not overwritten). This is the formal form of "I overwrite precisely what I can see."* `mvrOK`

Theorem 34.2 (the discovered invariant: overwritten stamps are staked)**.** *For every* ρ *with* mvrOK ρ*:*

$$\forall n.\ (\text{seqFold MVR } \rho).2\ n = \mathsf{true} \ \to\ \text{mvrTag}\,(\text{seqFold MVR } \rho)\ n,$$

every overwritten stamp is a staked tag (`mvrTag`), which is what makes a fresh stamp provably not-yet-overwritten. The campaign's first genuinely discovered invariant. `mvr_over_tags`

Theorem 34.3 (MVR, sequentially a last-write register)**.** *For every* ρ *with* mvrOK ρ *and every value* v*:*

$$\text{mvrView}\,(\text{seqFold MVR } \rho)\ v \iff \text{mvrSpecFold } \rho = \mathsf{some}\ v,$$

the visible-value view of the fold is the last write. `mvr_seq_sound`

34.4 Gap: dequeue() becomes delete-what-I-saw

The mergeable queue, the cleanest specimen (Figure 14). The sequential queue is

$$\mathsf{enq}\, v : S \mapsto S +\!\!+ [v] \qquad\qquad \mathsf{deq}() : S \mapsto \mathrm{tail}(S).$$

The RDT operation is $\mathsf{deq}\, t$: remove *by tag*, t being the tag of the element the issuing replica read as the head of its own queue; the implementation filters t out of the whole list, wherever it sits.

Definition 34.4 (queue sequential honesty, and the FIFO spec)**.** qOK ρ*: for every decomposition* $\rho = \sigma +\!\!+ o :: \tau$*,*

$$\forall v.\ o \textit{ carries } \mathsf{enq}\, v \to o.1 \notin \mathrm{qTags}\,(\mathrm{seqFold}\ Q\ \sigma),\ \textit{and}$$
$$\forall t.\ o \textit{ carries } \mathsf{deq}\, t \to \exists v\ \textit{rest}.\ \mathrm{seqFold}\ Q\ \sigma = (t, v) :: \textit{rest} :$$

enqueue stamps are fresh, and a dequeue's witnessed tag is literally the current head. The spec program keeps the sequential shape, blind to the tag: qSpecStep *appends the value on* $\mathsf{enq}\, v$ *and takes* tail *on* $\mathsf{deq}\, t$*;* qSpecFold *folds it from* $[\,]$*.* `qOK, qSpecStep, qSpecFold`

Theorem 34.5 (the discovered invariant: tags stay duplicate-free)**.** *For every* ρ *with* qOK ρ*:* $(\mathrm{qTags}\,(\mathrm{seqFold}\ Q\ \rho)).\mathrm{Nodup}$*. Fresh stakes append disjointly, dequeue filters, and nodup survives sublists; this is exactly what makes filter-by-tag remove one element.* `q_tags_nodup`

Theorem 34.6 (tier 2: the mergeable queue, sequentially a plain FIFO)**.** *For every* ρ *with* qOK ρ*:*

$$(\mathrm{seqFold}\ Q\ \rho).\mathrm{map\ snd}\ =\ \mathrm{qSpecFold}\ \rho,$$

delete-what-I-saw implements pop-the-head. `queue_seq_sound`

The theorem is kernel-checked list induction end to end; the campaign's in-file `#print axioms` audit trail lives in the sided, Peritext, and refutation files (§39 collects the provenance). The same witness discipline is what the convergence side already consumed (`qHonest_of_applicable`); nothing was invented for the occasion.

34.5 Gap: insert-at-index becomes insert-after-who-I-saw

The embedded-chain RGA. The sequential text buffer is $\mathsf{insert}(i, e)$ and $\mathsf{delete}(i)$, index-addressed. The RDT operation is $\mathsf{ins}(e, \pi, a)$: a is the *id* of the element the issuer saw at position $i - 1$ (its anchor; the sentinel 0 for the head), and π is the anchor's stored coordinate as the issuer read it (the proof-carrying prefix); $\mathsf{del}(x)$ names the id seen at position i.

Definition 34.7 (the buffer in id-clothing)**.** *The spec program over* (id, element) *pairs:*

$$\mathrm{eSpecStep}\ S\ (t, r, \mathsf{ins}(e, \pi, a)) = \begin{cases} (t, e) :: S & a = 0 \\ \mathrm{eInsAfter}\ a\ (t, e)\ S & \textit{otherwise,} \end{cases}$$
$$\mathrm{eSpecStep}\ S\ (t, r, \mathsf{del}(x)) = S.\mathrm{filter}\,(p.1 \neq x),$$

with eInsAfter *a p splicing p immediately after the entry carrying id a;* eSpecFold *folds it from* $[\,]$*. On a sequential history the anchor id determines the index uniquely, so this is the positional program with positions named by their occupants.* `eInsAfter, eSpecStep, eSpecFold`

Definition 34.8 (embed sequential honesty)**.** eSeqOK Γ ρ*: for every decomposition* $\rho = \sigma +\!\!+ o :: \tau$*,*

$$\big(\forall x \in \text{eInsIds } \sigma.\ x < o.1\big) \ \wedge\ \text{eApplicable } o\ (\text{eFold}_\Gamma\ \sigma),$$

the stamp exceeds every prior insert stamp (the Lamport condition), and the op is applicable at the current fold. `eApplicable` *is Part II's issuer-side guard: an insert's anchor precedes its stamp ($a < t$) and either is the sentinel with the empty prefix ($a = 0$, $\pi = [\,]$) or is live carrying exactly π as its stored coordinate; a delete's target is live.* `eSeqOK`

The theorem (Theorem 35.2) is the strongest form in the campaign: the canonical sorted state equals the spec buffer *record for record*. The same spec, taken at the rehoming op alphabet (ids only, values dropped), is what the rehoming RGA is refuted against (§36); its gap is not the witness (the rehoming op carries one too, plus a path) but the implementation moving *other* elements' referents on delete.

34.6 Gap: one mark becomes a boundary pair

Fused Peritext. The sequential marked-text editor has an *atomic* $\mathsf{mark}(i, j, m)$. The fused RDT has no such operation: a mark is **two** ordinary inserts of boundary elements $\langle m\rangle$ and $\langle /m\rangle$ sharing a fresh mark id, anchored at the range ends; mark removal is two deletes. An atomic positional operation is traded for a *pair* of witnessed inserts, which is the atomicity horn of the mark-positioning trilemma, accepted knowingly. The spec therefore lives at the token level.

Definition 34.9 (the naive marked-text editor)**.** *The editor's document is a plain buffer of* (id, element) *entries over* char $\oplus$ boundary*:* editorFold ρ = eSpecFold ρ *at the rich payload, Definition 34.7 verbatim. Its screen is one left-to-right walk maintaining the open-mark set: a start boundary opens its mark, the matching close removes it, characters display with what is open, boundaries are invisible (*editorRender*); the id-tagged variant* editorRenderIds *additionally reports each character's id.* `EditorBuf`*,* `editorFold`*,* `editorRender`*,* `editorRenderIds`

Theorem 35.6 says the datatype's render is that editor's screen, id for id in the `_ids` variant; the range-level reading is recovered by the positional intent theorems (`render_id_active_iff_between`: a character carries mark m iff it sits, in reading order, strictly between m's boundaries; an element-list-level theorem of the shared read layer, `Peritext_Rehoming/Peritext_Read.lean`, applying to this instance's `docElts` verbatim). The pair encoding is also why the character/boundary distinction is load-bearing in the delete theorem: deleting a *character* re-formats nothing (`renderIds_del`), while deleting a *boundary* is mark removal and must re-format its span (machine-checked necessary: `boundary_delete_breaks_filter_eq`). The boundary-pair encoding is one of the repository's two mark models; the other, the document-order model of separate mark records that the runtime ships, with its own read rules, no-leak theorem, and garbage collector, is §27.

35 The proofs: four tiers, three kinds of invariant

The campaign ran easiest to hardest, and the difficulty gradient turned out to be a gradient of invariant *kinds*.

Tier 1 (the thirteen flat datatypes, fourteen theorems), `SeqSpec_Flat.lean`. Table 3 is the display. Mostly view homomorphisms, no invariant at all. The one exception needs *identity hygiene*: a fact about tag bookkeeping that holds on every reachable single-replica state and is exactly what the induction consumes: the MVR's `mvr_over_tags` (Theorem 34.2, overwritten stamps are staked). The AWPQ, despite its two-part spec, needed none: both of its theorems are invariant-free fold inductions.

Tier 2 (the mergeable queue), `MergeableQueue_SeqSpec.lean`. One identity-hygiene invariant, `q_tags_nodup` (Theorem 34.5), feeding the capstone Theorem 34.6 of §34.4.

Tier 3 (the embedded-chain RGA), `EmbedRGA_SeqSpec.lean`. The invariant becomes *geometric*. The spec splices an insert immediately after its anchor; the datatype sorts records by birth-chain coordinates; the bridge is the adjacency lemma (Figure 15), whose proof costs only sum-telescoping: a chain's deltas sum to its id, and every existing id is below the fresh stamp.

Theorem 35.1 (the adjacency lemma)**.** *For chains $c_r \neq c_a$ and a delta δ exceeding every delta c_r hangs directly off c_a (whenever $c_r = c_a +\!\!+ d :: rest$, $d < \delta$):*

$$\text{chainBefore } c_r \ (c_a +\!\!+ [\delta]) \iff \text{chainBefore } c_r \ c_a,$$

appending the fresh delta changes no display verdict against the anchor: the newcomer lands immediately after it. ***`chainBefore_snoc_iff`***

Theorem 35.2 (tier 3: the embed RGA, sequentially the naive buffer)**.** *For every ordered prefix code Γ and every ρ with* eSeqOK Γ ρ *(Definition 34.8):*

$$(\text{eFold}_\Gamma \ \rho).\text{map eProj} \ = \ \text{eSpecFold } \rho,$$

the canonical sorted state is the spec buffer, record for record. ***`embed_seq_sound`***

Two transports come for free. The published tombstoned RGA is read-equivalent to the embed on honest event sets (the compaction theorem, §30), so it inherits the buffer spec:

Theorem 35.3 (the tombstoned-RGA transport)**.** *For every Γ, every ρ with* eSeqOK Γ ρ*, and every id list L that reads the published RGA's fold, i.e. L pairwise-sorted by* visible_lt (rgaFold ρ) *and $\forall t.\ t \in L \iff$* visible (rgaFold ρ) t*:*

$$L \ = \ (\text{eSpecFold } \rho).\text{map fst},$$

and the conclusion never mentions the code. Element agreement is the companion ***`rga_seq_read_element`****: every buffer entry (t, e) is read visible by the published RGA at id t with element e. (`EmbedRGA_SeqSpec_RGA.lean` is a standalone build target: the two RGA models share top-level names.)* ***`rga_seq_read_eq_buffer`***

And the sided generalization simulates the one-sided instance on all-R histories unconditionally (Theorem 23.6, `sFold_liftOp`), so the buffer spec transports to the sided datatype's all-R fragment at zero proof cost. The full two-sided theorem needs the two sided adjacency lemmas at the kernel order:

Theorem 35.4 (the sided adjacency pair)**.** *For sided chains $c_r \neq c_a$ and a fresh delta δ:*

(R) if $d < \delta$ whenever $c_r = c_a +\!\!+ (\mathrm{R}, d) :: \mathit{rest}$, then

$$\text{schainBefore } c_r \ (c_a +\!\!+ [(\mathrm{R}, \delta)]) \iff \text{schainBefore } c_r \ c_a;$$

(L) if $d < \delta$ whenever $c_r = c_a +\!\!+ (\mathrm{L}, d) :: \mathit{rest}$, then

$$\text{schainBefore } (c_a +\!\!+ [(\mathrm{L}, \delta)]) \ c_r \iff \text{schainBefore } c_a \ c_r :$$

a fresh R-extension displays immediately after its anchor, a fresh L-extension immediately before it (the L band is mirrored: the newest L-child hugs the node from above). `schainBefore_snocR_iff`, `schainBefore_snocL_iff`

Theorem 35.5 (the sided capstone)**.** *The two-sided spec is sentinel-rooted:* sSpecFold *folds* sSpecStep *from the seed* $[(0, 0)]$*, where an R-insert splices immediately after its anchor's entry (*`sInsAfter`*), an L-insert immediately before it (*`sInsBefore`*), and delete filters;* sRFold_Γ *is the canonical fold seeded with the sentinel record* $(0, 0, [\,])$ *(*`rootRec`*), so both insert directions have a splice point and the front insert is no special case. For every* Γ *and every* ρ *with* sSeqOK Γ ρ *(monotone insert stamps and* `sApplicable` *at each unrooted fold, the prefix-split shape of Definition 33.1):*

$$(\text{sRFold}_\Gamma \ \rho).\text{map sProj} \ = \ \text{sSpecFold } \rho.$$

The read corollary `sided_seq_read` *drops the sentinel on both sides. Both carry in-file audits: axioms* {propext, Quot.sound}. `sided_seq_sound`, `sSeqOK`

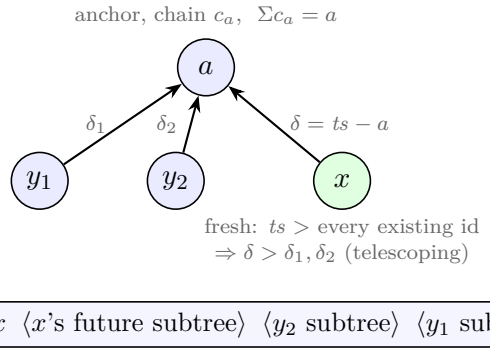


Figure 15: The adjacency lemma, the geometric heart of tier 3. Siblings display newest-first; a fresh insert's delta beats every existing child's because deltas telescope to ids and every existing id is below the fresh stamp. Hence the newcomer lands *immediately* after its anchor: exactly what the sequential buffer's splice does.

Tier 4 (fused Peritext), `PeritextEmbed_SeqSpec.lean`**.** The invariant kind is *composition*, and that is the finding.

Theorem 35.6 (tier 4: the render is the editor's screen)**.** *For every* Γ *and every rich-payload history* ρ *with* eSeqOK Γ ρ *(Definition 34.8 at* $\alpha :=$ PeritextElt*):*

$$\text{renderRichText}\,(\text{eFold}_\Gamma \ \rho) = \text{editorRender}\,(\text{editorFold } \rho),$$
$$\text{renderIds}\,(\text{eFold}_\Gamma \ \rho) = \text{editorRenderIds}\,(\text{editorFold } \rho),$$

screen for screen and, in the second form, id for id. In-file audits printed for both. `peritextEmbed_seq_sound`, `peritextEmbed_seq_sound_ids`

The proof is Theorem 35.2 made payload-generic (the fold's (id, element) sequence is the naive buffer) composed with one shared render fold (`editorRenderIdsAux_map_eProj`: the datatype's render and the editor's screen are literally the same pure fold over that sequence). No new invariant was needed. This is also why the tier had to wait for the re-based Peritext: against the rehoming kernel the same statement is false (§36).

36 The refutations

The statement shape is falsifiable per datatype, and three refutations landed, all machine-checked, each with its honesty guards discharged on the witness so the failure is never an artifact of a malformed trace.

The rehoming RGA fails at `do`. Figure 16. The refuted statement is displayed in full, then negated; its honesty guard is the datatype's *own* conditioning, so the failure is no artifact of a malformed trace.

Definition 36.1 (the refuted statement)**.** RehomingSeqSound*: for all op lists ops and id lists ids,*

$$\begin{aligned}&\text{StepsHonest } \mathit{ops} \to \mathit{ids} \text{ pairwise descending} \to\\ &\quad\big(\forall i.\ \text{contains}\,(\text{replay } \mathit{ops})\ i \to i \in \mathit{ids}\big) \to\\ &\quad\text{document}\,(\text{replay } \mathit{ops})\ \mathit{ids} \;=\; \text{bufFold } \mathit{ops},\end{aligned}$$

the exact statement shape tiers 1–3 prove: on every honest single-replica history, read with a complete descending candidate list, the document is the naive buffer. Here replay *is the datatype's own* `do_` *fold,* bufFold *the §34.5 buffer program at this op alphabet (*`bufIns` *after the named anchor, head for the sentinel, filter delete), and* StepsHonest *requires every op* `accurate` *(the claimed path is the true ancestor chain) and* `fresh_ts` *at the state it executes against: the same predicates the datatype's commutation layer conditions on.* `RehomingSeqSound`, `StepsHonest`, `bufFold`

Proposition 36.2 (refutation 1: the rehoming RGA is not the naive buffer)**.** ¬ RehomingSeqSound*. The witness opsW is four ops on one replica, stamps* $1, 2, 3, 9$*: insert a then b at the root (siblings re-sort newest-first, reading* $\langle b, a\rangle$*), insert c under a, delete a. Honesty is discharged on the witness (*`stepsHonest_opsW`*); the naive buffer keeps survivor order,* bufFold $\mathit{opsW} = \langle b, c\rangle$ *(*`bufFold_opsW`*), while the rehoming* `do` *re-homes c to the root, where the newest-first tiebreak leapfrogs it over b: the read is* $\langle c, b\rangle$ *(*`replay_opsW_read`*). The should-pass half (*`seq_agrees_before_delete`*) shows both sides agree on the delete-free prefix, isolating the delete as the exact point of departure. In-file audits printed for both halves.* `rehoming_seq_refuted`

The design's convergence capstone is untouched: it converges, to its own reordering fold. This refutation is what demoted the design from the repository's canonical seat.

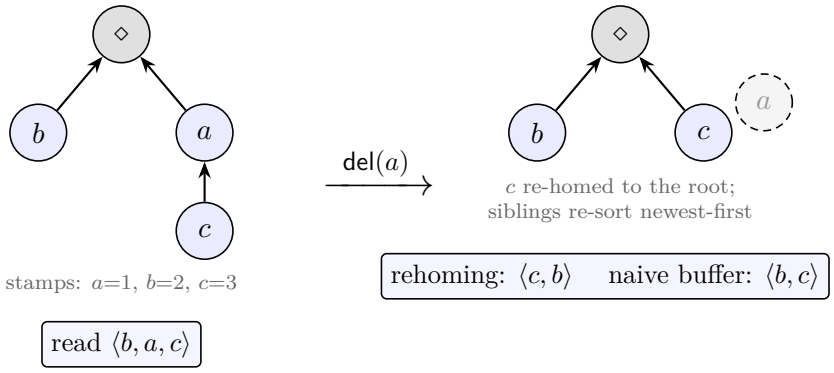


Figure 16: The four-op refutation. One replica, no merge. Deleting a swaps the surviving b and c in the rehoming RGA's read; the naive buffer keeps their order. The embedded-chain RGA reads $\langle b, c\rangle$ on the same trace: c's immutable coordinate still carries a's chain as pure geometry, so nothing moves.

Fused Peritext on the rehoming kernel fails at the render. The same reorder lifted to char $\oplus$ boundary.

Proposition 36.3 (refutation 2: a fused delete re-formats a survivor)**.** *On the rich-text witness* docResidual *(five* `do_` *steps; bold span exactly* $\{X\}$*, characters* P *and* C *plain), deleting the plain character* P *(id* 3*) and reading both sides at their complete descending candidate lists, projected at the bold mark:*

$$\begin{aligned}&\text{renderRichText}\,\big(\text{do\_ docResidual}\ (9,0,\mathsf{del}([2,1],3))\big)\\ &\quad\neq\ \big(\text{renderRichText docResidual}\big).\text{filter}\,(\mathit{codepoint}\neq P)\,:\end{aligned}$$

the post-delete render is not the pre-delete render minus the deleted character. The untouched C *re-homes past the close boundary into the bold span (*`fused_delete_moves_char_into_span`*), on one replica, no boundary touched. This upgraded the re-base of Peritext onto the embed kernel from housekeeping to a correctness fix.* `fused_delete_reformats_survivor`

On the embed kernel the corresponding *general* theorem holds, with the survivor claim quantified over every state:

Theorem 36.4 (character deletes never re-format, on the embed kernel)**.** *For every* Γ*, every state* s*, all* ts, r, x*, if* x *is a character in* s *(every record of* s *with id* x *carries some* char c*):*

$$\text{renderIds}\,\big(\text{eUpdate}_\Gamma\ s\ (ts, r, \mathsf{del}(x))\big)\ =\ (\text{renderIds}\ s).\text{filter}\,(e.1\neq x),$$

the post-delete render is the pre-delete render with exactly x*'s entries removed: every surviving character keeps codepoint and formatting predicate bitwise, because character entries are open-set-neutral and delete is a filter on an order-canonical state. In-file audit:* {propext, Quot.sound}*, no choice. The character hypothesis is machine-checked necessary: a boundary delete refutes the filter equation (*`boundary_delete_breaks_filter_eq`*), as it must, since a boundary delete is mark removal and re-formats its span. The user-facing corollary at the plain read is* `renderRichText_del`*.* `renderIds_del`

Shesha passes sequentially and fails at the merge. The mirror image.

Theorem 36.5 (Shesha's positive half: sequentially the naive linked list)**.** *Unconditionally, for every op list ops:*

$$\text{read}\,(\text{fold}\ \mathit{ops})\ =\ \text{seqFold}\ \mathit{ops},$$

the datatype's read is the naive-list fold: insert after the named anchor, filter delete. (This file's `seqFold` *is its own naive-list program, a name coincidence with Definition 32.1 that plays no role here.) No honesty hypothesis: the totality choices of both sides agree on every input. Corollary* `delete_preserves_survivor_order`*, on every state, not only reachable ones:* $\text{read}\,(\text{delete}\ s\ d) = (\text{read}\ s).\text{filter}\,(\cdot \neq d)$*; Shesha uniquely preserves survivor order under delete among the tombstone-free designs of its generation.* `sequential_soundness`

Proposition 36.6 (refutation 3: Shesha's merge admits no pre-splice row store)**.** *The row-store residue* `shesha_rows_residue` *(*`Shesha/Shesha_Presplice.lean`*, the one open obligation of the design's RA-linearizability plan) asserts: for every honest Shesha configuration with transitive irreflexive visibility, downward-closed branch event sets, canonical witnesses, and* `SCoh`*-aligned replays, some pre-splice row store (per-anchor rows, duplicate-free, id-bounded, reversing both branches' same-anchor insert orders) expands to every merged row. That statement is machine-checked* ***false****: its full quantifier prefix is negated wholesale by* `shesha_rows_residue_refuted` *(*`Shesha/Shesha_Rows_Refuted.lean`*, in-file audit printed), on the concrete core*

$$\text{row}\,(\text{mergeL}\ \mathit{st}_0\ \mathit{st}_1\ \mathit{st}_2)\ 0\ =\ [3,4,2]$$

(`cz_merge_read`*): the three-way merge splits the deleted marker* 1*'s children* $3, 2$ *around the concurrent sibling* 4*, an order no row store expands to, producing a read that is the fold of no delivery-respecting linearization.* `shesha_rows_residue_refuted`

37 The two-axis separation

Finding (do-faithfulness and merge-faithfulness are independent axes)**.** *Together the refutations witness the independence (Figure 17, every placement a named kernel theorem): the rehoming family occupies merge-yes/do-no (*`rga_ra_linearizable3_eq` *with Proposition 36.2), Shesha do-yes/merge-no (Theorem 36.5 with Proposition 36.6), and the embedded-chain family shows the top corner is achievable, by one design idea in both sequence datatypes: immutable birth coordinates, so that deletion moves nothing and the merge decides nothing.*

38 What composition does not give

It is tempting to compose the guarantees: RA-linearizability says every version is fold(ρ) for some delivery-respecting ρ over the event union; the campaign says folds of sequentially honest histories equal the spec program. May one conclude that every merged document is the naive program's output on *some* ordering of everyone's operations? **No**, and the failure is precise.

The campaign's theorems quantify over *sequentially honest* histories; an RA-linearizability witness is delivery-respecting but need not be sequentially honest on either count. Stamp monotonicity fails harmlessly: when the state is a function of the event set (fold-canonicity, Theorem 30.3, `e_fold_canon`, displayed in Part II and not re-displayed here), any enumeration can be re-sorted. Applicability is the real corner.

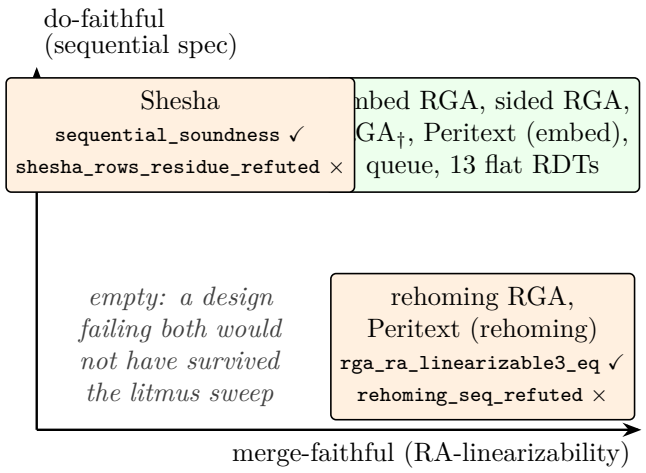


Figure 17: Every placement is a named kernel theorem. The two failure modes are orthogonal: converging to a reordering fold (bottom right) and folding correctly into a non-linearizable merge (top left).

Example 38.1 (the dead-anchor corner)**.** *Figure 18. In the union of two branches, an insert's anchor may be deleted by a concurrent operation carrying a smaller Lamport stamp: LCA* ins1*, branch A* ins2 *after* 1*, branch B* del1*. The merged fold keeps* 2 *(OR-set survival on immutable coordinates). The branch-order replay* [ins1, ins2, del1] *reproduces it, but in the stamp-sorted replay* [ins1, del1, ins2] *the delete precedes the insert, the anchor is dead at the insert's prefix, and the naive buffer no-ops the splice: buffer* $\langle\,\rangle \neq \langle 2\rangle$*, while the datatype (whose coordinates travel in the op) places the element correctly. Some delivery-respecting interleavings of the union match the spec program and some do not; deliberately, no Lean identifier is attached, because the openness is the point (Open Question 43.12).*

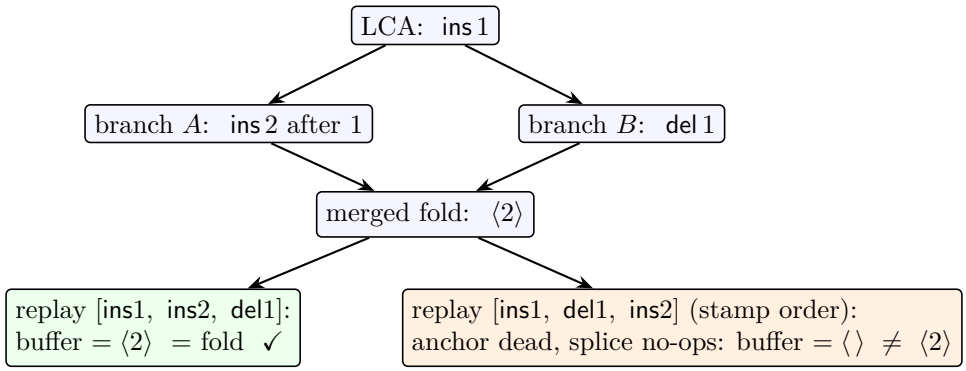


Figure 18: The dead-anchor corner. The merged fold keeps 2 (OR-set survival on immutable coordinates); whether a naive-buffer replay of the union agrees depends on which delivery-respecting order is chosen, and the stamp-sorted one can be wrong. This is the same corner the sequence-CRDT trilemma names for insert-after-already-deleted-anchor.

Whether a matching order always exists is genuinely open; it is recorded as Open Question 43.12 in the consolidated list.

Independently of the question, the composition that *is* sound and kernel-checked today: every version is the datatype's fold of a delivery-respecting enumeration (RA-linearizability); the fold is canonical per event set (Theorem 30.3); each replica's own contribution obeyed the sequential spec while it was being made (this campaign); and merge-level intent is delivered by separate, named theorems where it holds (delete-order preservation as sublist stability, subtree convexity as non-interleaving, read-equivalence to the published RGA as the compaction theorem). The campaign does not manufacture a merge-level user story; it pins the single-replica story so that merge-level claims have something sound to stand on.

39 The verified matrix

The anomaly-comparison matrix (task #64, `whiteboard/anomaly-matrix/`) ran six implementations and four production libraries through 25,917 merges per row, every cell backed by executed code. Its in-repo rows have since been upgraded from machine-run to kernel-checked; the table below is the matrix's load-bearing core with each cell a Lean identifier. External libraries (Yjs, Automerge, Loro, list-positions) remain empirical by nature; their rows live in the matrix report unchanged.

design	sequential spec (do)	RA-lin (merge)
embedded-chain RGA	`embed_seq_sound`	`embed_ra_linearizable3`
sided RGA (two-sided)	`sided_seq_sound`	`sided_embed_ra_linearizable3`
published RGA$_\dagger$	`rga_seq_read_eq_buffer`	`rgaWithTombstones_ra_linearizable3_eq`
Peritext (embed)	`peritextEmbed_seq_sound`	`peritextEmbed_ra_linearizable3`
mergeable queue	`queue_seq_sound`	`queue_ra_linearizable3`
thirteen flat RDTs	the fourteen theorems of Table 3	`*_ra_linearizable3_eq` (eleven; see note)
rehoming RGA	**refuted**: `rehoming_seq_refuted`	`rga_ra_linearizable3_eq`
Peritext (rehoming)	**refuted** (render): `fused_delete_reformats_survivor`	`peritext_ra_linearizable_up_to_eq`
Shesha	`sequential_soundness`	**refuted**: `shesha_rows_residue_refuted`

Table 4: The verified matrix: the load-bearing core of the anomaly comparison, every cell a resolving Lean identifier. Positive rows above the rule, the three refutations below it (§36).

Every positive theorem above is kernel-clean (axioms $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$, convention fold (iii) of §19); where an in-file audit is printed and recorded, its result can be stronger: `renderIds_del` audits at $\{\text{propext}, \text{Quot.sound}\}$, no choice (re-run and observed); the sided capstones audit at the full kernel-clean triple, choice included. Refutations by concrete witness follow the SPOT convention (`native_decide`, adding the compiler axioms); the refuted *statements* are quantified properties whose honesty guards are discharged on the witness. On the flat row: the sequential names are the lowercase per-datatype theorems of Table 3; the RA-lin capstones are capitalized (`ORSet_ra_linearizable3_eq`, …, `LWW_ra_linearizable3_eq`) and match the `*_ra_linearizable3_eq` pattern for eleven of the thirteen; the Counter and the conditioned G-set conclude through the conditioned-discharge route (`counter_ra_linearizable3_cd`, `gset_ra_linearizable3_cd`).

Part IV
The verified peer-to-peer runtime

The preceding parts are a metatheory and its instances: theorems about what a merge computes and what a compaction preserves. This part is a systems pillar built on them. The same design runs as a general peer-to-peer MRDT runtime, datatype-parametric over the framework signature, of which collaborative sequence editing is one instance; the runtime's garbage collectors are the executable form of Part I's GC-safety and stability theorems, its save format is the run table of Part II, and a benchmark measures it against production CRDT libraries on the standard editing-trace corpus. One boundary is drawn throughout: the Lean development is the verified artifact, and the JavaScript runtime is an *unverified transliteration* of the verified kernel that mirrors its semantics and whose GC callbacks correspond, named theorem by named theorem, to the mechanized results. The runtime does not re-prove convergence; it exercises the semantics the theorems certify and asserts the observable consequences (equal reads, reads unchanged by GC, git round-trip). Every performance number is a run of checked-in scripts on one machine, labeled measured or measured-in-model.

40 The runtime: one content-addressed replica

Datatype-parametric, proven over three datatypes. The core object is `DistributedReplica` (`runtime/src/replica.js`, tasks #94/#108): a single content-addressed commit store carrying three capabilities at once — delta gossip over the wire, the certified stability GC, and the keep-set commit GC — every one of them datatype-agnostic (`init`/`apply`/`merge3`/`read`) except state compaction, which a datatype opts into by additionally providing `compact`/`remapState`. The runtime is therefore not a text editor with a CRDT inside; sequence editing is one instance. The test suite runs convergence, the content-address round-trip and tamper gate, and the commit GC over the embedded-chain RGA and an OR-set, and the certified state GC over the RGA (refuse-then-fire) with the OR-set correctly declining it (an OR-set has no coordinate re-coding to do). The pieces that Part I proved generic run generically; the third datatype, rich text, follows the certified-GC paragraph below.

Content addressing: one hash for wire and disk. Separate peers assign different local ids to the same commit, so every commit is named by the SHA-256 of its canonical content, a Merkle DAG folding in parent ids (`runtime/src/hash.js`, #108): an authored commit hashes $\{\text{parents}, \text{replica}, \text{seq}, \text{payload}\}$, a merge commit hashes its *sorted* parent ids (so $\mathsf{merge}(a, b)$ and $\mathsf{merge}(b, a)$ are literally the same commit and never diverge into a spurious criss-cross), the root hashes $\{\text{root}\}$. Ingest is guarded: a peer recomputes each incoming commit's state (`apply` for authored, `merge3` for merges) and the recomputed content id must match the wire id, so a transmitted state is never trusted. The pure-JS SHA is checked bit-for-bit against the platform crypto library. The *same* id names a commit on the wire and on disk, which is what makes git persistence content-addressed with no second hash.

Delta sync, head-sync preserved. Two peers exchange their DAG frontiers (head content-ids), each computes the ancestor-set difference the other lacks, and ships it as a *delta*: per commit only the op payload, parent refs, and author, never a whole state. A whole-state snapshot (the

run-table serializer below) is used only for a genuine bulk catch-up, chosen when the delta would be larger (a brand-new or very-far-behind peer). Crucially a peer only ever merges its current head with a head another peer just advertised, never a stale interior commit: this is exactly the head-sync hypothesis the GC-safety theorem (Theorem 17.3, §17) consumes, made structural rather than advisory. Merges route through the same criss-cross gate as the in-process runtime.

The document is a git repository. Persistence writes one JSON file per commit named by its SHA, a heads file, and a materialized `doc.txt`, then commits them *in a git repository*; loading replays the commits into a fresh replica whose reads equal the original and which re-enters the live wire with identical SHAs. `git clone` of the repository therefore yields a self-contained, loadable document (clone is a join; a fresh replica id is a fork), and because `doc.txt` tracks the content, plain `git log -p` reads as sensible edit history. The content-address SHA is the git-object discipline reused: the same id on disk as on the wire.

Certified GC in the running system. The stability callback is the executable `settledAt_of_allHeard` (Theorem 18.4, §18). `compactStable` replaces asserted settledness with a *checked* certificate built from the frontier: if any registered replica has not been heard from since the cut, compaction is refused — a no-op reporting the missing replica — and fires only once that replica is heard from. Absence of evidence is refusal, never assumption; this is the runtime witness of the theorem's createReplica breaker. The directed test drives the discriminating-remove countermodel of §18: a lagging replica makes `compactStable` refuse, then fire when heard from, with reads identical to a never-compacted control throughout, and a FAIL companion in which the *old* asserted compaction at the same point diverges (a frozen-delta order flip), so the certificate's refusal is load-bearing, not pessimism. Under a certified cut the in-flight crutch vanishes: every op concurrent with the cut is already delivered (SettledAt), so passing an empty in-flight set is *proved* sufficient rather than asserted (a composite of Theorem 26.7 and Proposition 26.9; deliberately no single lemma). The keep-set commit GC runs off the same frontier read from above (§17), and the run-table serializer (§26.5) supplies the save format.

The third datatype: rich text, with a firing marks GC. Peritext ships as a ported datatype (`runtime/src/datatypes/peritext.js`): the document-order mark model of §27 layered on the embed shadow, plugging into `DistributedReplica` over the same `init/apply/merge3/read` contract with no Peritext-specific runtime code, so rich text inherits delta gossip, content addressing, and commit GC by parametricity. Its state GC no longer refuses: `compactStable` fires under the same certificate through the marks keep-set (`compact-peritext.js`), the executable face of Theorems 27.9 and 27.11, with the delete-must-settle refinement, the conservative deviations, and the measured suite and cost numbers of §27; the flip that justified the old refusal stays executable as a negative control.

Honest limits. The demo (`p2p-demo/`, task #95) shows several replicas editing one document over a real WebSocket transport, converging, saving to and cloning from git, and firing certified GC while the session is live. What it is not: the transport is a *star relay*, not a network-layer mesh (the transport interface is shaped so a WebRTC data-channel mesh drops in unchanged); there is no authenticated identity (the SHA gate bounds tampering with a commit's content, not who may author); open membership is cooperative, over the closed replica set the stability certificate and the commit GC both quantify over; and compaction is *linearized* by a coordinated

checkpoint barrier, because concurrent divergent compaction across replicas — two peers compacting incomparable cuts and then merging across epochs — is the deferred protocol half, the multi-replica layer above the single-replica lemma (∗) of §26.2 (closed there: Definition 26.21, Theorem 26.24; the residue is OQ 43.7). A cross-epoch merge throws rather than guess. And the datatype is the unverified transliteration noted above.

41 Performance

Where does the shipped artifact sit against production CRDT libraries? The benchmark (`benchmarks/`, task #98) runs the runtime against Yjs 13.6, Automerge 3.3 (wasm), Loro 1.13 (wasm), and list-positions 2.0 on three workloads, and every cell is a run of the checked-in scripts on one machine (node v26.3.1, Apple M4 Pro, 24 GB); nothing is quoted from literature, and each system's convergence and final text are gated before any number is reported. The corpus is the standard collaborative-editing replay set (the josephg traces: `automerge-paper` is Kleppmann's LaTeX-editing trace, `seph-blog1` is Gentle's blog draft, and `friendsforever/clownschool` are concurrent sessions, here also replayed single-author as the `_flat` twins). One honesty note up front: we benchmark *on* this corpus but not *against* Eg-walker's own implementation (§28), which is an event- graph replay engine rather than a state library; that comparison is a genuine gap.

Per-metric methodology, stated natively. *Apply* is timed with a high-resolution clock around each single-character op, reported as median and p95 after a 1000-op warmup (JIT and wasm lazy init); timer overhead of 30 to 60 ns per op is *not* subtracted and biases the sub-microsecond medians upward. For ours each op includes the adapter's position-to-id bookkeeping (an id-array splice) on top of the datatype `apply`. *Save size* is the bytes of each library's native serialization, and what a save *contains* differs, so every row is labeled: Automerge's `save` is the full change history (compressed; no state-only mode exists), Loro's `snapshot` carries state plus history while `shallow-snapshot` drops history at the frontiers, Yjs's update drops deleted content but not tombstone id structure (v2 run-length codes the same content), list-positions saves live characters plus position metadata as JSON, and ours saves live state only (delete is pure removal). Ours is reported in four labeled representations, discussed below. *Load* is from the primary save into a fresh document, median of five, including materializing the text once; for ours it includes re-deriving the display order (the coordinate sort). *Merge/sync* times the bidirectional exchange per round in the concurrent workload. *Heap* numbers are flagged not comparable across the wasm boundary (Automerge and Loro hold state in wasm linear memory, invisible to the JS heap counter). Labels throughout distinguish *measured* from *measured-in-model*.

The former headline loss: apply latency, and the experiment that closed it. As first shipped, per-character apply lost to production by three to four orders of magnitude, and the previous revision of this section diagnosed the loss as *interface-inherited*: the persistent `apply` returned a fresh `Map`, an O(live set) copy per keystroke, linear in the document (order machinery contributes nothing measurable: minting a coordinate is $O(1)$ and inserts never compare), and predicted that a transient or batched apply path would reach the microsecond class without touching the verified semantics. Task #111 ran that experiment: the state container is now a persistent hash-array-mapped trie (`pmap.js`, dependency-free; `set` returns a new root sharing all

but an $O(\log n)$ path, with transients for batched application and deterministic sorted iteration), the three-way merge became a delta merge from the LCA, and nothing else moved. The swap is licensed by `tableOf_congr` (the state is a function of the live-chain set, §26.3); the datatype interface, commit granularity, and wire format are unchanged, and every save byte, fingerprint, and content id is bit-identical (the full suites at the swap, 92 runtime plus 10 p2p tests, since grown to 103 runtime with the marks-GC tests of §27.6; the serializer gates and the Python projection cross-check, bit-for-bit). The diagnosis was confirmed with margin (median apply, by trace and final size):

apply, median (final chars)	`ff_flat` (21k)	`cs_flat` (21k)	`seph` (57k)	`am-paper` (105k)
ours (shipped, persistent HAMT)	0.50 µs	0.38 µs	3.96 µs	4.08 µs
ours, copied-Map interface (pre-#111)	364 µs	347 µs	1.66 ms	3.16 ms
Yjs	1.5 µs	1.7 µs	1.4 µs	1.4 µs
Automerge	24 µs	26 µs	27 µs	27 µs
Loro	0.6 µs	0.8 µs	0.6 µs	0.5 µs
list-positions	0.6 µs	0.7 µs	1.0 µs	0.7 µs

That is 418 to 913× on the medians (whole-trace replay on `automerge-paper`: 713 s down to 1.94 s), landing in the same decade as Yjs and Loro, with Automerge now the slowest applier in the table. The residual size trend in our row (0.4 to 4 µs) is not the datatype: instrumented separately, the datatype `apply` is a flat 0.49 µs mean on `automerge-paper`, and the growth is the benchmark adapter's own $O(n)$ position-to-id bookkeeping (an id-array splice per op), outside the datatype. Allocation behavior improved with the copies gone: peak heap on `automerge-paper` fell from 433 to 132 MB, with retained heap at 4 to 22 MB (the trie retains structurally what the copies used to churn; absolute coordinates still share prefixes as cons strings, so the bit cost materializes only at serialization). Load sits at 38 to 289 ms (JSON parse plus the coordinate sort) against Loro's sub-millisecond snapshot load, though within range of Automerge's full-history load (on `automerge-paper`, ours 289 ms versus Automerge's 319 ms).

Where we compete: merge and sync. The unique-LCA three-way merge under the head-sync runtime is Yjs-level and 17 to 59× faster than the wasm libraries in these sessions:

system	sync median freq	sync median bulk	wire payload/sync freq	wire payload/sync bulk
ours (embed RGA, shipped)	107 µs	754 µs	in-process†	
Yjs	90 µs	1.13 ms	1.6 KB	15.1 KB
Automerge	1.82 ms	18.3 ms	in-process	
Loro	6.29 ms	17.1 ms	0.4 KB	4.6 KB
list-positions	34 µs	626 µs	6.4 KB	123.6 KB

(`freq` = 60 rounds of 25 ops/replica, final 1804 chars; `bulk` = 6 rounds of 500 ops/replica, final 3604 chars; commit GC runs outside the timed window at millisecond totals.) Two honest caveats. First, the HAMT swap shows up here as a mild regression at this scale: the pre-#111 `freq` sync median was 73 µs, and per-id lookups are now $O(\log n)$ trie walks rather than $O(1)$ Map hits (the same swap cut the local op in this workload from 26 to 2.4 µs, so the trade buys three orders on typing for a fraction of one on small-doc sync). Second, these are small documents,

and while the delta merge no longer *rebuilds* the live set in memory, it still scans O(live set) in time, so the numbers do not extrapolate to 100k-character documents. †The shared-store benchmark merges in-process, so the payload column is not applicable; the separate-store wire protocol (§40) ships a delta that is a function of that round's ops, constant across rounds while a whole-state resync grows with the document, and the runtime's sync tests measure it at 0.28 to 0.60× the whole-state baseline in steady state.

Save size, four representations. Our shipped runtime stores absolute chain coordinates as bit-strings, so the naive save is far above production; the run table (§26.3) is the designed successor, and task #104 ships it as a real serializer. The four representations, in bytes per live character (final state of each sequential trace):

representation (ours)	`ff`	`cs`	`seph`	`am-p`
as-shipped JSON	1637	2503	2991	2320
+ settled-cut compaction	871	1486	933	1295
run-table serialized, SHIPPED	1.5	1.6	1.6	1.2
+ compaction, SHIPPED	1.1	1.1	1.3	1.1
run-table projection (model)	3.8	3.8	3.9	4.0

In absolute terms the shipped, compacted run-table save is **24 / 22 / 73 / 120 KB** for the four traces, down from the as-shipped JSON's **35 / 53 / 170 / 243 MB**: four orders of magnitude, lossless (`decode(encode(s))` reads identically, gated on every trace and by a 120-trial merge/delete property test). Its save-time cost is the honest column: the run-table encode takes 0.2 to 1.7 s across the traces where production saves are milliseconds, pre-existing and unchanged by the apply fast path. The serializer is anchored to the shipped state, not a different datatype: its `accountingBits` routine reproduces the projection's bit totals bit-for-bit, and the shipped bytes sit *below* the projection because the model deliberately charges the recoverable positional fields (per-record run id and offset, and the tail-attachment parent offset) that a real encoder drops. Against production, the shipped run-table save reaches rough parity with the smallest state-only saves:

system, variant	contains	`ff`	`cs`	`seph`	`am-p`
ours, run-table + compaction (SHIPPED)	state	1.1	1.1	1.3	1.1
ours, run-table projection	state (model)	3.8	3.8	3.9	4.0
Yjs update-v1	state + tombstone ids	3.8	4.4	6.0	3.0
Yjs update-v2	run-length coded	1.9	2.0	2.4	1.5
Automerge save	full history	1.3	1.2	3.6	1.2
Loro snapshot	state + history	2.9	3.1	5.9	2.4
Loro shallow-snapshot	state only	1.0	1.1	0.9	0.6
list-positions	state (JSON)	4.7	2.9	10.1	4.7

The shipped save is below Yjs update-v2 and Automerge's compressed history and on par with Loro's shallow-snapshot (0.6 to 1.1 B/char, still the smallest); history-carrying rows (Automerge always, Loro snapshot) are never compared against state-only rows without the label saying so.

The categorical win: storage on delete. Ours is the only save that shrinks when the document does. The churn workload runs five cycles of insert 2000 then delete 1800 characters,

then a final 200-character insert, and asks per row whether the save *grows across a delete phase* (bytes, ours in the packed binary estimate):

system, variant	c1-ins	c1-del	c5-ins	c5-del	final	grows?
ours (binary est.)	22278	2191	72322	25891	31596	never
ours, compacted (est.)	11576	1090	18884	6524	7882	never
Yjs update-v1	32931	31671	163331	161995	165395	never
Yjs update-v2	9539	8812	45776	44841	45856	never
Automerge save	8004	11387	56294	59724	60742	**always**
Loro snapshot	13349	15337	76144	78290	79679	**always**
Loro shallow-snapshot	6170	878	8677	3296	3882	never
list-positions	121933	94172	454518	428465	436314	never

The final document is 1200 live characters in every row. Automerge's save grows by roughly 3.4 KB across every delete phase (history-carrying, monotone by design) and Loro's full snapshot likewise; Yjs never grows but cannot shed tombstone structure (165 KB v1 for those 1200 characters against our compacted 7.9 KB and Loro's shallow 3.9 KB). This is the storage-on-delete axis of the retention taxonomy (§29) measured live: tombstone-freedom is the design's semantics showing up as the only save that gets smaller when the document does.

Caveats, so the numbers cannot be over-read. Timer overhead is not subtracted and biases the sub-microsecond medians; Automerge is driven one change per character (the automerge-perf convention; batching would lower its per-op cost and history size); list-positions is a positions library, not a full CRDT, and its sync row is its documented op-log integration with an unoptimized JSON payload; heap numbers are not comparable across the wasm boundary; and cross-system merged *order* may legitimately differ on concurrent insertions, so only intra-system convergence is gated.

42 Push-button discharge: an SMT translation-validation layer

This is a tool, not a theorem, and the section labels it as such. The eight verification conditions the flat capstone consumes (§13, `flat_ra_linearizable3_eq`) are, for a flat instance, algebraic enough to hand to an SMT solver, and the `smt/` layer (tasks #99/#49) does exactly that: per instance it discharges the eight by z3, via unconditional laws that each *imply* the corresponding conditioned VC (the `DeltaVCs3` on-ramp (`feasibleDeltaVCs3_of_delta`, Definition 6.1) for the feasible laws, `cdVC3_of_all_comm` for the causal-delta bound: the smt bundle's `all_comm` law is that lemma's hypothesis). This is *translation validation*: the Lean metatheory stays the source of truth, and the trust gap is the fidelity of the hand-mirrored z3 term to the Lean signature, closed by calibration rather than by a verified translator.

Calibration, exact against Lean. Four shapes span the space: Counter/PN (numeric group), GSet (set lattice), LWW (timestamp lex), and ORSet (instance-set with kill sets, non-commuting). The first four return UNSAT (VC valid) on all eight, matching their Lean discharges exactly. The OR-set is the informative one: six of eight are push-button, and the two that come back `sat` are precisely the two that are *configuration-conditioned in Lean* (its `feasible_local_redistribute`, Flat VC 6, which needs canonical-state tag-freshness, and

`CDVC3`, Flat VC 8, which needs the maximal-remove trichotomy). Their unconditional over-approximation is genuinely false, so the solver's `sat` is not a mismatch with Lean but a precise identification of the two VCs that required hand proof. Five deliberately broken mutants (drop the $-l$ in a counter merge, `max`-merge a PN, project a GSet merge, arbitrate LWW on the timestamp alone, drop the $\neg l$ guard in the OR-set) are each caught with a readable countermodel.

The #49 loop, and the normal form it forced. The success metric was a remove-wins set whose remove-records are garbage-collected against the LCA. Its literal specification comes back `sat` on both feasible-redistribute laws, and the loop's finding is why: the LCA-conditioned supersession drop is *order-sensitive* (whether a remove is collected depends on adds introduced by the very delta being redistributed), so the merge is not a convergent semilattice and the delta laws fail unconditionally, with a countermodel exhibited. The repair the loop forces is a *normal form*: normalize each element's remove-records to the per-element maximum timestamp (a remove superseded by a higher one is intrinsically absent), and likewise its adds; the merge becomes a product of two `max`-semilattices, LCA-blind, and all eight VCs then discharge push-button, zero hand proof, with the semantics preserved exactly (an element is present iff its maximum add strictly exceeds its maximum remove). Three further kill-tests (eager GC without the LCA guard; an add-wins tie-break; `min`-ing the remove timestamp) are each refuted by at least one genuine `sat` countermodel; a few individual kill queries hit the solver timeout instead of returning, but every killed variant carries `sat` verdicts independent of those (`smt/results/results.json`). The SMT loop therefore did not merely certify #49's spec; it refuted the literal one and pushed the design to the max-timestamp normal form under which the whole bundle is push-button.

Recommendation. The right integration is certificate replay: use the SMT layer as a pre-flight oracle that tells a new flat instance's author instantly which VCs hold and which need conditioning (the OR-set-style boundary) and produces the countermodel when a design is broken, with a thin Lean tactic re-proving the UNSAT cells natively (most already close by `omega` or `cases <;> rfl`). The residual hand proof then shrinks to exactly the configuration-conditioned cells the triage flags — for the whole commuting class, and for the repaired #49 design, empty.

Part V
Open questions and the mechanization map

43 Open questions

The consolidated list: the framework questions of the metatheory (Part I), the campaign's successor question (Part III), the retention question of the FugueMax arc (Part II), and the questions opened by the runtime and storage extension (virtual LCAs, GC, stability, the run table). Two resolved questions are kept at the end in compressed form, because the later parts depend on their corrections; the Weidner–Kleppmann Theorem-9 gap, listed as open in the previous revision of this document, is closed (§24) and no longer appears.

Open Question 43.1 (Is there a rely-guarantee program logic for MRDTs?)**.** *The framework already carries the skeleton of a verification-condition logic: the signature-plus-conditioning is the "program", RA-linearizability up to ≈ is the judgment, the eight VCs are local proof obligations, the metatheorem is their soundness theorem, and the product ⊗ with* `JoinLemma3At` *is a frame/composition rule (the coupling hierarchy L0–L3 stratifying it by interaction strength, as disjoint-vs-shared reasoning does in separation logic). The conditioning layer is specifically rely-guarantee: honest delivery is the rely (an assumption on other replicas' and clients' steps), convergence/safety up to ≈ is the guarantee, merges are environment steps, and the "stable under concurrent honest extension" condition the safety metatheorem turns on (§15: the bounded counter succeeds and BudgetCart/FWW fail on exactly it) is the rely-guarantee stability side-condition, arrived at independently. What is missing to make it a program logic proper is a syntax: a language building* do/merge *from primitives (a per-replica counter cell, OR-set membership, an LWW/FWW* max/min *cell, a sequence node) and combinators (the product ⊗, the indexed map, the payload-parametric wrap of* `ORSetCore`*), each carrying its local VC contribution and its join-species, such that well-formedness entails RA-linearizability by construction — so that writing an MRDT and proving it converge become one act, as separation logic made heap-program verification syntax-directed. The semantic combinators already exist and are each separately proven sound; what is open is the grammar together with a single soundness theorem for the grammar (not one per combinator). This question subsumes the two framework-theory questions below:* CDVC3*-derivability (43.2) asks whether the rule set is minimal, and the structure theorem asks to classify the primitives.*

Open Question 43.2 (CDVC3-derivability)**.** *Is* CDVC3 *derivable from VCs 1–7 (plus the unconditional delta laws where they hold)? Specializing the merge to an LCA-blind lattice join — the CRDT case — the question closes exactly: there, the corresponding causal-delta bound is equivalent to the Join Lemma and no strictly weaker bridge VC exists, with both attack routes characterized — the positive induction is circular at two identified case shapes, and the countermodel space is narrowed to non-atomic lattice states (all of this mechanized, in* `Sal/CRDTs/Metatheory/`*). The ternary question, by contrast, is now* ***closed****:* `CDVC3` *is an independent rule of the ternary bundle (*`cdvc3_not_derivable_from_core_delta`*,* `Refutations/CD_Not_Derivable_Ternary.lean`*, kernel-clean) — there is a* `ConditionedMRDTSig` *(*`AWSetF3`*, the LCA-blind lift of the binary inflation-separator, with a deflationary* rem*) satisfying every* `CoreVCs3` *and* `DeltaVCs3` *clause yet falsifying* `CDVC3`*, so* CoreVCs3 + DeltaVCs3 ⇏ CDVC3 *(and, via* `joinLemma3_iff_cdVC3`*,* ⇏ *the Join Lemma). The reason ternary closes where binary (b″) stays open is instructive:*

`CDVC3`'s $\sqsubseteq$-half demands update inflation ($A \sqsubseteq \mathsf{update}\, A\, e$), which the binary `LatticeVCsPlus` ships as a separate axiom but the ternary `DeltaVCs3` honestly omits — so the inflation-free delta laws cannot back `CDVC3`, and a deflationary update refutes it. The eight-VC bundle is therefore ***minimal at*** *`CDVC3`: the causal-delta rule is a genuine independent axiom, not a derived lemma (a load-bearing point for the rule-basis of Open Question 43.1)*

Open Question 43.3 (structure theorem)**.** *Empirically, the unconditional delta contract holds exactly on the group and lattice classes. Is every unconditional-*DeltaVCs3 *merge a torsor-like group$\otimes$ lattice combination? (The naïve two-sorted umbrella* $\mathsf{mergeL}(l, a, b) = a \boxplus (b \boxminus l)$ *does not derive the contract uniformly, so the question is genuinely open.)*

Open Question 43.4 (the automation layer)**.** *The published catalogue's seventeen induction-scheme VCs were an SMT-facing Skolemization of feasibility, aimed at the wrong target. Design the analogous per-data-type induction scheme whose induction implies* CDVC3 *and the feasible delta laws — restoring push-button discharge, now of VCs that actually imply RA-linearizability. The observed uniformity of the discharges (σ-characterization by sandwich invariant + absorber trichotomy) suggests the scheme exists.*

Open Question 43.5 (tightness of the MCA-closure keep set)**.** *Under virtual merges the keep set retains the upward event-set closure of* mcasClosure(heads)*, and safety is proved: every future virtual resolution reads only kept versions (`gc_safetyV`, §17). Tightness is the unexamined converse: is every closure member actually readable by some future virtual merge? The harness already shows non-members can be dead, but whether the closure itself over-retains is open; a negative answer (some closure members are unreadable in every future) would license a strictly smaller keep set, and the gateway lemma's structure suggests where to look: closure members reachable only through non-head prune-time versions.*

Open Question 43.6 (open membership and certificate transport for stability)**.** *The stability gate is no longer assumed. On an honestly reachable configuration* AllHeardSince *(a purely local frontier fact) implies* SettledAt *(`settledAt_of_allHeard`, §18), and the running system realizes the theorem as `compactStable`'s refuse-then-fire (§40). Two operational gaps remain. Open membership: the certificate quantifies over a known replica set, and a replica that joins after a compaction is the theorem's one breaker (*createReplica*), handled today by a policy gate; a model where the replica set is itself a grow-only MRDT could turn the gate into a theorem. Certificate transport: can a replica hand a peer a compact, checkable certificate of SettledAt (a frontier vector plus the evidence commits) so the callback fires at nodes that did not compute the frontier themselves? The commit-shaped-not-event-shaped erratum constrains the answer: the certificate must name commits, since an event-set summary can miss the pull that carries no new events yet opens the gate.*

Open Question 43.7 (beyond the closed honesty-rebasing lemma ($*$): the protocol face)**.** *The lemma this question originally named is closed. The single-epoch honesty gap (T3: the re-coded history is honest only modulo the coordinate order-embedding, §26.1) and the multi-epoch* CompatChain *residue (§26.2) were one obligation, ($*$): exhibit an epoch-2 configuration honest modulo the embedding F_1, so that anchored-existence re-derives at the boundary. Its discharge is the coded-forest cluster (Definition 26.21, Theorems 26.22, 26.23, 26.24, Corollary 26.25): honesty is not re-derived but re-based, the history-free invariant I crossing the epoch boundary where* EHonest *cannot. What remains open is the protocol half. Single replica, the closed cluster consumes construction-shaped hypotheses ($h_{\mathrm{RestShape}}$/$h_{\mathrm{MintShape}}$, true of the shipped compaction*

and fusion maps); the question is which delivery protocol discharges the per-continuation face: derive ContOK *and the straggler-declaration hypotheses (§27.5) from an honest transport layer rather than assuming them per continuation. Multi-replica, concurrent divergent compaction is now closed at* $s1$ *(Theorem 26.18, Open Question 43.8); the per-continuation face here is the sole single-replica residue. The marks-layer capstone (§27) consumes the same residue and adds nothing to it: its* `ContOK` *and straggler-declaration hypotheses are the per-continuation face of exactly this protocol half.*

Open Question 43.8 (concurrent compaction: the epoch DAG)**.** *The mechanized stability metatheorem covers compaction epochs that are linearized per replica (the v1 runtime, and the demo's checkpoint barrier, refuse compaction off the newest epoch), VC-S6 gives observational coherence for nested cuts on one lineage, and the single-replica multi-epoch composition is closed: the shared honesty-rebasing lemma* $(*)$ *it once waited on is discharged by the coded-forest cluster (Definition 26.21, Theorems 26.23 and 26.24), and what remains on the single-replica side is only the protocol face of Open Question 43.7 — deriving the per-continuation hypotheses (*ContOK*, the straggler declarations) from a delivery protocol. Concurrent divergent compaction, two replicas compacting at incomparable cuts* S, S' *and still merging, was the conjectured shape of this question: an epoch DAG with common-refinement translation. It is now done, at all three registers. The model validated it at* $s1$ *(bit-identical) across* 2400 *trials (*`whiteboard/epoch_diamond_check.py`*); the mechanization is Theorem 26.18 with its transported carried domain (*`transportedDom`*, the aliasing negative Proposition 26.19), the map-drop certificate (*`mapDrop_sound`*, ack-only refuted), and the join-epoch freshness discharge (*`contOK_root_key_fresh`*); the runtime ships it as a cut-indexed epoch DAG replacing the cross-epoch throw (Remark 26.20). Three residues remain, and they are honest. (i) The general derivation of leg-agreement for arbitrary histories from the renumber-and-fusion composition algebra is reduced to the pointwise certificate-agreement condition and validated on the directed cases plus the* 2400 *trials, not mechanized in full generality. (ii) The runtime realizes the join at the greatest lower bound rather than the join cut (Remark 26.20); its inverse-map garbage collection, distinct from the forward-map drop closed above, is a follow-on. (iii) The liveness half is untouched and may be genuinely hard: a single replica withholding its heard-from evidence blocks the certificate and so blocks compaction forever, which either is an impossibility result or wants an eviction rule, and lands with the Byzantine and liveness questions (both, to our knowledge, unclaimed for CRDT metadata management).*

Open Question 43.9 (rc-expressiveness: chains, LWW, and merge-recoverability)**.** *The catalogue's* rc *relations are all star-shaped: the conflicting pairs are cross-class (*rem/add*; add/checkout), so every edge runs from a class that is never second to a class that is never first. This is forced: the update layer's directionality law (*`rc_non_comm_directional`*) orients every non-commuting concurrent pair, while its companion* `no_rc_chain` *forbids length-2 paths, and any orientation of a self-conflicting class (concurrent writes; concurrent claims of one slot) contains one. So total-order policies — last-writer-wins first among them — are inexpressible in the* rc *mechanism as constrained by the VC set. Two questions, one of them already closed at its boundary. (i) Would a chain-tolerant generalization (the concurrent arm of* lo *as a transitive tiebreak) be sound? The semantics is coherent — with Lamport clocks the composite of* vis *and any timestamp- or priority-oriented tiebreak is acyclic, and canonical states remain well-defined — so the restriction appears to be an artifact of the swap-flavored proof machinery (*`cond_comm_lift` *cannot bubble past two consecutive* rc *edges), not of the theorem. The decisive experiment: re-prove the swap VC and the causal-delta peel over a transitive tiebreak; we expect the swap route*

to break and the canonical-state route (which peels a global maximum, and a total order has one everywhere) to survive. (ii) What would it buy? Not a metadata-free LWW register: whatever the order arbitrates, the converged outcome must be produced by $\mathsf{mergeL}(l, a, b)$ *— a function of three states — and for a plain-value LWW two executions present identical state triples with opposite timestamp orders, so no such function exists (`lww_merge_needs_timestamps`, `Refutations/LWW_Merge_Needs_Timestamps.lean`, kernel-checked). The arbitration key is forced into the state by the merge; once there, writes commute and* rc *is moot — which is why every LWW implementation stores its timestamp. What a chain-tolerant arm could buy: metadata-free priority policies across three or more op classes (outcomes state-recoverable, orientations chained). The general principle the kill-test isolates:* rc *is the mechanism for conflicts whose outcome is recoverable from branch states; clique conflicts pay for their arbitration key in state, and chains would extend* rc *only within the recoverable class.*

Resolved in framing by the arbitration recast (§7). Total-order policies are expressible in the framework after all, not by generalizing rc *but by demoting it: the adequacy engine runs over an abstract arbitration, and LWW instantiates it by its timestamp total order directly (Theorem 7.5), discharging the six clauses without an* rc *arm. So question (i)'s chain-tolerant-*rc *experiment is superseded for adequacy — the total order need never be squeezed into* rc *at all. Question (ii) stands unchanged: the recast still does not buy a metadata-free LWW (`lww_merge_needs_timestamps`); the arbitration key remains forced into the state by* mergeL.

Open Question 43.10 (Causal canonicity under general rc)**.** *The generic safety metatheorem consumes causal canonical witnesses — enumerations linearizing* $\xrightarrow{\mathsf{vis}}$ *and respecting* lo^E *jointly. With every concurrent pair unordered (Either), the two discharge species of §15 suffice. Under a general* rc *the existence of a joint linearization is open: a cycle would need at least two* rc*-edges separated by* $\xrightarrow{\mathsf{vis}}$*-paths — a single* rc*-edge closed by a* $\xrightarrow{\mathsf{vis}}$*-path contradicts concurrency via transitivity, and a non-commuting first* $\xrightarrow{\mathsf{vis}}$*-step is an absorber killing the edge — and `no_rc_chain` forbids adjacent* rc*-edges but not* $\xrightarrow{\mathsf{vis}}$*-separated ones. `CausalCanonical` is therefore a hypothesis with two discharge lemmas rather than a theorem. The budget cart (`BudgetCart`, `MRDT_Instances/BudgetCart/`) has now forced the question — and widened it: its convergence discharges along the OR-set route, but the ungated safety obligation `SafetyStepOn` is false for it, by a two-event refutation — `CausalFold` pins only* $\xrightarrow{\mathsf{vis}}$*-respect, and the fold of a causal past containing a concurrent same-item* add/rem *pair is enumeration-dependent (the orders disagree exactly where add-wins should arbitrate). So for* rc*-nontrivial datatypes the honesty fold needs* rc*-orientation too: the right notion is folds along jointly* $\xrightarrow{\mathsf{vis}}$*- and* lo^E*-respecting enumerations, for both the version witnesses (`CausalCanonical`) and the causal-past folds (`CausalFold`). The cart's safety theorem is delivered gated (`bcart_version_inv_gated`, with the missing transfer named explicitly as `BCartSpendMono`); closing the gate is this question.*

Open Question 43.11 (Does conditioning compose? — convergence: yes, now a theorem)**.** *The catalogue is built one data type at a time; practice builds data types from data types — the canonical case is the key-value map whose values are themselves MRDTs (the certified-MRDT line's own composite). The factored framework suggests composition should happen at the `JoinLemma3At` boundary — the map combinator would consume each value's per-configuration join, whatever species produced it, so a map of queues composes as readily as a map of counters — with a small once-only kit: folds and configurations project per key, cross-key pairs commute so* lo^E *localizes, and per-key joins re-interleave. Conjecturally the coupling strength stratifies what survives composition: payload parametricity (free), the indexed product = the grow-only-keys map (a theorem*

to prove), read coupling — one layer's app *reads another's state, as in a budgeted cart — (convergence composes; safety needs a monotone-transfer lemma), uniform coupling — key removal, reaching into the value layer the same way for every value type (one generic obligation: a reset element and a* remove ‖ *v-op policy) — and ad-hoc effect coupling (rich-text marks anchored in a sequence), where nothing is expected to compose automatically; the marks layer of §27 is that coupling made concrete, and its read model and GC are indeed discharged directly at the render level rather than through the product kit. The convergence half is now mechanized and stated in Part I: the binary heterogeneous product $D_1 \otimes D_2$ composes at exactly the* `JoinLemma3At` *boundary (Theorems 15.20 and 15.21, end of §15; consumability demo: a queue ⊗ counter capstone with zero bespoke proof,* `MRDT_Instances/ProductDemo/`*). Two boundary findings from the pen-and-paper pass (*`Development/COMPOSITION_PENPAPER.md`*): reachability does not project (a second-component apply is a version-duplicating stutter for the first's projection), so finished capstones never compose — only per-configuration certificates do, which retroactively forces the factored interface of §15; and two-sided causal-witness re-interleaving is refuted (a realizable four-event cross-*$\xrightarrow{\text{vis}}$ *cycle), with the sound repair — pin one side, re-sort the other — covering products with at most one* rc*-nontrivial component. The safety and $\approx$ kits are also mechanized: honesty,* `SafetyStep`*, and one-sided causal witnesses compose (*`Metatheory/Product_Safety.lean`*: the pinned-extension lemma,* `safetyStepOn_prod`*,* `causalCanonical_prod_of_one_sided`*, composite* `prod_version_inv_on_of_one_sided`*), and the eq-quotient layer lifts at the pragmatic cut $\approx_2 =$ = (*`Metatheory/ProductEq.lean`*: every* `GenericEqQuotient` *obligation componentwise,* `eqJoinLemma3C_H_prod` *glued by concatenation with no congruence chasing, capstone* `prod_ra_linearizable_up_to_eq_H` *— one quotiented component, one flat, exactly the Peritext shape, with the quotiented side's reachability supplies as premises per the memo's cost-center analysis). Still open: the general $\approx_1 \times \approx_2$ lift (no new ideas, double the plumbing — deferred until a second quotiented component exists), the indexed (map-of-MRDTs) generalization, and the L2.5/L3 coupling boundaries, where composition breaking is expected to be as informative as where it holds.*

Open Question 43.12 (the sequential witness at merges)**.** *On every honestly reachable configuration of the embedded-chain RGA, does the event union admit a delivery-respecting enumeration that is also sequentially honest, so that the merged read is the naive buffer's output on it? Caution from adjacent territory: for the rehoming design the analogous "order dependency edges first" discipline is refuted (the* `DepComp` *refutation, via rehoming non-transitivity). The embed's immutable coordinates remove that refutation's mechanism, but no proof exists in either direction.*

Open Question 43.13 (the chain-price program: possibility and impossibility of retention)**.** *The fooling pair of §25 gives this document its first representation-level impossibility (previous negatives were all specification-level), and it suggests a program: for each intent tier (the RGA reads, Fugue, FugueMax; delete-order preservation), characterize the exact information a tombstone-free state must retain, fooling pairs as lower bounds, capstones as upper bounds. Three concrete instances. (i) Is the full dead chain necessary, or only its order type relative to live stamps? The renumber epoch already exploits rank-compressibility below settled cuts; is the settled gate exactly the boundary of compressibility, or can unsettled retention also be rank-coded against a fooling-pair obstruction? (ii) The FugueMax tag retention (Open Question 43.17) is this program's maximal-intent instance. (iii) Delete sensitivity: is every delete-sensitive read function unimplementable over retention-free states, the general form of "no policy reaches Shesha"?*

Open Question 43.14 (the run-table wire format: the one face still open)**.** *The run-table theorem bill is now closed on both sides (task #73), and the results are presented natively upstream: the four closed T-mut equations and the literal table-direct walk (`tableWalk_tableOf`) in §26.3; the sided mirror (the repr iso over labels, sides, and liveness, the offset-vs-tail comparator `scmpTable_eq_keyLt`, the in-order walk with its literal `stableWalk`, and the all-R lockstep bridge `sTableOf_liftR/swalk_liftR`) in §26.4. What remains is one engineering face: task #104's serializer turned the projection column into measured bytes (§41), and task #111's persistent-HAMT swap removed the interface-inherited per-op copy, so the open item is a delta/op wire format for sync (the concurrent payload column is still in-process).*

Open Question 43.15 (FugueMax tag re-mapping)**.** *The re-coding cluster (§26.1) rewrites coordinates at the record's top level. The FugueMax variant embeds keys inside R-entry tags, so an epoch re-map must rewrite coordinates occurring inside tag payloads, applying the stable-prefix factorization under a congruence over the tag structure. The TagsOK finding (§24) raises the stakes: tag wellformedness is convergence-load-bearing, so an ill-formed tag re-map would break convergence itself, not merely display intent. Neither the congruence nor its H2 argument exists yet.*

Open Question 43.16 (generation-discipline dependence of storage arguments)**.** *`AnchorsFactorBeyond` (§26.2) is a theorem under the generation discipline and refutable from bare honest reachability (the countermodel forges the carried split while keeping the minted coordinate honest). How much of the storage layer has this shape? The conjecture: every storage-layer hypothesis that quantifies over carried payloads (op prefixes, tags, in-flight declarations) is dischargeable from generation honesty and refutable without it, so that a uniform metatheorem ("payload facts come from the mint," the storage echo of Part I's `GenHonest` shape) would collapse the per-instance discharges. A refutation instance would be equally informative: a payload fact needed by some compaction that generation honesty does not pin.*

Open Question 43.17 (the retention lower bound for tombstone-free maximal non-interleaving)**.** *The FugueMax kernel variant realizes its sibling order by embedding the mint-time successor's key into contested R-entries, at $\Theta(|\text{origin key}|)$ coordinate cost where plain Fugue pays zero (§30). Is that retention a lower bound: must any tombstone-free design achieving maximal non-interleaving carry the right origin's ordering information in its keys? The question has a Myhill–Nerode shape (task #89): distinguish states that demand different verdicts, and count what any key scheme must remember. It is the entropy law of §20 meeting the ordering intent, and the fooling-pair technique of §25 is the designated tool: exhibit two histories whose tag-free states coincide while maximal non-interleaving owes them different orders.*

Resolved: criss-cross merges and virtual LCAs. The previous revision asked whether the eight VCs imply that a recursively computed virtual base equals $\sigma(E_1 \cap E_2)$, guessing inclusion-exclusion arguments per datatype and a candidate new store invariant. The answer landed stronger and cheaper than forecast (§16): the covering proposition makes the virtual base's event set *exactly* the intersection from invariants already carried (`StoreInv.origin` plus monotonicity, no new invariant), one fold induction from the per-configuration Join Lemma gives canonicity of the virtual state, and the per-datatype VC surface does not move. The per-datatype question dissolved into the hook the framework already demanded. The residue that remains open is operational, not semantic: keep-set tightness (Open Question 43.5).

Resolved: the self-referential-spec limit. RA-linearizability certifies that a version's state is the fold of some delivery-respecting linearization of its events: convergence to the datatype's *own* `do`-fold, with that fold as the reference. It says nothing about whether the fold's single-replica semantics is the intended one; a defect in `do` appears identically in the concurrent state and its fold, and is certified as agreement. Independent *intent* specifications are therefore a separate obligation, not a corollary of convergence. The question was resolved systematically by the sequential-specification campaign (Part III), which supplies the independent specification for every production RDT; its refutations also correct the earlier reading of the sharpest instance. The survivor reorder exhibited by the campaign's four-op trace (§36) is not intrinsic to tombstone-freedom, it is intrinsic to rehoming: the embedded-chain RGA is tombstone-free, satisfies the sequential list specification in the strongest form (`embed_seq_sound`: the canonical state is the naive buffer, record for record), and preserves delete order (`eUpdate_del_sublist`). The successor question is Open Question 43.12.

44 Mechanization map

Everything above is mechanized in Lean 4 (Mathlib; axioms: `propext`, `Classical.choice`, `Quot.sound`; no `sorryAx` anywhere in the cited chains; refutations by concrete witness follow the SPOT convention, `native_decide` adding the compiler axioms) in the Sal repository. Files are relative to `Sal/ConditionedMRDTs/` unless they carry an explicit `Sal/` prefix.

Artifact	Lean	File
Part I: framework and metatheorems		
signature, ternary lo	`MRDTSig`, `ConditionedMRDTSig`	`Framework/MRDTSig.lean`
store, DAG, transitions	`Configuration`, `Step3`	`Framework/ExecutionModel.lean`, `Metatheory/LCA_Lemma.lean`
Lemma 4.2	`lca_events_of_storeInv`	`Metatheory/LCA_Lemma.lean`
σ/lo^E layer (merge-independent)	`UpdateVCs`, `IsCanonicalState`	`Framework/Sigma_Lo0n3.lean`
the eight VCs	`CoreVCs3CD`, `FeasibleDeltaVCs3`, `CDVC3`	`Framework/VC_Set.lean`
Theorem 10.1 + bridges	`ra_linearizable_of_core_feasible_cd3`	`Metatheory/Adequacy.lean`
defeater vs. 2-OP (§4.2)	`no_inter_lca_2op_rem_peel_of_defeater`	`Refutations/InterLca2op_Defeater_Arbiter.lean`
closure-indexed contract (§10)	`JoinLemma3C`, `no_proper_back_block`	`Metatheory/JoinLemma3C.lean`
closure-indexed adequacy + instances	`ra_linearizable3_of_joinC`, `ConditionedContract`	`Metatheory/ConditionedContract.lean`
all 9 flat discharges via the contract	`*_adequate_viaContract` (9)	`Development/All_MRDTs_viaContract.lean`
per-configuration join hook	`JoinLemma3At`, `goodConfig3_merge_at`	`Framework/VC_Set.lean`, `Metatheory/Adequacy.lean`
honest-reachability metatheorem	`HonestReach`, `ra_linearizable3_of_honest_reach`	`Metatheory/HonestReach.lean`
generic honesty shape	`GenHonest`, `AppHonest`	`Metatheory/GenHonest.lean`
witness-class join lemmas	`JoinLemma3AtW`	`Metatheory/WitnessClass.lean`
generic safety (causal witness)	`SafetyStep`, `version_inv_of_causal_canonical`	`Metatheory/GenericSafety.lean`
escrow metatheorem	`Measured`, `escrow_version_inv`	`Metatheory/EscrowSafety.lean`
product composition (raw kit)	`joinLemma3At_prod`, `prod_ra_linearizable3_of_honest_reach`	`Metatheory/Product.lean`
product safety kit	`safetyStepOn_prod`, `prod_version_inv_on_of_one_sided`	`Metatheory/Product_Safety.lean`
product $\approx$-lift	`eqJoinLemma3C_H_prod`, `prod_ra_linearizable_up_to_eq_H`	`Metatheory/ProductEq.lean`
observational quotient $D \mapsto D_\approx$	`QSig`, `qapplicable`	`Metatheory/GenericEqQuotient*.lean`
canonical-witness layer, raw-$\approx$ target	`IsRALinearizable3Eq`, `RA_linearizable_up_to_eq_H`	`Metatheory/GoodConfig3H.lean`
flat collapse of the framework	`eqJoinH_of_joinC`, `flat_ra_linearizable3_eq`	`Metatheory/FlatGeneric_Bridge.lean`
LWW merge needs timestamps (OQ 43.9)	`lww_merge_needs_timestamps`	`Refutations/LWW_Merge_Needs_Timestamps.lean`
CDVC3 independent of core+delta (OQ 43.2)	`cdvc3_not_derivable_from_core_delta`	`Refutations/CD_Not_Derivable_Ternary.lean`
impossibility kill-tests	—	`Refutations/Impossibility.lean`
findings journal	—	`Development/MRDT_METATHEORY_DRAFT.md`
Part I: the runtime arc (§16–§18)		
virtual-merge step relation	`Step3V` (base embedding + `mergeVirtual`)	`Metatheory/LCA_Lemma.lean`
covering proposition (Prop. 16.4)	`mca_events_cover`	`Metatheory/LCA_Lemma.lean`
virtual fold induction	`virtualLCAState_canonical`, `goodConfig3_mergeVirtual_at`	`Metatheory/Adequacy.lean`
honest reachability over `Step3V`	`HonestReachV`, `ra_linearizable3_of_honest_reachV`	`Metatheory/HonestReach.lean`
single-MCA picks refuted (criss-cross pins)	`t1f_pick_u_resurrects_y`, `t1f_pick_v_resurrects_x`	`Metatheory/VirtualLCA_Spot.lean`
H-layer + flat catalogue over `Step3V`	`flat_ra_linearizable3_eq_V`	`Metatheory/GoodConfig3H_V.lean`, `Metatheory/FlatGeneric_Bridge_V.lean`
queue over virtual merges (§16)	`queue_ra_linearizable3_V`	`MRDT_Instances/MergeableQueue/`
rehoming Eq capstone over `Step3V` (stress test)	`rga_ra_linearizable3_eq_V`	`MRDT_Instances/RGA_Rehoming/RA_Lin_V.lean`
commit GC: keep set + safety (§17)	`keepSet`, `gc_safety`, `lca_not_dropped`	`Metatheory/GC_Safety.lean`
GC under virtual merges	`mcasClosure`, `keepSetV`, `gc_safetyV`	`Metatheory/GC_Safety.lean`
bounded-state GC: clock swap (§17)	`clockPrune`, `ClockCoherent`, `clockLCA_sound`, `gc_safety_bounded`	`Metatheory/GC_BoundedState.lean`
SettledAt discharged from the frontier (§18)	`AllHeardSince`, `settledAt_of_allHeard`	`Metatheory/EvidenceDischarge.lean`
SettledAt def + stability bundle + metatheorem	`SettledAt`, `StabilityVC`, `stability_simulation`, `stability_ra_inherited`	`Metatheory/Stability_VC.lean`
OR-set stability instance	`orStabilityVC`, `ORSet_stability_reads`	`MRDT_Instances/ORSet/ORSet_Stability.lean`
static drop set refuted	`static_drop_unsound`	`MRDT_Instances/ORSet/ORSet_Stability.lean`
Part I: instances (Table 1)		
catalogue instance theorems	all instance theorems	`MRDT_Instances/` (one directory per RDT)
the flat instances, generically	`*_ra_linearizable3_eq` (11)	`MRDT_Instances/` (per-RDT)
bounded-counter safety	`bc_version_inv`, `bc_value_nonneg`	`MRDT_Instances/BoundedCounter/`
mergeable queue (direct join)	`q_join_at`, `queue_ra_linearizable3`	`MRDT_Instances/MergeableQueue/`
FWW register (payload arbitration)	`fww_version_min`	`MRDT_Instances/FWWRegister/`
BudgetCart (gated safety)	`BCart_ra_linearizable3_eq`, `bcart_version_inv_gated`	`MRDT_Instances/BudgetCart/`
Part II: the embedded-chain family		
model-layer kernel	`rc_non_comm'`, `display_iff_chainBefore`, `subtree_convex`	`Sal/MRDTs/RGA_Embed/`
embedded-chain RGA (queue route)	`embed_ra_linearizable3`, `e_fold_canon`	`MRDT_Instances/EmbedRGA/`
Elias-δ inheritance corollary	(capstone at the δ-flip)	`MRDT_Instances/EmbedRGA/EmbedRGA_EliasDelta.lean`
compaction theorem	`rga_read_eq_embed_read`	`MRDT_Instances/EmbedRGA/EmbedRGA_ReadEquiv.lean`
compaction order core	`visible_lt` = chain-lex	`Sal/MRDTs/RGA_Embed/RGA_Embed_ReadEquiv.lean`
sided chain-lex kernel	`sdisplay_iff_schainBefore`, `schainBefore_liftR`	`Sal/MRDTs/RGA_Embed/Sided_ChainLex.lean`
sided RGA + per-policy intent	`sided_embed_ra_linearizable3`, `sFold_liftOp`	`MRDT_Instances/SidedRGA/`
Fugue/FugueMax vs. W–K Def. 4	`fugue_not_maximally_noninterleaving`, `fuguemax_same_origin_low_first`	`MRDT_Instances/SidedRGA/`, `Sal/MRDTs/RGA_Embed/SidedMax_ChainLex.lean`
Peritext on the embed kernel	`peritextEmbed_ra_linearizable3`, `renderIds_del`	`MRDT_Instances/Peritext_Embed/`
Part II: Theorem 9 and the storage layer (§24–§26)		
traversal theory, interval form	`schain_ext_after_is_R`, `schain_ext_before_is_L`	`Sal/MRDTs/RGA_Embed/Sided_Traversal.lean`
forward NI, FugueMax	`fuguemax_forward_ni`	`MRDT_Instances/SidedRGA/SidedRGA_NonInterleaving.lean`
forward NI, plain Fugue (as stated)	`fugue_forward_ni`	`MRDT_Instances/SidedRGA/SidedRGA_Fugue_ForwardNI.lean`
Theorem 9 capstone (§24)	`fuguemax_maximally_noninterleaving`	`MRDT_Instances/SidedRGA/SidedRGA_Backward.lean`
FugueMax convergence (TagsOK)	`fuguemax_ra_linearizable3`, `f_fold_canon`	`MRDT_Instances/SidedRGA/SidedRGA_FugueMax_RA_Lin.lean`
sibling-splice fooling pair (§25)	`sibling_splice_no_merge_function`	`Refutations/SiblingSplice_Fooling.lean`
re-coding cluster, T1–T3 + collapse (§26.1)	`eRecode_applySeq`, `eRecode_reads_identical`, `eRecode_ext_global_collapse`	`MRDT_Instances/EmbedRGA/EmbedRGA_Recoding.lean`
SettledAt bridge, remap species	`eRecode_settled_bridge`	`MRDT_Instances/EmbedRGA/EmbedRGA_Stability_Bridge.lean`
anchors factor at the cut	`anchorsFactorBeyond_of_anchored`, `eRecode_settled_bridge_honest`	`MRDT_Instances/EmbedRGA/EmbedRGA_AnchorsFactor.lean`
verified concrete compaction (§26.2)	`compactEliasDelta_settled_reads` + in-flight SPOTs	`MRDT_Instances/EmbedRGA/EmbedRGA_CompactEliasDelta.lean`
spine fusion (§26.2)	`compactThenFuse_reads`	`MRDT_Instances/EmbedRGA/EmbedRGA_Fusion.lean`
merge-vs-remap (VC-S4, §26.2)	`merge_remap_congr`	`MRDT_Instances/EmbedRGA/EmbedRGA_MergeCongr.lean`
multi-epoch composition (§26.2)	`StablePrefixMap.comp`, `multiEpoch_settled_reads`, `naive_composition_collides`	`MRDT_Instances/EmbedRGA/EmbedRGA_MultiEpoch.lean`
nested settled cuts ((*)'s closed halves)	`nested_cuts_settled`, `rED_hocc_fresh`	`MRDT_Instances/EmbedRGA/EmbedRGA_CompatChain.lean`
run table, one-sided (§26.3)	`tail_attachment`, `canonical_tableOf`, `cmpTable_eq_keyLt`, `walk_display`, T-mut	`Sal/MRDTs/RGA_Embed/RunTable.lean`, `MRDT_Instances/EmbedRGA/EmbedRGA_RunTable.lean`
run table, sided (§26.4: iso, comparator, walks, all-R lockstep)	`sTableOf_sStateOf`, `scmpTable_eq_keyLt`, `stableWalk_sTableOf`, `sTableOf_liftR`	`Sal/MRDTs/RGA_Embed/SidedRunTable.lean`
document-order mark read model (§27)	`doc_no_backward_leak`, `doc_delete_can_respan`, `tree_backward_leak_refutes_noleak`	`MRDT_Instances/Peritext_Embed/PeritextEmbed_MarkIntent.lean`
marks GC O1–O3 (§27)	`keepSPM_A1`, `scanRight_filter`, `marksGC_render_congr` (single- and two-epoch)	`MRDT_Instances/Peritext_Embed/PeritextEmbed_MarksGC.lean`
marks GC O4, guarded pair-drop (§27)	`a3_guarded_drop` + α/β flip SPOTs	`MRDT_Instances/Peritext_Embed/PeritextEmbed_MarksGC_A3.lean`
Part III: the sequential-specification campaign		
tier 1: eleven flat RDTs	`*_seq_sound` (per RDT)	`MRDT_Instances/SeqSpec_Flat.lean`
tier 2: mergeable queue	`queue_seq_sound`	`MRDT_Instances/MergeableQueue/MergeableQueue_SeqSpec.lean`
tier 3: embedded-chain RGA	`embed_seq_sound`	`MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec.lean`
tier 3, two-sided: sided RGA	`sided_seq_sound`	`MRDT_Instances/SidedRGA/SidedRGA_SeqSpec.lean`
tier 3 corollary: RGA_1	`rga_seq_read_eq_buffer`	`MRDT_Instances/EmbedRGA/EmbedRGA_SeqSpec_RGA.lean` (standalone)
tier 4: fused Peritext	`peritextEmbed_seq_sound`	`MRDT_Instances/Peritext_Embed/PeritextEmbed_SeqSpec.lean`
render fix theorem	`renderIds_del`	`MRDT_Instances/Peritext_Embed/PeritextEmbed.lean`
negative: rehoming RGA	`rehoming_seq_refuted`	`MRDT_Instances/RGA_Rehoming/RGA_SeqSpec_Refuted.lean`
negative: rehoming Peritext	`fused_delete_reformats_survivor`	`MRDT_Instances/Peritext_Rehoming/Peritext_Read.lean` §10
negative: Shesha merge	`Shesha_Rows_Refuted`	`MRDT_Instances/Shesha/Shesha_Rows_Refuted.lean`
The runtime pillar: engineering artifacts (unverified, mirror the verified kernel)		
p2p runtime replica (§40)	`DistributedReplica`, `compactStable`	`runtime/src/replica.js`
peritext marks GC (§27, §40)	`compactPeritext`, `compactiblePeritext`	`runtime/src/compact-peritext.js`
run-table serializer (§41)	`encode`/`decode`, `accountingBits`	`runtime/src/serialize.js`
git persistence + p2p demo (§40)	`persist`/`load`, `WsTransport`	`p2p-demo/src/`
cross-system benchmark (§41)	the matrix	`benchmarks/`
SMT discharge — a *tool*, not a theorem (§42)	z3 translation-validation of the 8 flat VCs	`smt/`

The update-side layer (lo^E, convergence, σ) is merge-independent — it is stated over the update signature $\langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{rc} \rangle$ alone — and is shared with the binary (CRDT) specialization of the theory; in the repository it lives in `Sal/CRDTs/Metatheory/`, the directory that also holds the binary exactness results quoted in Open Question 43.2. The production data types of Table 1 are mirrored faithfully from `Sal/MRDTs/`$*$ (encoding deviations documented per instance); their original 24-VC discharges are untouched.

The foundations live under `Framework/`, the metatheorems under `Metatheory/`, the machine-checked negative results under `Refutations/`, and the per-RDT conditioned instances under `MRDT_Instances/`; findings journals and superseded routes remain under `Development/`, which nothing imports. Each cited theorem is kernel-checked with axioms in {`propext`, `Classical.choice`, `Quot.sound`} (no `sorryAx`). They import one linearization

file carrying two unrelated `sorry`s that the cited theorems do not transitively depend on; `Development/CONDITIONED_METATHEORY_PLAN.md` is their roadmap.